



Technical Specifications

DigitalAgency

1. INTRODUCTION

1.1 EXECUTIVE SUMMARY

1.1.1 Project Overview

AgencyOS represents a comprehensive digital transformation initiative designed to address the critical operational challenges facing modern digital design and development agencies. The platform serves as a hybrid CRM and project management solution purpose-built for agencies, consultancies, and service-driven businesses that manage high-touch client relationships. This unified system consolidates the entire agency workflow from initial lead capture through final project delivery and ongoing support.

1.1.2 Core Business Problem

Scope creep is a dreaded thing that can happen on any project, wasting money, decreasing satisfaction, and causing the expected project value to not be met. Most projects seem to suffer from scope creep, and both project teams and stakeholders are consistently frustrated by it. The digital agency landscape is plagued by fragmented toolsets, inefficient approval processes, uncontrolled scope expansion, asset management challenges, and opaque budget tracking. According to a study by PMI (Project Management Institute), 52% of projects experience scope creep, with 43% of those significantly impacting project success metrics such as schedule, budget, and quality.

Current agency operations suffer from:

- **Tool Fragmentation:** Multiple disconnected systems for CRM, project management, design review, and billing

- **Approval Bottlenecks:** Slow, manual approval processes that delay project delivery
- **Scope Creep:** Uncontrolled project expansion resulting in delays, budget overruns, and decreased overall quality of work and morale
- **Asset Sprawl:** Disorganized digital assets across various platforms and storage systems
- **Budget Opacity:** Limited visibility into project profitability and resource utilization

1.1.3 Key Stakeholders and Users

Stakeholder Group	Primary Users	Key Responsibilities
Agency Internal	Owners, Project Managers, Designers, Engineers, QA, Finance, Contractors	Project execution, resource management, delivery oversight
Client Organizations	Client Admin, Stakeholder/Reviewer, Billing Only, Viewer	Project approval, feedback provision, budget authorization

1.1.4 Expected Business Impact and Value Proposition

AgencyOS delivers measurable business value through operational efficiency and client satisfaction improvements:

Operational Efficiency Gains:

- Reduce approval cycle times to <3 days (p50)
- Minimize unbilled scope to <5% of total project value
- Achieve >90% on-time milestone delivery
- Decrease Days Sales Outstanding (DSO) to <25 days

Strategic Advantages:

- **Unified System of Record:** Single platform eliminating tool switching and data silos
- **Design-Native Reviews:** Integrated Figma workflows with annotated feedback systems
- **Git-Aware Delivery:** Automated CI/CD preview integration for seamless development handoffs
- **Budget Guardrails:** Real-time budget tracking with automated overage alerts at 80/100/120% thresholds
- **Two-Way Integrations:** Seamless connectivity with Figma, Jira, GitHub, Slack, and other essential tools

1.2 SYSTEM OVERVIEW

1.2.1 Project Context

Business Context and Market Positioning

The global project management software market size was valued at USD 6.59 billion in 2022 and is projected to reach USD 20.47 billion by 2030, growing at a CAGR of 15.7% from 2023 to 2030. Large enterprises controlled 61.1% of 2024 spend, but SMEs chart a 17.2% CAGR that reshapes the project management software market size trajectory. Growth centers on Asia-Pacific, where local governments fund digital upskilling grants.

AgencyOS positions itself within the specialized agency management software segment, competing with solutions like Productive, an all-in-one project management software designed for agencies that offers comprehensive tools for managing projects, resources, budgets, and client relationships, all within a single platform.

Current System Limitations

Existing agency management solutions suffer from:

- **Limited Integration Depth:** Surface-level connections that require manual data synchronization
- **Generic Project Management:** Lack of agency-specific workflows for design reviews and client approvals
- **Inadequate Financial Controls:** Poor budget tracking and profitability analysis capabilities
- **Weak Client Experience:** Limited client portal functionality and collaboration features

Integration with Existing Enterprise Landscape

AgencyOS integrates with the modern agency technology stack through:

- **Design Tools:** Native Figma integration, Adobe Creative Cloud connectivity
- **Development Platforms:** GitHub/GitLab repository management, automated CI/CD preview capture
- **Communication Systems:** Slack/Teams notifications, Zoom/Calendly scheduling integration
- **Financial Systems:** Stripe payment processing, QuickBooks/Xero accounting synchronization
- **Productivity Suites:** Google Drive/Dropbox file management, Notion/Confluence knowledge bases

1.2.2 High-Level Description

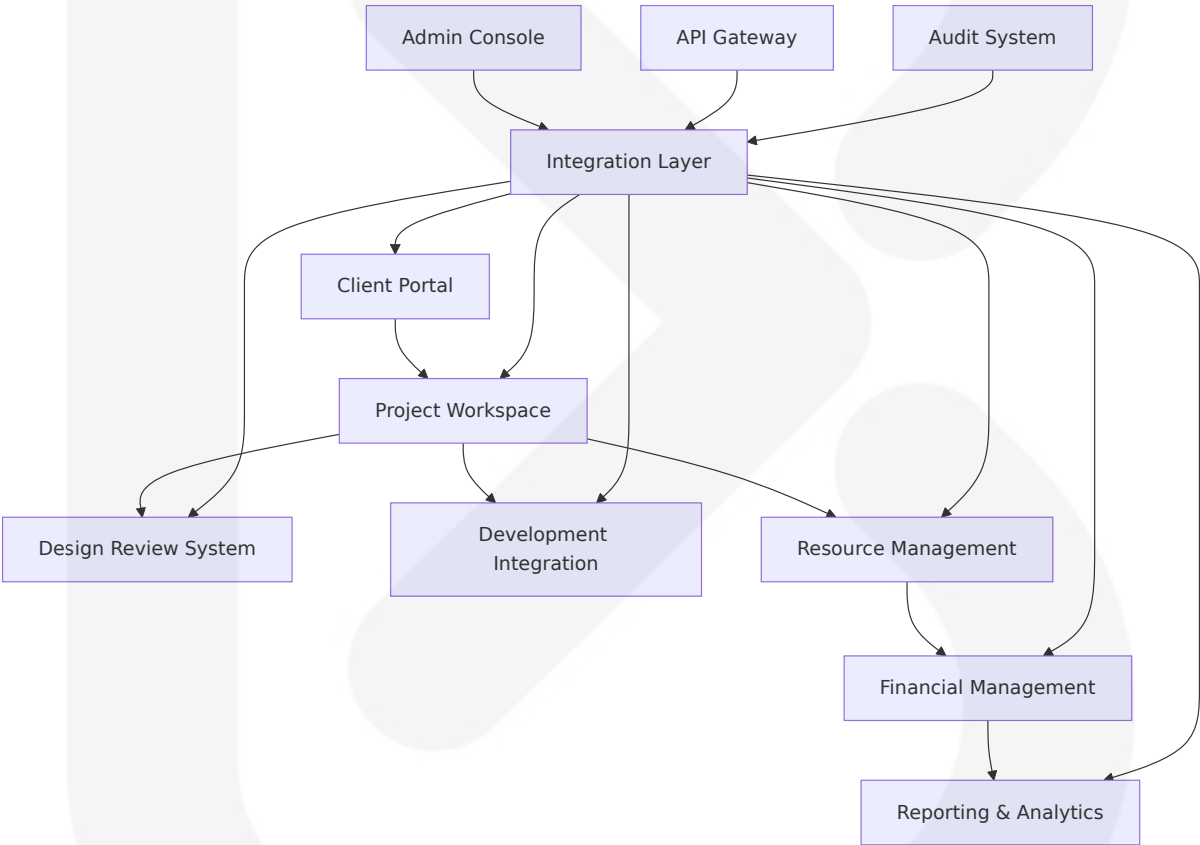
Primary System Capabilities

AgencyOS provides comprehensive agency management capabilities across four core domains:

Domain	Core Capabilities
Client Relationship Management	Lead capture, proposal generation, e-signature workflows, deposit collection

Domain	Core Capabilities
Project Execution	Sprint planning, task management, design review, UAT coordination, change request processing
Resource Management	Time tracking, utilization monitoring, capacity planning, margin analysis
Financial Operations	Automated invoicing, payment processing, accounting integration, profitability reporting

Major System Components



Core Technical Approach

The system employs a modern, cloud-native architecture built on:

- **Frontend:** React 18+ with Next.js App Router for optimal performance and SEO

- **Backend:** Node.js 20+ with NestJS/Fastify for scalable API development
- **Database:** PostgreSQL 15+ with row-level security for multi-tenant data isolation
- **Integration:** Event-driven architecture with webhook-based synchronization
- **AI Enhancement:** Vector embeddings for intelligent project insights and risk detection

1.2.3 Success Criteria

Measurable Objectives

Metric Category	Target	Measurement Method
Performance	p95 API response <300ms, p95 page load TTI <2.5s	Application Performance Monitoring
Reliability	99.9% uptime SLO, RTO 60 min, RPO 15 min	Infrastructure monitoring and incident tracking
User Adoption	>80% daily active users within 90 days	User analytics and engagement metrics

Critical Success Factors

1. **Seamless Integration:** All specified third-party integrations must function reliably with <1% sync failure rate
2. **User Experience:** Intuitive interface requiring <2 hours training for basic proficiency
3. **Data Security:** SOC 2 Type II compliance with zero security incidents
4. **Scalability:** Support for 10,000+ concurrent users with linear performance scaling

Key Performance Indicators (KPIs)

Operational Efficiency KPIs:

- Approval cycle time: p50 <3 days
- Scope creep incidents: <5% of total project value
- On-time delivery rate: >90%
- Resource utilization: 75-85% optimal range

Financial Performance KPIs:

- Days Sales Outstanding: <25 days
- Project margin accuracy: ±5% variance from estimates
- Invoice processing time: <24 hours automated
- Client retention rate: >95%

1.3 SCOPE

1.3.1 In-Scope

Core Features and Functionalities

Must-Have Capabilities:

Feature Category	Included Capabilities
Multi-Tenant Architecture	Workspace isolation, project-level RBAC, tenant-specific customization
Client Lifecycle Management	CRM, proposal/SOW builder, e-signature, deposit collection, client portal
Project Management	Task/sprint management, Jira/Linear sync, budget tracking, time management
Design & Review	Figma integration, annotated reviews, version control, approval workflows

Primary User Workflows:

- Lead intake → proposal → SOW e-signature → deposit → project kickoff

- Design delivery → client review → feedback incorporation → approval
- Development → UAT → defect tracking → release → client sign-off
- Change request → impact assessment → approval → scope update → billing

Essential Integrations:

- **Design:** Figma, Adobe Creative Cloud
- **Development:** GitHub/GitLab, CI/CD preview systems
- **Communication:** Slack/Teams, email notifications
- **Financial:** Stripe, QuickBooks/Xero
- **Project Management:** Jira/Linear task synchronization

Implementation Boundaries

System Boundaries:

- Web-based application with mobile-responsive design
- Cloud-hosted infrastructure with global CDN distribution
- API-first architecture supporting third-party integrations
- Multi-region deployment capability with data residency options

User Groups Covered:

- Agency teams: 5-500 users per organization
- Client organizations: 1-100 users per project
- External contractors: Limited access with project-specific permissions

Geographic Coverage:

- Primary markets: North America, Europe, Australia
- Compliance: GDPR, CCPA, SOC 2 Type II
- Localization: English, with framework for additional languages

1.3.2 Out-of-Scope

Explicitly Excluded Features

Phase 1 Exclusions:

- Advanced AI forecasting and anomaly detection
- Multi-region data residency controls
- On-premises deployment options
- Public case study generation from project artifacts
- Advanced social media scheduling and publishing
- Built-in video conferencing capabilities

Integration Points Not Covered:

- Legacy ERP systems requiring custom connectors
- Proprietary agency-specific tools without public APIs
- Real-time collaboration editing (beyond commenting)
- Advanced business intelligence and data warehousing

Future Phase Considerations

Medium Priority (Phase 2):

- Resource planning and capacity forecasting
- Advanced utilization and margin dashboards
- Knowledge base with AI-powered search
- White-labeling and custom branding options
- SCIM provisioning for enterprise SSO

Lower Priority (Phase 3):

- AI-powered risk forecasting and recommendations
- Cost anomaly detection and automated alerts
- Multi-region deployment with data sovereignty
- On-premises installation options
- Advanced reporting and business intelligence

Unsupported Use Cases

- **Enterprise Resource Planning:** Full ERP functionality beyond project-focused financial tracking
- **Human Resources Management:** Employee lifecycle management, payroll, benefits administration
- **Marketing Automation:** Email campaigns, lead nurturing, marketing analytics beyond project context
- **Content Management:** Website CMS, blog management, SEO optimization tools
- **E-commerce:** Online store functionality, product catalog management, order processing

2. PRODUCT REQUIREMENTS

2.1 FEATURE CATALOG

2.1.1 Core Platform Features

Feature ID	Feature Name	Category	Priority	Status
F-001	Multi-Tenant Workspace Management	Platform Core	Critical	Proposed
F-002	Role-Based Access Control (RBAC)	Security	Critical	Proposed
F-003	Client Relationship Management	CRM	Critical	Proposed
F-004	Project Management System	Project Management	Critical	Proposed
F-005	Design Review & Approval System	Design Management	Critical	Proposed

2.1.2 Client Lifecycle Management Features

Feature ID	Feature Name	Category	Priority	Status
F-006	Lead Capture & Management	CRM	High	Proposed
F-007	Proposal & SOW Builder	Sales	Critical	Proposed
F-008	E-Signature Integration	Legal/Compliance	Critical	Proposed
F-009	Deposit Collection System	Financial	Critical	Proposed
F-010	Client Portal	Client Experience	Critical	Proposed

2.1.3 Project Execution Features

Feature ID	Feature Name	Category	Priority	Status
F-011	Task & Sprint Management	Project Management	Critical	Proposed
F-012	Budget Tracking & Management	Financial	Critical	Proposed
F-013	Time Tracking & Utilization	Resource Management	High	Proposed
F-014	Change Request Management	Project Management	High	Proposed
F-015	UAT & QA Management	Quality Assurance	High	Proposed

2.1.4 Integration & Automation Features

Feature ID	Feature Name	Category	Priority	Status
F-016	Figma Integration	Design Tools	Critical	Proposed
F-017	GitHub/GitLab Integration	Development Tools	Critical	Proposed
F-018	Jira/Linear Sync	Project Management	High	Proposed
F-019	Slack/Teams Notifications	Communication	High	Proposed
F-020	Automated Invoicing	Financial	Critical	Proposed

2.1.5 Asset & Knowledge Management Features

Feature ID	Feature Name	Category	Priority	Status
F-021	Asset Library & Versioning	Asset Management	High	Proposed
F-022	Knowledge Base System	Knowledge Management	Medium	Proposed
F-023	Audit Trail & Logging	Compliance	Critical	Proposed
F-024	Reporting & Analytics	Business Intelligence	High	Proposed
F-025	AI-Powered Insights	AI/ML	Medium	Proposed

2.2 FUNCTIONAL REQUIREMENTS

2.2.1 Multi-Tenant Workspace Management (F-001)

Requirement ID	Description	Acceptance Criteria	Priority	Complexity
F-001-R Q-001	System must support isolated workspaces for multiple organizations with project-level access control	Each tenant has completely isolated data, users can be assigned to specific projects within workspace, workspace admin can manage all tenant settings	Must-Have	High
F-001-R Q-002	Workspace customization including branding and domain configuration	Custom logos, color schemes, and subdomain support implemented	Should-Have	Medium
F-001-R Q-003	Cross-workspace user management for contractors and external collaborators	Users can be invited to multiple workspaces with different roles, single sign-on across workspaces	Should-Have	High

Technical Specifications:

- **Input Parameters:** Workspace configuration, user assignments, branding assets
- **Output/Response:** Isolated tenant environment with custom branding
- **Performance Criteria:** Workspace switching <2 seconds, tenant isolation 100% effective
- **Data Requirements:** PostgreSQL row-level security, tenant-specific schemas

Validation Rules:

- **Business Rules:** No cross-tenant data leakage, workspace admin controls all tenant access
- **Security Requirements:** Complete data isolation, encrypted tenant boundaries
- **Compliance Requirements:** SOC 2 Type II multi-tenancy controls

2.2.2 Role-Based Access Control (F-002)

Requirement ID	Description	Acceptance Criteria	Priority	Complexity
F-002-RQ-001	Implement comprehensive RBAC system with predefined roles for agency and client users	11 predefined roles implemented (Super Admin, Admin, PM, Designer, Engineer, QA, Finance, Contractor, Client Admin, Client Reviewer, Viewer), role permissions enforced across all features	Must-Have	High
F-002-RQ-002	Project-level permission overrides and granular access control	Users can have different permissions per project, fine-grained feature access control	Must-Have	High
F-002-RQ-003	SSO integration with OIDC/SAML providers	Support for major SSO providers, MFA enforcement, SCIM provisioning	Should-Have	Medium

Technical Specifications:

- **Input Parameters:** User credentials, role assignments, permission matrices

- **Output/Response:** Authenticated user session with appropriate permissions
- **Performance Criteria:** Authentication <500ms, permission checks <100ms
- **Data Requirements:** User roles, permissions, project assignments stored securely

Validation Rules:

- **Business Rules:** Least privilege principle, role inheritance, project-specific overrides
- **Security Requirements:** MFA required for admin roles, session management, audit logging of all access
- **Compliance Requirements:** SOC 2 access controls, GDPR user consent

2.2.3 Design Review & Approval System (F-005)

Require ment ID	Description	Acceptance Cri teria	Priority	Comple xity
F-005-R Q-001	Native Figma i ntegration wit h annotated f eedback and version comp arison	Direct Figma file embedding, thre aded comments on design eleme nts, version diff v isualization, appr oval workflow int egration	Must-Ha ve	High
F-005-R Q-002	Multi-format d esign review s upporting PDF s, images, an d video with a nnotation tool s	Support for PDF, JPG, PNG, MP4 fo rmats with drawi ng tools, text co mments, approv al stamps	Must-Ha ve	Medium

Requirement ID	Description	Acceptance Criteria	Priority	Complexity
F-005-R Q-003	Structured approval workflow with stakeholder notifications and deadline tracking	Configurable approval chains, automated reminders, approval status dashboard, SLA tracking	Must-Have	Medium

Technical Specifications:

- **Input Parameters:** Design files, reviewer assignments, approval criteria
- **Output/Response:** Annotated designs, approval status, consolidated feedback
- **Performance Criteria:** File loading <3 seconds, annotation saving <1 second
- **Data Requirements:** Design versions, annotations, approval history

Validation Rules:

- **Business Rules:** Only Client Admin can reopen approved designs, all feedback must be addressed
- **Security Requirements:** Watermarked shareable links, access expiry controls
- **Compliance Requirements:** Audit trail of all design changes and approvals

2.2.4 Project Management System (F-004)

Requirement ID	Description	Acceptance Criteria	Priority	Complexity
F-004-R Q-001	Comprehensive project planning with scope definition, milestones, and tasks	Project templates, Gantt charts, milestone tracking, dependency management	Must-Have	High

Requirement ID	Description	Acceptance Criteria	Priority	Complexity
	k dependencies	nt, resource allocation		
F-004-RQ-002	Sprint management with task assignment and progress tracking	Sprint planning interface, task boards, burndown charts, velocity tracking	Must-Have	Medium
F-004-RQ-003	Real-time budget tracking with automated overage alerts	Budget vs. actual reporting, 80/100/120% threshold alerts, margin analysis	Must-Have	Medium

Technical Specifications:

- **Input Parameters:** Project scope, budget, timeline, resource assignments
- **Output/Response:** Project dashboard, progress reports, budget status
- **Performance Criteria:** Dashboard loading <2 seconds, real-time updates <500ms
- **Data Requirements:** Project data, task relationships, budget allocations

Validation Rules:

- **Business Rules:** No work without signed SOW, budget alerts at defined thresholds
- **Security Requirements:** Project-level access controls, audit logging
- **Compliance Requirements:** Financial tracking accuracy, change management controls

2.2.5 Automated Invoicing System (F-020)

Requirement ID	Description	Acceptance Criteria	Priority	Complexity
F-020-RQ-001	Milestone-based invoice generation with Stripe payment processing	Automatic invoice creation on milestone completion, Stripe integration, payment tracking	Must-Have	Medium
F-020-RQ-002	Integration with QuickBooks/Xero for accounting synchronization	Two-way sync with accounting systems, chart of accounts mapping, tax handling	Should-Have	Medium
F-020-RQ-003	Automated dunning process with configurable payment terms	Payment reminders at 3/7/14 days, late fee calculation, collection workflow	Should-Have	Low

Technical Specifications:

- **Input Parameters:** Milestone completion, billing rates, payment terms
- **Output/Response:** Generated invoices, payment confirmations, accounting entries
- **Performance Criteria:** Invoice generation <30 seconds, payment processing <5 seconds
- **Data Requirements:** Billing data, payment history, accounting integration

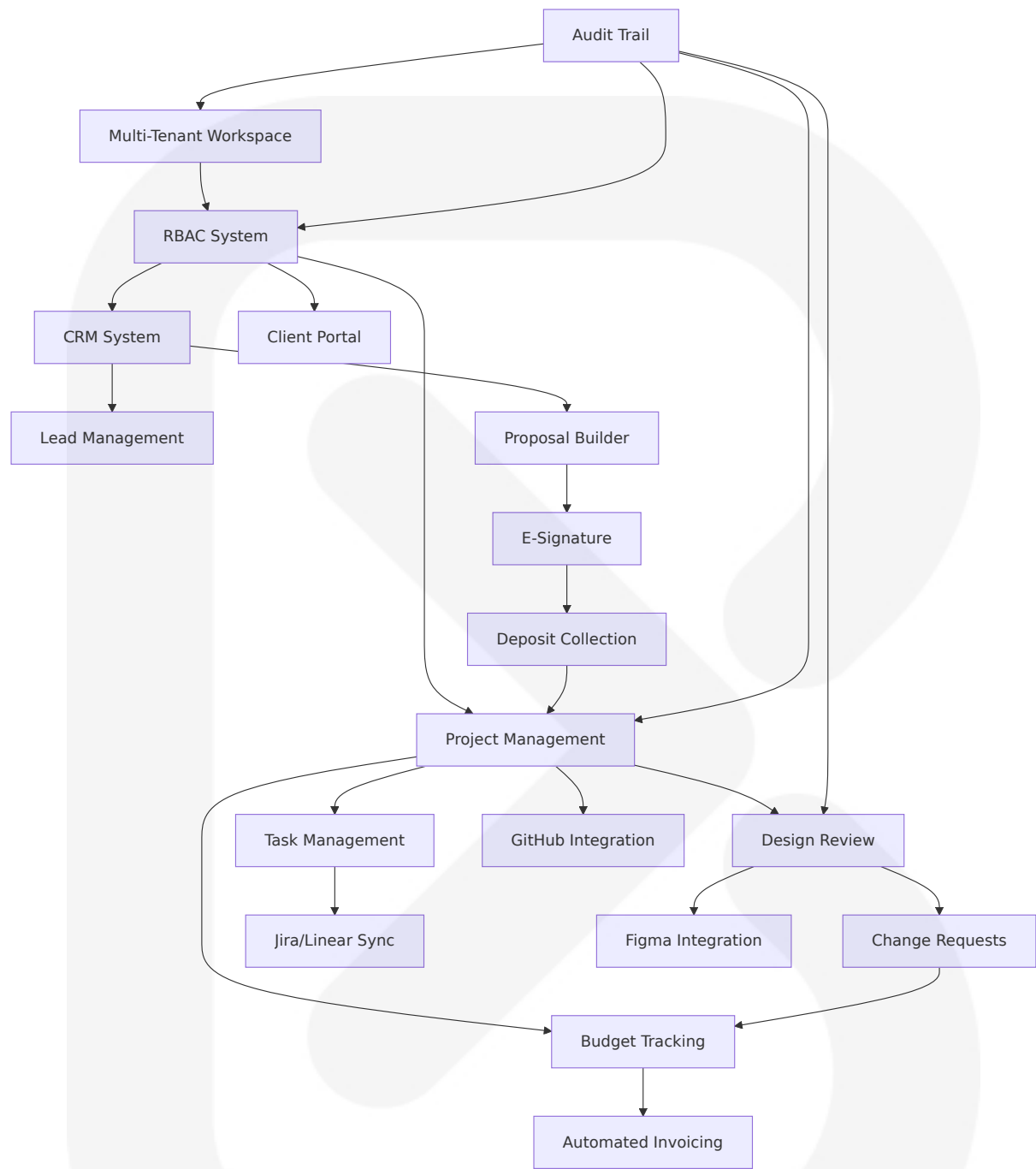
Validation Rules:

- **Business Rules:** Net payment terms enforced, automated late fees, milestone-based billing only
- **Security Requirements:** PCI DSS compliance for payment processing, encrypted financial data

- **Compliance Requirements:** Tax compliance, financial audit trail, GDPR data handling

2.3 FEATURE RELATIONSHIPS

2.3.1 Core Dependencies Map



2.3.2 Integration Points

Integration Category	Primary Features	Shared Components	Dependencies
Authentication & Authorization	F-001, F-002, F-010	SSO service, permission engine	Identity provider, session manager

Integration Category	Primary Features	Shared Components	Dependencies
Financial Operations	F-009, F-012, F-020	Payment processor, accounting sync	Stripe API, QuickBooks/Xero APIs
Design Workflow	F-005, F-016, F-021	File storage, version control	Figma API, S3-compatible storage
Project Execution	F-004, F-011, F-017, F-018	Task engine, notification service	GitHub/GitLab APIs, Jira/Linear APIs

2.3.3 Shared Services

Service Name	Supporting Features	Technical Requirements
Notification Engine	F-005, F-012, F-014, F-019, F-020	Email, Slack, in-app notifications
File Management	F-005, F-010, F-021, F-022	S3-compatible storage, CDN
Audit Service	F-001, F-002, F-004, F-005, F-023	Immutable logging, compliance reporting
Search Engine	F-003, F-021, F-022, F-024	Meilisearch/OpenSearch integration

2.4 IMPLEMENTATION CONSIDERATIONS

2.4.1 Technical Constraints

Feature Category	Constraints	Mitigation Strategies
Multi-Tenancy	Database isolation, performance at scale	Row-level security, connection pooling, caching
Real-Time Features	WebSocket connections, state synchronization	Event-driven architecture, Redis pub/sub
File Processing	Large design files, format compatibility	Streaming uploads, background processing
Integration Reliability	Third-party API limits, failure handling	Circuit breakers, retry mechanisms, fallback modes

2.4.2 Performance Requirements

Feature	Performance Target	Measurement Method
API Response Times	p95 <300ms for all endpoints	Application Performance Monitoring
Page Load Times	p95 TTI <2.5s for all pages	Real User Monitoring
File Upload	Streaming for files > 10MB	Progress tracking, resumable uploads
Search Performance	<500ms p95 for all queries	Search analytics, query optimization

2.4.3 Scalability Considerations

Component	Scaling Strategy	Implementation
Application Tier	Horizontal scaling with load balancing	Kubernetes auto-scaling, stateless services
Database	Read replicas, connection pooling	PostgreSQL streaming replication
File Storage	CDN distribution, regional caching	S3 + CloudFront architecture

Component	Scaling Strategy	Implementation
Background Jobs	Queue partitioning, worker scaling	BullMQ with Redis clustering

2.4.4 Security Implications

Security Domain	Requirements	Implementation
Data Protection	AES-256 encryption at rest, TLS 1.2+ in transit	Database encryption, SSL certificates
Access Control	Role-based permissions, least privilege	JWT tokens, permission middleware
Audit & Compliance	SOC 2 Type II, GDPR compliance	Immutable audit logs, data retention policies
Integration Security	OAuth 2.0, API key management	Secure credential storage, token rotation

2.4.5 Maintenance Requirements

Maintenance Category	Frequency	Automation Level
Database Backups	Daily with quarterly restore testing	Fully automated
Security Updates	Weekly for critical, monthly for standard	Semi-automated with approval gates
Performance Monitoring	Continuous with alerting	Fully automated
Integration Health Checks	Every 15 minutes	Fully automated with failover

3. TECHNOLOGY STACK

3.1 PROGRAMMING LANGUAGES

3.1.1 Frontend Languages

Language	Version	Platform	Justification
TypeScript	5.3+	Web Frontend	Next.js 15 supports both React 18 and the React 19 release candidate, providing type safety, enhanced developer experience, and seamless integration with React ecosystem. TypeScript ensures code reliability and maintainability across the complex multi-tenant architecture.
JavaScript (ES2022)	ES2022+	Web Frontend	Modern JavaScript features for dynamic functionality, with TypeScript compilation target ensuring broad browser compatibility while leveraging latest language features.

3.1.2 Backend Languages

Language	Version	Platform	Justification
TypeScript	5.3+	Backend Services	Unified language stack reduces context switching, provides compile-time type checking for API contracts, and ensures consistency between frontend and backend data models. Critical for maintaining type safety across complex integration points.
Node.js	20.x LTS	Runtime Environment	Node.js 20.x is in Long Term Support (LTS) with the codename 'Iron' and will remain in "Active LTS" until October 2024, then move into "Maintenance" until end of life in April 2025.

Language	Version	Platform	Justification
			I 2026. Provides stable foundation with up to 30% faster file handling, 15% faster web processing, and 10% quicker startup times.

3.1.3 Language Selection Criteria

Performance Requirements:

- TypeScript compilation optimizations for production builds
- Node.js 20 performance improvements align with p95 API response <300ms targets
- Shared language reduces serialization overhead between frontend/backend

Developer Experience:

- Single language stack reduces cognitive load and team specialization requirements
- Rich ecosystem support for required integrations (Figma, Stripe, GitHub)
- Strong tooling support for debugging, testing, and deployment

Ecosystem Compatibility:

- Native support for all required third-party APIs and SDKs
- Extensive library ecosystem for specialized requirements (PDF processing, image manipulation)
- Strong community support and documentation

3.2 FRAMEWORKS & LIBRARIES

3.2.1 Frontend Framework Stack

Framework/Library	Version	Purpose	Justification
React	18.2+	UI Library	Next.js 15 maintains backward compatibility for the Pages Router with React 18, providing stable foundation for complex UI components while supporting future React 19 migration path.
Next.js	15.x	Full-Stack Framework	Next.js 15 App Router uses React 19 RC but maintains React 18 compatibility. Provides SSR, API routes, file-based routing, and optimized performance for agency portal requirements.
TanStack Query	5.x	Data Fetching	Advanced caching, synchronization, and background updates essential for real-time project status updates and multi-user collaboration features.
React Hook Form	7.x	Form Management	Performant form handling with minimal re-renders, critical for complex proposal builders and project configuration forms.
Zod	3.x	Schema Validation	Runtime type validation ensuring data integrity across client-server boundaries and third-party integrations.

3.2.2 UI Component Libraries

Library	Version	Purpose	Justification
Radix UI	1.x	Headless Components	Accessible, unstyled components providing foundation for custom design system while ensuring WCAG 2.2 AA compliance.
Tailwind CSS	3.4+	Styling Framework	Utility-first approach enables rapid UI development and consistent

Library	Version	Purpose	Justification
			ent design system implementati on across multi-tenant customiz ation requirements.
Framer Motion	11.x	Animation Library	Smooth transitions and micro-in teractions enhancing user exper ience for design review workflo ws and approval processes.

3.2.3 Backend Framework Stack

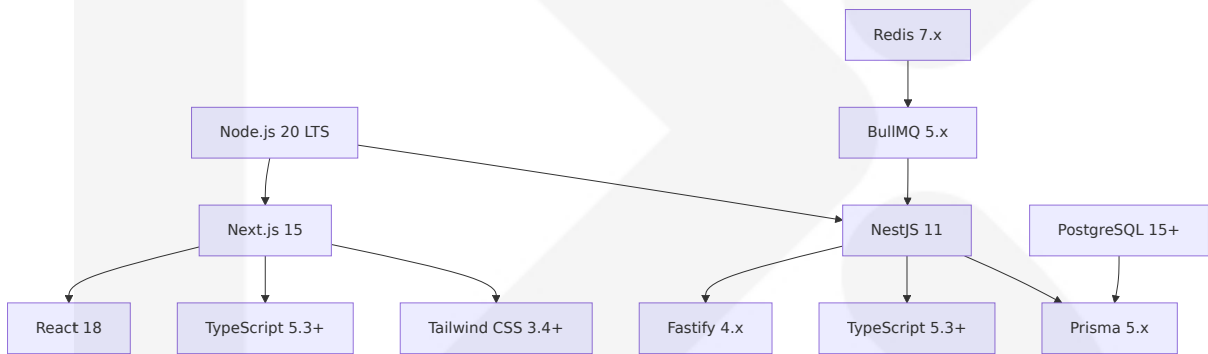
Framewo rk/Librar y	Version	Purpose	Justification
NestJS	11.x	Backend Framewor k	Latest version: 11.1.6, providing enterprise-grade architecture wit h dependency injection, decorat ors, and modular structure ideal for complex agency managemen t requirements.
Fastify	4.x	HTTP Ser ver	Nest.js currently utilizes Fastify v 4, which will exit LTS on June 30, 2025. High-performance alternat ive to Express with over 25% red uction in p95 latency for some A PIs, supporting performance targ ets.
Prisma	5.x	Database ORM	Type-safe database access with excellent TypeScript integration, automated migrations, and robu st query optimization for comple x multi-tenant data models.

3.2.4 Real-time & Background Processing

Library	Version	Purpose	Justification
BullMQ	5.x	Job Queue	Redis-based job processing for automated invoicing, email no

Library	Version	Purpose	Justification
			tifications, and background sync operations with third-party services.
Socket.io	4.x	WebSocket Management	Real-time updates for design review comments, project status changes, and collaborative features across client and agency users.
Server-Sent Events	Native	Live Updates	Lightweight alternative to WebSockets for one-way real-time updates like build status and approval notifications.

3.2.5 Framework Compatibility Matrix



3.3 OPEN SOURCE DEPENDENCIES

3.3.1 Core Runtime Dependencies

Package	Version	Registry	Purpose
@nestjs/core	^11.1.6	npm	Core NestJS framework
@nestjs/platform-fastify	^11.1.6	npm	Fastify adapter for NestJS

Package	Version	Registry	Purpose
next	^15.1.0	npm	React framework with SSR
react	^18.2.0	npm	UI library
@tanstack/react-query	^5.x	npm	Data fetching and caching
prisma	^5.x	npm	Database ORM
zod	^3.x	npm	Schema validation

3.3.2 Authentication & Security

Package	Version	Registry	Purpose
@auth0/nextjs-auth0	^3.x	npm	Auth0 integration for Next.js
jsonwebtoken	^9.x	npm	JWT token handling
bcryptjs	^2.x	npm	Password hashing
helmet	^7.x	npm	Security headers middleware
rate-limiter-flexible	^5.x	npm	Rate limiting implementation

3.3.3 Third-Party Integrations

Package	Version	Registry	Purpose
stripe	^19.0.0	npm	Latest version: 19.0.0, last published: 2 days ago Payment processing
@octokit/rest	^20.x	npm	GitHub API integration

Package	Version	Registry	Purpose
@slack/web-api	^7.x	npm	Slack notifications
figma-api	^1.x	npm	Figma API supports read access and interactions with Figma files, with OpenAPI specification available
docusign-esign	^6.x	npm	E-signature integration

3.3.4 Data Processing & Utilities

Package	Version	Registry	Purpose
bullmq	^5.x	npm	Redis-based job queue
ioredis	^5.x	npm	Redis client
sharp	^0.33.x	npm	Image processing
pdf-lib	^1.x	npm	PDF manipulation
date-fns	^3.x	npm	Date utilities
lodash	^4.x	npm	Utility functions

3.3.5 Development & Testing Dependencies

Package	Version	Registry	Purpose
typescript	^5.3.0	npm	TypeScript compiler
@types/node	^20.x	npm	Node.js type definitions
eslint	^9.x	npm	Code linting
prettier	^3.x	npm	Code formatting
jest	^29.x	npm	Testing framework

Package	Version	Registry	Purpose
@testing-library/react	^14.x	npm	React testing utilities

3.3.6 Package Management Strategy

Version Pinning Policy:

- Major versions pinned for stability (e.g., ^11.1.6 for NestJS)
- Security-critical packages use exact versions
- Development dependencies allow minor updates for tooling improvements

Registry Configuration:

- Primary registry: npm (registry.npmjs.org)
- Private packages: Scoped to organization namespace
- Dependency scanning: Automated vulnerability detection via GitHub Dependabot

Update Strategy:

- Monthly dependency reviews for security updates
- Quarterly major version evaluations
- Automated testing pipeline validates all dependency updates

3.4 THIRD-PARTY SERVICES

3.4.1 Design & Development Integrations

Service	API Version	Purpose	Integration Method
Figma	REST API v1	Design file access, image rendering, comments, team proj	OAuth 2.0, Webhooks

Service	API Version	Purpose	Integration Method
		ects and project files	
GitHub	REST API v4	Repository management, CI/CD integration, PR previews	GitHub Apps, Webhooks
GitLab	REST API v4	Alternative repository management	OAuth 2.0, System hooks
Jira	REST API v3	Task synchronization, issue tracking	OAuth 2.0, Webhooks
Linear	GraphQL API	Modern task management integration	API Keys, Webhooks

3.4.2 Communication & Collaboration

Service	API Version	Purpose	Integration Method
Slack	Web API v1	Team notifications, file sharing	OAuth 2.0, Bot tokens
Microsoft Teams	Graph API v1.0	Enterprise communication	OAuth 2.0, Webhooks
Zoom	REST API v2	Meeting scheduling, recordings	OAuth 2.0, Webhooks
Calendly	REST API v2	Appointment scheduling	OAuth 2.0, Webhooks

3.4.3 Financial & Legal Services

Service	API Version	Purpose	Integration Method
Stripe	API v1	Current version is 2025-09-30.clover Payment processing, invoicing	API Keys, Webhooks
QuickBooks	v3 API	Accounting synchronization	OAuth 2.0, Webhooks

Service	API Version	Purpose	Integration Method
Xero	API v2	Alternative accounting platform	OAuth 2.0, Webhooks
DocuSign	eSignature API v2.1	Contract e-signatures	OAuth 2.0, Connect webhooks
Adobe Sign	REST API v6	Alternative e-signature solution	OAuth 2.0, Webhooks

3.4.4 Cloud Storage & Content

Service	API Version	Purpose	Integration Method
Google Drive	Drive API v3	File storage, collaboration	OAuth 2.0, Push notifications
Dropbox	API v2	Alternative file storage	OAuth 2.0, Webhooks
Notion	API v1	Knowledge base integration	OAuth 2.0, Internal integrations
Confluence	REST API v1	Enterprise documentation	OAuth 2.0, Webhooks

3.4.5 Authentication & Identity

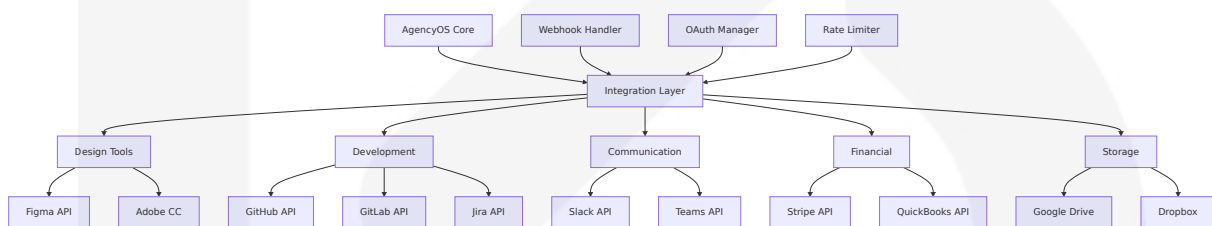
Service	API Version	Purpose	Integration Method
Auth0	Management API v2	Identity management, SSO	Machine-to-Machine tokens
Okta	API v1	Enterprise identity provider	OAuth 2.0, SCIM
Azure AD	Microsoft Graph v1.0	Microsoft ecosystem SSO	OAuth 2.0, SCIM

Service	API Version	Purpose	Integration Method
Google Workspace	Admin SDK	Google ecosystem integration	Service accounts, OAuth 2.0

3.4.6 Monitoring & Analytics

Service	API Version	Purpose	Integration Method
Sentry	REST API	Error tracking, performance monitoring	DSN, API tokens
Datadog	API v2	Infrastructure monitoring	API Keys, Agent
Mixpanel	API v2	User analytics, event tracking	API Keys, Client Libraries
PostHog	API v1	Product analytics, feature flags	API Keys, Client Libraries

3.4.7 Integration Architecture



3.5 DATABASES & STORAGE

3.5.1 Primary Database

Component	Technology	Version	Purpose
Primary Database	PostgreSQL	15.x	Current minor version: 15.14 (2025-08-14) EOL: 2027-11 (estimated) Multi-tenant data storage with row-level security
Connection Pooling	PgBouncer	1.21+	Connection management for high-concurrency workloads
Read Replicas	PostgreSQL	15.x	Read scaling for reporting and analytics queries

PostgreSQL Configuration:

- **Row-Level Security (RLS):** Tenant isolation at database level
- **JSONB Columns:** Flexible metadata storage for project configurations
- **Full-Text Search:** Built-in search capabilities for projects and assets
- **Partitioning:** Time-based partitioning for audit logs and analytics data

3.5.2 Caching Layer

Component	Technology	Version	Purpose
Primary Cache	Redis	7.2+	Session storage, job queues, real-time data
Clustering	Redis Cluster	7.2+	High availability and horizontal scaling
Persistence	RDB + AOF	-	Durability for critical cached data

Redis Usage Patterns:

- **Session Store:** User authentication and workspace context
- **Job Queues:** BullMQ for background processing
- **Real-time Data:** WebSocket connection management
- **Rate Limiting:** API throttling and abuse prevention

- **Cache Invalidation:** Event-driven cache updates

3.5.3 Object Storage

Component	Technology	Version	Purpose
Primary Storage	AWS S3	-	Design files, assets, document storage
CDN	CloudFront	-	Global asset delivery and caching
Backup Storage	S3 Glacier	-	Long-term archival and compliance

Storage Architecture:

- **Bucket Structure:** Tenant-isolated prefixes for multi-tenancy
- **Lifecycle Policies:** Automated archival of inactive project assets
- **Versioning:** Design file version control and rollback capabilities
- **Encryption:** AES-256 server-side encryption for all stored objects

3.5.4 Search Engine

Component	Technology	Version	Purpose
Search Engine	Meilisearch	1.5+	Full-text search across projects, assets, and knowledge base
Indexing	Real-time	-	Immediate search availability for new content
Faceted Search	Built-in	-	Advanced filtering by project status, client, date ranges

Search Implementation:

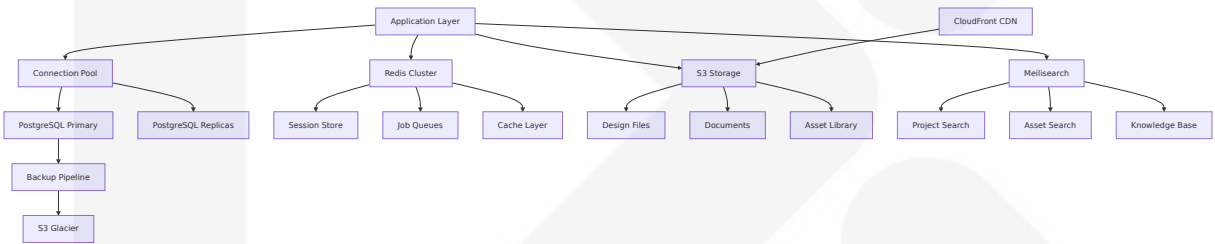
- **Multi-tenant Indexes:** Isolated search per workspace
- **Typo Tolerance:** Fuzzy matching for user-friendly search

- **Ranking:** Custom relevance scoring based on project activity
- **Synonyms:** Business-specific terminology mapping

3.5.5 Analytics & Reporting

Component	Technology	Version	Purpose
Time-Series DB	InfluxDB	2.7+	Performance metrics, usage analytics
Data Warehouse	PostgreSQL	15.x	Aggregated reporting data
ETL Pipeline	Custom Node.js	-	Data transformation and aggregation

3.5.6 Data Architecture



3.5.7 Data Persistence Strategy

Backup & Recovery:

- **PostgreSQL:** Daily full backups, continuous WAL archiving
- **Redis:** RDB snapshots every 6 hours, AOF for durability
- **S3:** Cross-region replication for critical assets
- **Recovery Time Objective (RTO):** 60 minutes
- **Recovery Point Objective (RPO):** 15 minutes

Data Retention:

- **Transactional Data:** 7 years for compliance
- **Audit Logs:** 24 months active, archived thereafter

- **Design Files:** Indefinite with lifecycle management
- **Analytics Data:** 2 years detailed, 5 years aggregated

Compliance & Security:

- **Encryption at Rest:** AES-256 for all databases
- **Encryption in Transit:** TLS 1.2+ for all connections
- **Access Controls:** Role-based database permissions
- **Audit Logging:** All data access and modifications logged

3.6 DEVELOPMENT & DEPLOYMENT

3.6.1 Development Tools

Tool	Version	Purpose	Configuration
Node.js	20.x LTS	Runtime environment	Node.js 20.x LTS with codename 'Iron' in Active LTS until October 2024
pnpm	8.x	Package manager	Workspace support, faster installs, disk space efficiency
TypeScript	5.3+	Type checking	Strict mode, path mapping, incremental compilation
ESLint	9.x	Code linting	Next.js 15 introduces support for ESLint 9, with backwards compatibility for ESLint 8
Prettier	3.x	Code formatting	Consistent formatting across team

3.6.2 Build System

Component	Technology	Purpose	Configuration
Frontend Build	Next.js	SSR, static generation, bundling	App Router, TypeScript, Tailwind integration

Component	Technology	Purpose	Configuration
			gration
Backend Build	NestJS CLI	TypeScript compilation, module bundling	Production optimizations, tree shaking
Asset Processing	Sharp	Image optimization, format conversion	WebP generation, responsive images
Bundle Analysis	Webpack Bundle Analyzer	Bundle size optimization	Automated size monitoring

3.6.3 Containerization

Component	Technology	Version	Purpose
Base Images	Node.js Alpine	20-alpine	Minimal attack surface, smaller image size
Multi-stage Builds	Docker	24.x	Optimized production images
Container Registry	AWS ECR	-	Private image storage and scanning
Security Scanning	Trivy	Latest	Vulnerability detection in images

Docker Configuration:

```
# Multi-stage build example
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

FROM node:20-alpine AS production
```



```
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

3.6.4 CI/CD Pipeline

Stage	Tool	Purpose	Configuration
Source Control	GitHub	Code repository, PR management	Branch protection, required reviews
CI Pipeline	GitHub Actions	Automated testing, building	Matrix builds, parallel execution
Code Quality	SonarCloud	Static analysis, security scanning	Quality gates, coverage thresholds
Deployment	AWS CodeDeploy	Blue-green deployments	Automated rollback, health checks

GitHub Actions Workflow:

```
name: CI/CD Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [20.x]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
```

```
cache: 'pnpm'
- run: pnpm install --frozen-lockfile
- run: pnpm test:ci
- run: pnpm build
```

3.6.5 Infrastructure as Code

Component	Technology	Purpose	Configuration
IaC Framework	Terraform	Infrastructure provisioning	Modular, reusable configurations
State Management	Terraform Cloud	Remote state, collaboration	Encrypted state, access controls
Configuration	AWS Systems Manager	Environment variables, secrets	Parameter Store, encrypted values
Monitoring	CloudWatch	Logs, metrics, alerting	Custom dashboards, automated alerts

Terraform Module Structure:

```
module "vpc" {
  source = "../modules/vpc"
  environment = var.environment
}

module "eks" {
  source = "../modules/eks"
  vpc_id = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnet_ids
}

module "rds" {
  source = "../modules/rds"
  vpc_id = module.vpc.vpc_id
  subnet_ids = module.vpc.database_subnet_ids
}
```

3.6.6 Kubernetes Deployment

Component	Technology	Version	Purpose
Orchestration	Amazon EKS	1.28+	Managed Kubernetes service
Ingress	AWS Load Balancer Controller	2.6+	Application load balancing
Service Mesh	Istio	1.19+	Traffic management, security
Monitoring	Prometheus + Grafana	Latest	Metrics collection and visualization

Kubernetes Manifests:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: agencyos-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: agencyos-api
  template:
    metadata:
      labels:
        app: agencyos-api
    spec:
      containers:
        - name: api
          image: agencyos/api:latest
          ports:
            - containerPort: 3000
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
```

name: **database-secret**
key: **url**

3.6.7 Development Workflow



Development Environment:

- **Local Setup:** Docker Compose for full stack development
- **Hot Reloading:** Next.js and NestJS development servers
- **Database Seeding:** Automated test data generation
- **API Documentation:** OpenAPI/Swagger auto-generation

Quality Assurance:

- **Unit Tests:** Jest with >80% coverage requirement
- **Integration Tests:** Supertest for API endpoints
- **E2E Tests:** Playwright for critical user journeys
- **Performance Tests:** k6 for load testing
- **Security Tests:** OWASP ZAP for vulnerability scanning

Deployment Strategy:

- **Blue-Green Deployments:** Zero-downtime releases
- **Canary Releases:** Gradual rollout for major features
- **Feature Flags:** Runtime feature toggling
- **Rollback Procedures:** Automated rollback on health check failures
- **Database Migrations:** Backward-compatible schema changes

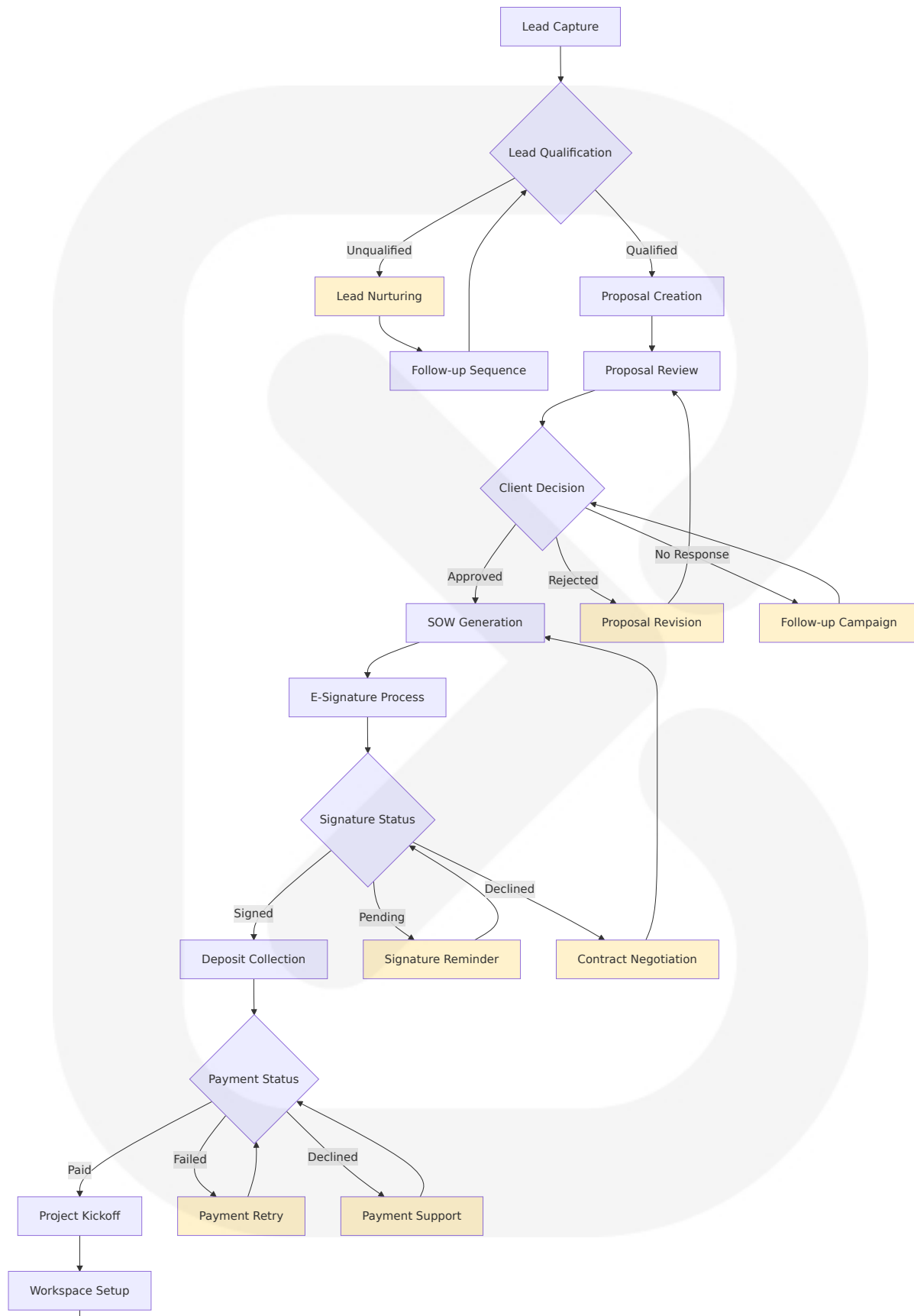
4. PROCESS FLOWCHART

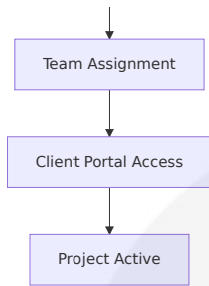
4.1 SYSTEM WORKFLOWS

4.1.1 Core Business Processes

Client Onboarding Workflow

The AgencyOS client onboarding process follows a structured flowchart approach using nodes (geometric shapes) and edges (arrows or lines) to define the complete journey from initial lead capture through project kickoff.





Business Rules:

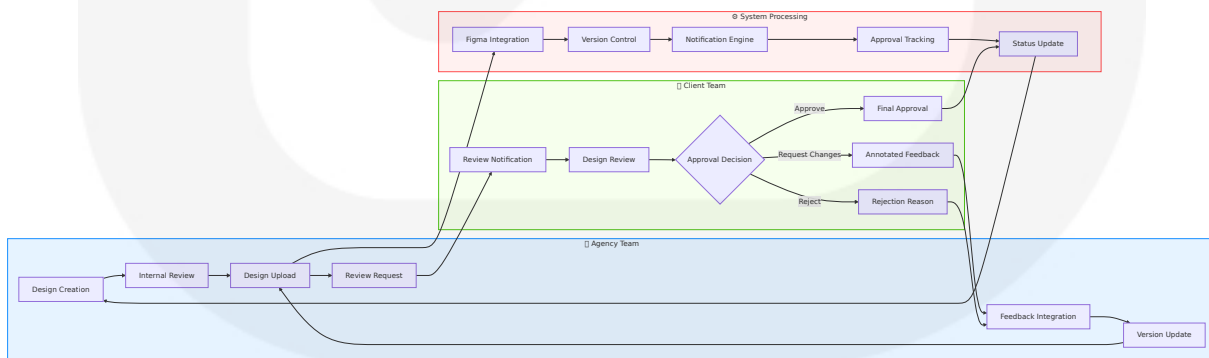
- Lead qualification must score >70% on predefined criteria
- Proposals expire after 30 days without response
- SOW requires legal review for contracts >\$50K
- Deposit collection mandatory before project activation
- Client portal access granted only after successful payment

SLA Requirements:

- Proposal generation: <24 hours from qualification
- E-signature processing: <2 hours from SOW approval
- Deposit collection: Net 15 payment terms
- Project kickoff: <48 hours from deposit confirmation

Design Review & Approval Workflow

The design review process utilizes swimlane-style subgraphs to divide responsibilities between departments, with distinct visual clarity for each stakeholder group.



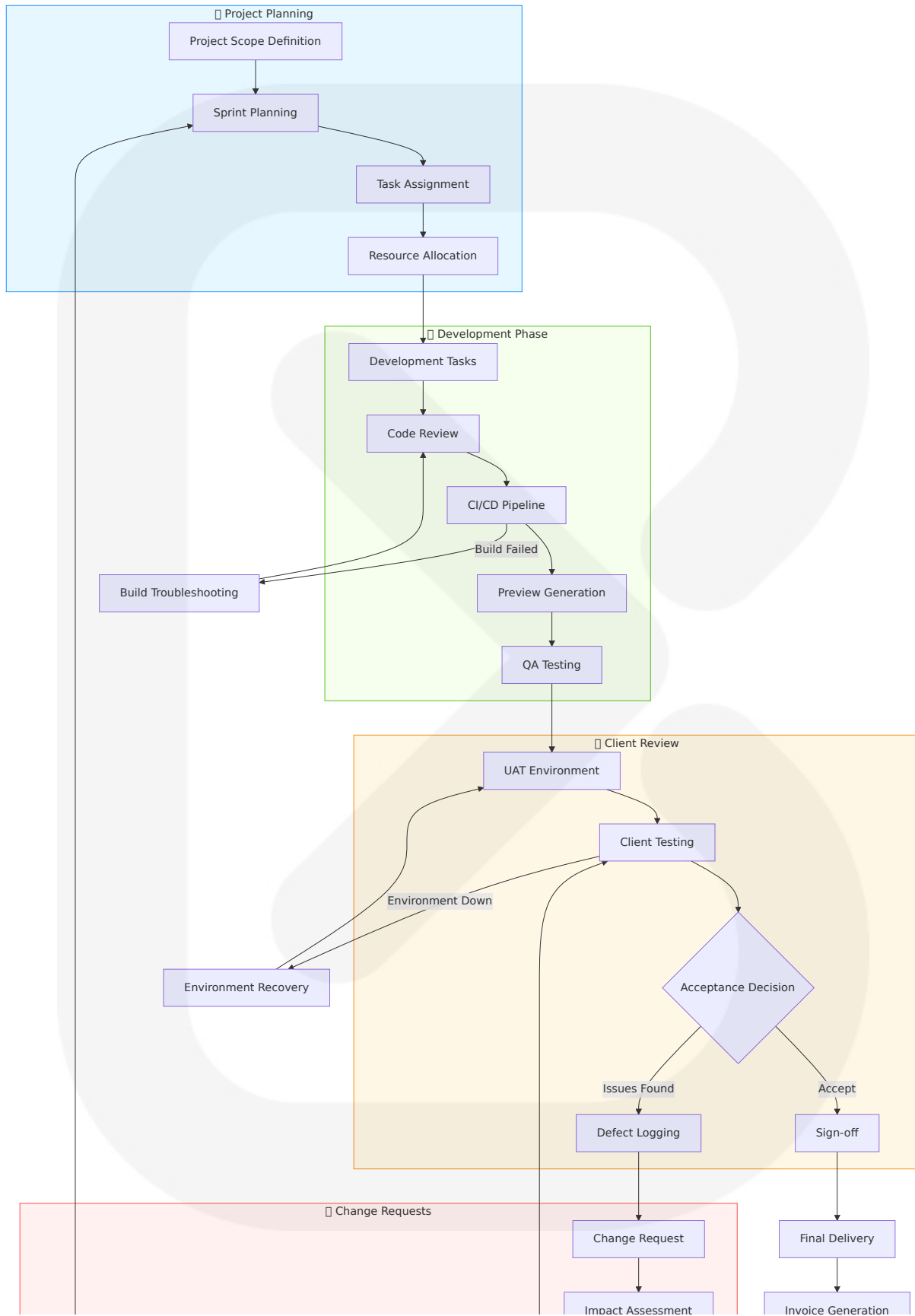
Validation Rules:

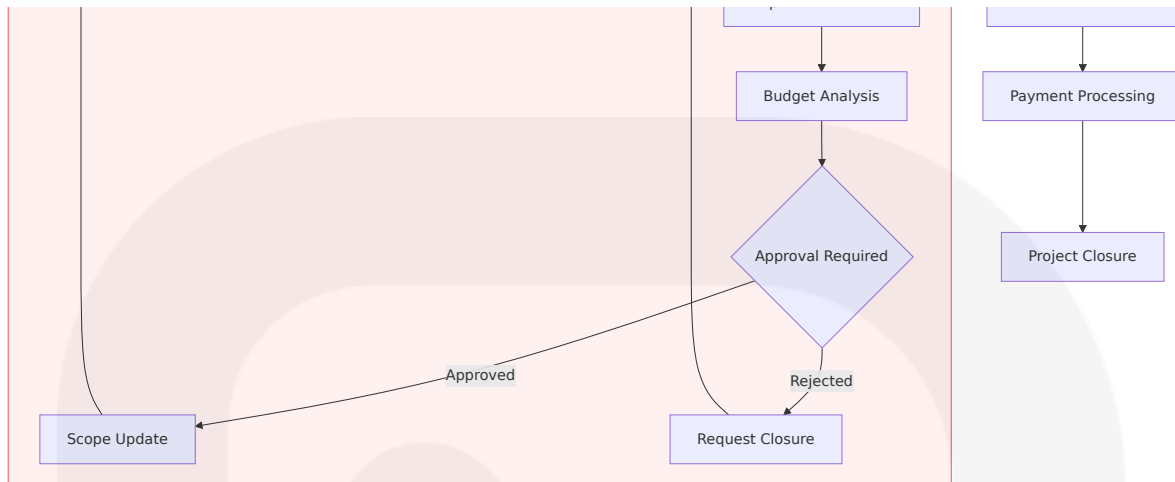
- Only Client Admin can provide final approval
- All feedback must include specific annotations
- Version history maintained for 24 months
- Approval decisions are immutable once confirmed
- Rejected designs require documented reasoning

Error Handling:

- Figma API failures trigger local backup storage
- Missing annotations prompt clarification requests
- Approval timeouts escalate to project manager
- Version conflicts resolved through merge workflows

Project Execution Workflow





State Transitions:

- Planning → Development: Requires signed SOW and resource confirmation
- Development → Client Review: All QA tests must pass
- Client Review → Change Management: Triggered by defect discovery
- Change Management → Planning: Approved changes update project scope

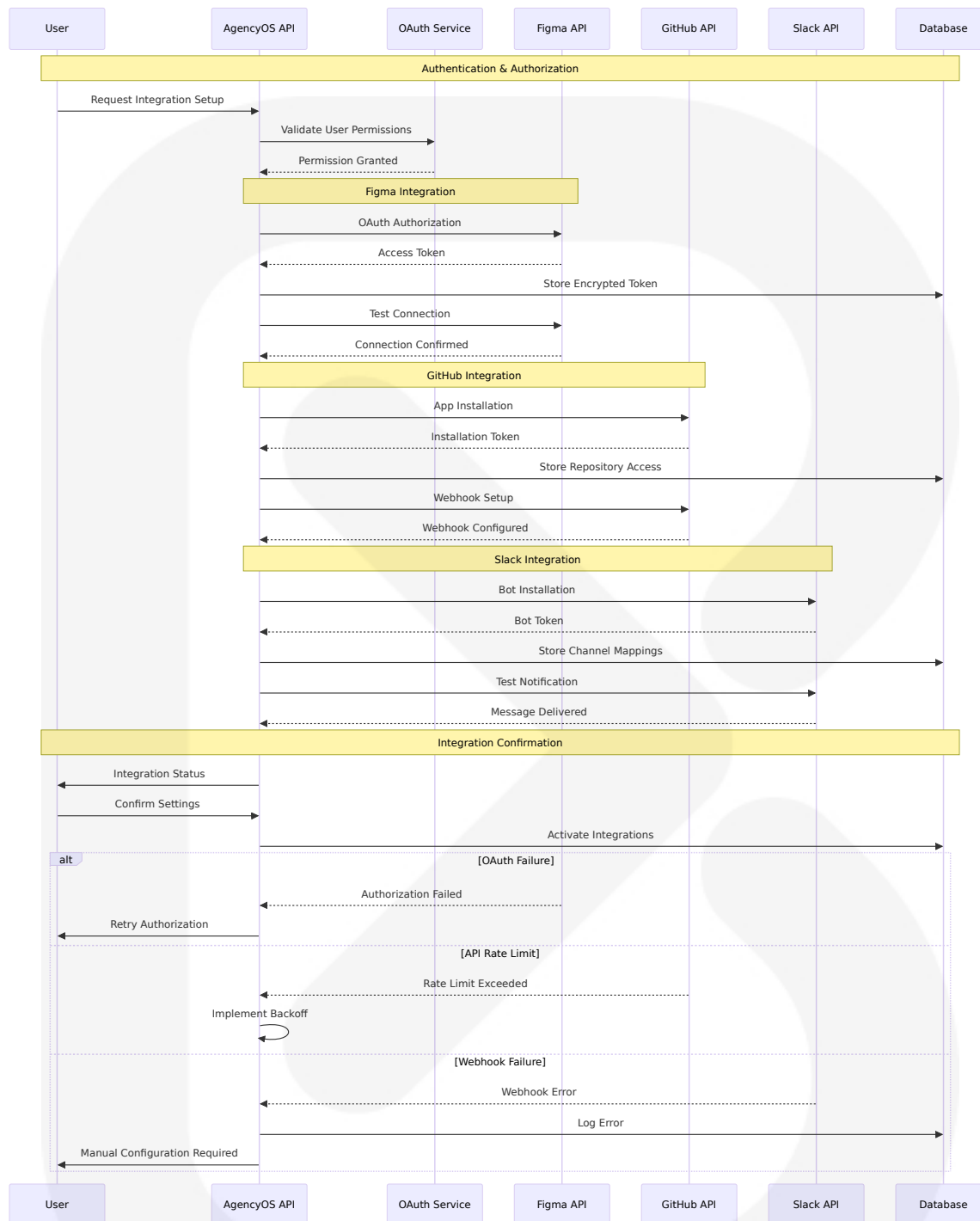
Transaction Boundaries:

- Sprint completion commits all task updates
- Client sign-off creates immutable approval record
- Change request approval updates budget and timeline atomically
- Invoice generation locks project financial data

4.1.2 Integration Workflows

Third-Party Service Integration Flow

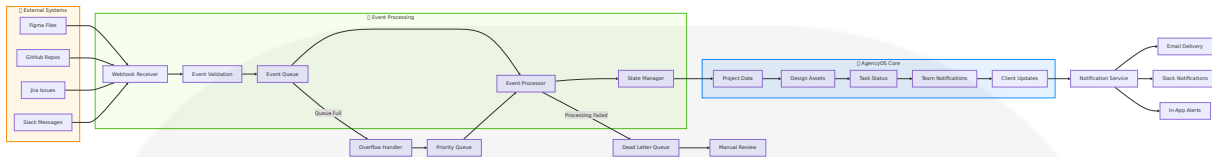
Integration workflows utilize version tracking capabilities and maintain clear change history through Git-compatible systems.



Integration Constraints:

- OAuth tokens refresh automatically 24 hours before expiration
- API rate limits enforced with exponential backoff
- Webhook failures trigger circuit breaker after 3 consecutive errors
- Integration health checks run every 15 minutes

Real-Time Data Synchronization



Event Processing Rules:

- High priority events (security, payments) processed within 30 seconds
- Design file updates batched and processed every 5 minutes
- Failed events retry with exponential backoff (1s, 2s, 4s, 8s, 16s)
- Dead letter queue reviewed daily for manual intervention

4.2 FLOWCHART REQUIREMENTS

4.2.1 Decision Points and Validation

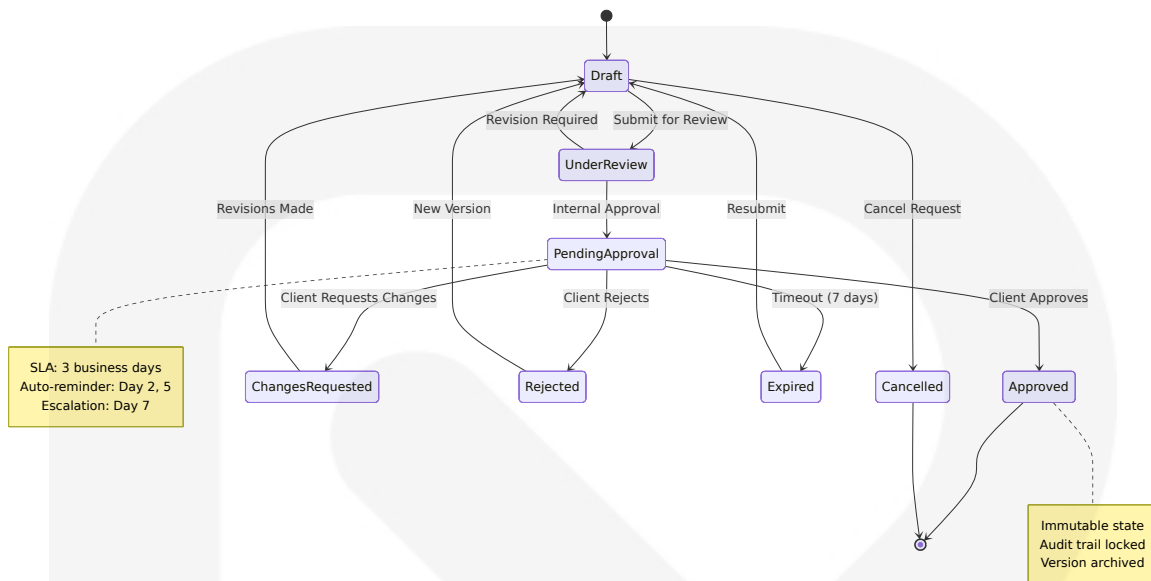
Budget Threshold Management



Authorization Checkpoints:

- Time entry validation requires active project status
- Budget threshold alerts sent to PM and Finance roles
- Override permissions limited to Project Manager and above
- Emergency overrides require Admin approval and audit justification

Client Approval Workflow States



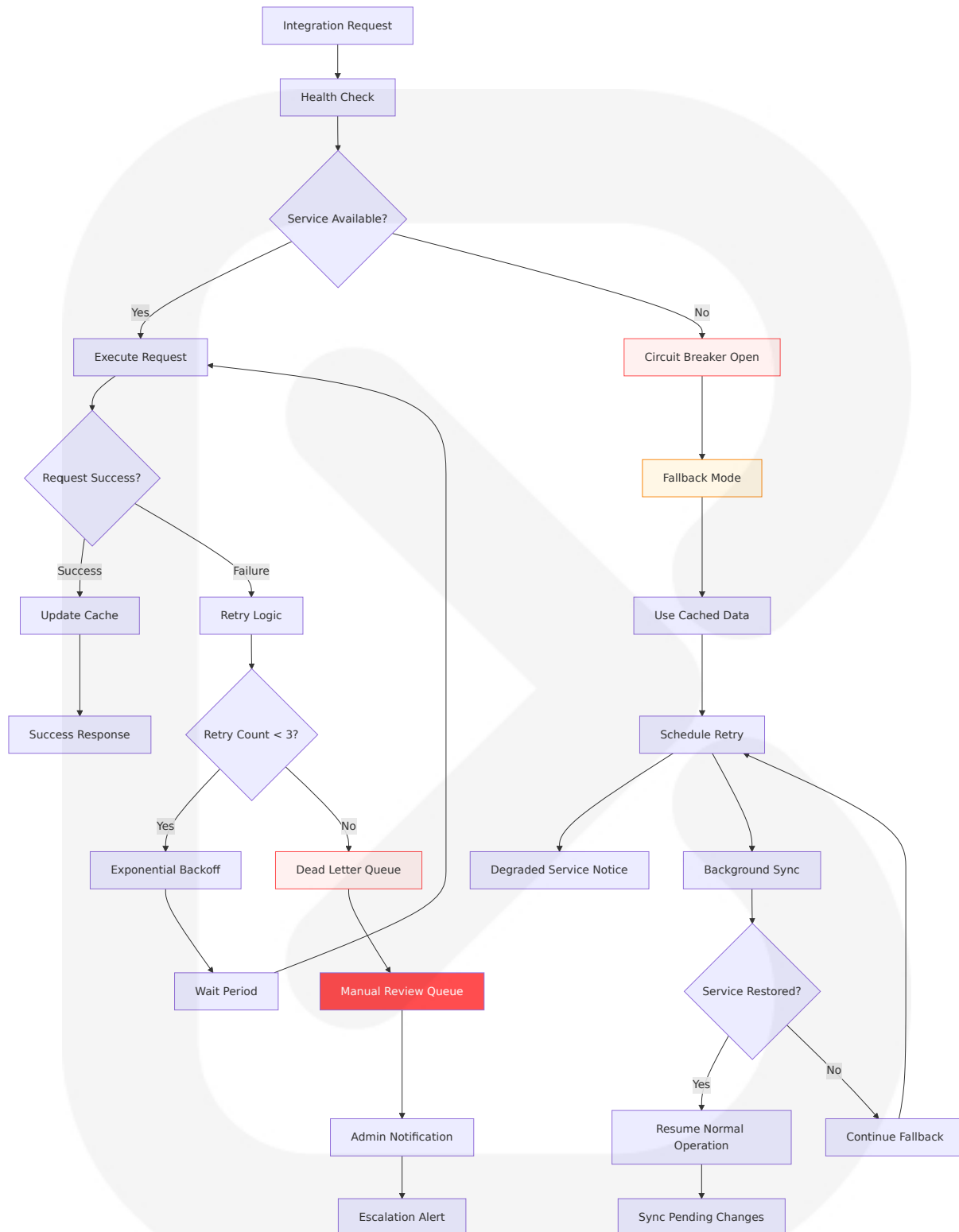
State Transition Rules:

- Only Client Admin can transition to Approved state
- Rejected items require documented reasoning
- Expired approvals automatically notify stakeholders
- State changes trigger webhook notifications to integrated systems

4.2.2 Error Handling and Recovery

Integration Failure Recovery

Version tracking and change history capabilities ensure reliable recovery from integration failures.

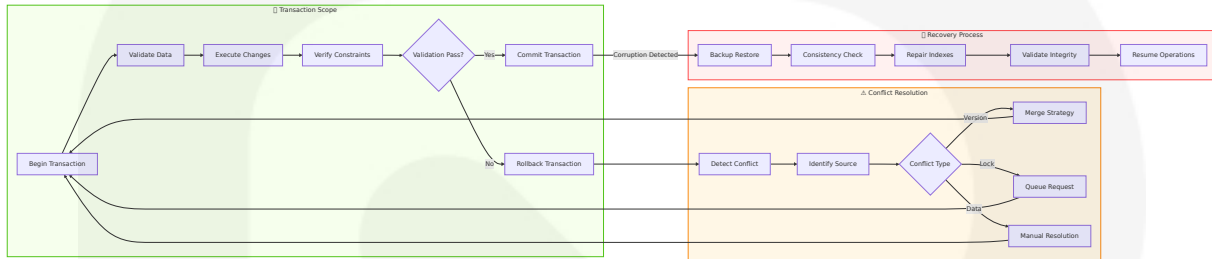


Recovery Procedures:

- Circuit breaker opens after 5 consecutive failures

- Exponential backoff: 1s, 2s, 4s, 8s, 16s intervals
- Fallback mode maintains 95% functionality using cached data
- Manual review queue processed within 4 hours during business hours

Data Consistency Management



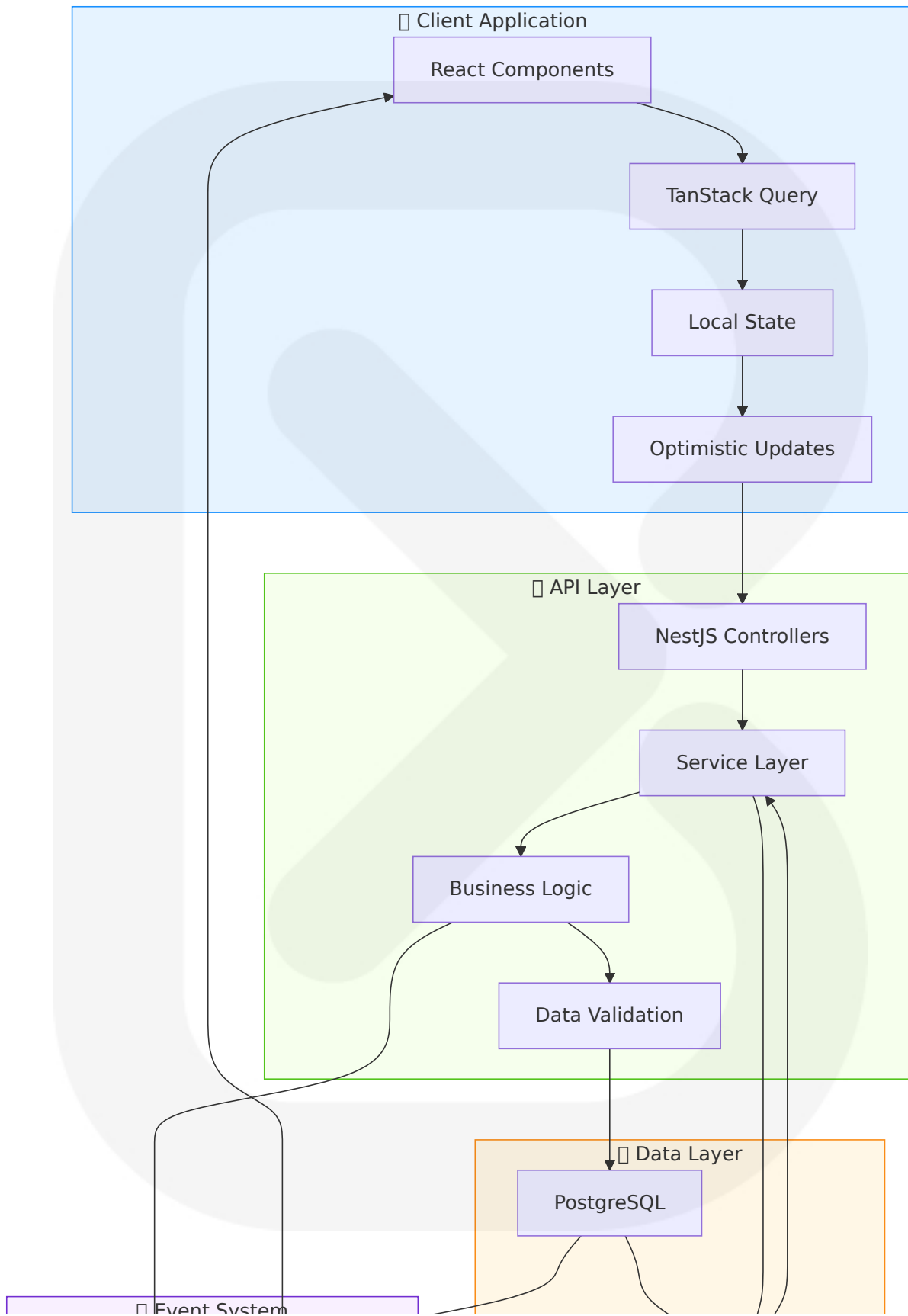
Consistency Rules:

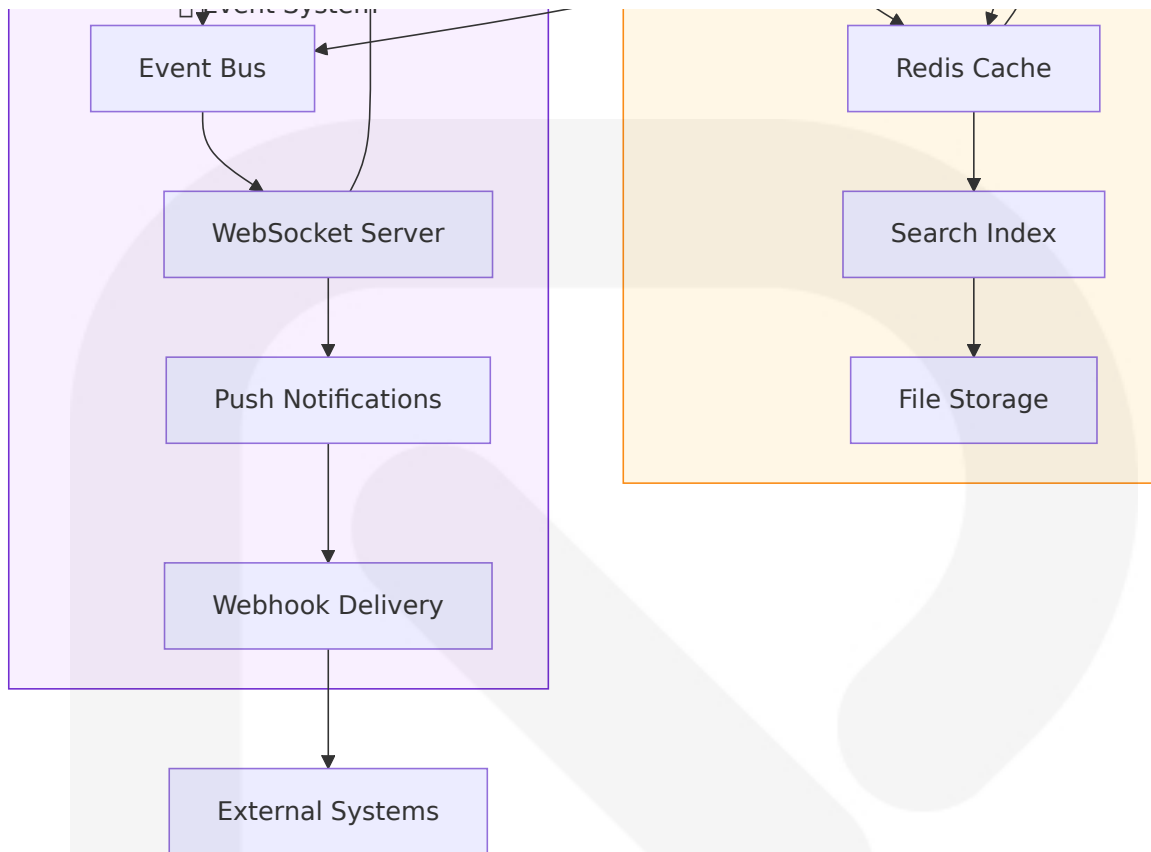
- All financial transactions use ACID compliance
- Project data changes require optimistic locking
- Concurrent edits resolved using last-writer-wins with conflict detection
- Backup restoration triggers full system consistency validation

4.3 TECHNICAL IMPLEMENTATION

4.3.1 State Management Architecture

Application State Flow

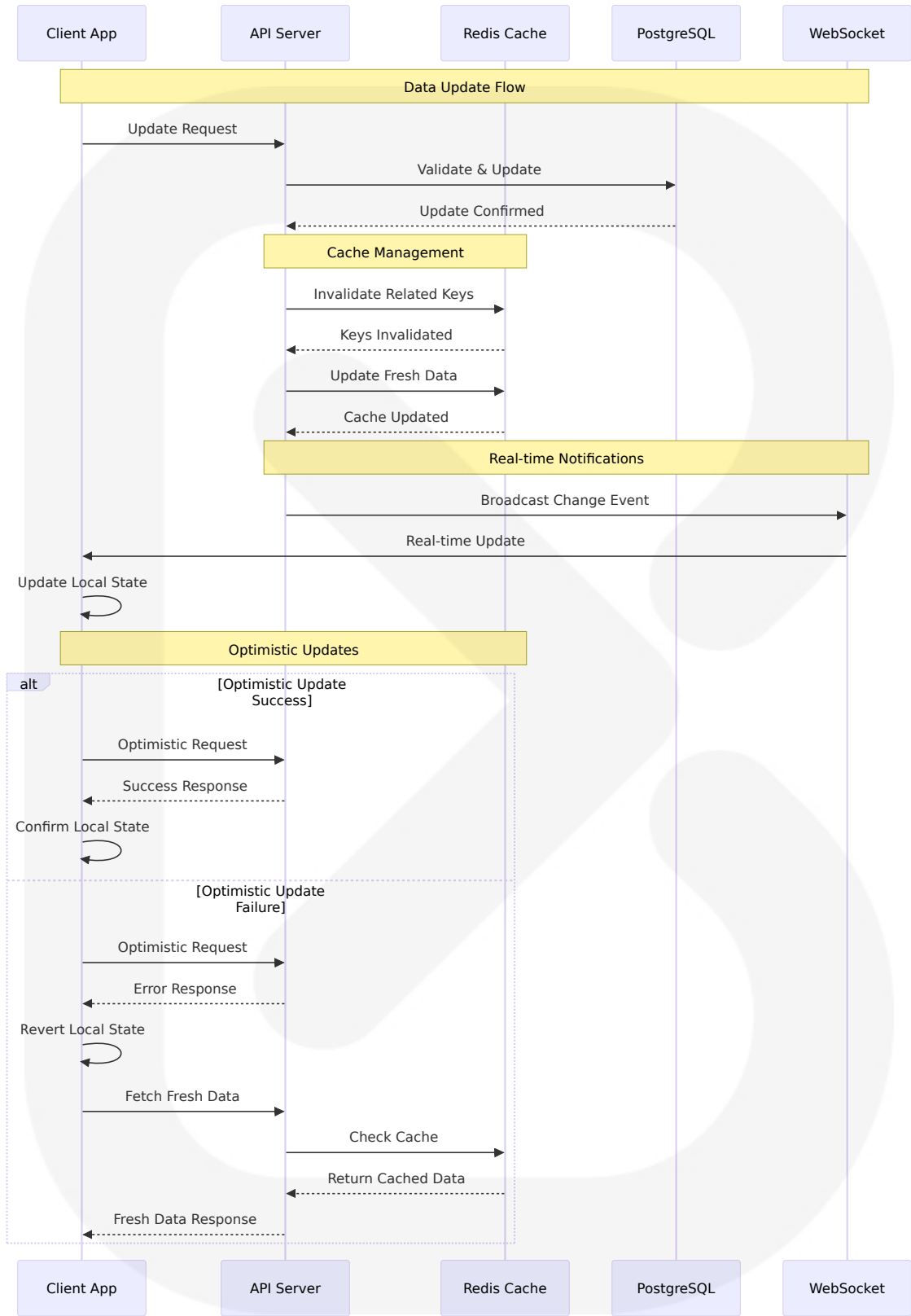




State Persistence Strategy:

- Client state persisted to localStorage for offline capability
- Server state cached in Redis with 1-hour TTL for frequently accessed data
- Database transactions ensure consistency across related entities
- Event sourcing maintains audit trail for all state changes

Cache Invalidation Flow



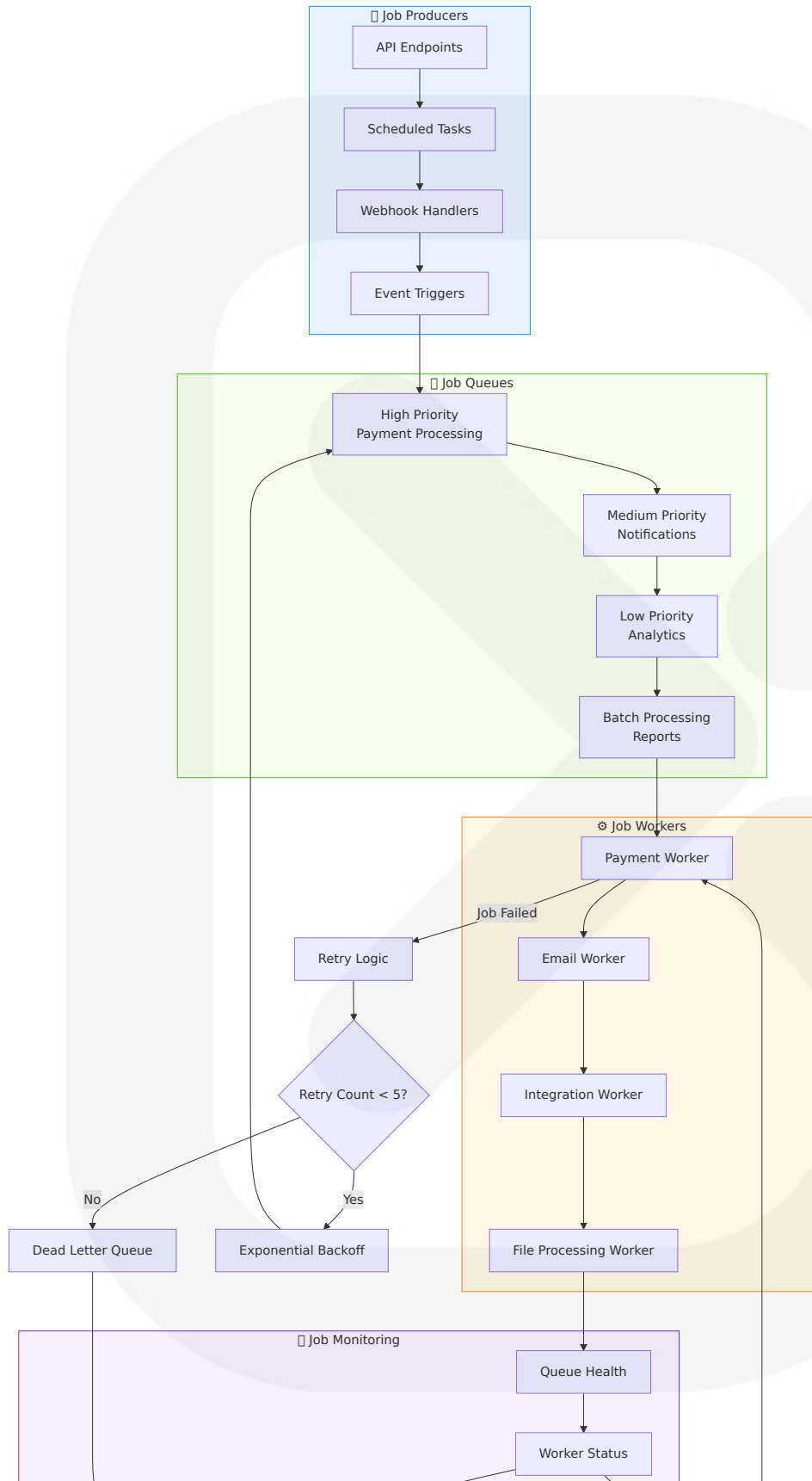
Cache Strategy:

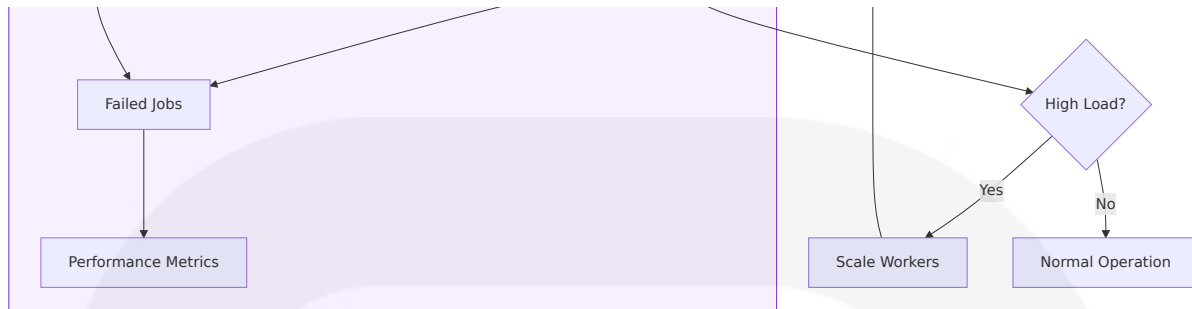
- Write-through caching for critical data (user sessions, project status)
- Write-behind caching for analytics and reporting data
- Cache warming for frequently accessed project dashboards
- Distributed cache invalidation using Redis pub/sub

4.3.2 Background Processing

Job Queue Management

Background processing utilizes BullMQ with Redis for reliable job queue management, following the geometric shapes and edges pattern for clear process flow visualization.





Job Processing Rules:

- High priority jobs (payments, security) processed within 30 seconds
- Medium priority jobs (notifications) processed within 5 minutes
- Low priority jobs (analytics) processed within 1 hour
- Failed jobs retry with exponential backoff: 1min, 2min, 4min, 8min, 16min

Automated Invoice Generation



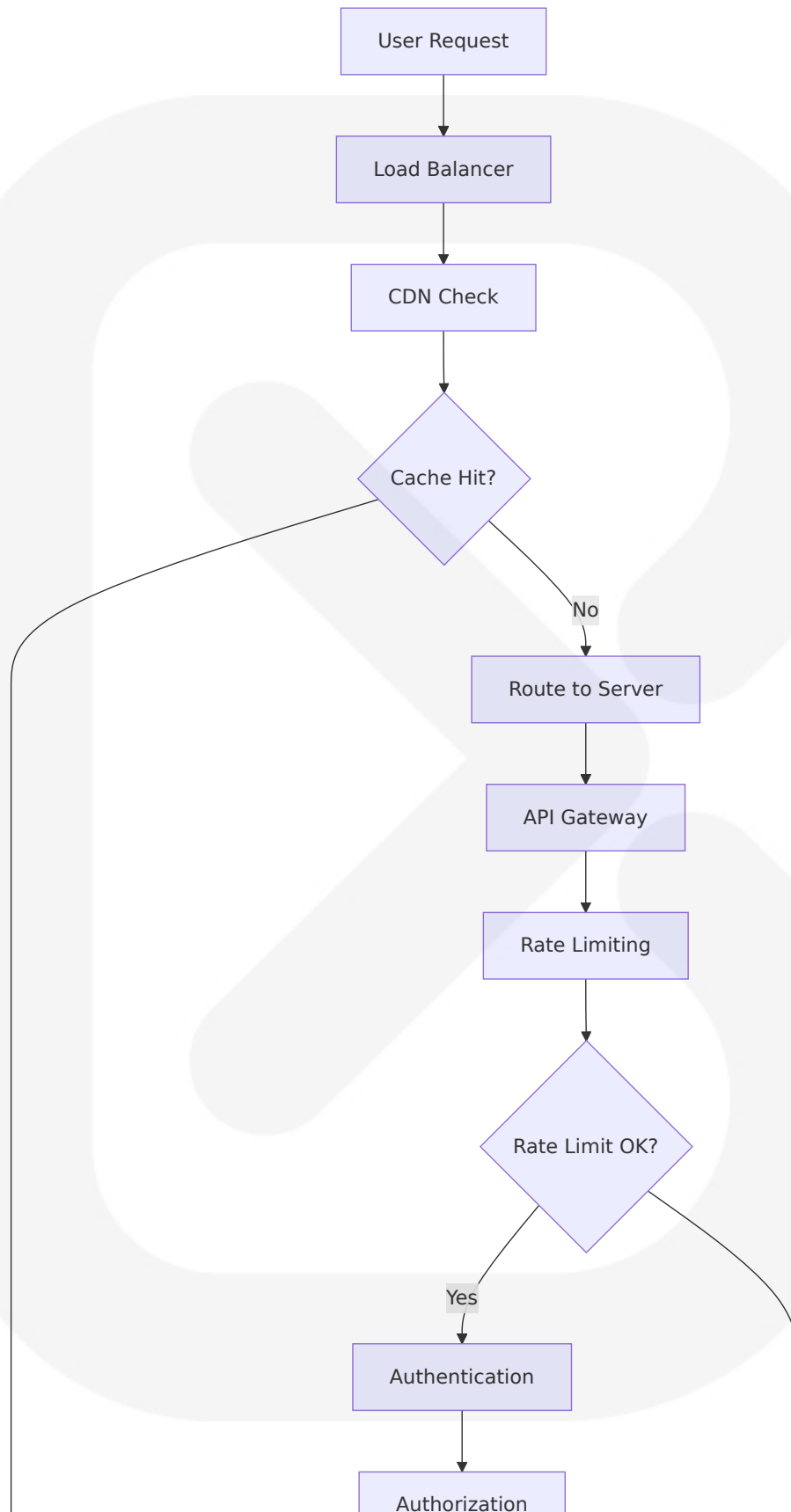
Automation Rules:

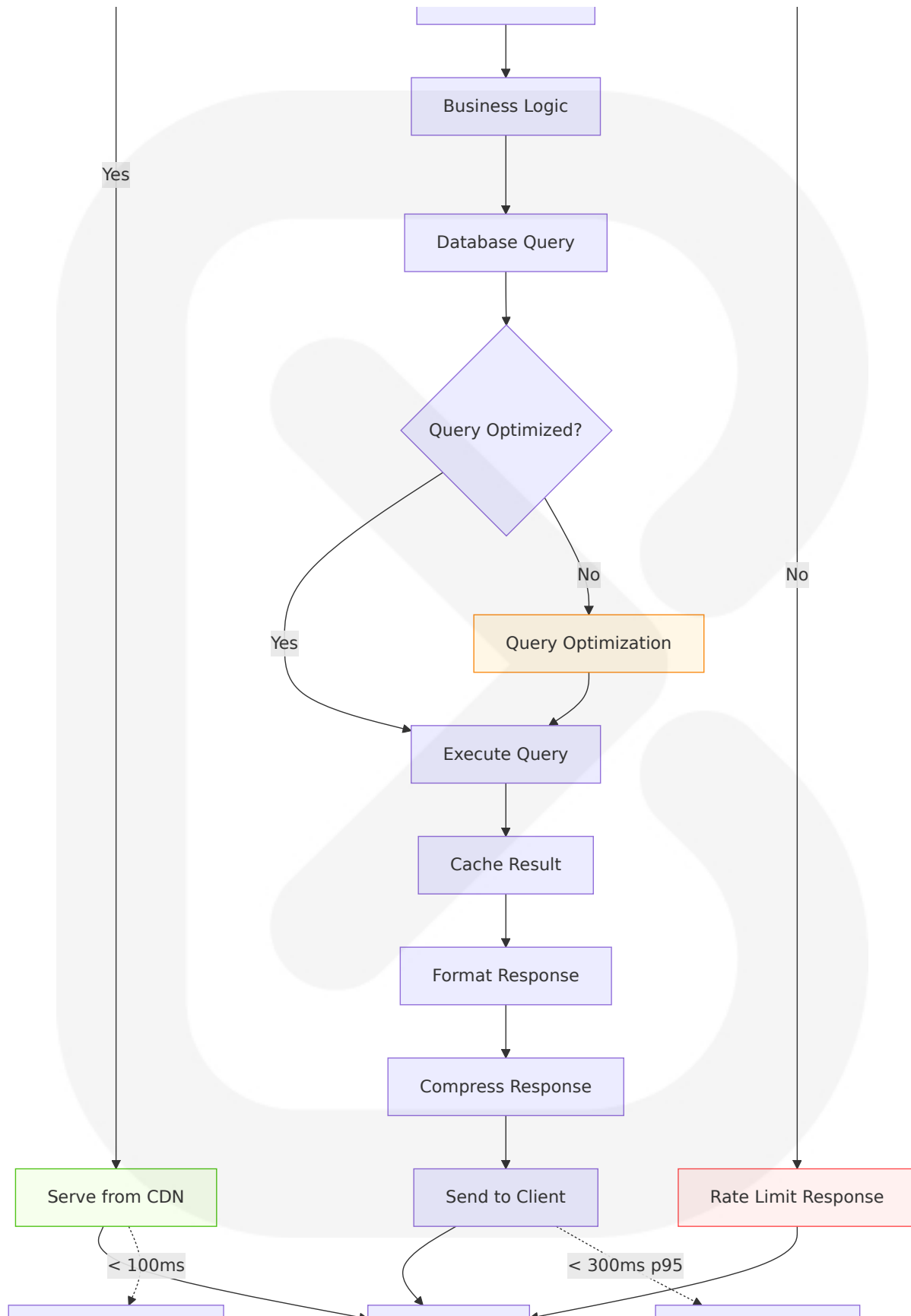
- Invoice generation triggered within 1 hour of milestone completion
- PDF generation includes project branding and custom terms
- Failed deliveries retry 3 times over 24 hours
- Accounting sync failures escalate to finance team after 48 hours

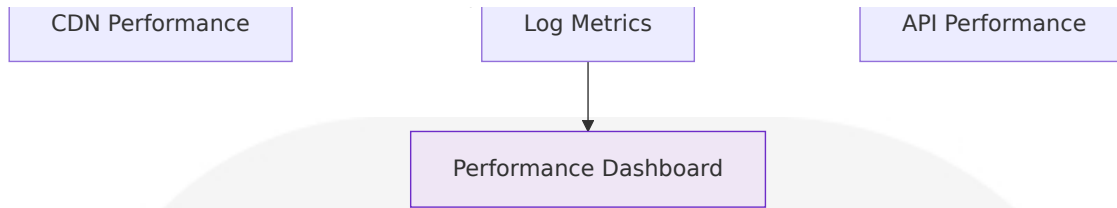
4.4 PERFORMANCE AND MONITORING

4.4.1 System Performance Flow

Response Time Optimization



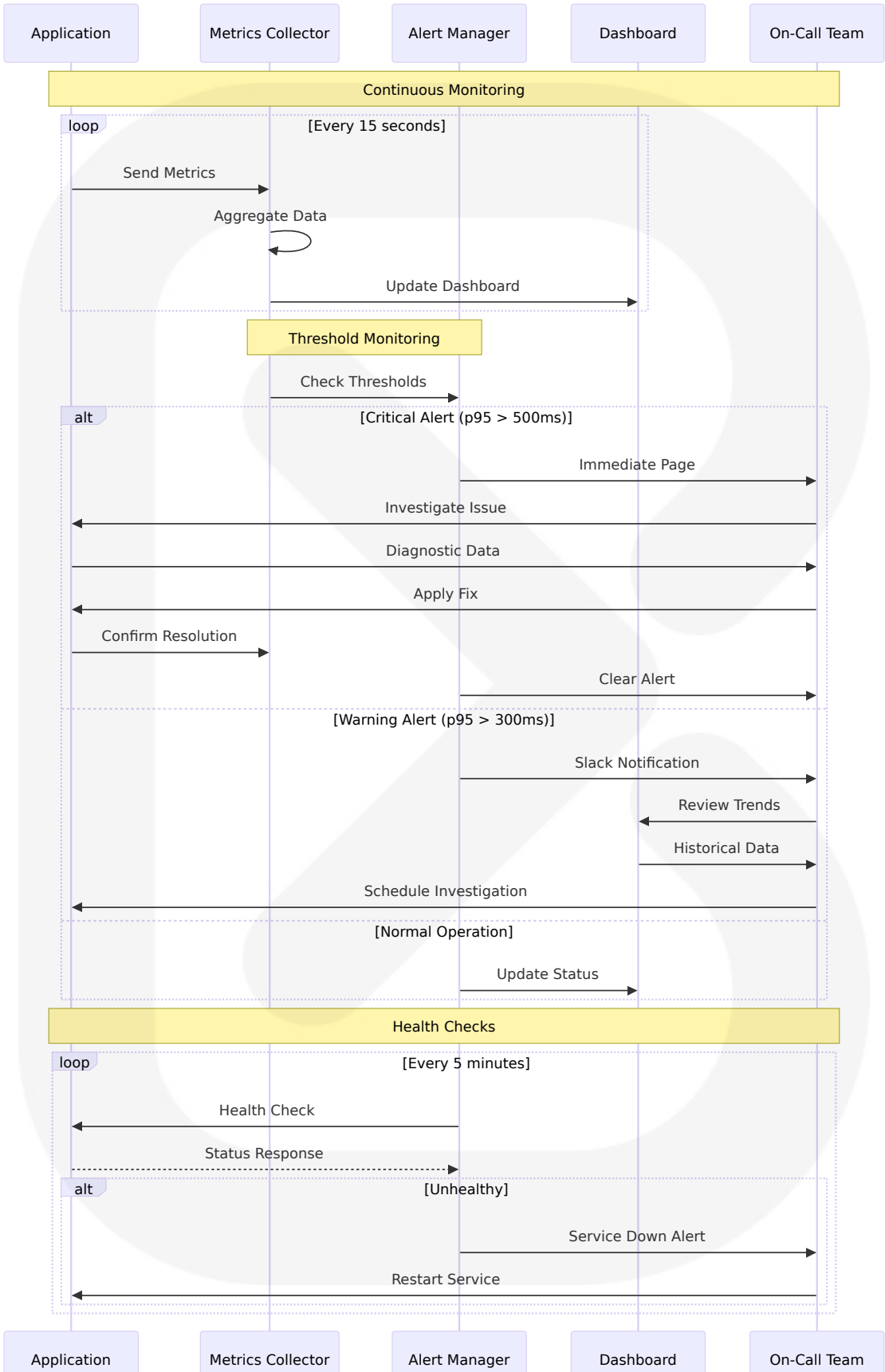




Performance Targets:

- CDN cache hit ratio >90% for static assets
- API response time p95 <300ms for all endpoints
- Database query time p95 <50ms
- Page load time p95 TTI <2.5s

Monitoring and Alerting Flow



Alerting Thresholds:

- Critical: API p95 >500ms, Error rate >5%, Service down
- Warning: API p95 >300ms, Error rate >1%, High memory usage >80%
- Info: Deployment events, Configuration changes, Scheduled maintenance

Monitoring Stack:

- Application metrics: Prometheus + Grafana
- Error tracking: Sentry for exception monitoring
- Infrastructure: DataDog for system metrics
- User experience: Real User Monitoring (RUM) for client-side performance

5. SYSTEM ARCHITECTURE

5.1 HIGH-LEVEL ARCHITECTURE

5.1.1 System Overview

AgencyOS employs a modern, scalable microservices-inspired architecture built on Next.js 15 and NestJS 11, designed to support multi-tenant agency operations with strict data isolation and high performance. The system follows a **hybrid monolithic-microservices pattern** where core business logic remains consolidated for development efficiency while maintaining clear service boundaries for future decomposition.

The architecture is built on **event-driven principles** with asynchronous processing capabilities, enabling real-time collaboration features essential for design review workflows and project management. Next.js 15 introduces significant caching changes, moving from cached-by-default to

uncached-by-default for GET Route Handlers and Client Router Cache, which aligns perfectly with AgencyOS's need for real-time data consistency across multi-tenant environments.

Key Architectural Principles:

- **Multi-tenant by design:** PostgreSQL Row-Level Security (RLS) provides database-level tenant isolation, restricting which rows are returned by SELECT queries or affected by INSERT, UPDATE, and DELETE commands
- **API-first approach:** RESTful APIs with OpenAPI specifications enable seamless third-party integrations
- **Event-driven communication:** Asynchronous messaging patterns support real-time features and integration reliability
- **Horizontal scalability:** Stateless services with shared-nothing architecture enable elastic scaling

The system boundaries encompass the complete agency workflow from lead capture through project delivery, with clear interfaces to external design tools (Figma), development platforms (GitHub/GitLab), communication systems (Slack/Teams), and financial services (Stripe, QuickBooks).

5.1.2 Core Components Table

Component Name	Primary Responsibility	Key Dependencies	Integration Points
Frontend Application	User interface, client-side logic, real-time updates	Next.js 15, React 18, TanStack Query	WebSocket server, API Gateway, CDN
API Gateway	Request routing, authentication, rate limiting, API orchestration	NestJS 11, Fastify, JWT validation	All backend services, external APIs

Component Name	Primary Responsibility	Key Dependencies	Integration Points
Project Management Service	Task management, sprint planning, budget tracking, milestone management	PostgreSQL, Redis, BullMQ	Jira/Linear APIs, GitHub webhooks
Design Review Service	Figma integration, annotation system, approval workflows	Figma API, S3 storage, Web Socket	File storage, notification service

5.1.3 Data Flow Description

The primary data flow follows a **request-response pattern** for synchronous operations and **event-driven messaging** for asynchronous processes. Client requests enter through the Next.js frontend, which communicates with the NestJS API Gateway via RESTful endpoints. The API Gateway performs authentication, authorization, and request routing to appropriate service modules.

Critical Data Flows:

- **Authentication Flow:** JWT tokens validated at API Gateway, user context propagated to all services via request headers
- **Multi-tenant Data Access:** Tenant context set dynamically using SET LOCAL, enforcing strict data isolation at the database level without scattering tenant filters across application code
- **Real-time Updates:** WebSocket connections maintain persistent client connections for live design review comments and project status updates
- **Integration Data Sync:** Webhook receivers process external system events, transforming and routing data through event queues

Data Transformation Points:

- API Gateway normalizes external webhook payloads into internal event formats

- Service layer transforms business objects into database entities with tenant context
- Integration adapters convert between internal data models and third-party API schemas

Key Data Stores:

- **PostgreSQL Primary:** Transactional data with row-level security for tenant isolation
- **Redis Cache:** Session storage, job queues, and frequently accessed project data
- **S3 Object Storage:** Design files, documents, and asset library with CDN distribution
- **Meilisearch:** Full-text search across projects, assets, and knowledge base content

5.1.4 External Integration Points

System Name	Integration Type	Data Exchange Pattern	Protocol/Format	SLA Requirements
Figma API	Real-time sync	Webhook + polling	REST/JSON, OAuth 2.0	<5min sync latency
GitHub/GitLab	Event-driven	Webhooks + API calls	REST/JSON, OAuth 2.0	<30sec PR p review
Stripe Payments	Transactional	API calls + webhooks	REST/JSON, API keys	<10sec payment processing
Slack/Teams	Notification	Bot API + webhooks	REST/JSON, Bot tokens	<5sec notification delivery

5.2 COMPONENT DETAILS

5.2.1 Frontend Application Architecture

Purpose and Responsibilities:

The frontend application serves as the primary user interface for both agency teams and clients, built on Next.js 15 with React 19 support and enhanced hydration error improvements. It manages client-side state, handles real-time updates, and provides responsive interfaces for project management, design review, and client collaboration.

Technologies and Frameworks:

- **Next.js 15 App Router:** Emphasizes App Router architecture, React Server Components, and seamless full-stack capabilities
- **React 18+:** Component-based UI with concurrent features and Suspense boundaries
- **TanStack Query v5:** Advanced data fetching with optimistic updates and background synchronization
- **Tailwind CSS:** Utility-first styling with design system consistency

Key Interfaces and APIs:

- RESTful API consumption through typed client libraries
- WebSocket connections for real-time collaboration features
- Server-Sent Events for one-way live updates (build status, notifications)
- File upload APIs with progress tracking and resumable uploads

Data Persistence Requirements:

- Client-side state persisted to localStorage for offline capability
- Optimistic updates with rollback mechanisms for failed operations
- Session storage for temporary form data and navigation state

Scaling Considerations:

- Static asset optimization with Next.js Image component and automatic WebP generation

- Code splitting and lazy loading for large component trees
- CDN distribution for global performance optimization
- Service Worker implementation for offline functionality

5.2.2 API Gateway Service

Purpose and Responsibilities:

The API Gateway serves as the central entry point for all backend operations, implementing authentication, authorization, request routing, and rate limiting. It acts as an entry point for clients and routes requests to the appropriate microservice, simplifying tasks like load balancing, authentication and rate limiting.

Technologies and Frameworks:

- **NestJS 11:** Latest version 11.1.6 providing application architecture for highly testable, scalable, and loosely coupled applications
- **Fastify 4.x:** High-performance HTTP server with 25% better p95 latency than Express
- **JWT Authentication:** Stateless authentication with refresh token rotation
- **Rate Limiting:** Redis-backed rate limiting with sliding window algorithm

Key Interfaces and APIs:

- OpenAPI 3.0 specification with automated documentation generation
- GraphQL endpoint for complex data fetching requirements
- WebSocket gateway for real-time communication management
- Webhook receivers for external system integration

Data Persistence Requirements:

- Stateless design with no local data persistence
- Session data stored in Redis with configurable TTL
- Request/response logging for audit and debugging purposes

Scaling Considerations:

- Horizontal scaling with load balancer distribution
- Connection pooling for database and Redis connections
- Circuit breaker patterns for external service calls
- Graceful shutdown handling for zero-downtime deployments

5.2.3 Multi-Tenant Data Layer

Purpose and Responsibilities:

The data layer implements comprehensive multi-tenant isolation using PostgreSQL's row level security feature to create SaaS applications that use a pool model to share database resources while reducing the risk and overhead of enforcing isolation policies.

Technologies and Frameworks:

- **PostgreSQL 15+:** Row Level Security with `security_invoker = true` for views to obey RLS policies of underlying tables
- **Prisma ORM:** Type-safe database access with automated migrations
- **Connection Pooling:** PgBouncer for high-concurrency connection management

Key Interfaces and APIs:

- Database connection factory with tenant context injection
- Migration system with tenant-aware schema changes
- Backup and restore APIs with tenant-specific data handling

Data Persistence Requirements:

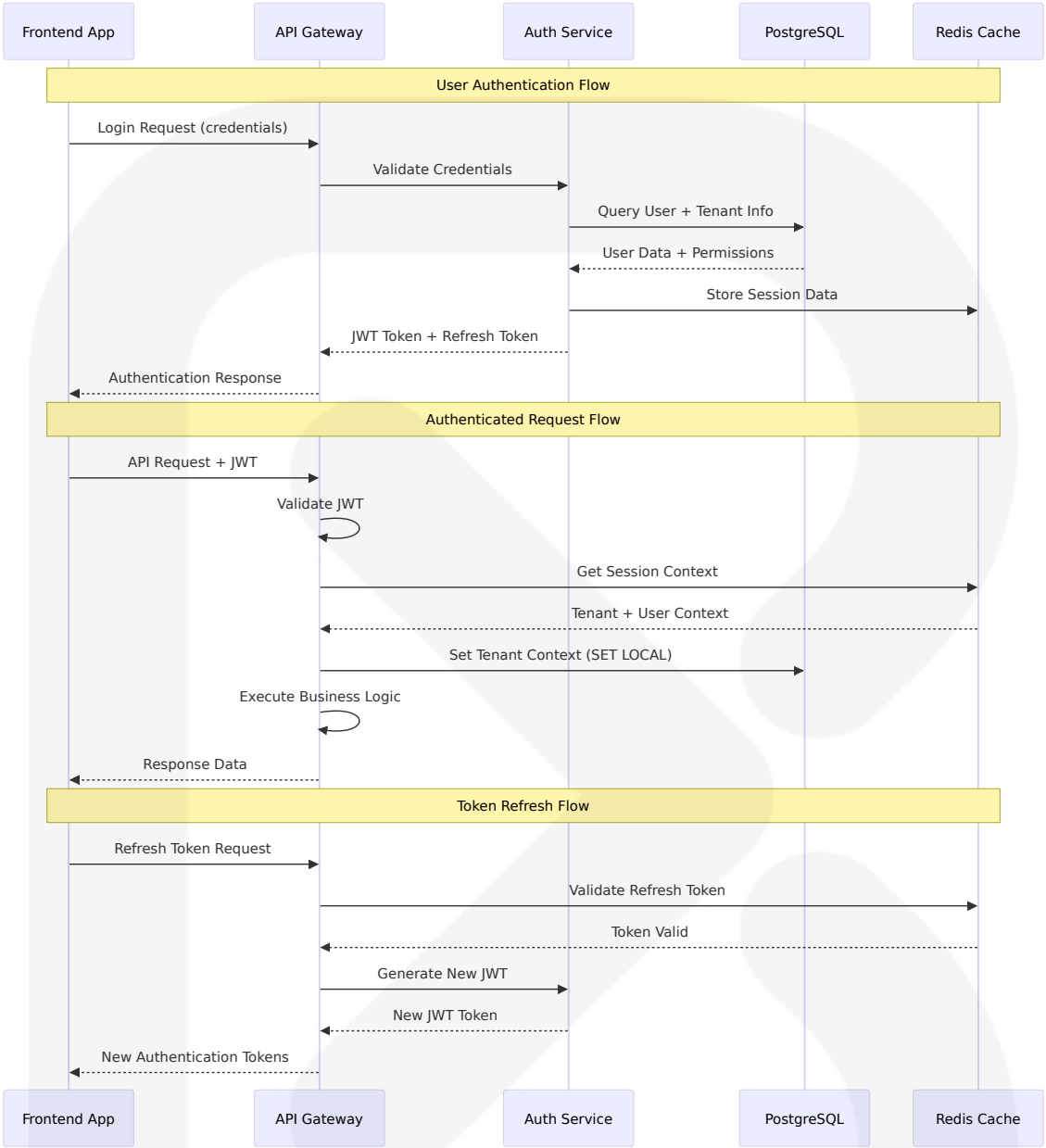
- Tenant context set using `SET LOCAL public.current_account`, with RLS policies pushing access control responsibility to the database
- Audit logging for all data modifications with immutable trail
- Automated backup with point-in-time recovery capabilities

Scaling Considerations:

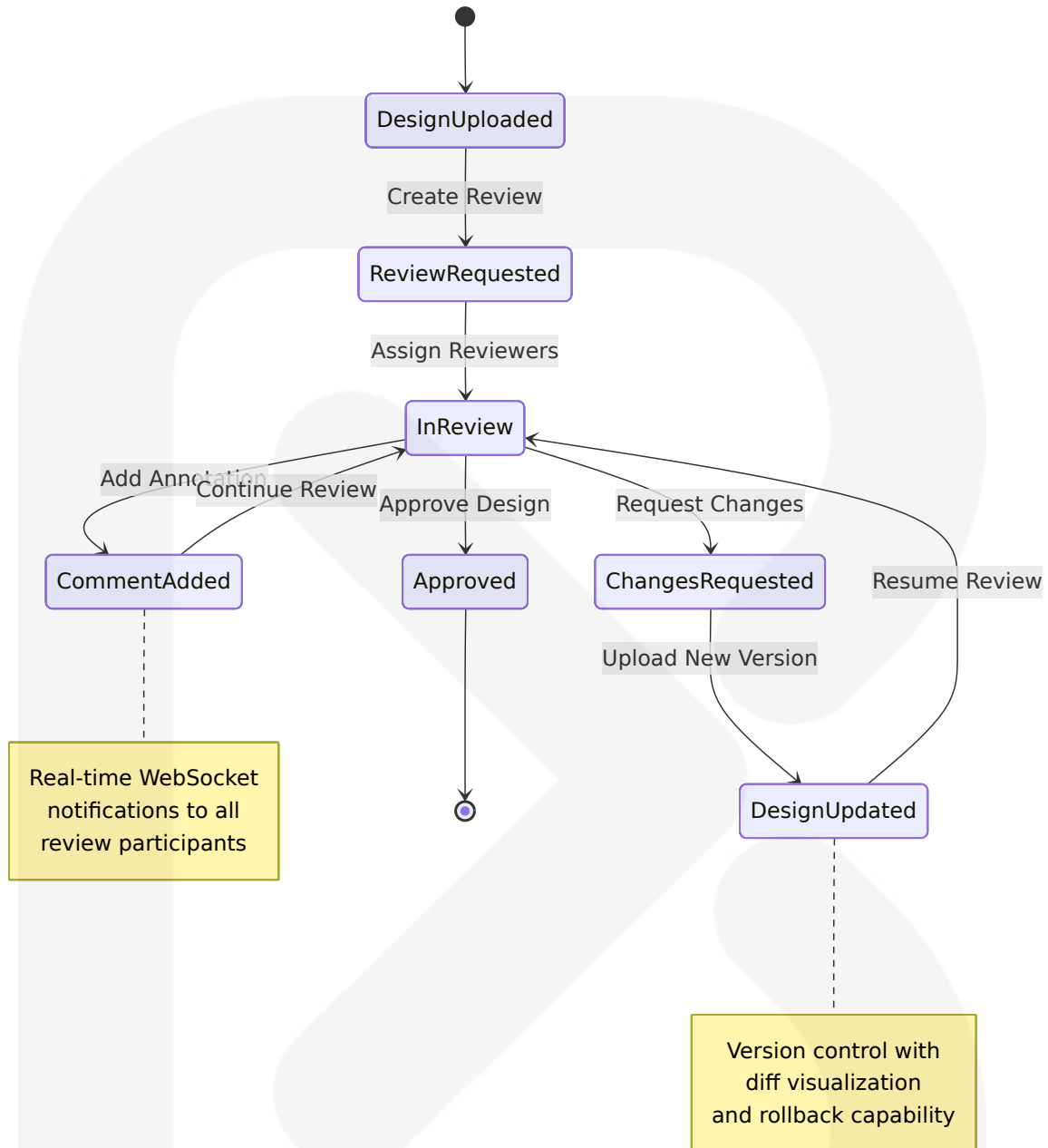
- Read replica configuration for reporting and analytics queries
- Partitioning strategy for time-series data (audit logs, analytics)
- Connection pool sizing based on tenant activity patterns

5.2.4 Component Interaction Diagrams

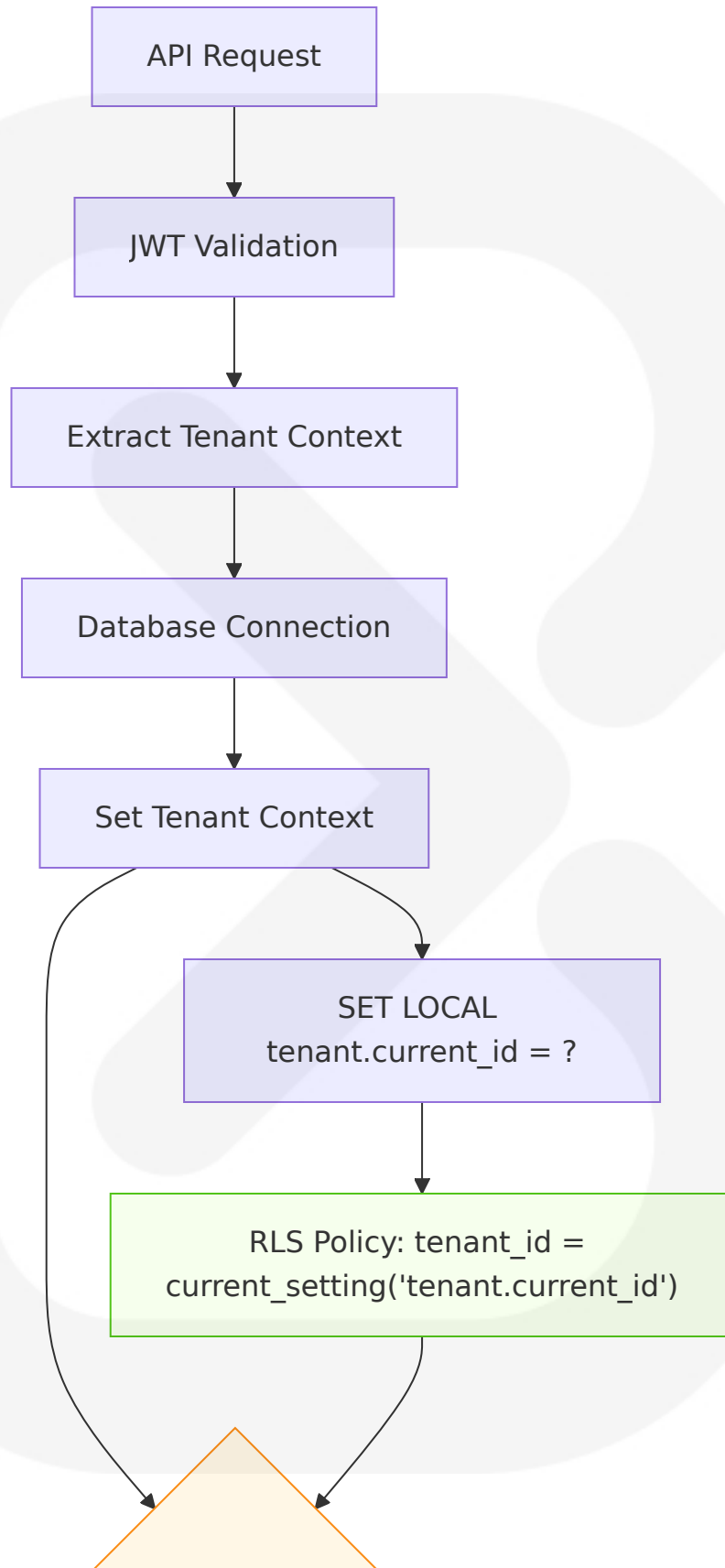
Authentication and Authorization Flow

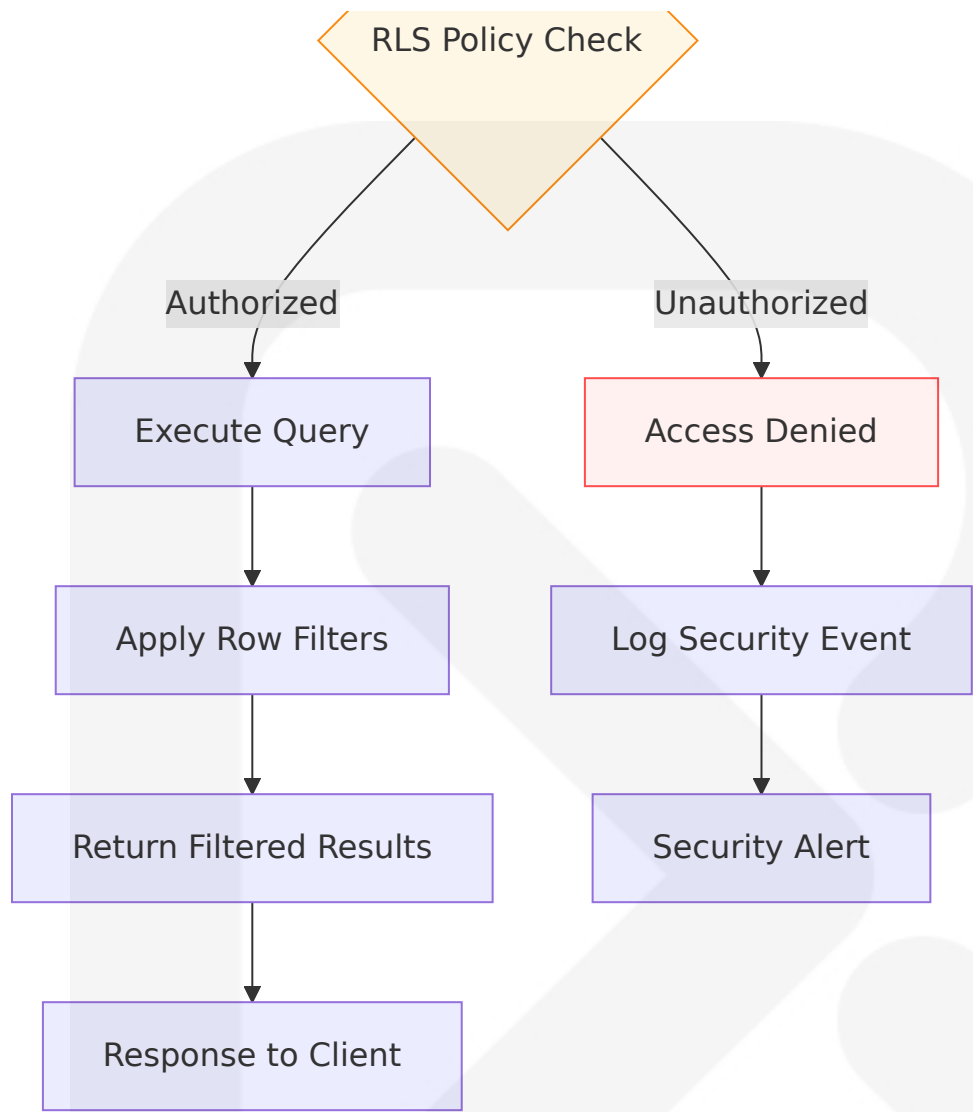


Real-Time Design Review Workflow



Multi-Tenant Data Access Pattern





5.3 TECHNICAL DECISIONS

5.3.1 Architecture Style Decisions

Hybrid Monolithic-Microservices Architecture

Decision Factor	Monolithic Benefits	Microservices Benefits	AgencyOS Choice
Development Velocity	Faster initial development, simpler debugging	Independent team scaling	Hybrid: Monolithic core with service boundaries

Decision Factor	Monolithic Benefits	Microservices Benefits	AgencyOS Choice
Data Consistency	ACID transactions, strong consistency	Eventual consistency challenges	Monolithic data layer with event-driven sync
Deployment Complexity	Single deployment unit	Complex orchestration	Monolithic deployment with microservice readiness
Technology Diversity	Unified technology stack	Technology per service	Unified stack with pluggable integration adapters

Rationale: AgencyOS adopts a hybrid approach that provides monolithic development benefits while maintaining clear service boundaries for future decomposition. This enables rapid feature development during the initial phase while preserving the option to extract services as the system scales.

Event-Driven Communication Pattern

The system implements event-driven architecture for asynchronous operations while maintaining synchronous request-response for user-facing interactions. Event-based patterns are more flexible, offering the opportunity to create complex architectures that scale more easily and are highly responsive.

Benefits:

- Loose coupling between components enables independent scaling
- Resilient to temporary service failures through message queuing
- Supports complex workflows like approval chains and notification cascades
- Enables audit trails and event sourcing for compliance requirements

5.3.2 Data Storage Solution Rationale

PostgreSQL with Row-Level Security for Multi-Tenancy

Row-Level Security enforces tenant-specific policies at the database level, ensuring isolation even if application code misses a filter, with every SQL operation automatically filtered according to defined policies.

Approach	Pros	Cons	AgencyOS Decision
Separate Databases	Complete isolation, simple queries	High operational overhead, resource waste	Not selected
Schema-per-Tenant	Good isolation, shared resources	Complex migrations, limited scalability	Not selected
Shared Schema + RLS	Efficient resources, database-enforced security	Complex policy management	Selected
Application-Level Filtering	Simple implementation, full control	Error-prone, security risks	Not selected

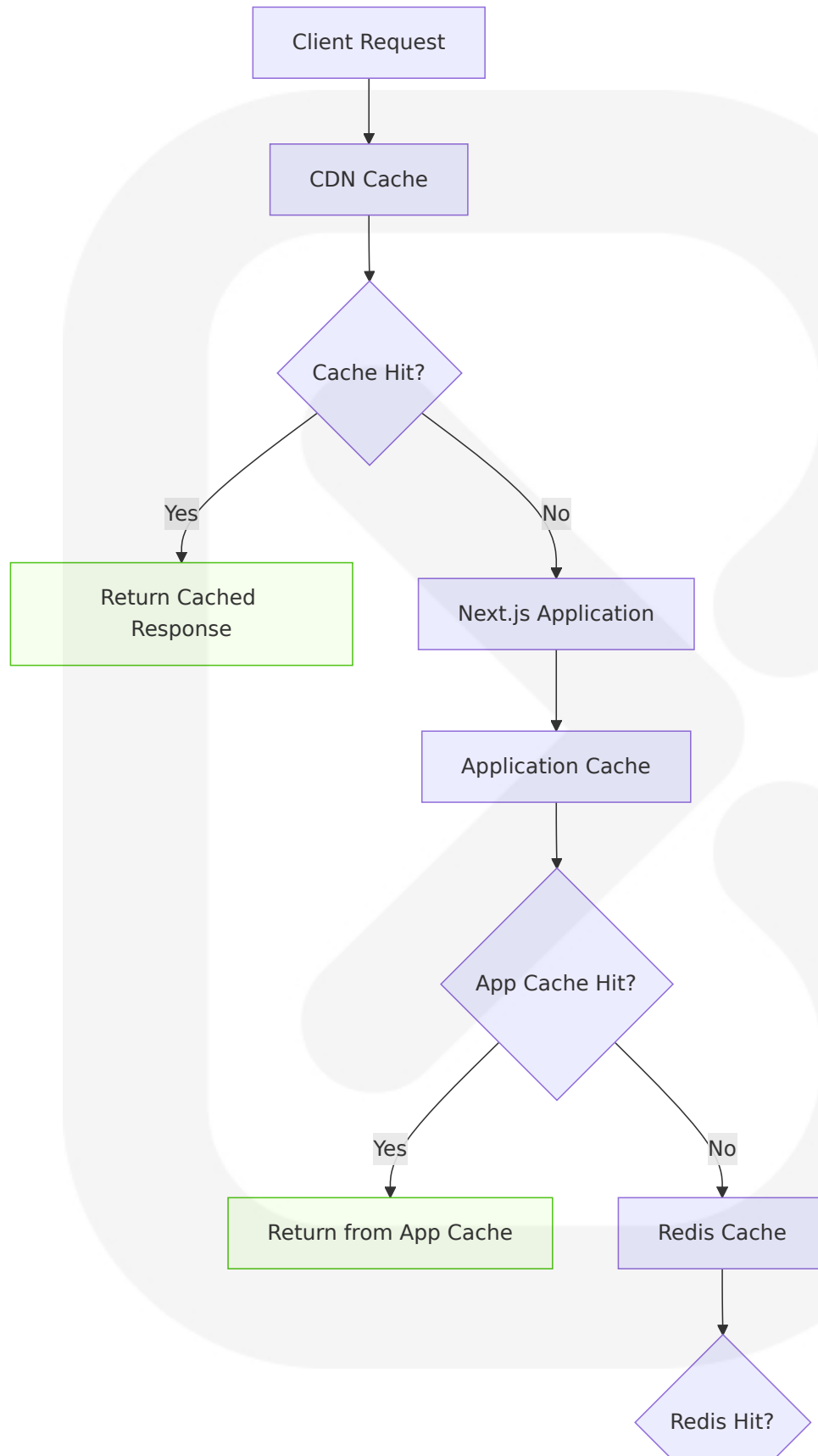
Implementation Strategy:

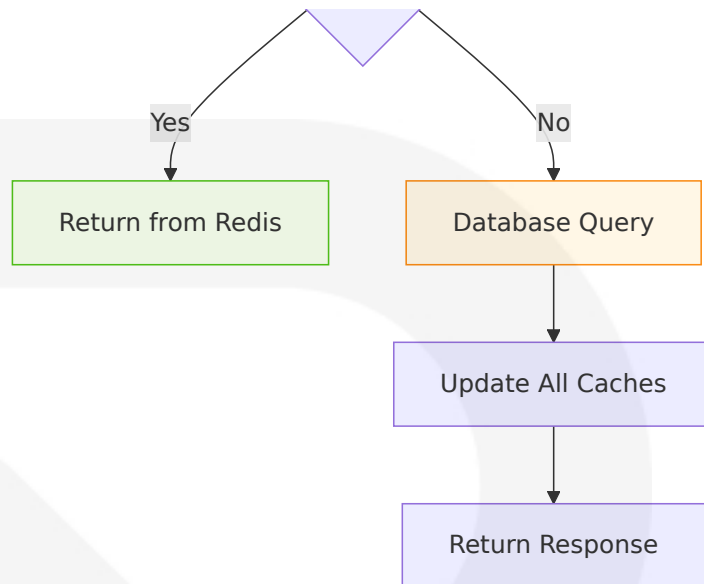
- Tenant context set dynamically using SET LOCAL, with RLS policies providing data isolation at the database level
- Automated policy creation for new tables through database migrations
- Comprehensive testing suite to validate tenant isolation across all data access patterns

5.3.3 Caching Strategy Justification

Multi-Layer Caching Architecture

The caching strategy addresses Next.js 15's caching changes from cached-by-default to uncached-by-default, requiring explicit opt-in for caching behavior.





Cache Invalidation Strategy:

- **Write-through:** Critical data (user sessions, project status) immediately updated in cache
- **Write-behind:** Analytics and reporting data updated asynchronously
- **Event-driven invalidation:** Cache keys invalidated based on business events (project updates, design changes)

5.3.4 Security Mechanism Selection

Defense-in-Depth Security Architecture

Security Layer	Technology	Purpose	Implementation
Network Security	WAF + DDoS Protection	Traffic filtering, attack mitigation	CloudFlare/AWS WAF rules
Application Security	JWT + OAuth 2.0	Authentication and authorization	NestJS Guards and Decorators
Data Security	PostgreSQL RLS	Row-level access control	Tenant-aware database policies
Transport Security	TLS 1.2+	Encryption in transit	SSL certificates with HSTS

Multi-Factor Authentication Strategy:

- Primary authentication via OAuth 2.0 providers (Auth0, Google, Microsoft)
- MFA enforcement for administrative roles and sensitive operations
- JWT function used to check for Multi-Factor Authentication, restricting user actions unless they have at least 2 levels of authentication (Assurance Level 2)

5.4 CROSS-CUTTING CONCERNS

5.4.1 Monitoring and Observability Approach

Three Pillars of Observability Implementation

The monitoring strategy implements comprehensive observability across metrics, logs, and traces to ensure system reliability and performance optimization.

Metrics Collection:

- **Application Metrics:** API response times, error rates, throughput via Prometheus
- **Business Metrics:** Project completion rates, approval cycle times, revenue tracking
- **Infrastructure Metrics:** CPU, memory, disk usage, network performance via DataDog
- **User Experience Metrics:** Page load times, interaction latency via Real User Monitoring

Distributed Tracing:

- OpenTelemetry instrumentation across all services and external integrations
- Trace correlation for multi-service request flows (design review, approval workflows)
- Performance bottleneck identification in complex integration chains

Alerting Strategy:

- **Critical Alerts:** Service downtime, payment processing failures, security incidents
- **Warning Alerts:** Performance degradation, high error rates, capacity thresholds
- **Business Alerts:** SLA violations, budget overruns, approval delays

5.4.2 Logging and Tracing Strategy

Structured Logging Architecture

All application logs follow structured JSON format with consistent field naming and correlation IDs for request tracing across service boundaries.

Log Levels and Retention:

- **ERROR:** System errors, integration failures, security events (retained 2 years)
- **WARN:** Performance issues, deprecated API usage, business rule violations (retained 1 year)
- **INFO:** Business events, user actions, system state changes (retained 6 months)
- **DEBUG:** Detailed execution flow, variable states (retained 30 days)

Audit Trail Requirements:

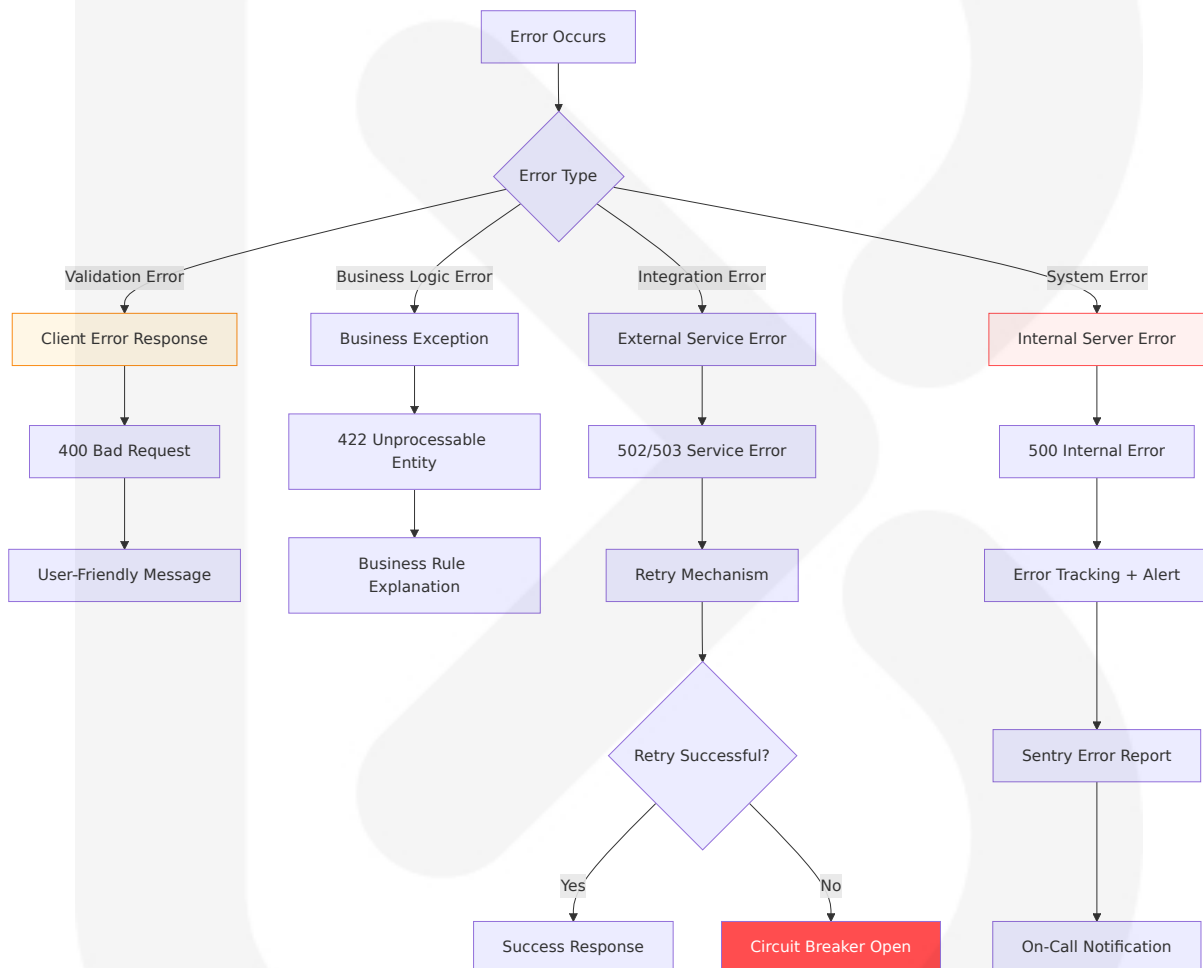
- All data modifications logged with user context, timestamp, and change details
- Immutable audit log storage with cryptographic integrity verification

- Compliance reporting capabilities for SOC 2 and GDPR requirements

5.4.3 Error Handling Patterns

Hierarchical Error Handling Strategy

The system implements consistent error handling patterns across all layers, from database constraints to user interface feedback.



Error Recovery Mechanisms:

- **Circuit Breaker Pattern:** Prevents cascade failures in external integrations
- **Exponential Backoff:** Retry logic for transient failures with increasing delays

- **Graceful Degradation:** Fallback modes for non-critical functionality
- **Dead Letter Queues:** Failed message handling with manual review processes

5.4.4 Authentication and Authorization Framework

Role-Based Access Control (RBAC) with Attribute-Based Extensions

The authorization framework implements hierarchical role-based permissions with project-level overrides and attribute-based fine-grained controls.

Role Hierarchy:

- **Super Admin:** Full system access, tenant management, security configuration
- **Admin:** Tenant-wide administration, user management, billing oversight
- **Project Manager:** Project lifecycle management, resource allocation, client communication
- **Team Members:** Task execution, time tracking, asset management (Designer, Engineer, QA)
- **Client Roles:** Project visibility, approval workflows, invoice access (Admin, Reviewer, Viewer)

Permission Enforcement Points:

- **API Gateway:** Route-level permissions and rate limiting
- **Service Layer:** Business logic authorization with context-aware rules
- **Database Layer:** Row-level security policies for data access control
- **Frontend:** UI element visibility and interaction permissions

5.4.5 Performance Requirements and SLAs

Service Level Objectives (SLOs)

Metric Category	Target SLO	Measurement Window	Alerting Threshold
API Response Time	p95 < 300ms	5-minute rolling window	p95 > 500ms for 2 minutes
Page Load Performance	p95 TTI < 2.5s	15-minute rolling window	p95 > 4s for 5 minutes
System Availability	99.9% uptime	Monthly measurement	< 99.5% in any 24-hour period
Data Consistency	100% tenant isolation	Continuous validation	Any cross-tenant data access

Performance Optimization Strategies:

- **Database Query Optimization:** Automated query analysis and index recommendations
- **CDN Distribution:** Global edge caching for static assets and API responses
- **Connection Pooling:** Optimized database connection management for multi-tenant workloads
- **Background Processing:** Asynchronous job processing for non-critical operations

5.4.6 Disaster Recovery Procedures

Recovery Time and Point Objectives

The disaster recovery strategy ensures business continuity with minimal data loss and service interruption.

Backup Strategy:

- **Database:** Continuous WAL archiving with point-in-time recovery capability

- **File Storage:** Cross-region replication for critical design assets and documents
- **Configuration:** Infrastructure as Code with automated environment recreation
- **Application State:** Redis persistence with backup restoration procedures

Failover Procedures:

- **Automated Failover:** Database replica promotion within 5 minutes of primary failure
- **Manual Failover:** Complete environment recreation within 60 minutes
- **Data Recovery:** Point-in-time restoration with maximum 15-minute data loss
- **Service Restoration:** Graduated service restoration prioritizing critical business functions

Business Continuity Planning:

- **Communication Plan:** Automated status page updates and stakeholder notifications
- **Escalation Procedures:** On-call rotation with defined response time requirements
- **Recovery Validation:** Automated testing of backup restoration and failover procedures
- **Post-Incident Review:** Comprehensive analysis and improvement implementation

6. SYSTEM COMPONENTS DESIGN

6.1 COMPONENT ARCHITECTURE OVERVIEW

6.1.1 Component-Based Design Philosophy

AgencyOS employs a component-based architecture where an application is constructed by putting together separate, reusable parts that each contain a particular purpose. This architectural approach aligns with modern software development practices, where systems are easier to develop, modify, and scale through modularity, flexibility, and reusability in software development.

The system's component architecture follows key principles that ensure scalability and maintainability:

Core Design Principles:

- **Modularity:** Divide the system into cohesive, reusable components that encapsulate specific functionality or behavior. Each component should have a well-defined purpose and interface
- **Encapsulation:** Components should encapsulate both data and behavior, enforcing information hiding and reducing dependencies between components
- **Reusability:** Design components to be reusable across different parts of the application or in other projects. This reduces development time and improves consistency
- **Separation of Concerns:** Divide the system into components based on distinct responsibilities, such as UI components, data access components, and business logic components

6.1.2 Component Communication Patterns

The AgencyOS component architecture implements multiple communication patterns to ensure efficient data flow and system

integration:

Interface-Based Communication:

Components communicate through well-defined interfaces that specify how they interact with each other. Interfaces define methods, events, or messages that components can use to send and receive information.

Asynchronous Messaging:

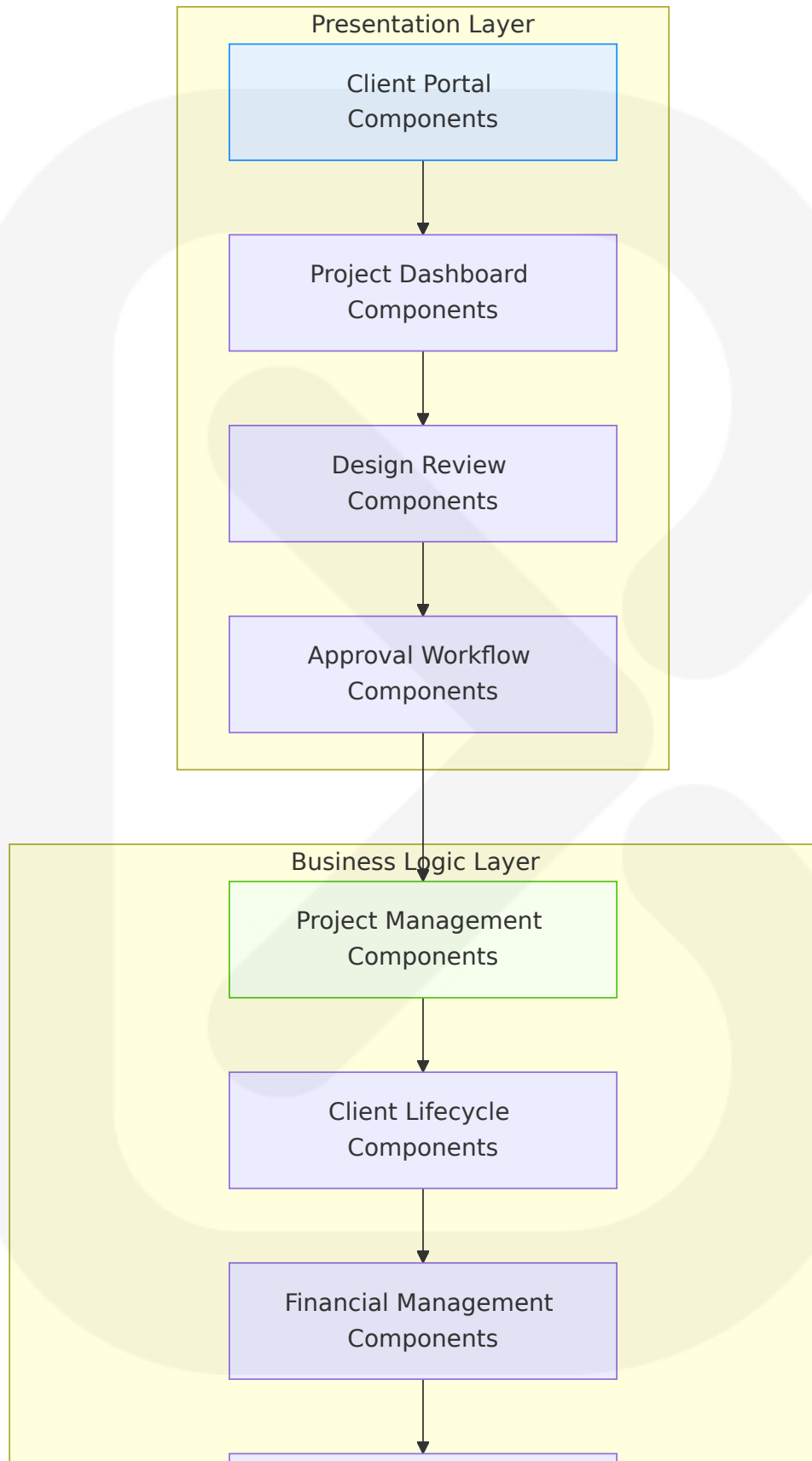
Components can communicate asynchronously using messaging systems such as message queues, publish-subscribe patterns, or event-driven architectures. This decouples components and allows them to operate independently.

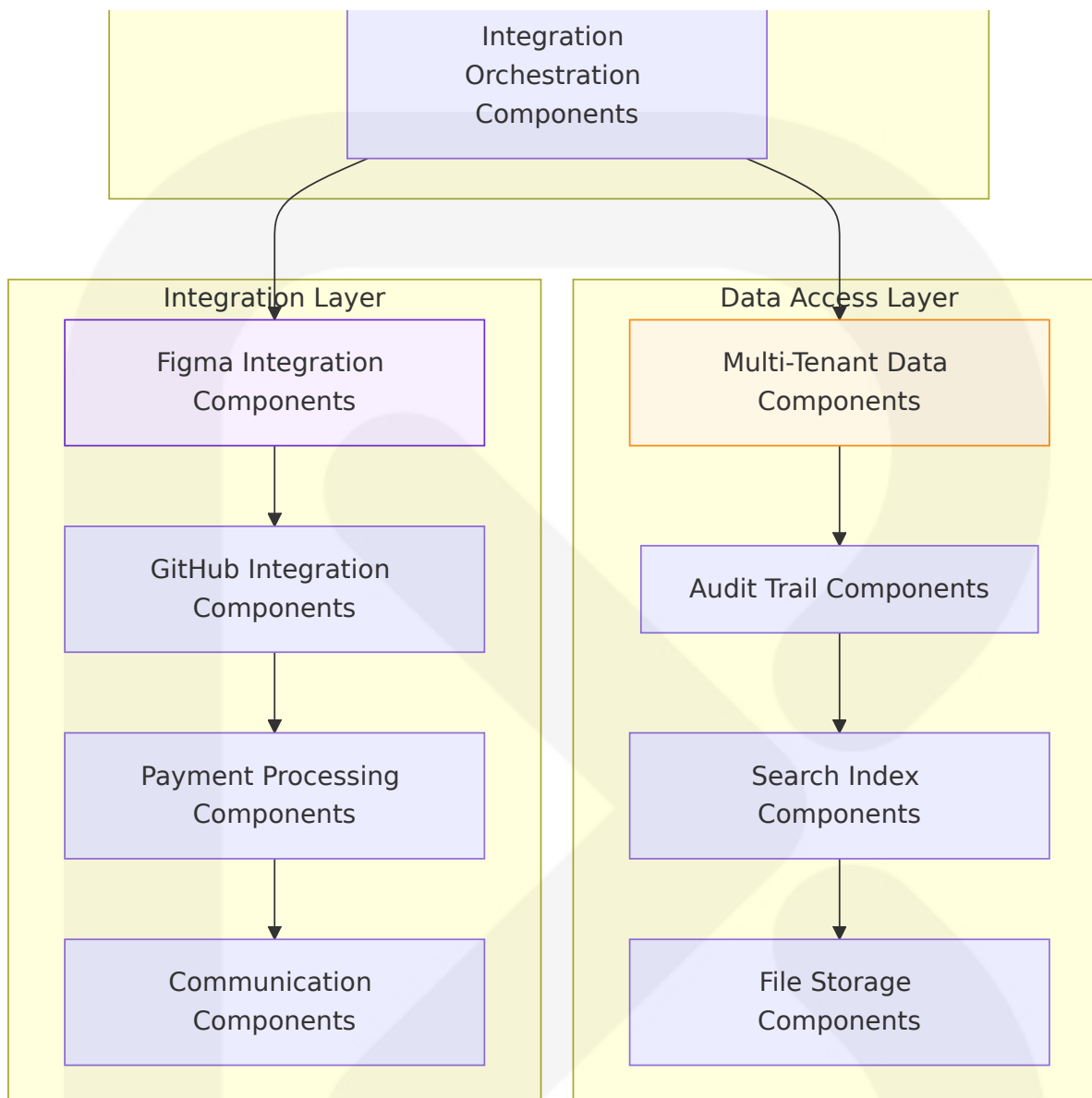
Remote Procedure Calls:

Components can invoke methods or functions on remote components through RPC mechanisms, enabling distributed system capabilities essential for multi-tenant operations.

6.1.3 Component Hierarchy and Organization

The system organizes components in a hierarchical structure that reflects business domains and technical responsibilities:





6.2 CORE SYSTEM COMPONENTS

6.2.1 Multi-Tenant Workspace Management Component

Component Purpose and Scope:

The Multi-Tenant Workspace Management Component serves as the foundational layer for AgencyOS's multi-tenant architecture, providing

complete data isolation and workspace customization capabilities for agency organizations.

Key Responsibilities:

- Tenant provisioning and configuration management
- Workspace-level branding and customization
- Cross-workspace user access coordination
- Tenant-specific feature flag management
- Resource allocation and usage monitoring

Technical Implementation:

Aspect	Implementation Details
Data Isolation	PostgreSQL Row-Level Security with dynamic tenant context setting
Workspace Storage	Tenant-prefixed S3 buckets with IAM policy enforcement
Configuration Management	JSON-based workspace settings with schema validation
User Context	JWT token embedding with tenant claims and project assignments

Component Interfaces:

```
interface WorkspaceManagementComponent {  
  // Workspace lifecycle management  
  createWorkspace(config: WorkspaceConfig): Promise<Workspace>  
  updateWorkspace(id: string, updates: Partial<WorkspaceConfig>):  
  Promise<Workspace>  
  deleteWorkspace(id: string): Promise<void>  
  
  // User access management  
  assignUserToWorkspace(userId: string, workspaceId: string, role:  
  WorkspaceRole): Promise<void>  
  revokeUserAccess(userId: string, workspaceId: string): Promise<void>  
  
  // Customization and branding
```

```
    updateBranding(workspaceId: string, branding: BrandingConfig):  
    Promise<void>  
    configureFeatures(workspaceId: string, features: FeatureFlags):  
    Promise<void>  
  }
```

Data Models:

```
interface WorkspaceConfig {  
  id: string  
  name: string  
  subdomain: string  
  branding: BrandingConfig  
  features: FeatureFlags  
  billing: BillingConfig  
  createdAt: Date  
  updatedAt: Date  
}  
  
interface BrandingConfig {  
  logo: string  
  primaryColor: string  
  secondaryColor: string  
  customDomain?: string  
  emailTemplates: EmailTemplateConfig[]  
}
```

Security Considerations:

- Tenant context validation on every database operation
- Encrypted storage of workspace configuration data
- Audit logging of all workspace management operations
- Rate limiting on workspace creation and modification

6.2.2 Role-Based Access Control (RBAC) Component

Component Purpose and Scope:

The RBAC Component implements comprehensive authorization and permission management across the multi-tenant AgencyOS platform, supporting both agency-internal and client-facing access patterns.

Key Responsibilities:

- User authentication and session management
- Role definition and permission assignment
- Project-level access control overrides
- SSO integration and MFA enforcement
- Permission evaluation and enforcement

Role Hierarchy and Permissions:

Role Category	Roles	Key Permissions
Agency Internal	Super Admin, Admin, PM, Designer, Engineer, QA, Finance, Contractor	Full workspace access, project management, resource allocation
Client Roles	Client Admin, Stakeholder/Reviewer, Billing Only, Viewer	Project visibility, approval workflows, invoice access
System Roles	System Admin, Audit Viewer	Cross-tenant administration, compliance reporting

Permission Matrix:

```
interface PermissionMatrix {  
  // Project management permissions  
  PROJECT_CREATE: RolePermission[]  
  PROJECT_EDIT: RolePermission[]  
  PROJECT_DELETE: RolePermission[]  
  PROJECT_VIEW: RolePermission[]  
  
  // Design review permissions  
  DESIGN_UPLOAD: RolePermission[]  
  DESIGN_REVIEW: RolePermission[]  
}
```

```
DESIGN_APPROVE: RolePermission[]

// Financial permissions
INVOICE_CREATE: RolePermission[]
INVOICE_VIEW: RolePermission[]
PAYMENT_PROCESS: RolePermission[]

// Administrative permissions
USER_MANAGE: RolePermission[]
WORKSPACE_CONFIGURE: RolePermission[]
AUDIT_VIEW: RolePermission[]
}
```

Component Interfaces:

```
interface RBACComponent {
  // Authentication
  authenticate(credentials: AuthCredentials): Promise<AuthResult>
  refreshToken(refreshToken: string): Promise<AuthResult>
  logout(sessionId: string): Promise<void>

  // Authorization
  checkPermission(userId: string, permission: Permission, context?:
PermissionContext): Promise<boolean>
  getUserPermissions(userId: string, projectId?: string):
Promise<Permission[]>

  // Role management
  assignRole(userId: string, role: Role, scope?: RoleScope):
Promise<void>
  revokeRole(userId: string, role: Role, scope?: RoleScope):
Promise<void>

  // SSO integration
  initiateSSOLogin(provider: SSOProvider): Promise<SSORedirect>
  completeSSOLogin(token: string): Promise<AuthResult>
}
```

Security Implementation:

- JWT tokens with short expiration and refresh rotation

- MFA enforcement for administrative roles
- Session management with concurrent session limits
- Audit logging of all authentication and authorization events

6.2.3 Project Management Core Component

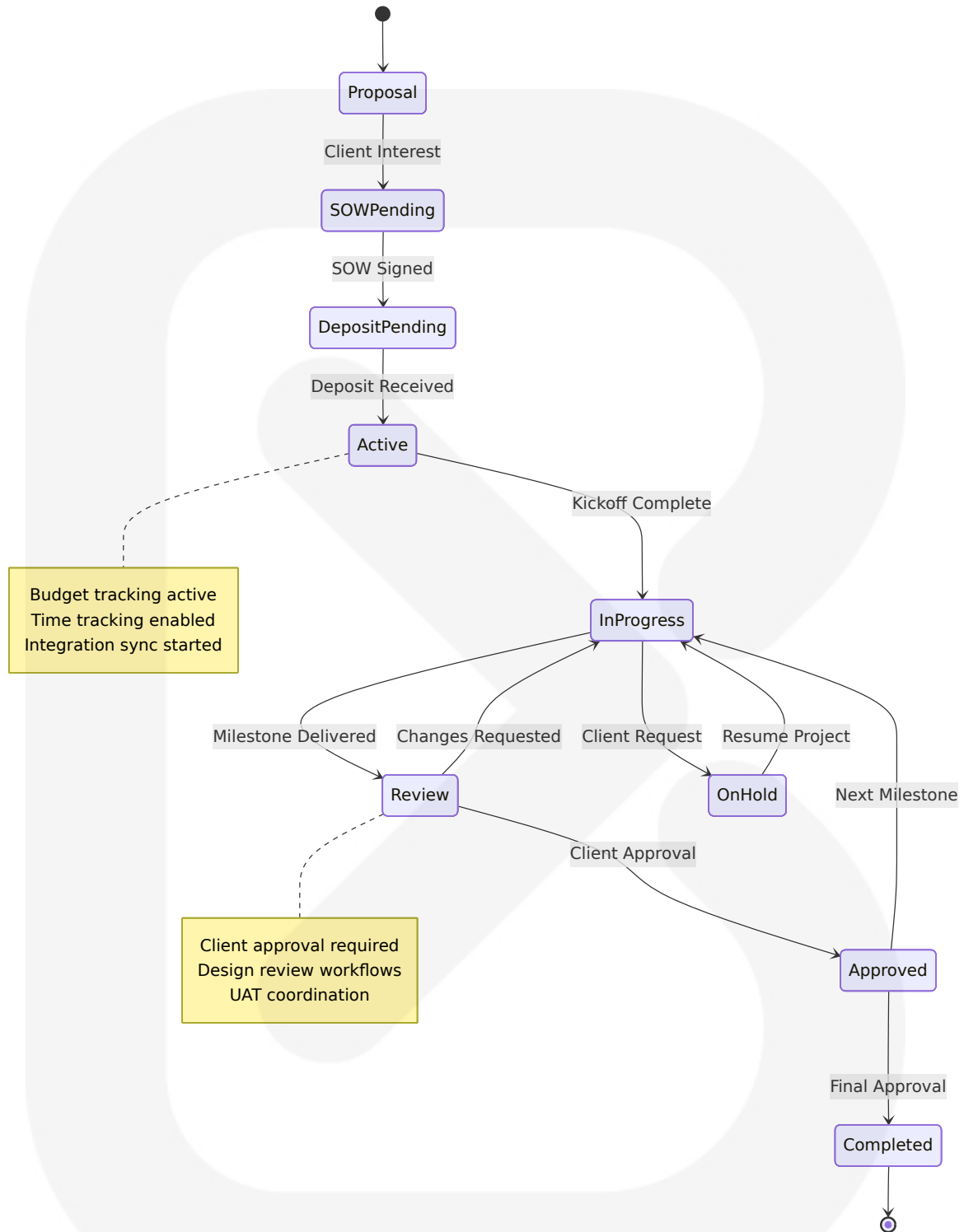
Component Purpose and Scope:

The Project Management Core Component orchestrates the complete project lifecycle from initial proposal through final delivery, integrating with external tools and maintaining comprehensive project state management.

Key Responsibilities:

- Project lifecycle orchestration and state management
- Task and sprint planning with external tool synchronization
- Budget tracking and resource allocation
- Milestone management and progress reporting
- Change request processing and approval workflows

Project State Machine:



Component Interfaces:

```

interface ProjectManagementComponent {
    // Project lifecycle

```

```

    createProject(proposal: ProposalData): Promise<Project>
    updateProjectStatus(projectId: string, status: ProjectStatus):
Promise<Project>
    completeProject(projectId: string, deliverables: Deliverable[]):
Promise<Project>

    // Task and sprint management
    createSprint(projectId: string, sprintData: SprintData):
Promise<Sprint>
    assignTask(taskId: string, assigneeId: string): Promise<Task>
    updateTaskStatus(taskId: string, status: TaskStatus): Promise<Task>

    // Budget and resource management
    trackTime(projectId: string, timeEntry: TimeEntry): Promise<void>
    updateBudget(projectId: string, budgetUpdate: BudgetUpdate):
Promise<Budget>
    generateUtilizationReport(projectId: string, period: DateRange):
Promise<UtilizationReport>

    // Change request management
    submitChangeRequest(projectId: string, changeRequest:
ChangeRequest): Promise<ChangeRequest>
    approveChangeRequest(changeRequestId: string, approval:
ChangeApproval): Promise<ChangeRequest>
  }

```

Integration Points:

External System	Integration Type	Data Synchronization
Jira/Linear	Bi-directional sync	Task status, assignments, comments
GitHub/GitLab	Webhook-based	PR status, deployment previews, commits
Time Tracking	Real-time updates	Time entries, utilization metrics
Financial Systems	Batch synchronization	Budget updates, invoice generation

Performance Considerations:

- Asynchronous processing for external system synchronization
- Caching of frequently accessed project data
- Background job processing for resource-intensive operations
- Real-time updates via WebSocket connections

6.2.4 Design Review and Approval Component

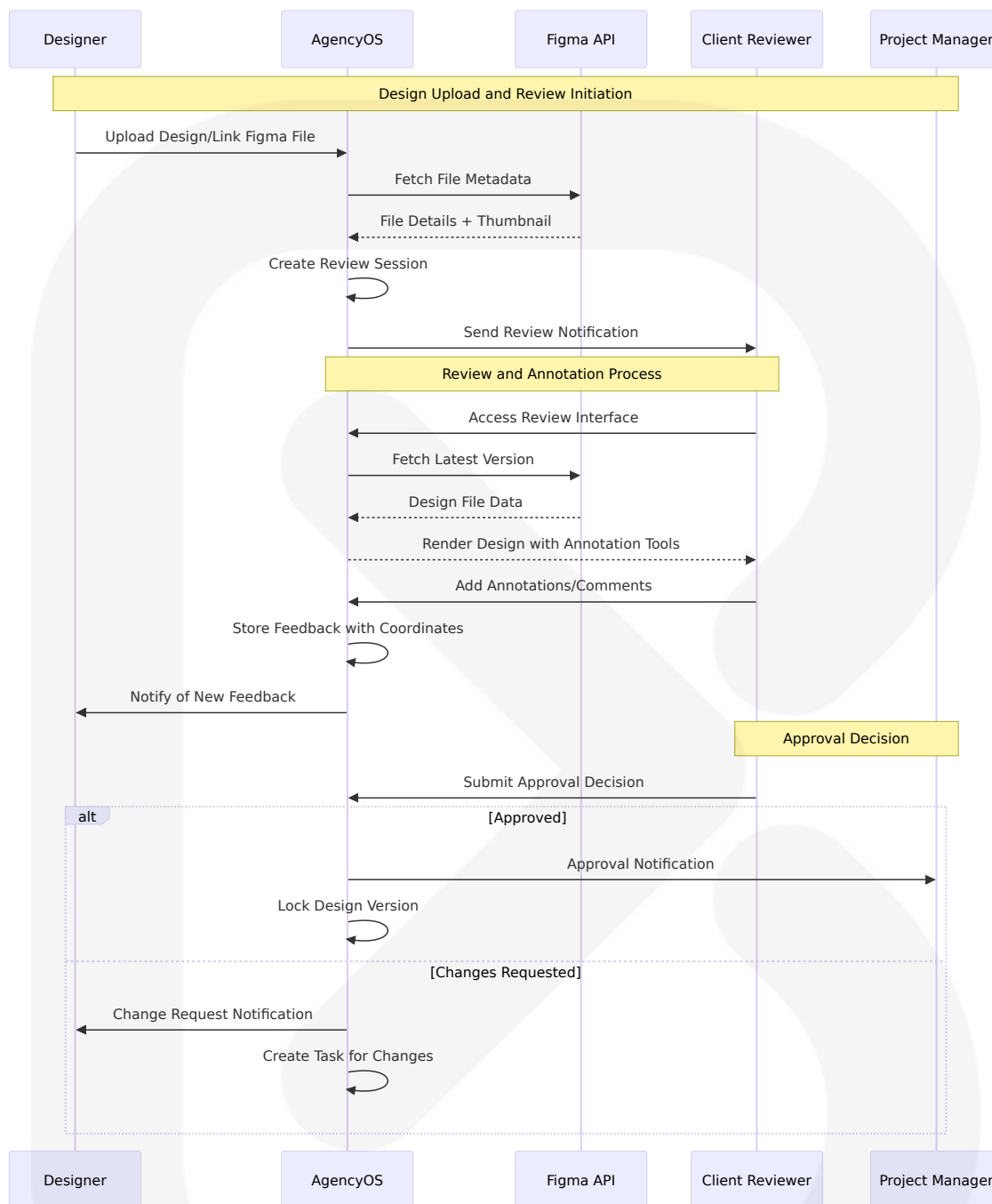
Component Purpose and Scope:

The Design Review and Approval Component facilitates collaborative design feedback and structured approval workflows, integrating natively with Figma and supporting multiple file formats for comprehensive design review processes.

Key Responsibilities:

- Figma integration with real-time design file synchronization
- Multi-format design review (PDF, images, video) with annotation tools
- Structured approval workflows with stakeholder management
- Version control and comparison capabilities
- Approval audit trail and compliance reporting

Design Review Workflow:



Component Interfaces:

```

interface DesignReviewComponent {
    // Design file management
    uploadDesignFile(projectId: string, file: DesignFile):
    Promise<DesignVersion>
    linkFigmaFile(projectId: string, figmaUrl: string):
    Promise<DesignVersion>
  
```

```
syncFigmaChanges(designVersionId: string): Promise<DesignVersion>

// Review session management
createReviewSession(designVersionId: string, reviewers: Reviewer[]):
Promise<ReviewSession>
  addAnnotation(sessionId: string, annotation: Annotation):
Promise<Annotation>
  submitReview(sessionId: string, decision: ReviewDecision):
Promise<ReviewSession>

// Version control
compareVersions(version1Id: string, version2Id: string):
Promise<VersionComparison>
rollbackToVersion(designId: string, versionId: string):
Promise<DesignVersion>

// Approval workflow
configureApprovalChain(projectId: string, approvers: ApprovalChain):
Promise<void>
  processApproval(approvalId: string, decision: ApprovalDecision):
Promise<Approval>
}
```

Figma Integration Architecture:

```
interface FigmaIntegration {
  // File operations
  getFileMetadata(fileKey: string): Promise<FigmaFileMetadata>
  getFileImages(fileKey: string, options: ImageOptions):
Promise<FigmaImages>
  getFileVersions(fileKey: string): Promise<FigmaVersion[]>

  // Comment operations
  getComments(fileKey: string): Promise<FigmaComment[]>
  postComment(fileKey: string, comment: CommentData):
Promise<FigmaComment>

  // Webhook handling
  handleWebhook(payload: FigmaWebhookPayload): Promise<void>
  subscribeToUpdates(fileKey: string): Promise<WebhookSubscription>
}
```

Annotation System:

```
interface AnnotationSystem {  
    // Annotation types  
    createTextAnnotation(coordinates: Point, text: string, author:  
User): Promise<TextAnnotation>  
    createDrawingAnnotation(path: DrawingPath, author: User):  
Promise<DrawingAnnotation>  
    createVoiceAnnotation(coordinates: Point, audioFile: File, author:  
User): Promise<VoiceAnnotation>  
  
    // Annotation management  
    resolveAnnotation(annotationId: string): Promise<void>  
    replyToAnnotation(annotationId: string, reply: string):  
Promise<AnnotationReply>  
    exportAnnotations(sessionId: string, format: ExportFormat):  
Promise<AnnotationExport>  
}
```

Security and Access Control:

- Watermarked shareable links with expiration controls
- Project-level access restrictions for design files
- Audit trail of all design access and modifications
- Encrypted storage of sensitive design assets

6.2.5 Client Portal and Communication Component

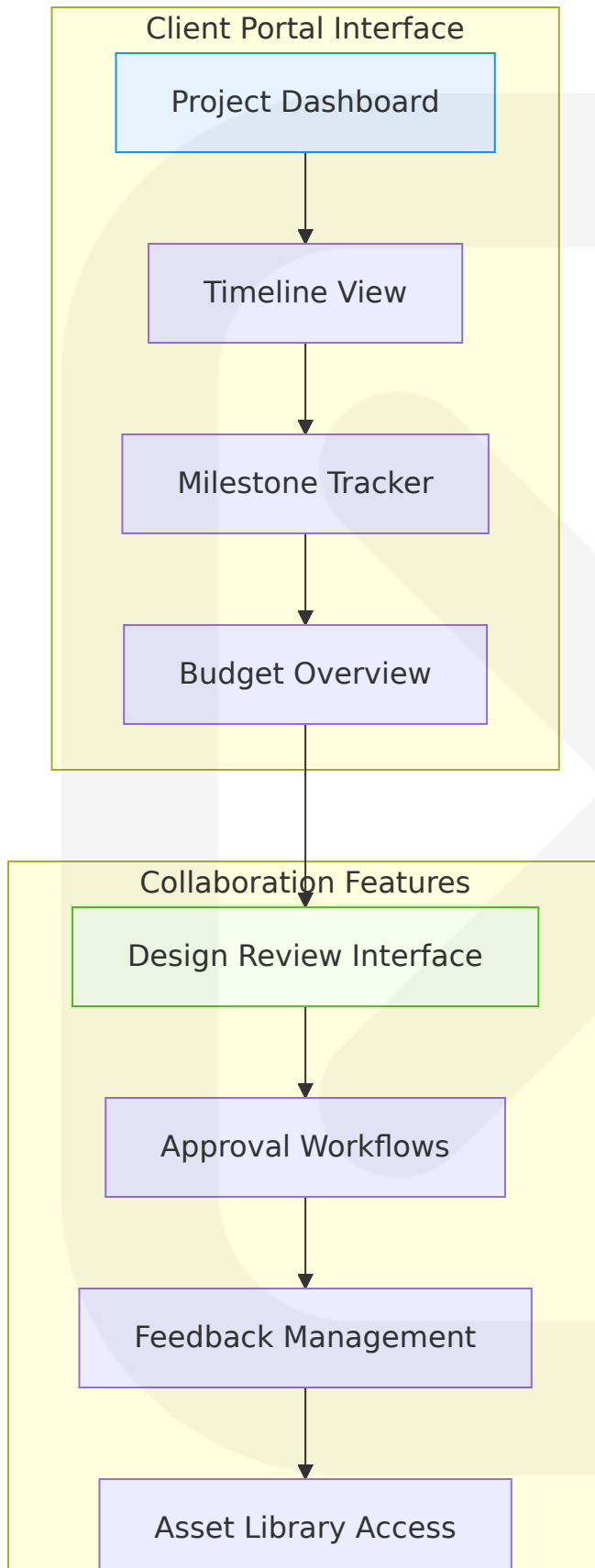
Component Purpose and Scope:

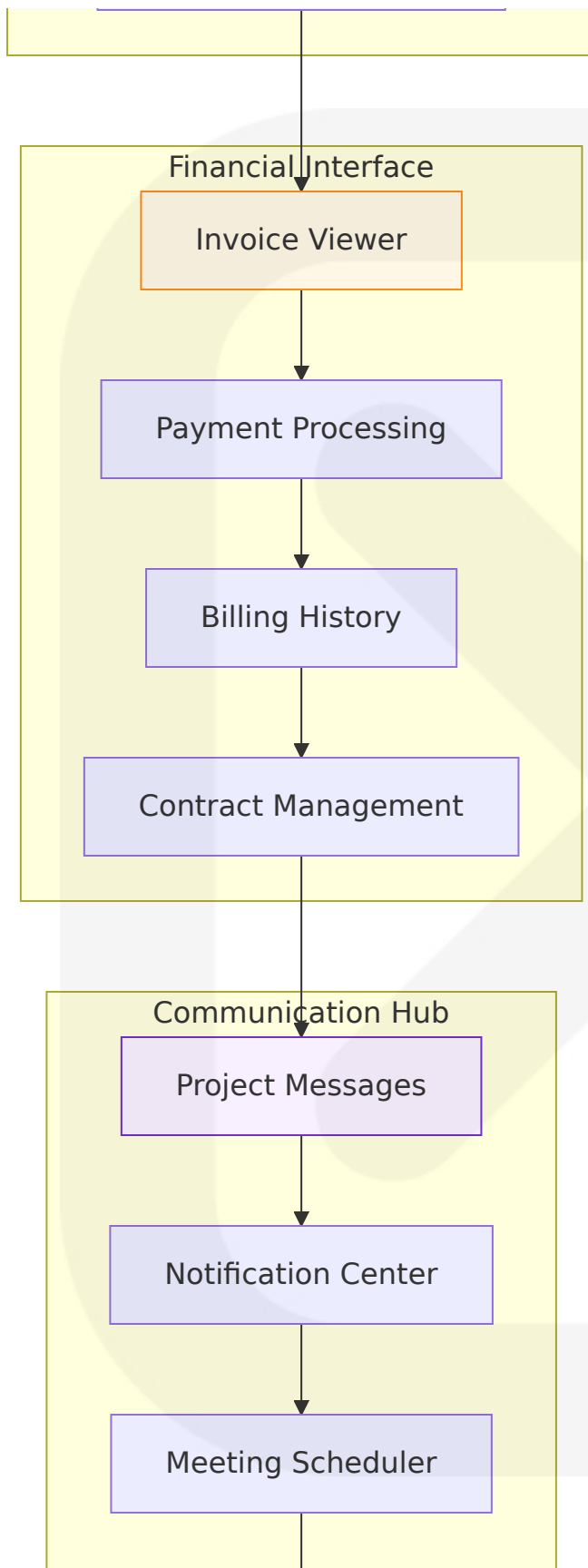
The Client Portal and Communication Component provides a comprehensive client-facing interface for project visibility, collaboration, and communication, while maintaining clear boundaries between client and agency operational data.

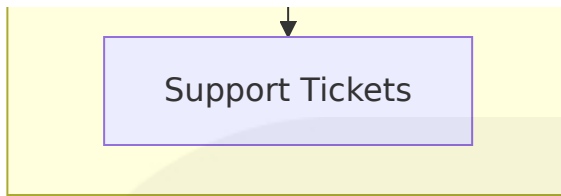
Key Responsibilities:

- Client-specific project dashboard and status reporting
- Document and asset sharing with access controls
- Invoice presentation and payment processing
- Communication hub for project-related discussions
- Mobile-responsive interface for client accessibility

Client Portal Architecture:







Component Interfaces:

```

interface ClientPortalComponent {
  // Dashboard and reporting
  getProjectDashboard(clientId: string, projectId: string):
  Promise<ClientDashboard>
  getProjectTimeline(projectId: string): Promise<ProjectTimeline>
  getMilestoneStatus(projectId: string): Promise<MilestoneStatus[]>

  // Document and asset access
  getSharedDocuments(projectId: string): Promise<SharedDocument[]>
  downloadAsset(assetId: string, clientId: string):
  Promise<AssetDownload>
  requestAssetAccess(assetId: string, justification: string):
  Promise<AccessRequest>

  // Communication features
  sendMessage(projectId: string, message: ClientMessage):
  Promise<Message>
  getMessageHistory(projectId: string): Promise<Message[]>
  scheduleClientMeeting(projectId: string, meeting: MeetingRequest):
  Promise<Meeting>

  // Approval workflows
  getPendingApprovals(clientId: string): Promise<PendingApproval[]>
  submitApproval(approvalId: string, decision: ApprovalDecision):
  Promise<Approval>
}
  
```

Client Data Model:

```

interface ClientDashboard {
  project: {
    id: string
    name: string
    status: ProjectStatus
  }
}
  
```

```
    progress: number
    nextMilestone: Milestone
  }
  timeline: {
    milestones: MilestoneStatus[]
    upcomingDeadlines: Deadline[]
    recentActivity: Activity[]
  }
  budget: {
    totalBudget: number
    spentAmount: number
    remainingBudget: number
    invoiceStatus: InvoiceStatus
  }
  approvals: {
    pending: PendingApproval[]
    completed: CompletedApproval[]
    overdue: OverdueApproval[]
  }
}
```

Mobile Responsiveness:

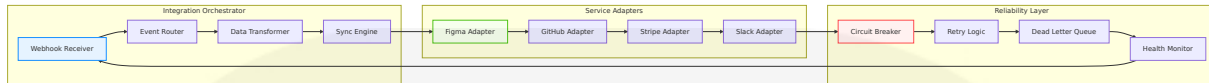
- Progressive Web App (PWA) capabilities for offline access
- Touch-optimized interfaces for design review and approval
- Push notifications for critical project updates
- Responsive design patterns for various screen sizes

6.3 INTEGRATION COMPONENTS

6.3.1 External Service Integration Architecture

The integration layer implements a standardized approach to external service connectivity, ensuring reliable data synchronization and fault tolerance across all third-party integrations.

Integration Patterns:



6.3.2 Figma Integration Component

Component Purpose and Scope:

The Figma Integration Component provides seamless connectivity with Figma's design platform, enabling real-time design file synchronization, collaborative review workflows, and automated design asset management.

Key Integration Features:

- Real-time design file synchronization with webhook support
- Automated image generation and optimization for web display
- Comment synchronization between AgencyOS and Figma
- Version tracking and change detection
- Team and project access management

Technical Implementation:

```

interface FigmaIntegrationComponent {
  // File synchronization
  syncDesignFile(fileKey: string): Promise<SyncResult>
  generatePreviewImages(fileKey: string, options: PreviewOptions):
  Promise<PreviewImage[]>
  detectFileChanges(fileKey: string, lastSyncTime: Date):
  Promise<ChangeDetection>

  // Comment integration
  syncComments(fileKey: string): Promise<CommentSync>
  postCommentToFigma(fileKey: string, comment: AgencyOSComment):
  Promise<FigmaComment>

  // Access management
  validateFileAccess(fileKey: string, userId: string):
  Promise<AccessValidation>

```

```
    getTeamProjects(teamId: string): Promise<FigmaProject[]>
  }
```

Webhook Processing:

```
interface FigmaWebhookHandler {
  handleFileUpdate(payload: FigmaFileUpdatePayload): Promise<void>
  handleCommentUpdate(payload: FigmaCommentPayload): Promise<void>
  handleVersionUpdate(payload: FigmaVersionPayload): Promise<void>

  // Webhook validation
  validateWebhookSignature(payload: string, signature: string):
  boolean
  processWebhookQueue(): Promise<ProcessingResult>
}
```

6.3.3 GitHub/GitLab Integration Component

Component Purpose and Scope:

The GitHub/GitLab Integration Component connects development workflows with project management, providing automated deployment preview capture, issue synchronization, and development progress tracking.

Key Integration Features:

- Repository webhook processing for PR and deployment events
- Automated preview URL capture and client notification
- Issue and PR synchronization with project tasks
- Commit tracking and developer activity monitoring
- Branch and release management integration

Component Interfaces:

```
interface GitIntegrationComponent {
  // Repository management
  connectRepository(projectId: string, repoConfig: RepositoryConfig):
```

```

Promise<RepoConnection>
  syncRepositoryData(repoId: string): Promise<SyncResult>

  // Pull request integration
  handlePullRequestEvent(webhook: PRWebhookPayload): Promise<void>
  capturePreviewURL(prId: string): Promise<PreviewCapture>
  notifyClientOfPreview(projectId: string, previewUrl: string):
Promise<void>

  // Issue synchronization
  syncIssuesWithTasks(repoId: string): Promise<IssueSyncResult>
  createIssueFromTask(taskId: string): Promise<GitIssue>
  updateTaskFromIssue(issueId: string): Promise<Task>

  // Deployment tracking
  trackDeploymentStatus(deploymentId: string):
Promise<DeploymentStatus>
  generateDeploymentReport(projectId: string, period: DateRange):
Promise<DeploymentReport>
}

```

Preview Capture System:

```

interface PreviewCaptureSystem {
  // Automated preview detection
  detectPreviewURLs(prData: PullRequestData): Promise<PreviewURL[]>
  validatePreviewAccess(url: string): Promise<AccessValidation>
  captureScreenshot(url: string, options: ScreenshotOptions):
Promise<Screenshot>

  // Client notification
  generatePreviewNotification(preview: PreviewData):
Promise<ClientNotification>
  trackPreviewEngagement(previewId: string):
Promise<EngagementMetrics>
}

```

6.3.4 Payment Processing Integration Component

Component Purpose and Scope:

The Payment Processing Integration Component handles all financial transactions, invoice generation, and accounting system synchronization, ensuring secure and compliant payment processing workflows.

Key Integration Features:

- Stripe payment processing with webhook handling
- Automated invoice generation and delivery
- QuickBooks/Xero accounting synchronization
- Subscription and recurring payment management
- Payment failure handling and retry logic

Component Architecture:

```
interface PaymentProcessingComponent {  
    // Payment processing  
    processPayment(paymentData: PaymentRequest): Promise<PaymentResult>  
    handlePaymentWebhook(webhook: StripeWebhookPayload): Promise<void>  
    refundPayment(paymentId: string, amount?: number):  
    Promise<RefundResult>  
  
    // Invoice management  
    generateInvoice(projectId: string, milestoneId: string):  
    Promise<Invoice>  
    sendInvoiceToClient(invoiceId: string): Promise<DeliveryResult>  
    processInvoicePayment(invoiceId: string, paymentData: PaymentData):  
    Promise<PaymentResult>  
  
    // Accounting integration  
    syncWithQuickBooks(transactionData: TransactionData[]):  
    Promise<SyncResult>  
    syncWithXero(transactionData: TransactionData[]):  
    Promise<SyncResult>  
    generateFinancialReport(period: DateRange): Promise<FinancialReport>  
  
    // Subscription management  
    createSubscription(customerId: string, planId: string):  
    Promise<Subscription>  
    updateSubscription(subscriptionId: string, updates:
```



```
SubscriptionUpdate): Promise<Subscription>
  cancelSubscription(subscriptionId: string):
  Promise<CancellationResult>
}
```

Security Implementation:

- PCI DSS compliance for payment data handling
- Webhook signature validation for all payment events
- Encrypted storage of sensitive financial information
- Audit logging of all financial transactions

6.4 DATA MANAGEMENT COMPONENTS

6.4.1 Multi-Tenant Data Access Component

Component Purpose and Scope:

The Multi-Tenant Data Access Component implements secure, scalable data access patterns with complete tenant isolation using PostgreSQL's Row-Level Security features and optimized query performance.

Key Responsibilities:

- Tenant context management and enforcement
- Database connection pooling and optimization
- Query performance monitoring and optimization
- Data migration and schema management
- Backup and recovery coordination

Technical Implementation:

```
interface MultiTenantDataComponent {
  // Tenant context management
  setTenantContext(tenantId: string): Promise<void>
}
```

```
    getCurrentTenant(): Promise<TenantContext>
    validateTenantAccess(userId: string, tenantId: string):
    Promise<boolean>

    // Data access operations
    executeQuery<T>(query: QueryBuilder, params: QueryParams):
    Promise<T[]>
    executeTransaction<T>(operations: TransactionOperation[]):
    Promise<T>

    // Schema management
    applyMigration(migration: Migration): Promise<MigrationResult>
    validateSchema(tenantId: string): Promise<SchemaValidation>

    // Performance optimization
    analyzeQueryPerformance(query: string): Promise<QueryAnalysis>
    optimizeIndexes(tableNames: string[]): Promise<IndexOptimization>
}
```

Row-Level Security Implementation:

```
-- Example RLS policy for project data
CREATE POLICY tenant_isolation_policy ON projects
    FOR ALL
    TO application_role
    USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Dynamic tenant context setting
SET LOCAL app.current_tenant = '550e8400-e29b-41d4-a716-446655440000';
```

6.4.2 Audit Trail and Compliance Component

Component Purpose and Scope:

The Audit Trail and Compliance Component maintains comprehensive audit logs for all system operations, ensuring regulatory compliance and providing detailed activity tracking for security and business intelligence purposes.

Key Responsibilities:

- Immutable audit log creation and storage
- Compliance reporting and data retention management
- Security event detection and alerting
- Data lineage tracking and impact analysis
- Regulatory compliance validation (SOC 2, GDPR)

Component Interfaces:

```
interface AuditTrailComponent {  
    // Audit logging  
    logUserAction(action: UserAction): Promise<AuditEntry>  
    logSystemEvent(event: SystemEvent): Promise<AuditEntry>  
    logDataChange(change: DataChangeEvent): Promise<AuditEntry>  
  
    // Compliance reporting  
    generateComplianceReport(type: ComplianceType, period: DateRange):  
    Promise<ComplianceReport>  
    validateDataRetention(): Promise<RetentionValidation>  
    processDataDeletionRequest(request: DeletionRequest):  
    Promise<DeletionResult>  
  
    // Security monitoring  
    detectAnomalousActivity(userId: string, timeWindow: TimeWindow):  
    Promise<AnomalyDetection>  
    generateSecurityAlert(event: SecurityEvent): Promise<SecurityAlert>  
  
    // Data lineage  
    trackDataLineage(dataId: string): Promise<DataLineage>  
    analyzeDataImpact(changeId: string): Promise<ImpactAnalysis>  
}
```

Audit Data Model:

```
interface AuditEntry {  
    id: string  
    timestamp: Date  
    tenantId: string  
    userId: string  
    action: AuditAction  
    resourceType: string
```

```
resourceId: string
changes: ChangeRecord[]
metadata: AuditMetadata
ipAddress: string
userAgent: string
sessionId: string
}

interface ChangeRecord {
  field: string
  oldValue: any
  newValue: any
  changeType: 'CREATE' | 'UPDATE' | 'DELETE'
}
```

6.4.3 Search and Analytics Component

Component Purpose and Scope:

The Search and Analytics Component provides comprehensive search capabilities across all project data, assets, and knowledge base content, while generating business intelligence insights for agency operations.

Key Responsibilities:

- Full-text search across projects, assets, and documents
- Real-time search index maintenance and optimization
- Business analytics and reporting dashboard data
- Performance metrics collection and analysis
- Predictive analytics for project risk assessment

Search Architecture:

```
interface SearchAnalyticsComponent {
  // Search operations
  searchProjects(query: SearchQuery, filters: SearchFilters):
  Promise<SearchResults<Project>>
  searchAssets(query: SearchQuery, filters: AssetFilters):
  Promise<SearchResults<Asset>>
```

```

    searchKnowledgeBase(query: SearchQuery):
    Promise<SearchResults<KnowledgeArticle>>

    // Index management
    reindexTenant(tenantId: string): Promise<IndexingResult>
    optimizeSearchIndex(): Promise<OptimizationResult>

    // Analytics and reporting
    generateProjectAnalytics(projectId: string):
    Promise<ProjectAnalytics>
    generateUtilizationReport(period: DateRange):
    Promise<UtilizationReport>
    generateRevenueAnalytics(period: DateRange):
    Promise<RevenueAnalytics>

    // Predictive analytics
    assessProjectRisk(projectId: string): Promise<RiskAssessment>
    predictResourceNeeds(period: DateRange): Promise<ResourcePrediction>
}

```

Analytics Data Models:

```

interface ProjectAnalytics {
  projectId: string
  metrics: {
    budgetUtilization: number
    timelineAdherence: number
    scopeCreepIndex: number
    clientSatisfactionScore: number
  }
  trends: {
    velocityTrend: TrendData[]
    budgetTrend: TrendData[]
    qualityTrend: TrendData[]
  }
  predictions: {
    completionDate: Date
    finalBudget: number
    riskFactors: RiskFactor[]
  }
}

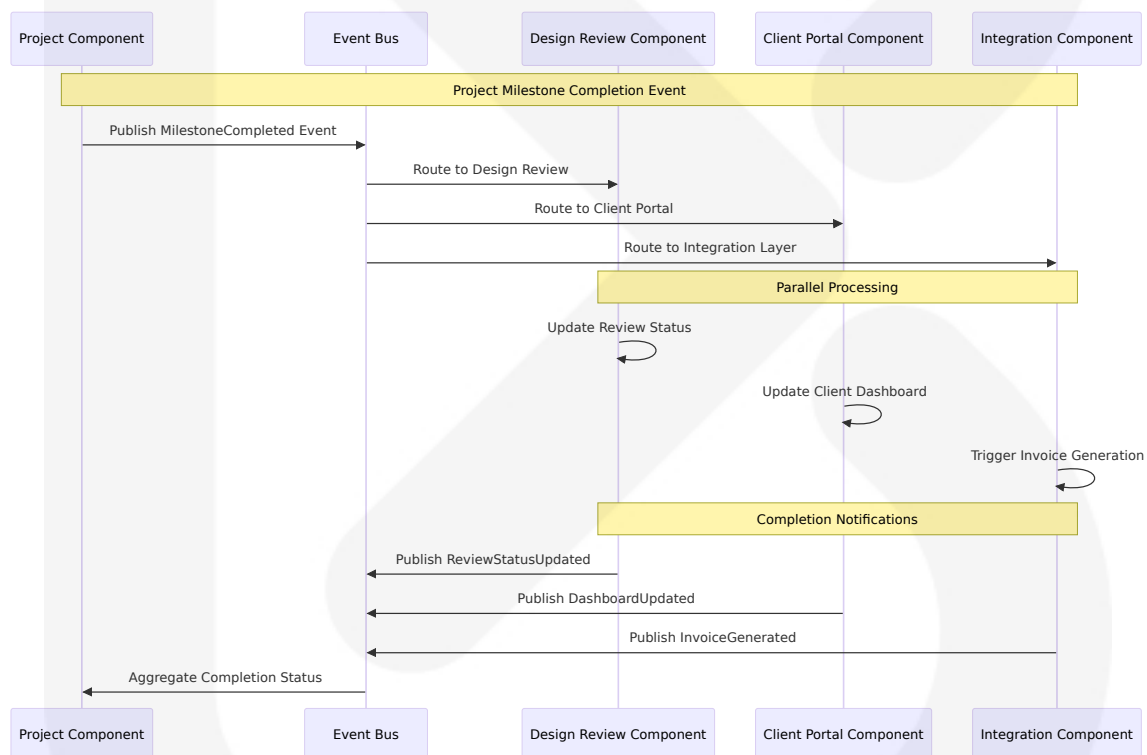
```

6.5 COMPONENT INTERACTION PATTERNS

6.5.1 Event-Driven Communication

The system implements event-driven architecture where the flow of a system is driven by events, allowing components to communicate by producing and consuming events, acting independently and asynchronously.

Event Bus Architecture:



6.5.2 Component Dependency Management

Dependency Injection Pattern:

```
interface ComponentContainer {
  // Core components
  workspaceManager: WorkspaceManagementComponent
  rbacManager: RBACComponent
  projectManager: ProjectManagementComponent

  // Integration components
  figmaIntegration: FigmaIntegrationComponent
  gitIntegration: GitIntegrationComponent
  paymentProcessor: PaymentProcessingComponent

  // Data components
  dataAccess: MultiTenantDataComponent
  auditTrail: AuditTrailComponent
  searchAnalytics: SearchAnalyticsComponent
}

// Component initialization with dependency injection
class ComponentFactory {
  static createProjectManager(dependencies: {
    dataAccess: MultiTenantDataComponent
    auditTrail: AuditTrailComponent
    eventBus: EventBus
  }): ProjectManagementComponent {
    return new ProjectManagementComponent(dependencies)
  }
}
```

6.5.3 Error Handling and Resilience Patterns

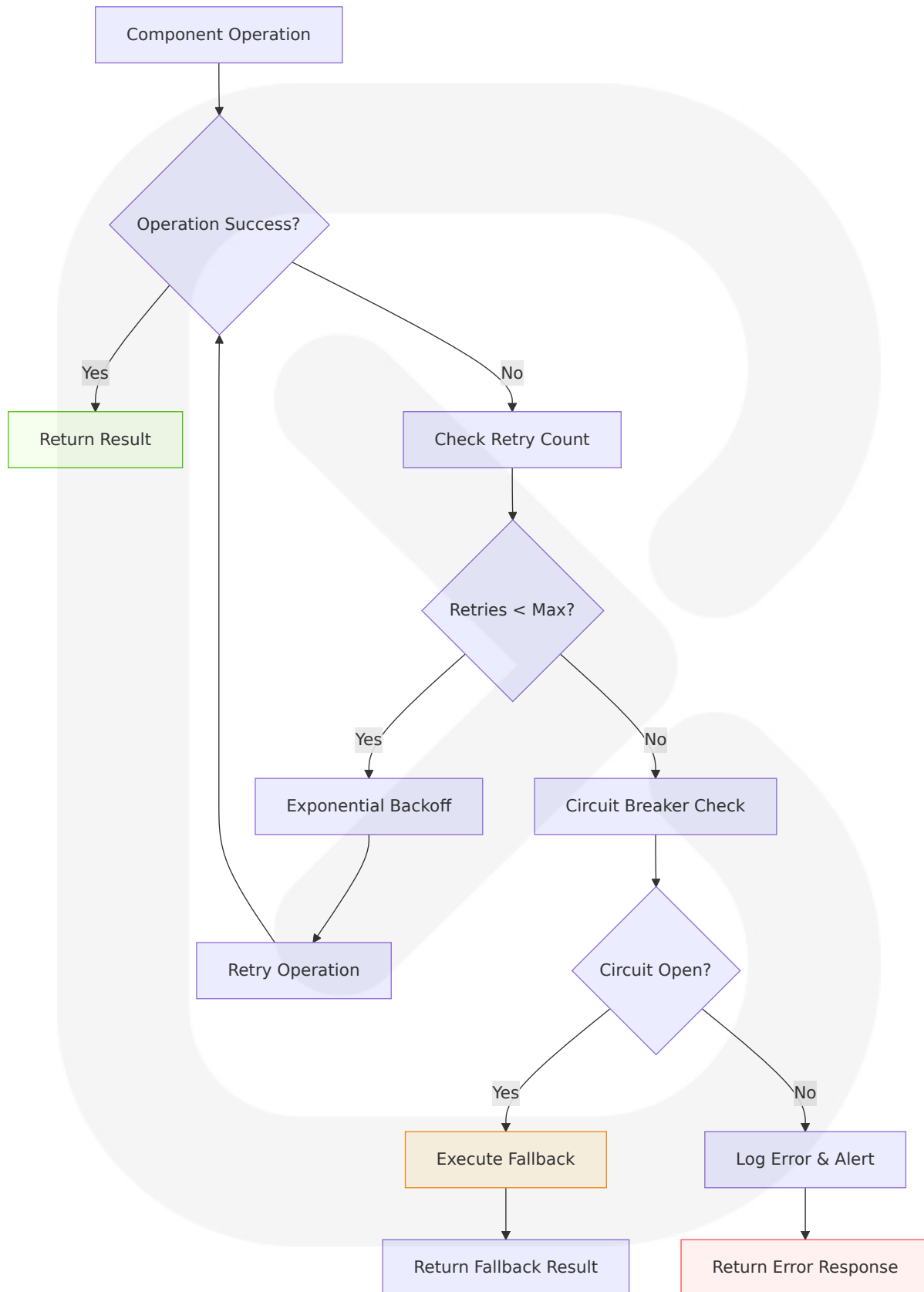
Circuit Breaker Implementation:

```
interface CircuitBreakerComponent {
  // Circuit breaker states
  executeWithCircuitBreaker<T>(
    operation: () => Promise<T>,
    fallback: () => Promise<T>
  ): Promise<T>

  // Health monitoring
```

```
monitorComponentHealth(componentId: string): Promise<HealthStatus>  
  
  // Failure recovery  
  handleComponentFailure(componentId: string, error: ComponentError):  
  Promise<RecoveryAction>  
}
```

Retry and Fallback Patterns:



This comprehensive component design ensures reusability, modularity, scalability, flexibility, collaboration, and maintenance benefits, with components being isolated for easier debugging and reduced risk of system-wide failures, while maintaining the architectural principles essential for AgencyOS's multi-tenant, integration-heavy environment.

6.1 CORE SERVICES ARCHITECTURE

Based on the comprehensive analysis of AgencyOS requirements and the current technology landscape, **Core Services Architecture is not applicable for this system** in the traditional microservices sense. Instead, AgencyOS employs a **hybrid monolithic-microservices architecture** that provides the benefits of both approaches while avoiding the complexity overhead of a full distributed system.

6.1.1 Architectural Decision Rationale

AgencyOS adopts a hybrid approach for several strategic reasons:

Why Not Pure Microservices:

- The digital agency landscape requires rapid feature development and iteration, where attempting a complete migration to microservices can be daunting or impractical
- Managing inter-service communication and deployment pipelines adds unnecessary complexity for projects at this stage
- As Martin Fowler points out, "almost all the successful microservice stories have started with a monolith that got too big and was broken up"

Why Hybrid Architecture:

- By strategically utilizing microservices and monoliths, organizations can strike a balance between flexibility, scalability, and maintaining

existing functionality

- By isolating specific components into standalone applications, we combine the strengths of modularity and scalability without overhauling the entire system
- A system may have a monolithic core that handles core operations and microservices developed for peripheral features for maximum flexibility and scalability

6.1.2 SERVICE COMPONENTS

6.1.2.1 Core Monolithic Services

The AgencyOS core operates as a **modular monolith** with clear service boundaries that can be extracted as microservices when scaling demands require it.

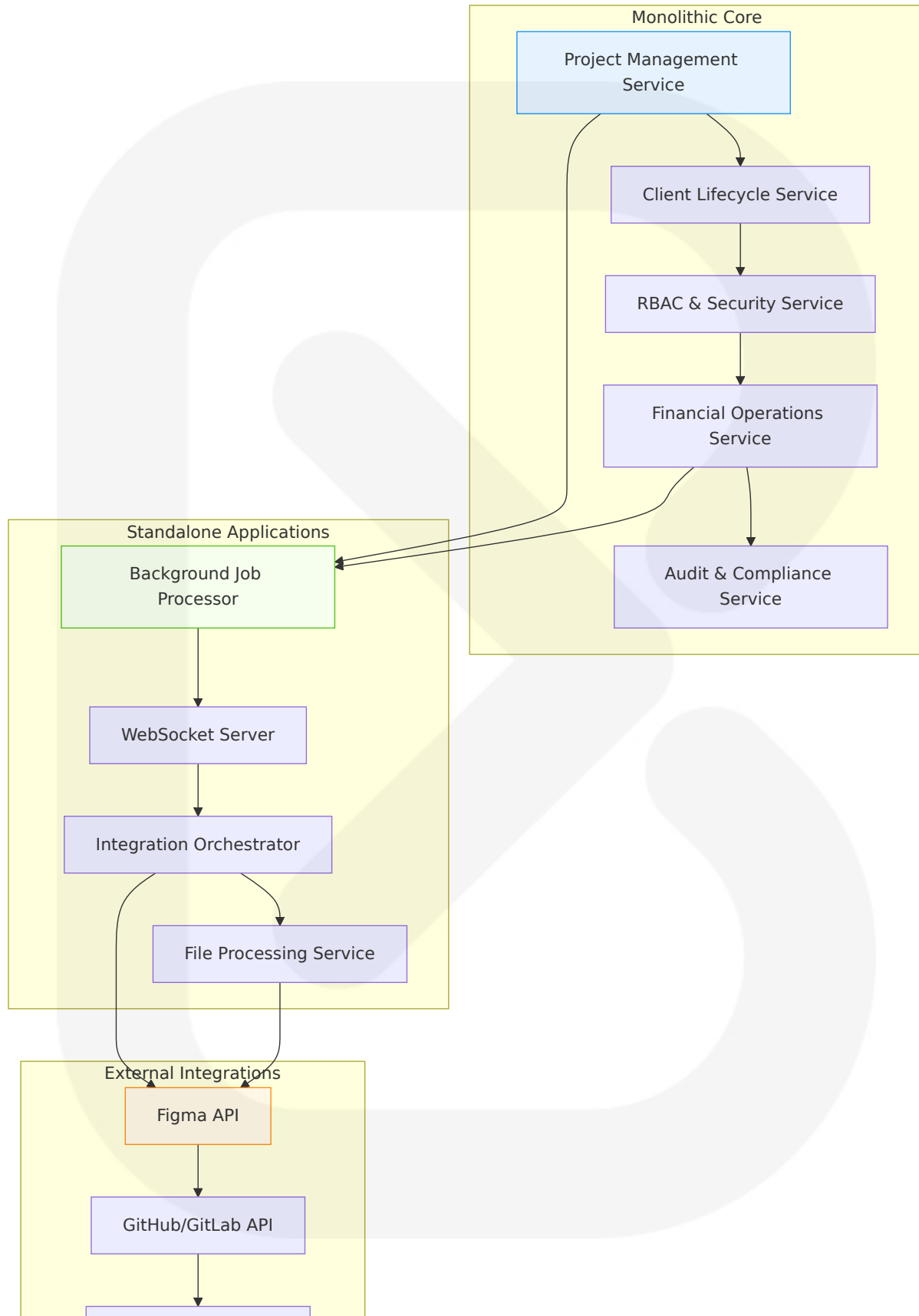
Service Domain	Responsibilities	Scaling Strategy	Future Extraction Readiness
Core Business Logic	Project management, client lifecycle, RBAC, audit trails	Vertical scaling with horizontal read replicas	High - clear domain boundaries
Design Review Engine	Figma integration, annotation system, approval workflows	Horizontal scaling via standalone apps	Medium - integration dependencies
Financial Operations	Invoicing, payment processing, accounting sync	Vertical scaling with queue-based processing	High - transactional boundaries

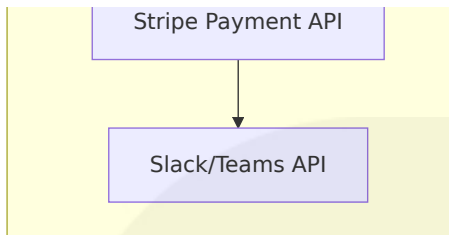
6.1.2.2 Standalone Application Components

Standalone applications in NestJS allow you to create modular, isolated services without a full application context, enabling lightweight scripts or specialized services without introducing the overhead of a complete microservices setup

Standalone Component	Purpose	Communication Pattern	Scaling Approach
Background Job Processor	Email notifications, file processing, integration sync	Redis pub/sub with BullMQ	Independent horizontal scaling
Real-time WebSocket Server	Live design reviews, project updates, chat	Socket.io with Redis adapter	Horizontal scaling with session affinity
Integration Orchestrator	Third-party API management, webhook processing	Event-driven with circuit breakers	Auto-scaling based on API load

6.1.2.3 Service Boundaries and Responsibilities





6.1.2.4 Inter-Service Communication Patterns

Within Monolithic Core:

- Direct method calls with dependency injection
- Shared database transactions for consistency
- Event-driven internal messaging for loose coupling

Between Core and Standalone Apps:

- Redis database publisher that announces new requests received by all other instances and a subscriber that listens to these requests
- BullMQ job queues for asynchronous processing
- RESTful API calls for synchronous operations

External Integration Communication:

- Communication between services should not be direct but should happen through APIs or message brokers like RabbitMQ or Kafka
- Circuit breaker patterns for fault tolerance
- Webhook receivers with signature validation

6.1.2.5 Service Discovery Mechanisms

AgencyOS implements a **simplified service discovery** approach suitable for its hybrid architecture:

Discovery Method	Use Case	Implementation
Environment-Based Configuration	Standalone app coordination	APP_ROLE environment variable for dynamic role switching

Discovery Method	Use Case	Implementation
Redis Service Registry	WebSocket server instances	Instance registration with health checks
DNS-Based Discovery	External API endpoints	Kubernetes service discovery for containerized deployments

6.1.3 SCALABILITY DESIGN

6.1.3.1 Horizontal/Vertical Scaling Approach

Monolithic Core Scaling:

- Improve performance by reducing the load on any single instance and allowing you to scale horizontally by adding more instances as needed
- Clustering allows you to run multiple instances of your application on a single machine, creating multiple worker processes to take advantage of the full capacity and handle more requests simultaneously

Standalone Application Scaling:

- Using a single codebase simplified development and deployment, while enabling the standalone app to scale independently when needed
- To scale your app, you will need to scale horizontally by creating multiple instances of your NestJS app

6.1.3.2 Auto-Scaling Triggers and Rules

Component	Scaling Trigger	Scaling Rule	Maximum Instances
Core Application	CPU > 70% for 5 minutes	Add 1 instance, max 10	10
WebSocket Server	Active connections > 1000 per instance	Add 1 instance, max 20	20

Component	Scaling Trigger	Scaling Rule	Maximum Instances
Background Processor	Queue depth > 100 jobs	Add 1 instance, max 15	15
Integration Service	API error rate > 5%	Add 1 instance, circuit breaker activation	8

6.1.3.3 Resource Allocation Strategy

Memory Management:

- Techniques for efficient memory management, such as using streams for handling large volumes of data, can prevent memory bottlenecks and improve the overall speed of your application

CPU Allocation:

- Core application: 2-4 CPU cores per instance
- WebSocket server: 1-2 CPU cores per instance (I/O intensive)
- Background processor: 2-8 CPU cores per instance (CPU intensive)

Database Connection Pooling:

- NestJS framework for building efficient, scalable Node.js server-side applications uses progressive JavaScript and combines elements of OOP, FP, and FRP
- Connection pool size: 10-20 connections per application instance
- Read replica routing for reporting queries

6.1.3.4 Performance Optimization Techniques



6.1.4 RESILIENCE PATTERNS

6.1.4.1 Fault Tolerance Mechanisms

Circuit Breaker Implementation:

- Implementation of microservices architecture using NestJS and popular patterns like CQRS, Saga, Event Bus, and Circuit Breaker to build robust, scalable, and resilient systems

Integration Point	Failure Threshold	Recovery Time	Fallback Strategy
Figma API	5 failures in 60 seconds	30 seconds	Cached design data
Payment Processing	3 failures in 30 seconds	60 seconds	Queue for retry
Email Notifications	10 failures in 300 seconds	120 seconds	SMS fallback

6.1.4.2 Disaster Recovery Procedures

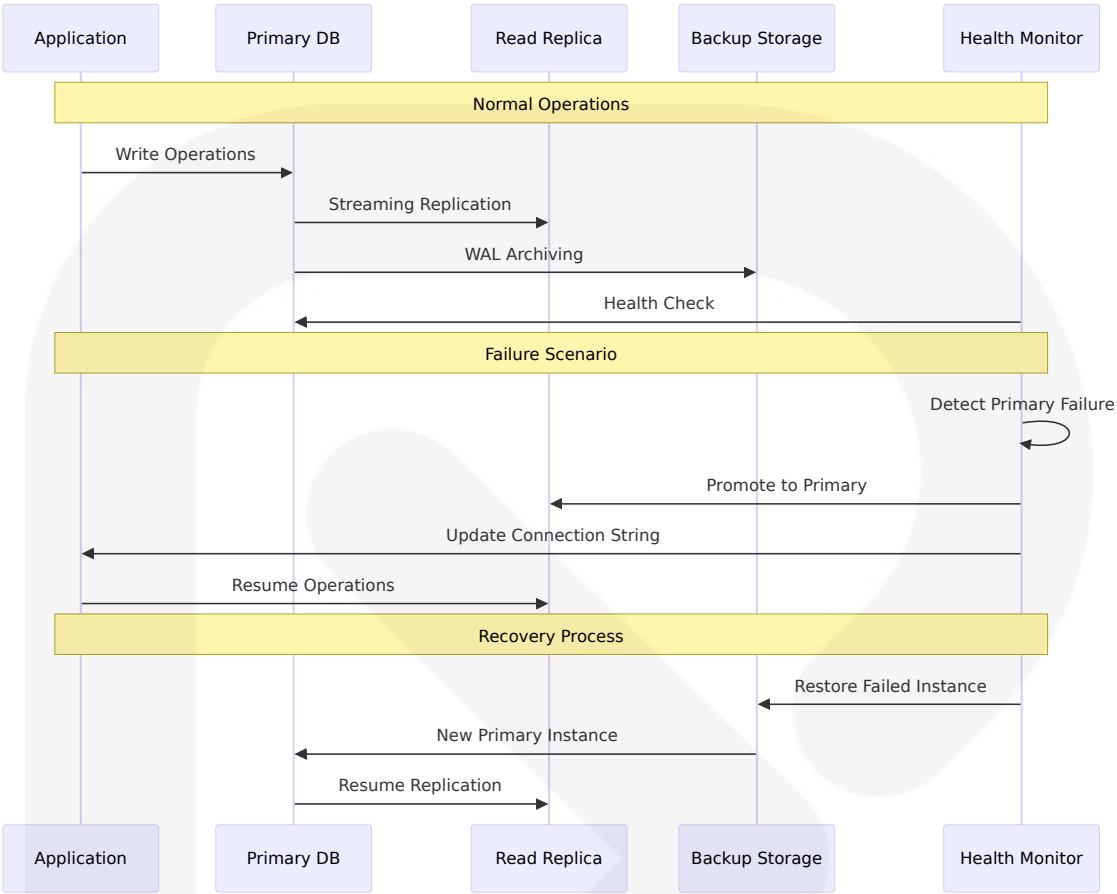
Recovery Time Objectives (RTO):

- Core application: 60 minutes maximum downtime
- Database restoration: 15 minutes maximum data loss (RPO)
- File storage: Cross-region replication with 99.9% availability

Backup Strategy:

- Database: Continuous WAL archiving with point-in-time recovery
- Application state: Redis persistence with RDB + AOF
- File assets: S3 cross-region replication

6.1.4.3 Data Redundancy Approach



6.1.4.4 Service Degradation Policies

Graceful Degradation Levels:

Service Level	Available Features	Disabled Features	User Impact
Full Service	All features operational	None	Normal operation
Degraded Mode	Core project management, basic design review	Real-time notifications, advanced integrations	Minor delays
Essential Only	Project viewing, basic file access	Design uploads, payment processing	Read-only access
Emergency Mode	User authentication, critical data access	All non-essential features	Maintenance notice

6.1.4.5 Failover Configurations

Database Failover:

- Automatic promotion of read replica to primary within 60 seconds
- Connection string updates via service discovery
- Data consistency validation before resuming operations

Application Failover:

- Load balancer health checks every 30 seconds
- Automatic traffic rerouting to healthy instances
- Session state recovery from Redis cluster

Integration Failover:

- Messaging systems often include built-in retry policies for handling failures, and with SQS and NestJS, you can build scalable and resilient microservices
- Exponential backoff retry logic: 1s, 2s, 4s, 8s, 16s
- Dead letter queue for manual intervention after 5 failed attempts

6.1.5 IMPLEMENTATION ROADMAP

6.1.5.1 Phase 1: Monolithic Foundation (Months 1-6)

- Implement core business logic as modular monolith
- Establish clear service boundaries within codebase
- Set up basic horizontal scaling with clustering
- Implement essential resilience patterns

6.1.5.2 Phase 2: Standalone Applications (Months 4-8)

- Extract background job processing to standalone app
- Implement WebSocket server as separate scalable component

- Add integration orchestrator for third-party APIs
- Establish Redis-based inter-service communication

6.1.5.3 Phase 3: Advanced Scaling (Months 8-12)

- Implement auto-scaling policies and monitoring
- Add advanced circuit breaker patterns
- Optimize database performance with read replicas
- Establish comprehensive disaster recovery procedures

This hybrid architecture approach ensures AgencyOS can start with rapid development velocity while maintaining clear paths for future scaling and potential microservices extraction as business demands evolve.

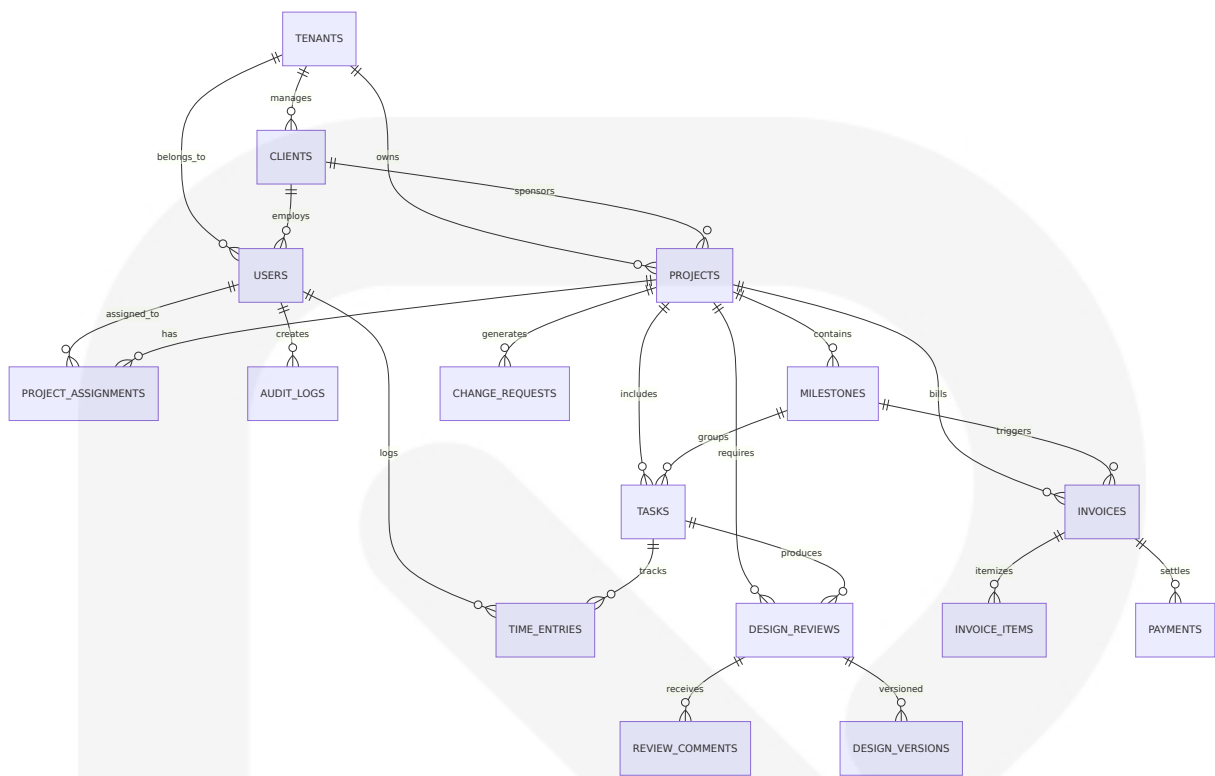
6.2 Database Design

6.2.1 Schema Design

6.2.1.1 Entity Relationships

AgencyOS implements a comprehensive multi-tenant database schema designed around PostgreSQL's row level security feature to create SaaS applications that use a pool model to share database resources and also reduce the risk and overhead of enforcing isolation policies. The schema follows a hub-and-spoke pattern with tenant isolation as the central organizing principle.

Core Entity Relationships:



6.2.1.2 Data Models and Structures

Core Multi-Tenant Tables:

Table Name	Primary Purpose	Key Relationships	RLS Policy
tenants	Workspace isolation and configuration	Root entity for all tenant data	N/A (no tenant_id)
users	Authentication and role management	Belongs to tenant, assigned to projects	tenant_id = current_tenant()
projects	Project lifecycle and metadata	Owned by tenant, assigned to client	tenant_id = current_tenant()
clients	Client organization management	Managed by tenant, sponsors projects	tenant_id = current_tenant()

Project Management Tables:

```

-- Core project structure
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  client_id UUID NOT NULL REFERENCES clients(id),
  name VARCHAR(255) NOT NULL,
  status project_status_enum NOT NULL DEFAULT 'proposal',
  budget_amount DECIMAL(12,2),
  budget_currency CHAR(3) DEFAULT 'USD',
  start_date DATE,
  end_date DATE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Task management with sprint organization
CREATE TABLE tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  project_id UUID NOT NULL REFERENCES projects(id),
  milestone_id UUID REFERENCES milestones(id),
  title VARCHAR(255) NOT NULL,
  description TEXT,
  status task_status_enum NOT NULL DEFAULT 'todo',
  priority task_priority_enum NOT NULL DEFAULT 'medium',
  estimated_hours DECIMAL(5,2),
  actual_hours DECIMAL(5,2) DEFAULT 0,
  assignee_id UUID REFERENCES users(id),
  external_id VARCHAR(100), -- For Jira/Linear sync
  external_system VARCHAR(50),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

Design Review and Asset Management:

```

-- Design review system
CREATE TABLE design_reviews (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  project_id UUID NOT NULL REFERENCES projects(id),

```

```
task_id UUID REFERENCES tasks(id),
title VARCHAR(255) NOT NULL,
status review_status_enum NOT NULL DEFAULT 'pending',
figma_file_key VARCHAR(100),
figma_node_id VARCHAR(100),
review_deadline TIMESTAMPTZ,
created_by UUID NOT NULL REFERENCES users(id),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Asset library with version control
CREATE TABLE assets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  project_id UUID REFERENCES projects(id),
  name VARCHAR(255) NOT NULL,
  file_type VARCHAR(50) NOT NULL,
  file_size BIGINT NOT NULL,
  storage_path VARCHAR(500) NOT NULL,
  version_number INTEGER DEFAULT 1,
  parent_asset_id UUID REFERENCES assets(id),
  metadata JSONB,
  created_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

6.2.1.3 Indexing Strategy

You should add the account identifier as the first column of any index you create. This will in effect first narrow down the search to rows belonging to that account. AgencyOS implements a comprehensive indexing strategy optimized for multi-tenant query patterns.

Primary Indexing Rules:

Index Type	Pattern	Purpose	Example
Tenant-First Composite	(tenant_id, business_key)	Multi-tenant query optimization	(tenant_id, project_id, status)

Index Type	Pattern	Purpose	Example
Partial Indexes	WHERE status = 'active'	Reduce index size for common filters	Active projects, pending reviews
Expression Indexes	LOWER(column)	Case-insensitive searches	User email lookups
GIN Indexes	JSONB columns	Metadata and configuration searches	Project settings, asset metadata

Critical Performance Indexes:

```
-- Multi-tenant project queries
CREATE INDEX idx_projects_tenant_status
ON projects (tenant_id, status, created_at DESC);

-- Time tracking performance
CREATE INDEX idx_time_entries_tenant_project_date
ON time_entries (tenant_id, project_id, entry_date DESC);

-- Design review workflows
CREATE INDEX idx_design_reviews_tenant_status_deadline
ON design_reviews (tenant_id, status, review_deadline)
WHERE status IN ('pending', 'in_review');

-- Asset search optimization
CREATE INDEX idx_assets_tenant_project_type
ON assets (tenant_id, project_id, file_type);

-- Full-text search on project content
CREATE INDEX idx_projects_search
ON projects USING GIN (to_tsvector('english', name || ' ' ||
COALESCE(description, '')));
```

6.2.1.4 Partitioning Approach

AgencyOS implements time-based partitioning for high-volume tables to maintain query performance as data grows. For partitioned tables, consider

creating local indexes on each partition to improve query performance without affecting global write operations.

Partitioned Tables Strategy:

```
-- Audit logs partitioned by month
CREATE TABLE audit_logs (
  id UUID DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  user_id UUID NOT NULL,
  action VARCHAR(100) NOT NULL,
  resource_type VARCHAR(50) NOT NULL,
  resource_id UUID,
  changes JSONB,
  ip_address INET,
  user_agent TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

-- Create monthly partitions
CREATE TABLE audit_logs_2024_01 PARTITION OF audit_logs
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

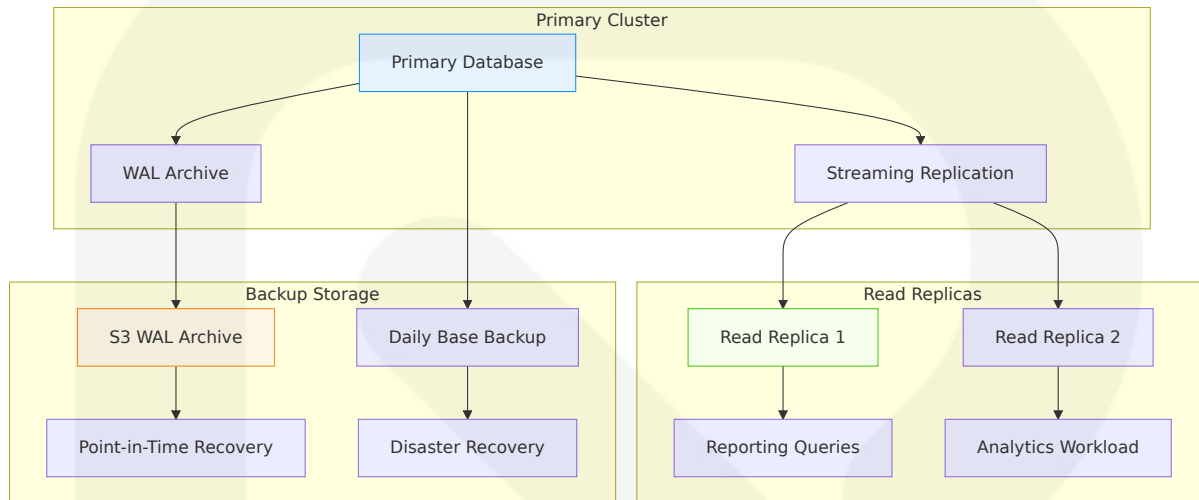
-- Time entries partitioned by quarter
CREATE TABLE time_entries (
  id UUID DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  user_id UUID NOT NULL,
  project_id UUID NOT NULL,
  task_id UUID,
  hours DECIMAL(5,2) NOT NULL,
  description TEXT,
  entry_date DATE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (entry_date);
```

6.2.1.5 Replication Configuration

PostgreSQL deployments can follow streaming replication: Replicates data from the primary node to the secondary, then copies data to Amazon S3 or

Azure Blob for backup storage. AgencyOS implements a robust replication strategy for high availability and read scaling.

Replication Architecture:



Replication Configuration:

```

-- Primary server configuration (postgresql.conf)
wal_level = replica
max_wal_senders = 10
wal_keep_size = 1GB
archive_mode = on
archive_command = 'aws s3 cp %p s3://agencyos-wal-archive/%f'

-- Read replica configuration
primary_conninfo = 'host=primary.agencyos.com port=5432
user=replicator'
restore_command = 'aws s3 cp s3://agencyos-wal-archive/%f %p'
recovery_target_timeline = 'latest'
  
```

6.2.1.6 Backup Architecture

We can combine a file-system-level backup with backup of the WAL files. If recovery is needed, we restore the file system backup and then replay from the backed-up WAL files to bring the system to a current state.

Comprehensive Backup Strategy:

Backup Type	Frequency	Retention	Purpose
Continuous WAL Archive	Real-time	30 days	Point-in-time recovery
Base Backup	Daily	90 days	Full database restoration
Logical Backup	Weekly	1 year	Schema migration and tenant export
Cross-Region Backup	Daily	30 days	Disaster recovery

6.2.2 Data Management

6.2.2.1 Migration Procedures

AgencyOS implements a robust database migration system that maintains tenant isolation while enabling schema evolution across all tenants simultaneously.

Migration Framework:

```
-- Migration tracking table
CREATE TABLE schema_migrations (
  version VARCHAR(50) PRIMARY KEY,
  applied_at TIMESTAMPTZ DEFAULT NOW(),
  rollback_sql TEXT
);

-- Tenant-aware migration execution
CREATE OR REPLACE FUNCTION apply_migration(
  migration_version VARCHAR(50),
  migration_sql TEXT,
  rollback_sql TEXT DEFAULT NULL
) RETURNS VOID AS $$
DECLARE
  tenant_record RECORD;
BEGIN
  -- Apply to shared tables first
```

```
EXECUTE migration_sql;

-- Record migration
INSERT INTO schema_migrations (version, rollback_sql)
VALUES (migration_version, rollback_sql);

-- Validate RLS policies still work
PERFORM validate_tenant_isolation();
END;
$$ LANGUAGE plpgsql;
```

Migration Validation:

```
-- Ensure tenant isolation after migrations
CREATE OR REPLACE FUNCTION validate_tenant_isolation() RETURNS BOOLEAN
AS $$
DECLARE
    test_tenant_id UUID;
    row_count INTEGER;
BEGIN
    -- Create test tenant context
    SELECT id INTO test_tenant_id FROM tenants LIMIT 1;
    PERFORM set_config('app.current_tenant', test_tenant_id::TEXT,
true);

    -- Verify RLS policies are active
    SELECT COUNT(*) INTO row_count FROM projects WHERE tenant_id !=
test_tenant_id;

    IF row_count > 0 THEN
        RAISE EXCEPTION 'Tenant isolation validation failed';
    END IF;

    RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

6.2.2.2 Versioning Strategy

Schema Version Management:

```
-- Track schema changes with semantic versioning
CREATE TABLE schema_versions (
  id SERIAL PRIMARY KEY,
  version VARCHAR(20) NOT NULL UNIQUE,
  description TEXT NOT NULL,
  migration_files TEXT[] NOT NULL,
  applied_at TIMESTAMPTZ DEFAULT NOW(),
  applied_by VARCHAR(100) NOT NULL
);

-- Feature flag integration for gradual rollouts
CREATE TABLE feature_flags (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID REFERENCES tenants(id),
  flag_name VARCHAR(100) NOT NULL,
  enabled BOOLEAN DEFAULT FALSE,
  config JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(tenant_id, flag_name)
);
```

6.2.2.3 Archival Policies

Data Lifecycle Management:

Data Category	Active Retention	Archive Retention	Deletion Policy
Project Data	2 years post-completion	5 years	Client contract dependent
Audit Logs	2 years	7 years	Regulatory compliance
Design Assets	Indefinite	Indefinite	Client ownership
Time Tracking	3 years	7 years	Financial reporting

```
-- Automated archival process
CREATE OR REPLACE FUNCTION archive_old_data() RETURNS VOID AS $$
```

```

BEGIN
    -- Archive completed projects older than 2 years
    UPDATE projects
    SET archived_at = NOW()
    WHERE status = 'completed'
    AND updated_at < NOW() - INTERVAL '2 years'
    AND archived_at IS NULL;

    -- Move old audit logs to archive partition
    CALL partition_old_audit_logs();

    -- Compress old design assets
    CALL compress_old_assets();
END;
$$ LANGUAGE plpgsql;

```

6.2.2.4 Data Storage and Retrieval Mechanisms

Optimized Query Patterns:

```

-- Tenant-aware query optimization
CREATE OR REPLACE FUNCTION set_tenant_context(tenant_uuid UUID)
RETURNS VOID AS $$
BEGIN
    PERFORM set_config('app.current_tenant', tenant_uuid::TEXT, true);
    PERFORM set_config('search_path', 'public', true);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Efficient project dashboard query
CREATE OR REPLACE VIEW project_dashboard AS
SELECT
    p.id,
    p.name,
    p.status,
    p.budget_amount,
    COUNT(t.id) as total_tasks,
    COUNT(t.id) FILTER (WHERE t.status = 'completed') as
completed_tasks,
    SUM(te.hours) as total_hours,
    c.name as client_name

```

```
FROM projects p
LEFT JOIN tasks t ON t.project_id = p.id AND t.tenant_id = p.tenant_id
LEFT JOIN time_entries te ON te.project_id = p.id AND te.tenant_id =
p.tenant_id
LEFT JOIN clients c ON c.id = p.client_id AND c.tenant_id =
p.tenant_id
WHERE p.tenant_id = current_setting('app.current_tenant')::UUID
GROUP BY p.id, p.name, p.status, p.budget_amount, c.name;
```

6.2.2.5 Caching Policies

Multi-Layer Caching Strategy:

```
-- Materialized views for expensive aggregations
CREATE MATERIALIZED VIEW tenant_utilization_summary AS
SELECT
    tenant_id,
    DATE_TRUNC('month', entry_date) as month,
    SUM(hours) as total_hours,
    COUNT(DISTINCT user_id) as active_users,
    COUNT(DISTINCT project_id) as active_projects
FROM time_entries
WHERE entry_date >= NOW() - INTERVAL '12 months'
GROUP BY tenant_id, DATE_TRUNC('month', entry_date);

-- Refresh policy for materialized views
CREATE OR REPLACE FUNCTION refresh_utilization_cache() RETURNS VOID AS
$$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY tenant_utilization_summary;
END;
$$ LANGUAGE plpgsql;
```

6.2.3 Compliance Considerations

6.2.3.1 Data Retention Rules

Regulatory Compliance Framework:

Regulation	Scope	Retention Period	Implementation
GDPR	EU personal data	Right to erasure	Automated deletion workflows
SOC 2	Audit trails	24 months minimum	Immutable audit logs
Financial Records	Invoicing, payments	7 years	Separate retention policies
Client Contracts	Project deliverables	Contract-specific	Configurable per client

```

-- GDPR compliance implementation
CREATE TABLE data_retention_policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  data_category VARCHAR(100) NOT NULL,
  retention_days INTEGER NOT NULL,
  auto_delete BOOLEAN DEFAULT FALSE,
  legal_basis TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Right to erasure implementation
CREATE OR REPLACE FUNCTION process_erasure_request(
  target_tenant_id UUID,
  target_user_id UUID
) RETURNS VOID AS $$
BEGIN
  -- Anonymize personal data
  UPDATE users
  SET email = 'deleted-' || id::TEXT || '@example.com',
      first_name = 'Deleted',
      last_name = 'User',
      phone = NULL,
      avatar_url = NULL
  WHERE tenant_id = target_tenant_id AND id = target_user_id;

  -- Log erasure action
  INSERT INTO audit_logs (tenant_id, action, resource_type,
resource_id)

```



```
VALUES (target_tenant_id, 'GDPR_ERASURE', 'user', target_user_id);
END;
$$ LANGUAGE plpgsql;
```

6.2.3.2 Backup and Fault Tolerance Policies

High Availability Configuration:

```
-- Backup validation and monitoring
CREATE TABLE backup_status (
    id SERIAL PRIMARY KEY,
    backup_type VARCHAR(50) NOT NULL,
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ,
    status VARCHAR(20) NOT NULL DEFAULT 'running',
    file_size BIGINT,
    checksum VARCHAR(64),
    error_message TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Automated backup verification
CREATE OR REPLACE FUNCTION verify_backup_integrity() RETURNS BOOLEAN
AS $$
DECLARE
    latest_backup RECORD;
    test_result BOOLEAN;
BEGIN
    -- Get latest backup
    SELECT * INTO latest_backup
    FROM backup_status
    WHERE status = 'completed'
    ORDER BY end_time DESC
    LIMIT 1;

    -- Verify backup can be restored (test environment)
    -- Implementation depends on backup storage system

    RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

6.2.3.3 Privacy Controls

Data Privacy Implementation:

```
-- Personal data classification
CREATE TABLE data_classification (
    table_name VARCHAR(100) NOT NULL,
    column_name VARCHAR(100) NOT NULL,
    classification VARCHAR(50) NOT NULL, -- PII, SENSITIVE, PUBLIC
    encryption_required BOOLEAN DEFAULT FALSE,
    retention_days INTEGER,
    PRIMARY KEY (table_name, column_name)
);

-- Encryption for sensitive data
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Encrypted storage for sensitive fields
ALTER TABLE users ADD COLUMN encrypted_ssn BYTEA;

CREATE OR REPLACE FUNCTION encrypt_sensitive_data(data TEXT) RETURNS
BYTEA AS $$
BEGIN
    RETURN pgp_sym_encrypt(data,
        current_setting('app.encryption_key'));
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

6.2.3.4 Audit Mechanisms

Comprehensive Audit Trail:

```
-- Audit trigger function
CREATE OR REPLACE FUNCTION audit_trigger_function() RETURNS TRIGGER AS
$$
DECLARE
    audit_data JSONB;
    tenant_id_val UUID;
BEGIN
    -- Extract tenant_id from the record
```

```
IF TG_OP = 'DELETE' THEN
    tenant_id_val := OLD.tenant_id;
    audit_data := to_jsonb(OLD);
ELSE
    tenant_id_val := NEW.tenant_id;
    audit_data := to_jsonb(NEW);
END IF;

-- Insert audit record
INSERT INTO audit_logs (
    tenant_id,
    user_id,
    action,
    resource_type,
    resource_id,
    changes,
    ip_address,
    user_agent
) VALUES (
    tenant_id_val,
    current_setting('app.current_user_id', true)::UUID,
    TG_OP,
    TG_TABLE_NAME,
    COALESCE(NEW.id, OLD.id),
    audit_data,
    inet_client_addr(),
    current_setting('app.user_agent', true)
);

RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Apply audit triggers to all tenant tables
CREATE TRIGGER audit_projects_trigger
AFTER INSERT OR UPDATE OR DELETE ON projects
FOR EACH ROW EXECUTE FUNCTION audit_trigger_function();
```

6.2.3.5 Access Controls

Row-Level Security Implementation:

When you define security policies on a table, these policies restrict which rows in that table are returned by SELECT queries or which rows are affected by INSERT, UPDATE, and DELETE commands. The policies are defined with a USING clause that returns a Boolean value, which indicates whether to process a given row in the table.

```
-- Enable RLS on all tenant tables
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE design_reviews ENABLE ROW LEVEL SECURITY;

-- Create tenant isolation policies
CREATE POLICY tenant_isolation_policy ON projects
  FOR ALL
  TO application_role
  USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON users
  FOR ALL
  TO application_role
  USING (tenant_id = current_setting('app.current_tenant')::UUID);

-- Project-level access control
CREATE POLICY project_access_policy ON tasks
  FOR ALL
  TO application_role
  USING (
    tenant_id = current_setting('app.current_tenant')::UUID
    AND (
      -- User is assigned to the project
      EXISTS (
        SELECT 1 FROM project_assignments pa
        WHERE pa.project_id = tasks.project_id
        AND pa.user_id =
current_setting('app.current_user_id')::UUID
        AND pa.tenant_id = tasks.tenant_id
      )
      -- Or user has admin role
      OR current_setting('app.user_role') IN ('admin',
'super_admin')
    )
  )
```

```
)  
);
```

6.2.4 Performance Optimization

6.2.4.1 Query Optimization Patterns

Tenant-Optimized Query Patterns:

```
-- Optimized project listing with proper index usage  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT p.id, p.name, p.status, c.name as client_name,  
       COUNT(t.id) as task_count  
FROM projects p  
JOIN clients c ON c.id = p.client_id AND c.tenant_id = p.tenant_id  
LEFT JOIN tasks t ON t.project_id = p.id AND t.tenant_id = p.tenant_id  
WHERE p.tenant_id = current_setting('app.current_tenant')::UUID  
      AND p.status = 'active'  
GROUP BY p.id, p.name, p.status, c.name  
ORDER BY p.updated_at DESC  
LIMIT 20;  
  
-- Query plan optimization for multi-tenant patterns  
CREATE STATISTICS tenant_project_stats ON tenant_id, status FROM  
projects;  
ANALYZE projects;
```

6.2.4.2 Caching Strategy

Redis Integration for Application Caching:

```
-- Cache invalidation triggers  
CREATE OR REPLACE FUNCTION invalidate_project_cache() RETURNS TRIGGER  
AS $$  
BEGIN  
    -- Notify application to invalidate cache  
    PERFORM pg_notify('cache_invalidation',  
                      json_build_object(
```

```
        'type', 'project',
        'tenant_id', COALESCE(NEW.tenant_id, OLD.tenant_id),
        'project_id', COALESCE(NEW.id, OLD.id)
    )::TEXT
);

RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER project_cache_invalidation
AFTER INSERT OR UPDATE OR DELETE ON projects
FOR EACH ROW EXECUTE FUNCTION invalidate_project_cache();
```

6.2.4.3 Connection Pooling

PgBouncer Configuration for Multi-Tenant Workloads:

```
# pgbouncer.ini optimized for multi-tenant applications
[databases]
agencyos = host=localhost port=5432 dbname=agencyos

[pgbouncer]
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 25
min_pool_size = 5
reserve_pool_size = 5
max_db_connections = 100

#### Multi-tenant optimization
server_reset_query = DISCARD ALL
server_check_query = SELECT 1
server_check_delay = 30
```

6.2.4.4 Read/Write Splitting

Application-Level Read Replica Routing:

```

-- Read-only queries routing configuration
CREATE OR REPLACE FUNCTION is_read_only_query(query_text TEXT) RETURNS
BOOLEAN AS $$
BEGIN
    RETURN query_text ~* '^(SELECT|WITH.*SELECT)'
        AND query_text !~*
        '(INSERT|UPDATE|DELETE|CREATE|ALTER|DROP)';
END;
$$ LANGUAGE plpgsql IMMUTABLE;

-- Connection routing logic (implemented in application layer)
-- Read queries → Read replicas
-- Write queries → Primary database
-- Transactions → Primary database

```

6.2.4.5 Batch Processing Approach

Efficient Bulk Operations:

```

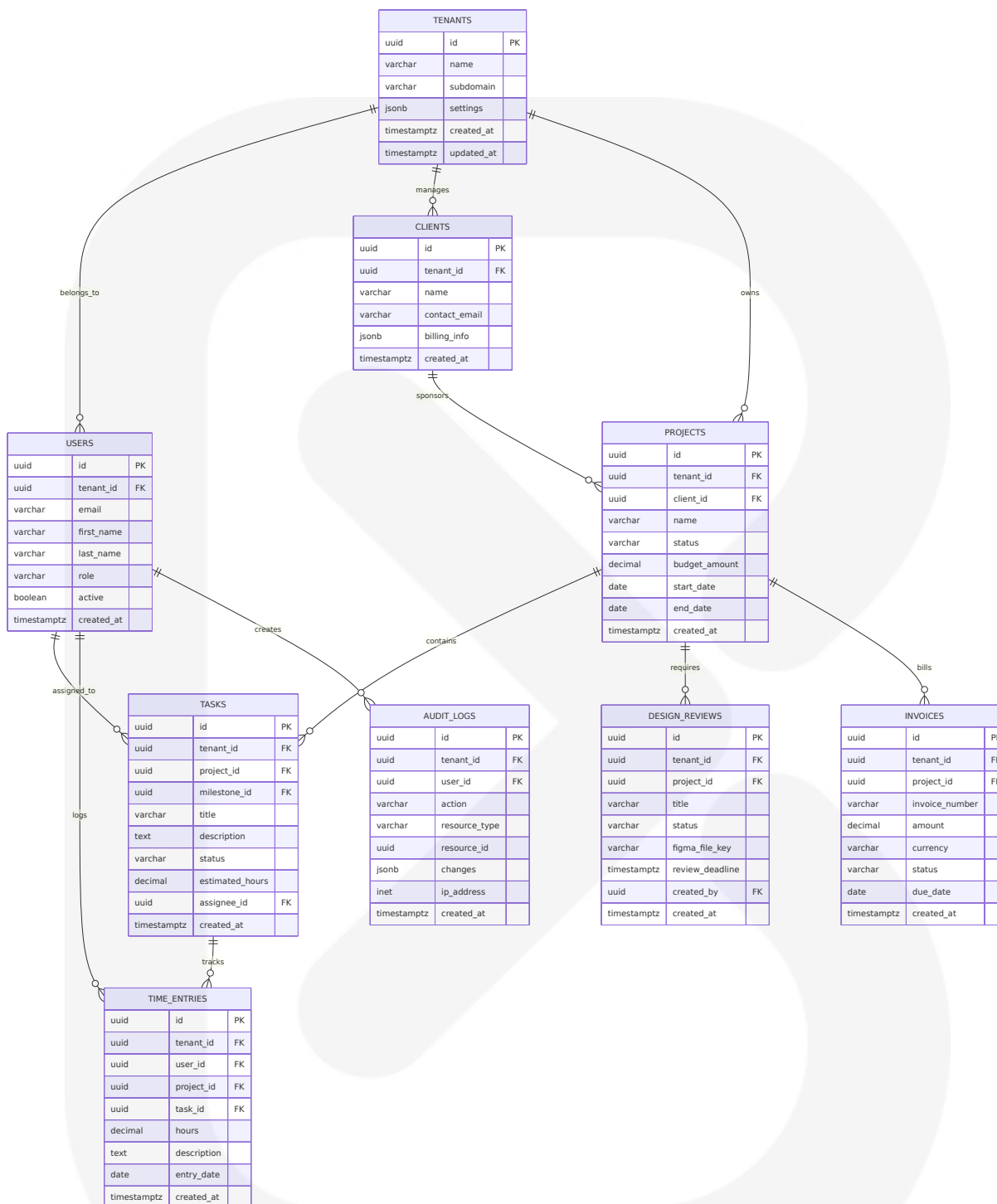
-- Batch time entry processing
CREATE OR REPLACE FUNCTION batch_insert_time_entries(
    entries JSONB
) RETURNS INTEGER AS $$
DECLARE
    inserted_count INTEGER;
BEGIN
    WITH entry_data AS (
        SELECT
            (entry->>'tenant_id')::UUID as tenant_id,
            (entry->>'user_id')::UUID as user_id,
            (entry->>'project_id')::UUID as project_id,
            (entry->>'task_id')::UUID as task_id,
            (entry->>'hours')::DECIMAL(5,2) as hours,
            entry->>'description' as description,
            (entry->>'entry_date')::DATE as entry_date
        FROM jsonb_array_elements(entries) as entry
    )
    INSERT INTO time_entries (tenant_id, user_id, project_id, task_id,
        hours, description, entry_date)
    SELECT * FROM entry_data;

```

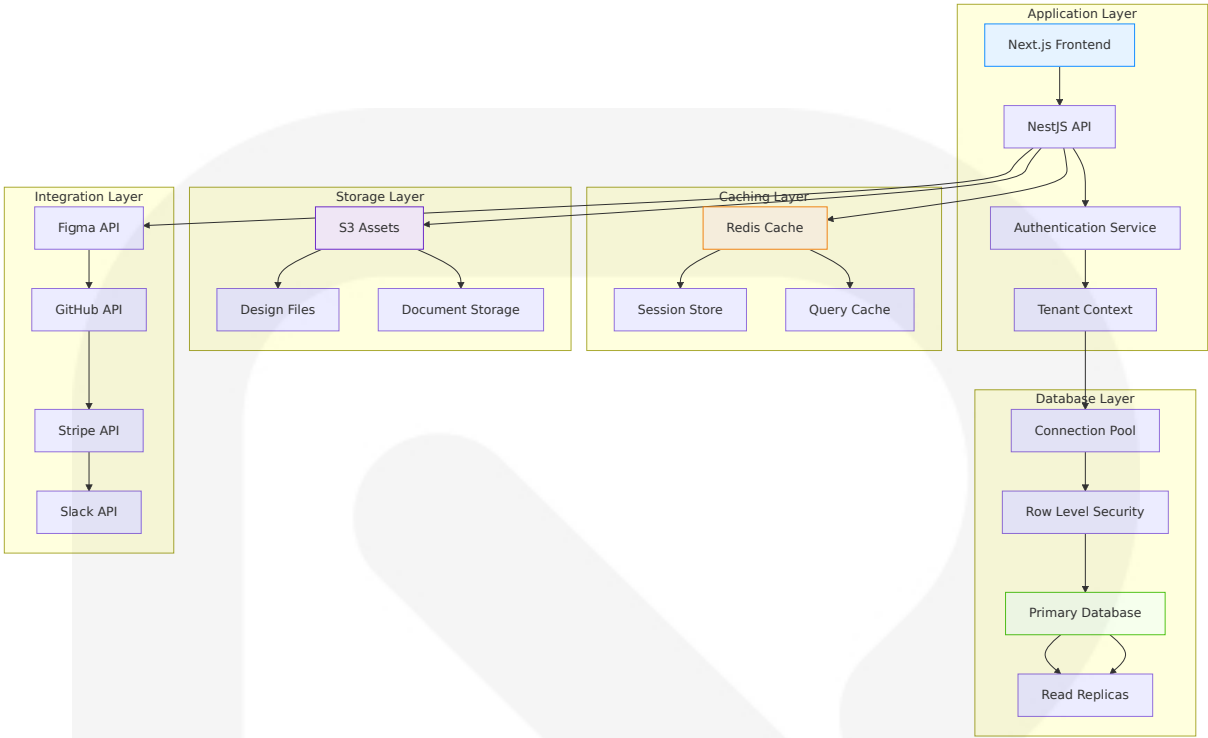
```
    GET DIAGNOSTICS inserted_count = ROW_COUNT;  
    RETURN inserted_count;  
END;  
$$ LANGUAGE plpgsql;
```

6.2.5 Database Schema Diagrams

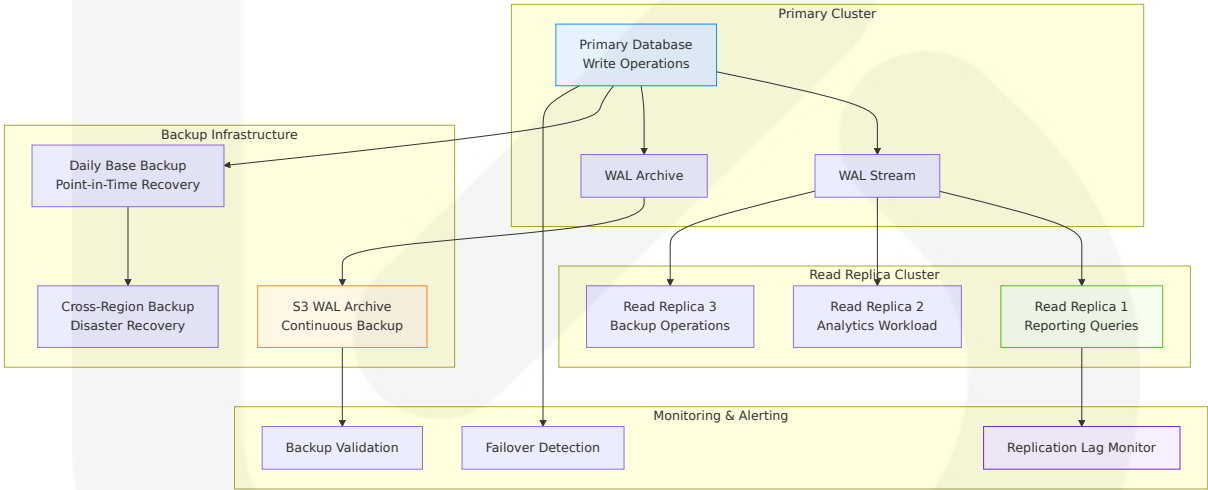
6.2.5.1 Core Entity Relationship Diagram



6.2.5.2 Data Flow Diagram



6.2.5.3 Replication Architecture Diagram



This comprehensive database design ensures Row-Level Security (RLS) is a powerful tool for implementing multi-tenancy in modern SaaS applications. By setting the tenant context dynamically using SET LOCAL, you can enforce strict data isolation at the database level—without scattering tenant filters across your application code, while providing the scalability, performance, and compliance features required for AgencyOS's multi-tenant architecture.

6.3 Integration Architecture

6.3.1 API Design

6.3.1.1 Protocol Specifications

AgencyOS implements a comprehensive RESTful API architecture built on modern web standards, designed to support both internal system operations and extensive third-party integrations. The API follows REST principles with REST structure, supporting authentication via access tokens and OAuth2, with requests made via HTTP endpoints with clear functions and appropriate response codes.

Core Protocol Standards:

Protocol	Version	Purpose	Implementation
HTTP/HTTPS	HTTP/2, TLS 1.2+	Primary communication protocol	HTTPS-only communication enforced for improved security, HTTP requests fail with 403 Forbidden status
REST	OpenAPI 3.0	API architecture pattern	Stateless, resource-based endpoints with standard HTTP methods
WebSocket	RFC 6455	Real-time communication	Design review collaboration, live project updates
Server-Sent Events	W3C Standard	One-way live updates	Build status notifications, approval alerts

API Endpoint Structure:

```
// Base API structure following Next.js 15 App Router patterns
interface APIEndpointStructure {
  // Core business endpoints
  '/api/v1/projects': ProjectEndpoints
  '/api/v1/clients': ClientEndpoints
  '/api/v1/design-reviews': DesignReviewEndpoints
}
```

```
'/api/v1/integrations': IntegrationEndpoints

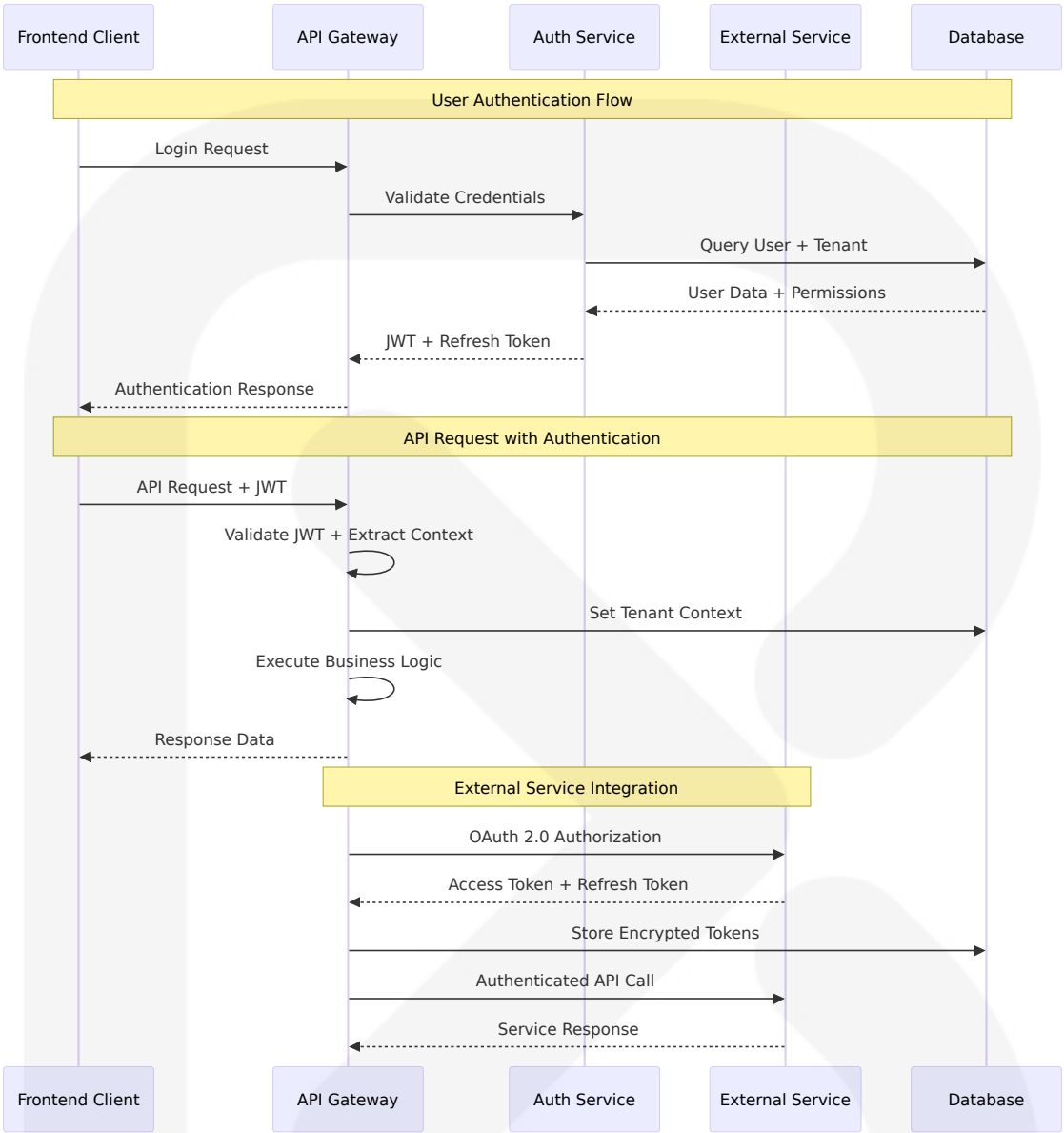
// Real-time endpoints
'/api/v1/ws': WebSocketEndpoints
'/api/v1/events': ServerSentEventEndpoints

// Webhook receivers
'/api/v1/webhooks/figma': FigmaWebhookHandler
'/api/v1/webhooks/github': GitHubWebhookHandler
'/api/v1/webhooks/stripe': StripeWebhookHandler
}
```

6.3.1.2 Authentication Methods

AgencyOS implements a multi-layered authentication strategy supporting both internal user authentication and external service integration patterns.

Authentication Flow Architecture:



Authentication Methods Matrix:

Method	Use Case	Token Type	Expiration	Refresh Strategy
JWT Bearer	Internal API access	Access token	15 minutes	Automatic refresh token rotation
OAuth 2.0	Third-party integrations	Bearer token	Service-specific	Background token refresh

Method	Use Case	Token Type	Expiration	Refresh Strategy
API Keys	Webhook verification	HMAC signature	N/A	Key rotation on security events
Session-based	Client portal access	HTTP-only cookies	24 hours	Sliding session extension

6.3.1.3 Authorization Framework

The authorization framework implements Role-Based Access Control (RBAC) with project-level permissions and tenant isolation enforcement.

Authorization Decision Flow:

```
interface AuthorizationFramework {  
  // Permission evaluation  
  checkPermission(  
    userId: string,  
    resource: string,  
    action: string,  
    context?: PermissionContext  
  ): Promise<AuthorizationResult>  
  
  // Role-based access  
  getUserRoles(userId: string, tenantId: string): Promise<Role[]>  
  getProjectPermissions(userId: string, projectId: string):  
  Promise<Permission[]>  
  
  // Tenant isolation  
  validateTenantAccess(userId: string, tenantId: string):  
  Promise<boolean>  
  setTenantContext(tenantId: string): Promise<void>  
}
```

Permission Matrix:

Resource	Admin	PM	Designer	Engineer	Client Admin	CR
Project Create	✓	✓	X	X	X	X
Design Upload	✓	✓	✓	X	X	X
Design Review	✓	✓	✓	✓	✓	✓
Design Approve	✓	✓	X	X	✓	X
Invoice View	✓	✓	X	X	✓	X

6.3.1.4 Rate Limiting Strategy

AgencyOS implements sophisticated rate limiting to protect against abuse while ensuring legitimate usage patterns remain unaffected.

Rate Limiting Tiers:

User Type	Requests/Minute	Requests/Hour	Burst Allowance	Enforcement Method
Authenticated Users	1000	10000	50 requests	Sliding window with Redis
Integration APIs	500	5000	25 requests	Token bucket algorithm
Webhook Endpoints	100	1000	10 requests	Fixed window with backoff
Anonymous Users	10	100	5 requests	IP-based limiting

Rate Limiting Implementation:

```
interface RateLimitingService {  
  // Check rate limit before processing  
  checkRateLimit(  
    identifier: string,  
    endpoint: string,  
    tier: RateLimitTier  
  ): Promise<RateLimitResult>  
  
  // Update rate limit counters  
  updateRateLimit(  
    identifier: string,  
    endpoint: string,  
    consumed: number  
  ): Promise<void>  
  
  // Handle rate limit exceeded  
  handleRateLimitExceeded(  
    identifier: string,  
    retryAfter: number  
  ): Promise<RateLimitResponse>  
}
```

6.3.1.5 Versioning Approach

AgencyOS implements URI versioning where the version is passed within the URI of the request, applying versions to controllers or individual routes using the `@Version` decorator.

API Versioning Strategy:

Versioning Method	Implementation	Example	Use Case
URI Versioning	/api/v1/resource	/api/v1/projects	Major API changes
Header Versioning	Accept-Version: 2.0	Custom version header	Gradual feature rollout
Media Type	Accept: application/vnd.api+json;version=2	Content negotiation	Backward compatibility

Version Lifecycle Management:

```
// NestJS versioning configuration
app.enableVersioning({
  type: VersioningType.URI,
  defaultVersion: '1',
  prefix: 'v'
})

// Controller-level versioning
@Controller('projects')
@Version('1')
export class ProjectsV1Controller {
  @Get()
  findAll(): Promise<Project[]> {
    return this.projectService.findAll()
  }
}

@Controller('projects')
@Version('2')
export class ProjectsV2Controller {
  @Get()
  findAll(): Promise<EnhancedProject[]> {
    return this.projectService.findAllEnhanced()
  }
}
```

6.3.1.6 Documentation Standards

AgencyOS maintains comprehensive API documentation using OpenAPI 3.0 specifications with automated generation and interactive testing capabilities.

Documentation Architecture:

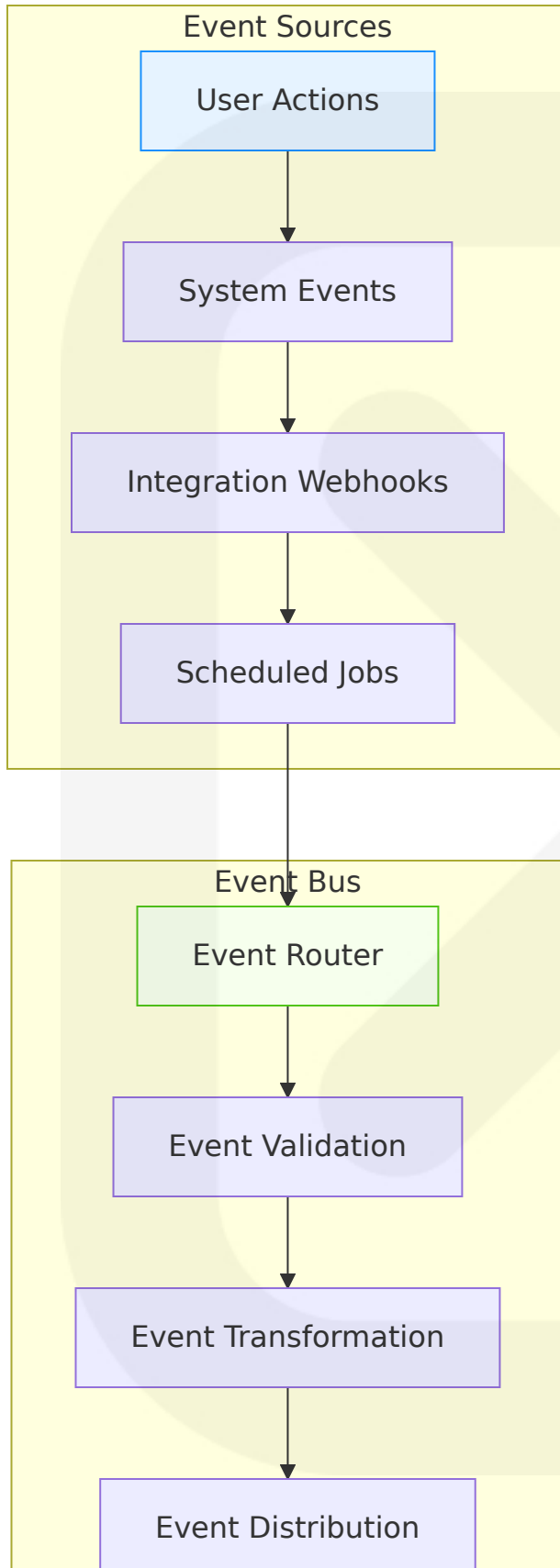
Component	Technology	Purpose	Update Frequency
OpenAPI Spec	OpenAPI 3.0	API contract definition	Automated on deployment
Interactive Docs	Swagger UI	API exploration and testing	Real-time updates
SDK Generation	OpenAPI Generator	Client library creation	Version-based releases
Integration Guides	Markdown + Code Examples	Developer onboarding	Monthly reviews

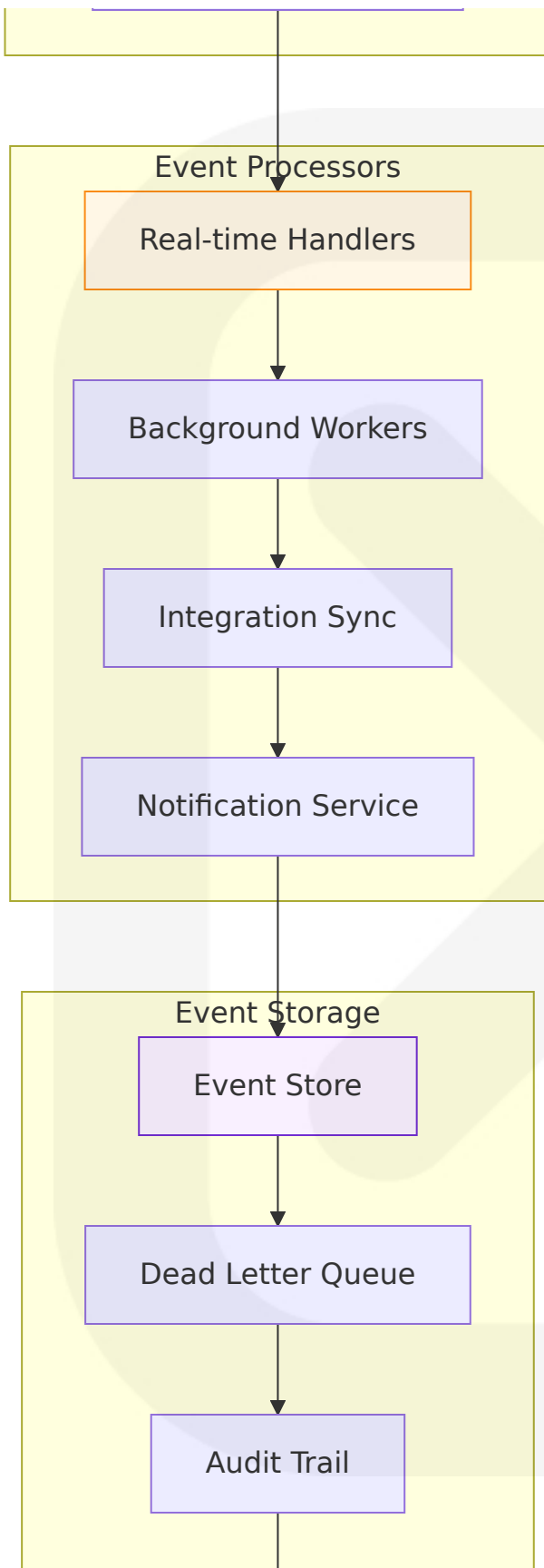
6.3.2 Message Processing

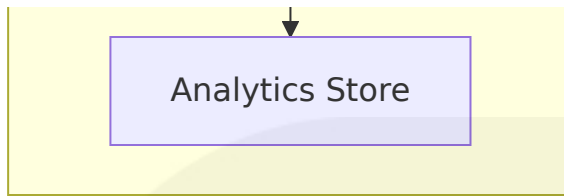
6.3.2.1 Event Processing Patterns

AgencyOS implements event-driven architecture patterns to handle asynchronous operations, real-time updates, and integration synchronization across the platform.

Event Processing Architecture:







Event Types and Handlers:

Event Category	Event Types	Processing Pattern	Handler Implementation
Project Events	ProjectCreated, MilestoneCompleted, BudgetExceeded	Immediate + Background	Real-time notifications + audit logging
Design Events	DesignUploaded, ReviewRequested, ApprovalReceived	Real-time + Async	WebSocket broadcast + integration sync
Integration Events	FigmaFileUpdated, GitHubPRCreated, StripePaymentReceived	Background Processing	Queue-based with retry logic
System Events	UserLogin, DataExport, SecurityAlert	Immediate Processing	Direct handling with audit trail

6.3.2.2 Message Queue Architecture

AgencyOS utilizes BullMQ, a lightweight, robust, and fast NodeJS library for creating background jobs and sending messages using queues, backed by Redis for horizontal scaling.

Queue Architecture Design:

```
interface MessageQueueArchitecture {  
  // Queue definitions  
  queues: {  
    'high-priority': HighPriorityQueue // Payments, security events  
    'notifications': NotificationQueue // Email, Slack, in-app  
  }  
  alerts:  
    'integrations': IntegrationQueue // Third-party API sync  
    'file-processing': FileProcessingQueue // Design file optimization  
    'analytics': AnalyticsQueue // Usage tracking, reporting  
}
```

```
}

// Worker configuration
workers: {
  concurrency: number
  rateLimiting: RateLimitConfig
  retryStrategy: RetryConfig
  errorHandling: ErrorHandlingConfig
}
}
```

BullMQ Queue Configuration:

```
// High-priority queue for critical operations
const highPriorityQueue = new Queue('high-priority', {
  connection: redisConnection,
  defaultJobOptions: {
    removeOnComplete: 100,
    removeOnFail: 50,
    attempts: 3,
    backoff: {
      type: 'exponential',
      delay: 2000
    }
  }
})

// Integration queue for third-party sync
const integrationQueue = new Queue('integrations', {
  connection: redisConnection,
  defaultJobOptions: {
    removeOnComplete: 500,
    removeOnFail: 100,
    attempts: 5,
    backoff: {
      type: 'exponential',
      delay: 5000
    },
  },
  delay: 1000 // Rate limiting for API calls
})
}
```

6.3.2.3 Stream Processing Design

AgencyOS implements stream processing for real-time data flows, particularly for design review collaboration and project status updates.

Stream Processing Patterns:

Stream Type	Technology	Use Case	Processing Model
Real-time Collaboration	WebSocket + Redis Streams	Design review comments, cursor tracking	Event streaming
Integration Data Flow	BullMQ + Redis Pub/Sub	Figma file changes, GitHub PR updates	Message streaming
Analytics Pipeline	Redis Streams + Time Windows	User activity, performance metrics	Batch streaming
Audit Trail	Append-only logs	Compliance, security monitoring	Log streaming

WebSocket Stream Implementation:

```
interface StreamProcessingService {  
  // Real-time collaboration streams  
  designReviewStream: {  
    subscribe(reviewId: string, userId: string):  
    Promise<StreamSubscription>  
    publish(reviewId: string, event: ReviewEvent): Promise<void>  
    unsubscribe(subscriptionId: string): Promise<void>  
  }  
  
  // Project status streams  
  projectStatusStream: {  
    subscribe(projectId: string, userRole: UserRole):  
    Promise<StreamSubscription>  
    publishStatusUpdate(projectId: string, update: StatusUpdate):  
    Promise<void>  
    getStreamHistory(projectId: string, since: Date):  
    Promise<StatusUpdate[]>  
  }  
}
```

```
}  
}
```

6.3.2.4 Batch Processing Flows

Batch processing handles resource-intensive operations that don't require immediate execution, such as report generation, data exports, and bulk integrations.

Batch Processing Categories:

Batch Type	Schedule	Processing Window	Resource Allocation
Daily Reports	2:00 AM UTC	2-hour window	4 CPU cores, 8GB RAM
Weekly Analytics	Sunday 1:00 AM UTC	4-hour window	8 CPU cores, 16GB RAM
Monthly Exports	1st of month, 3:00 AM UTC	6-hour window	12 CPU cores, 32GB RAM
Integration Sync	Every 15 minutes	5-minute window	2 CPU cores, 4GB RAM

Batch Job Implementation:

```
// Scheduled batch processing with BullMQ  
const batchQueue = new Queue('batch-processing', {  
  connection: redisConnection  
})  
  
// Daily report generation  
await batchQueue.add('generate-daily-reports', {  
  tenantIds: ['tenant-1', 'tenant-2'],  
  reportDate: new Date(),  
  reportTypes: ['utilization', 'revenue', 'project-status']  
}, {  
  repeat: { cron: '0 2 * * *' }, // Daily at 2 AM  
  jobId: 'daily-reports'  
})
```

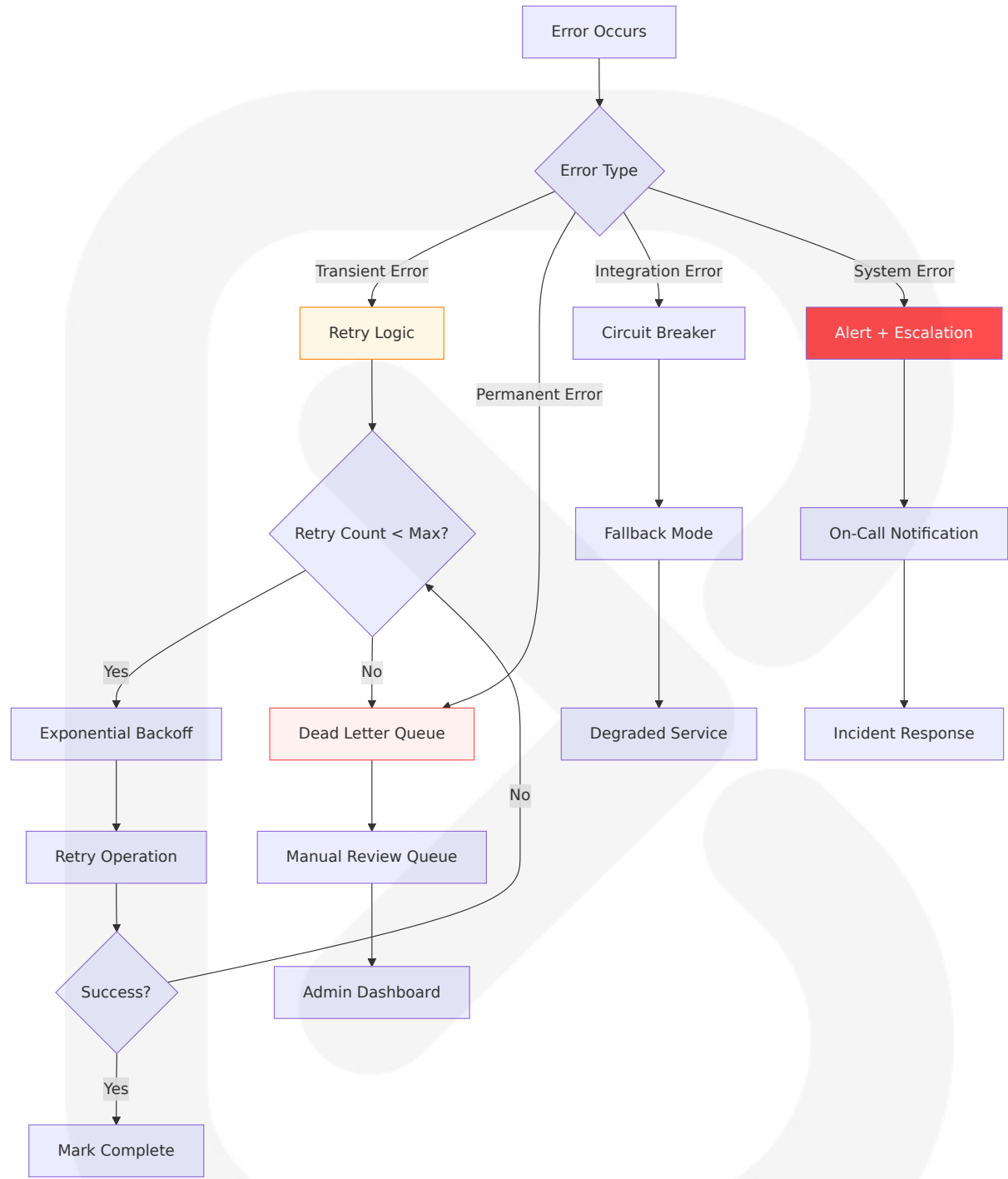


```
// Batch worker implementation
const batchWorker = new Worker('batch-processing', async (job) => {
  switch (job.name) {
    case 'generate-daily-reports':
      return await generateDailyReports(job.data)
    case 'sync-integration-data':
      return await syncIntegrationData(job.data)
    default:
      throw new Error(`Unknown batch job: ${job.name}`)
  }
}, {
  concurrency: 2,
  limiter: {
    max: 10,
    duration: 60000 // 10 jobs per minute
  }
})
```

6.3.2.5 Error Handling Strategy

Comprehensive error handling ensures system resilience and provides clear debugging information for both internal operations and external integrations.

Error Handling Hierarchy:



Error Recovery Patterns:

Error Category	Recovery Strategy	Retry Policy	Escalation Threshold
Network Timeouts	Exponential backoff retry	1s, 2s, 4s, 8s, 16s	5 consecutive failures

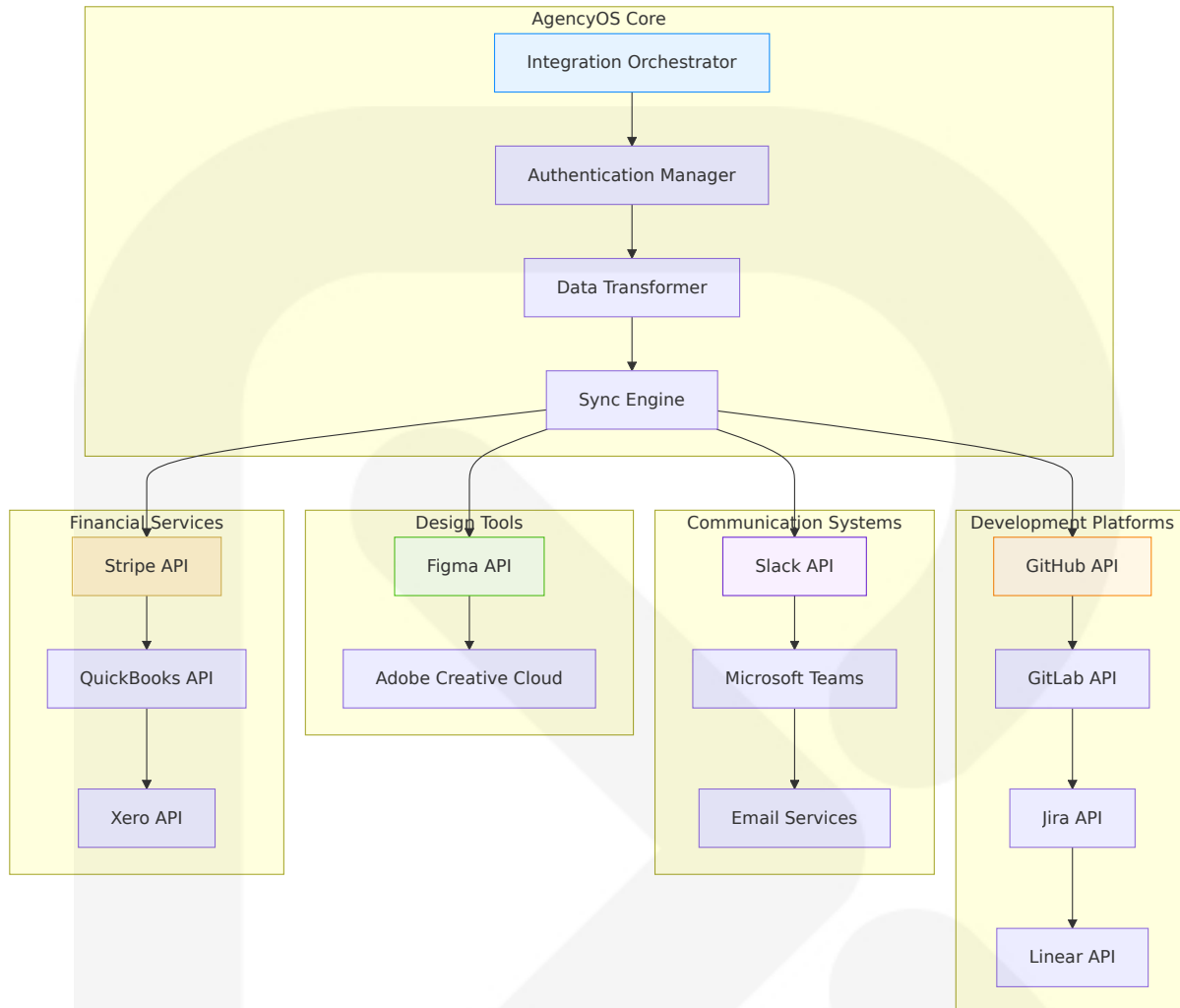
Error Category	Recovery Strategy	Retry Policy	Escalation Threshold
Rate Limit Exceeded	Respect retry-after header	Service-specific delays	3 rate limit violations
Authentication Failures	Token refresh + retry	Immediate refresh attempt	2 auth failures
Data Validation Errors	Dead letter queue	No retry	Immediate escalation

6.3.3 External Systems

6.3.3.1 Third-Party Integration Patterns

AgencyOS integrates with multiple external systems using standardized patterns that ensure reliability, security, and maintainability across all integration points.

Integration Architecture Overview:



6.3.3.2 Figma Integration Implementation

The Figma API supports read access and interactions with Figma files, providing the ability to view and extract any objects or layers and their properties.

Figma Integration Architecture:

```

interface FigmaIntegrationService {
    // File operations
    getFileMetadata(fileKey: string): Promise<FigmaFileMetadata>
    getFileImages(fileKey: string, options: ImageOptions):
    Promise<FigmaImages>
    syncFileChanges(fileKey: string): Promise<SyncResult>

```

```

// Comment operations
getComments(fileKey: string): Promise<FigmaComment[]>
postComment(fileKey: string, comment: CommentData):
Promise<FigmaComment>
syncComments(fileKey: string): Promise<CommentSyncResult>

// Webhook management
createWebhook(config: WebhookConfig): Promise<FigmaWebhook>
handleWebhook(payload: FigmaWebhookPayload): Promise<void>
validateWebhookSignature(payload: string, signature: string):
boolean
}

```

Figma Webhook Processing:

Figma webhooks support various event types like FILE_UPDATE and COMMENT, requiring proper configuration with event_type, team_id, endpoint, and passcode.

```

// Figma webhook handler implementation
@Controller('webhooks/figma')
export class FigmaWebhookController {
  @Post()
  async handleWebhook(
    @Body() payload: FigmaWebhookPayload,
    @Headers('x-figma-signature') signature: string
  ) {
    // Validate webhook signature
    if (!this.validateSignature(payload, signature)) {
      throw new UnauthorizedException('Invalid webhook signature')
    }

    // Process different event types
    switch (payload.event_type) {
      case 'FILE_UPDATE':
        await this.handleFileUpdate(payload)
        break
      case 'COMMENT':
        await this.handleCommentUpdate(payload)
        break
    }
  }
}

```

```
    case 'FILE_VERSION_UPDATE':  
      await this.handleVersionUpdate(payload)  
      break  
    }  
  
    return { status: 'processed' }  
  }  
}
```

6.3.3.3 GitHub/GitLab Integration Patterns

Development platform integrations enable automated workflow synchronization between project management and code repositories.

Git Platform Integration Matrix:

Feature	GitHub Integration	GitLab Integration	Implementation Pattern
Repository Sync	GitHub Apps API	GitLab System Hooks	Webhook-based updates
PR/MR Tracking	Pull Request API	Merge Request API	Status synchronization
Issue Management	Issues API	Issues API	Bi-directional sync
Deployment Tracking	Deployments API	Deployments API	Status monitoring

GitHub Integration Implementation:

```
interface GitHubIntegrationService {  
  // Repository management  
  connectRepository(projectId: string, repoConfig: RepoConfig):  
    Promise<Connection>  
  syncRepositoryData(repoId: string): Promise<SyncResult>  
  
  // Pull request integration  
  handlePullRequestEvent(webhook: PRWebhookPayload): Promise<void>  
  capturePreviewURL(prId: string): Promise<PreviewCapture>  
}
```

```
// Issue synchronization
syncIssuesWithTasks(repoId: string): Promise<IssueSyncResult>
createIssueFromTask(taskId: string): Promise<GitHubIssue>

// Deployment tracking
trackDeploymentStatus(deploymentId: string):
Promise<DeploymentStatus>
}
```

6.3.3.4 Payment Processing Integration

Stripe integration handles all financial transactions with comprehensive webhook processing for payment events.

Stripe Integration Architecture:

```
interface StripeIntegrationService {
  // Payment processing
  processPayment(paymentData: PaymentRequest): Promise<PaymentResult>
  createPaymentIntent(amount: number, currency: string):
Promise<PaymentIntent>

  // Invoice management
  createInvoice(invoiceData: InvoiceData): Promise<StripeInvoice>
  sendInvoice(invoiceId: string): Promise<void>

  // Webhook handling
  handleWebhook(payload: StripeWebhookPayload, signature: string):
Promise<void>
  validateWebhookSignature(payload: string, signature: string):
boolean

  // Subscription management
  createSubscription(customerId: string, priceId: string):
Promise<Subscription>
  updateSubscription(subscriptionId: string, updates:
SubscriptionUpdate): Promise<Subscription>
}
```

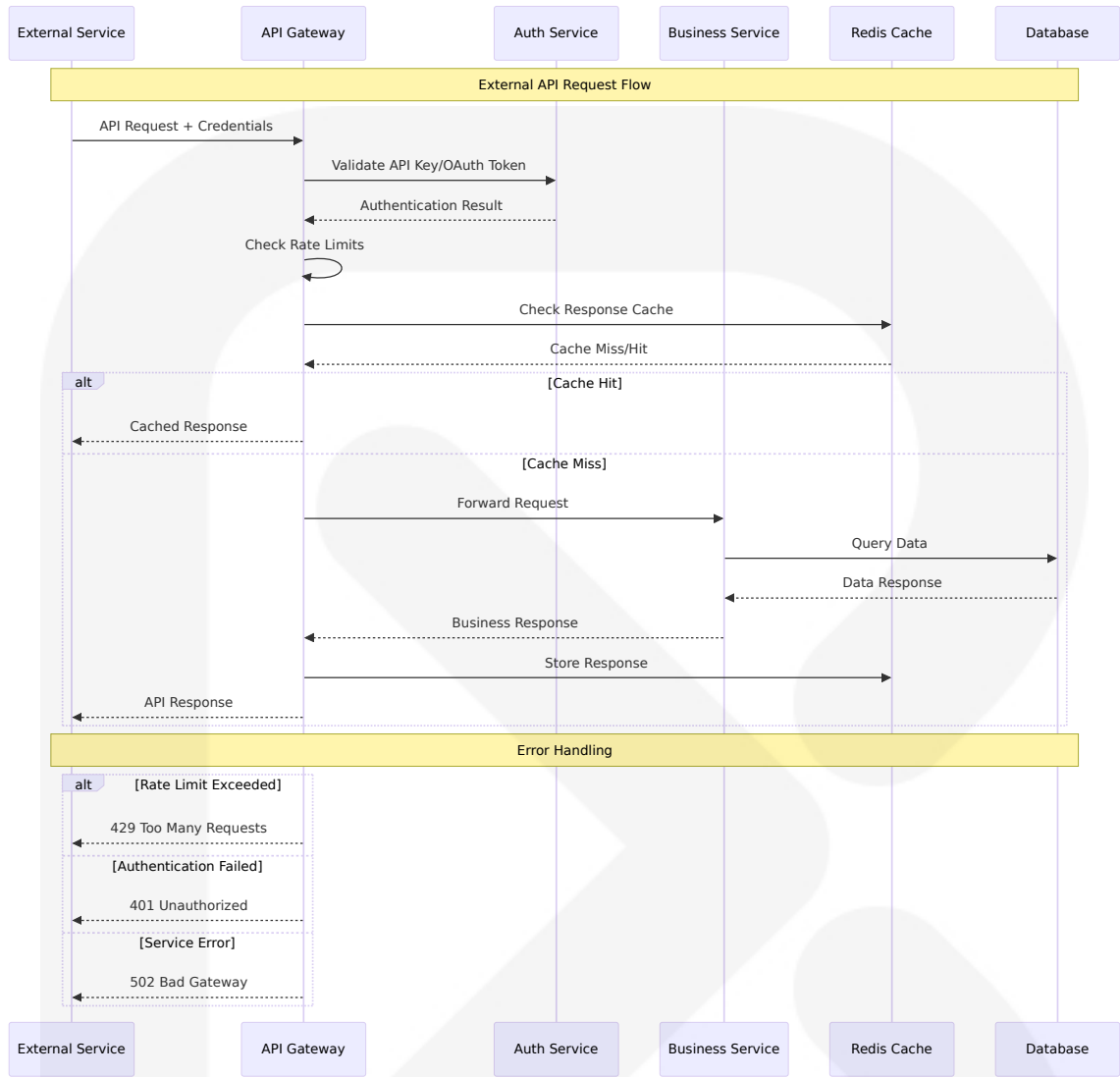
Stripe Webhook Event Processing:

Event Type	Processing Action	Business Impact	Error Handling
payment_intent.succeeded	Update invoice status, send confirmation	Revenue recognition	Retry with exponential backoff
payment_intent.payment_failed	Mark payment failed, send notification	Collections workflow	Alert finance team
invoice.payment_succeeded	Update project billing, trigger next milestone	Cash flow improvement	Sync with accounting system
customer.subscription.updated	Update client subscription status	Service level changes	Validate subscription limits

6.3.3.5 API Gateway Configuration

The API Gateway serves as the central integration hub, managing authentication, rate limiting, and request routing for all external service communications.

Gateway Architecture:



Gateway Configuration:

Configurati on	Value	Purpose	Implementatio n
Request Ti meout	30 seconds	Prevent hangin g connections	NestJS timeout i nterceptor
Rate Limiti ng	1000 req/min p er API key	Prevent abuse	Redis-based slidi ng window
Response C aching	5 minutes for G ET requests	Improve perfor mance	Redis cache with TTL
Circuit Bre aker	5 failures trigge r open state	Prevent cascad e failures	Hystrix pattern i mplementation

6.3.3.6 External Service Contracts

AgencyOS maintains formal service contracts with all external integrations to ensure reliability and manage dependencies.

Service Level Agreements:

Service Provider	Availability SLA	Response Time SLA	Rate Limits	Support Level
Figma	99.9% uptime	<500ms p95	1000 req/min	Business support
GitHub	99.95% uptime	<200ms p95	5000 req/hour	Enterprise support
Stripe	99.99% uptime	<100ms p95	No published limits	24/7 support
Slack	99.99% uptime	<300ms p95	1 req/second	Standard support

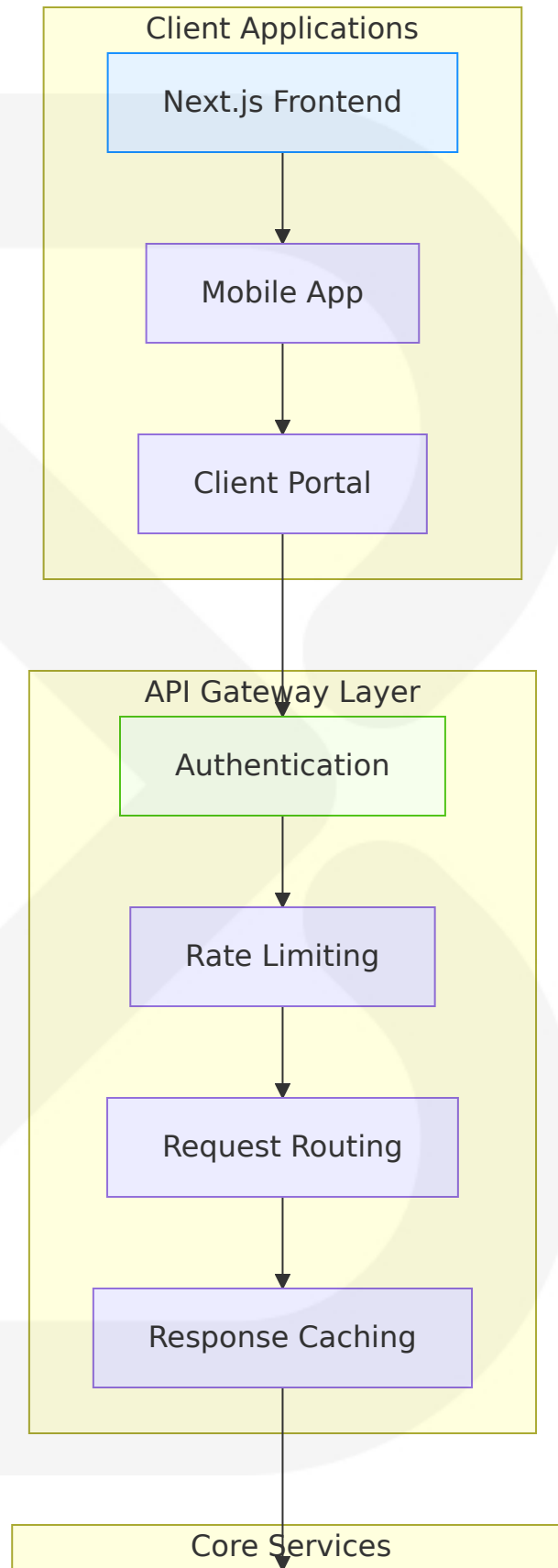
Integration Health Monitoring:

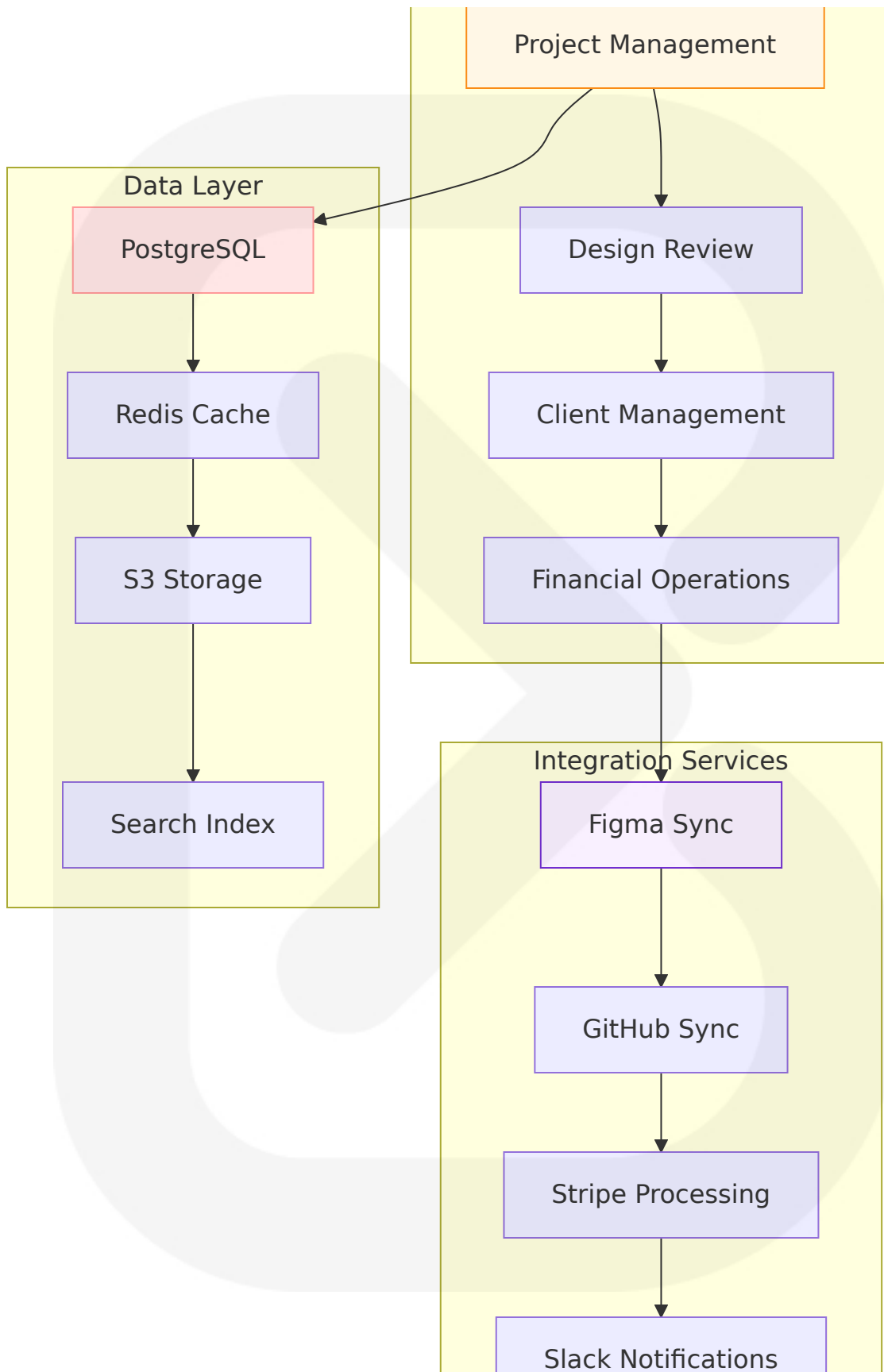
```
interface IntegrationHealthService {  
  // Health check implementation  
  checkServiceHealth(serviceId: string): Promise<HealthStatus>  
  
  // SLA monitoring  
  trackResponseTimes(serviceId: string, responseTime: number):  
  Promise<void>  
  calculateAvailability(serviceId: string, period: TimePeriod):  
  Promise<number>  
  
  // Alert management  
  triggerSLAAlert(serviceId: string, violation: SLAViolation):  
  Promise<void>  
  escalateServiceIssue(serviceId: string, severity: Severity):  
  Promise<void>  
}
```

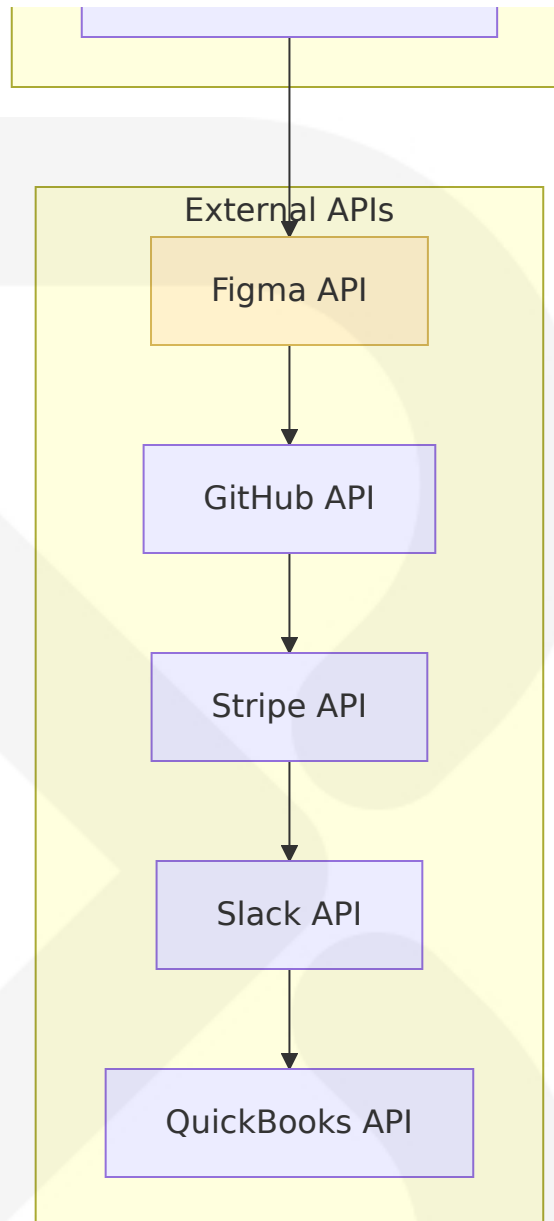
6.3.4 Integration Flow Diagrams

6.3.4.1 Complete Integration Architecture

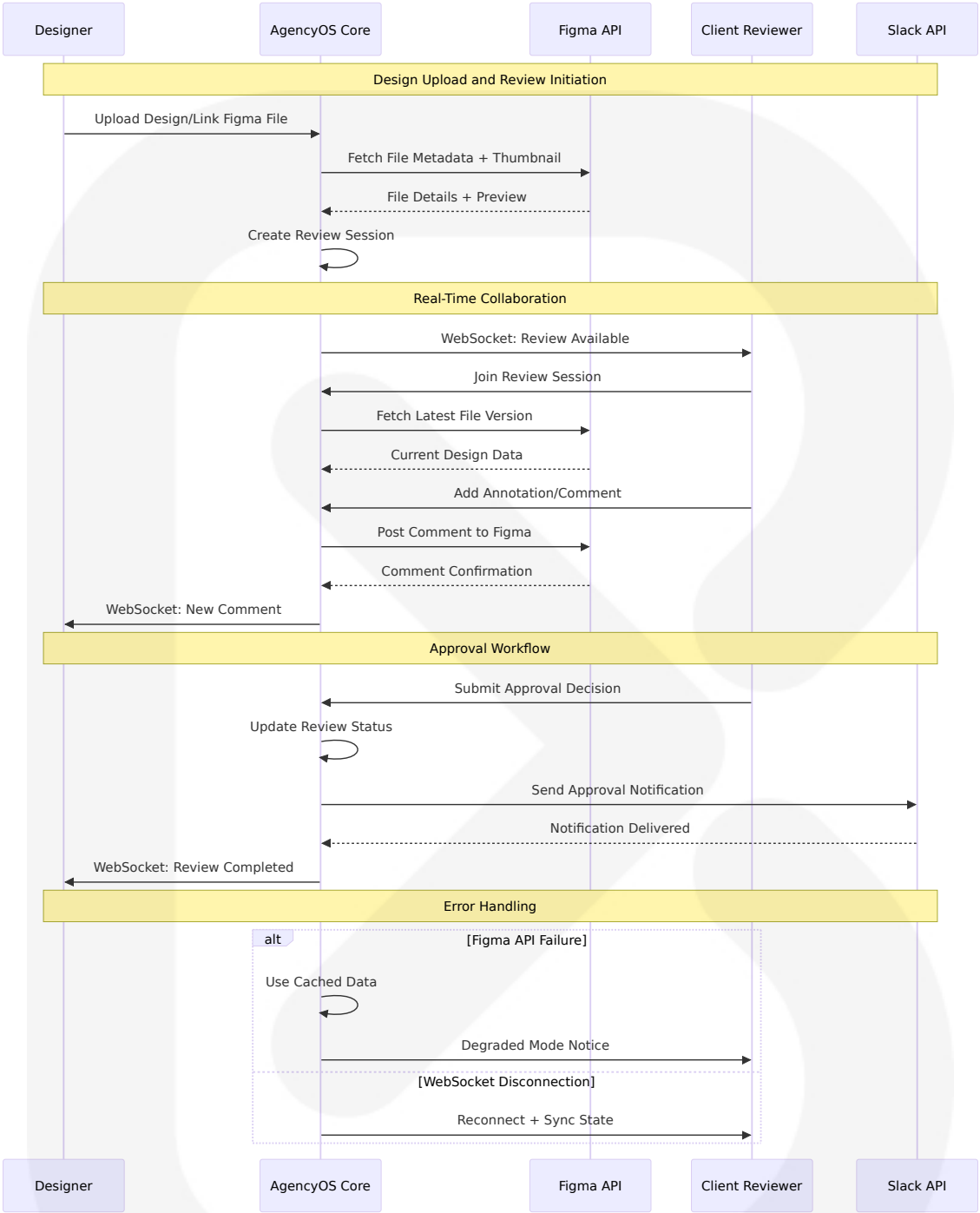




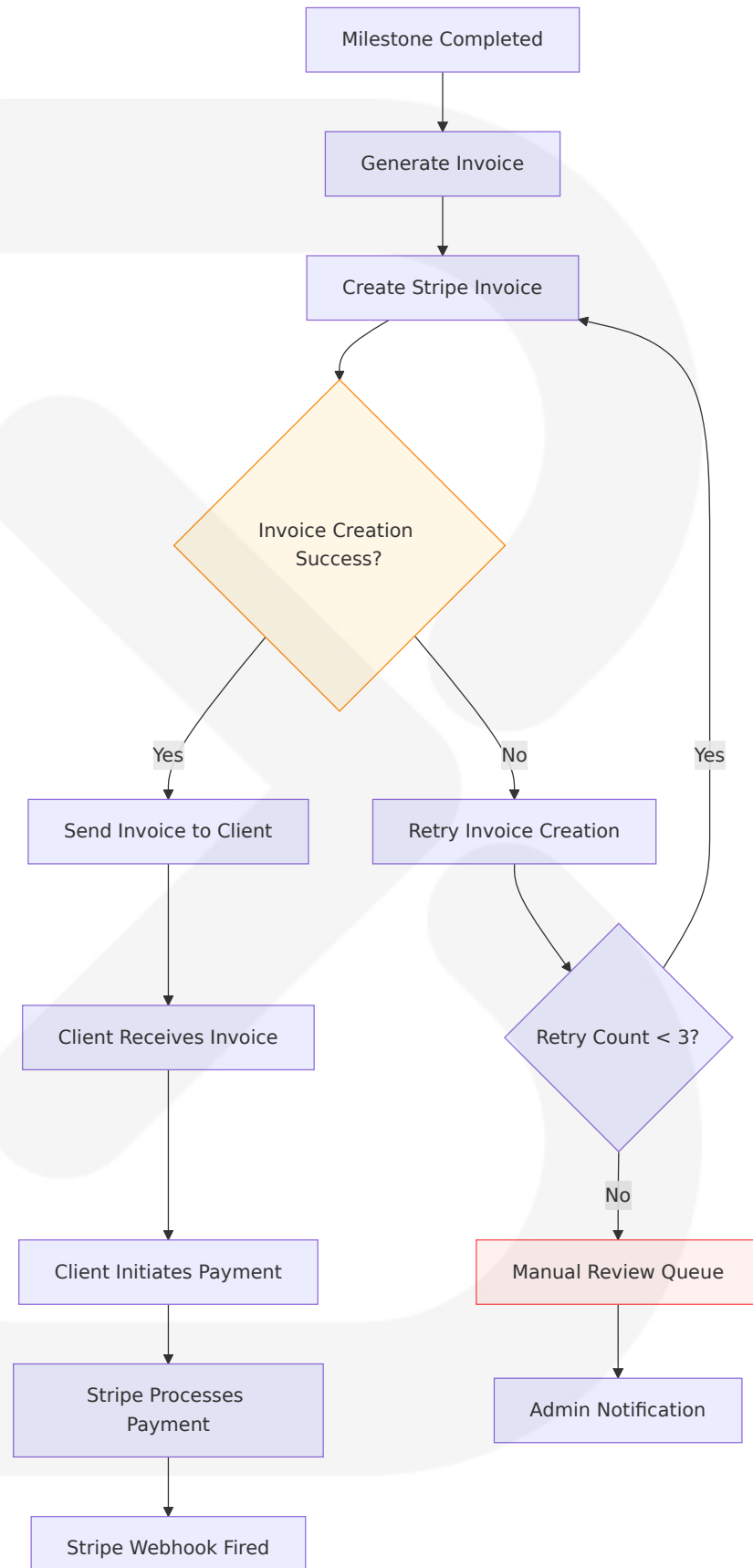


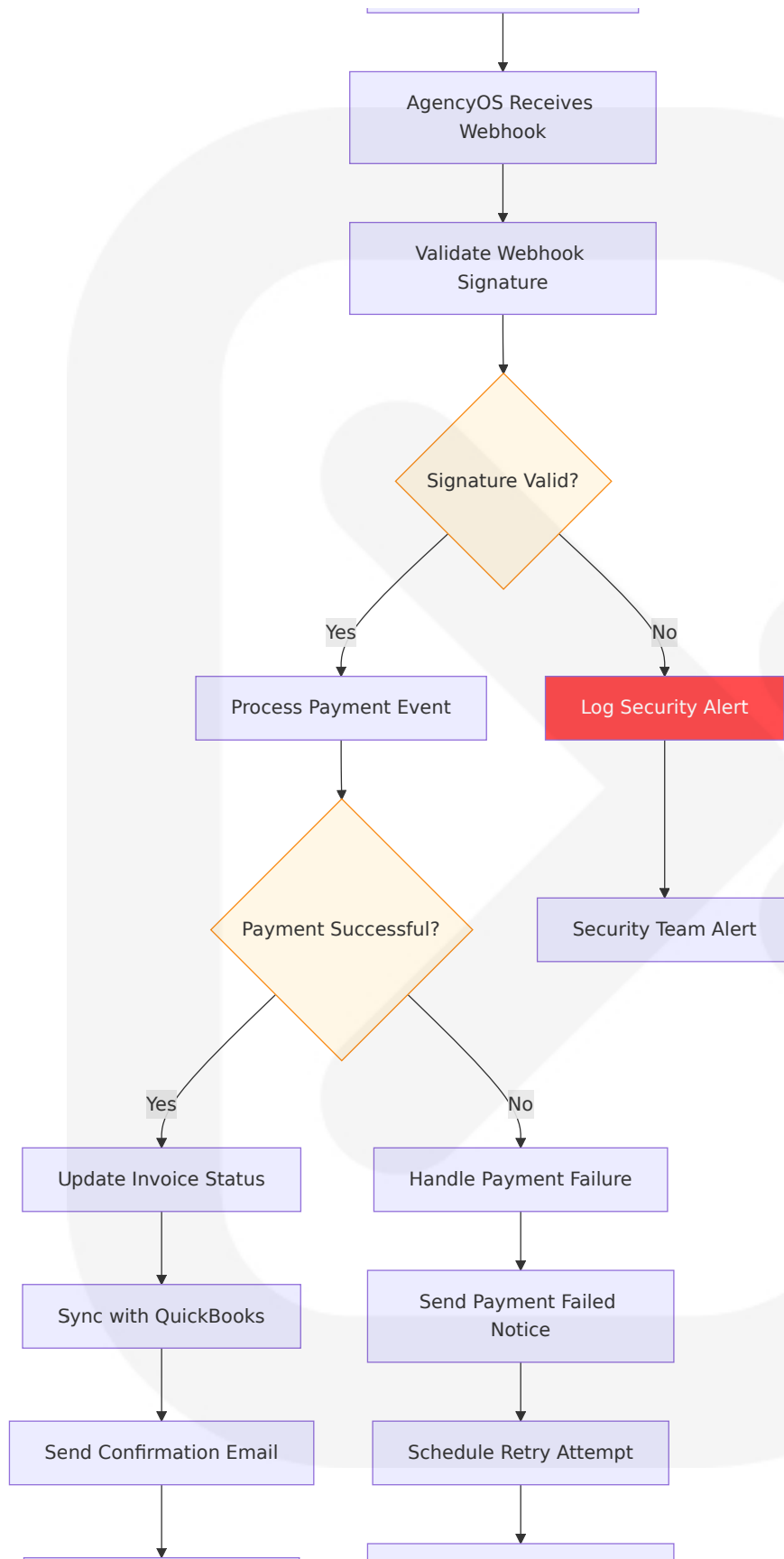


6.3.4.2 Real-Time Design Review Integration



6.3.4.3 Payment Processing Integration Flow





Update Project Status

Update Collections
Queue

This comprehensive Integration Architecture ensures AgencyOS can seamlessly connect with all required external systems while maintaining high reliability, security, and performance standards. The event-driven design patterns and robust error handling mechanisms provide the foundation for a scalable, maintainable integration ecosystem that supports the complex workflows required by modern digital agencies.

6.4 Security Architecture

6.4.1 Authentication Framework

6.4.1.1 Identity Management

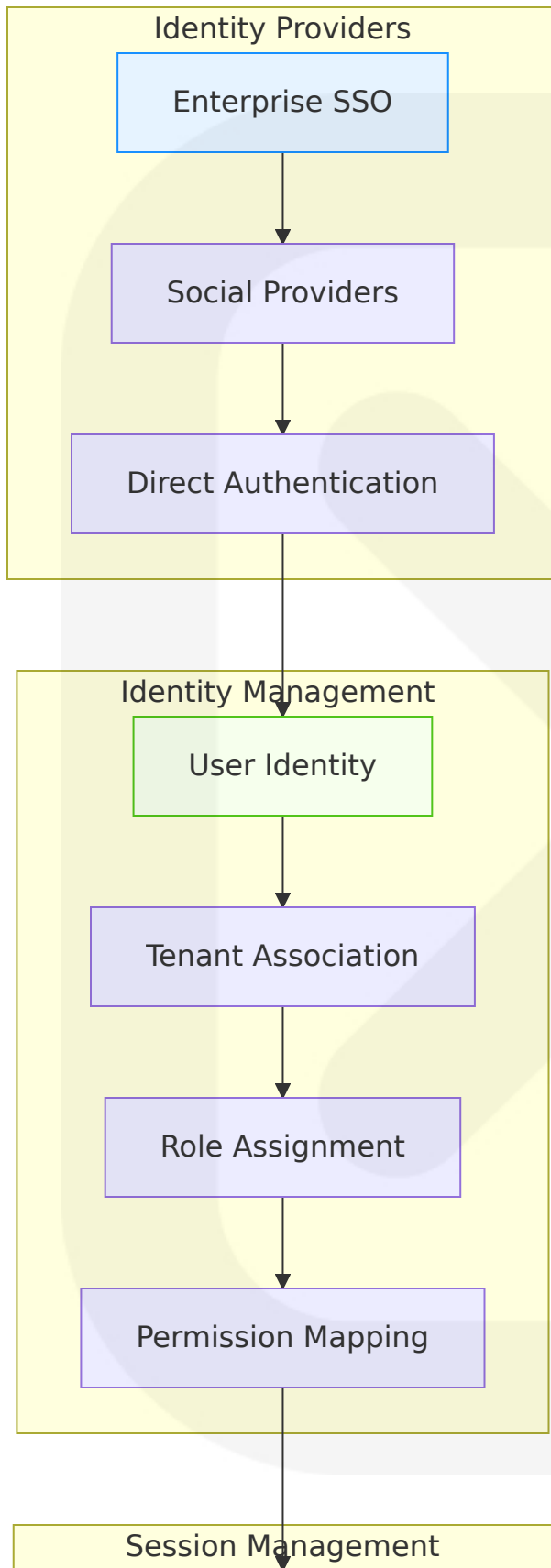
AgencyOS implements a comprehensive identity management system designed to support both internal agency users and external client organizations while maintaining strict tenant isolation and security controls.

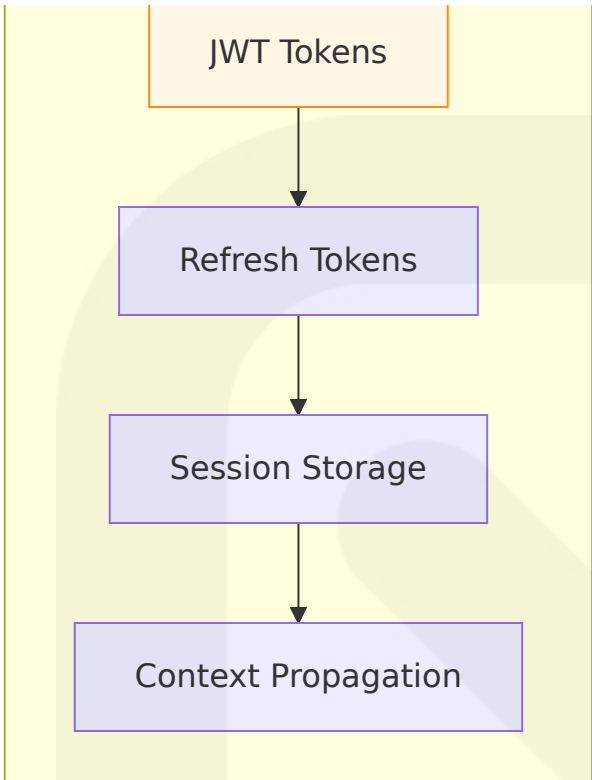
Identity Provider Integration:

Provider Type	Implementation	Use Case	Authentication Method
Enterprise SSO	OIDC and SAML support for enterprise identity providers like Okta, Auth0, Google Workspace, and Azure AD	Agency internal users	OIDC uses JSON-based web tokens (JWT) with industry standard claims for encrypted, sensitive user data
Social Identity	OAuth 2.0 + OIDC	Client user onboarding	OpenID Connect for customer-facing apps offering secure login via social providers or external IdPs, great for modern CIAM

Provider Type	Implementation	Use Case	Authentication Method
Direct Authentication	Username/password + MFA	Fallback authentication	Local credential storage with bcrypt hashing

Identity Data Model:





User Provisioning Workflow:

Provisioning Type	Trigger	Process	Deprovisioning
SCIM Automated	HR system integration	Real-time user creation with role assignment	Automatic account deactivation on termination
Just-in-Time (JIT)	First SSO login	Dynamic user creation from IdP claims	Manual deprovisioning required
Manual Invitation	Admin-initiated	Email invitation with role pre-assignment	Admin-controlled removal

6.4.1.2 Multi-Factor Authentication

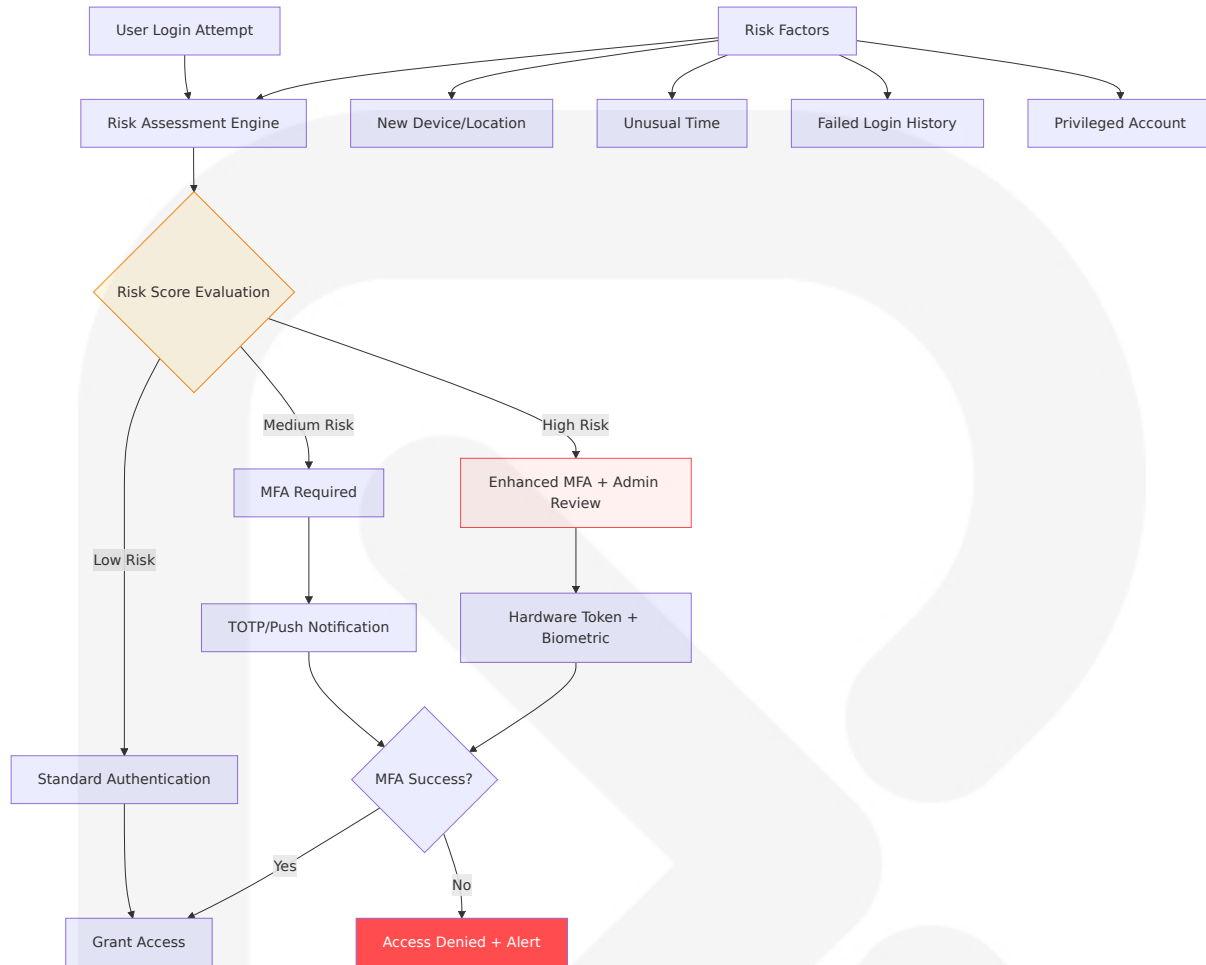
MFA blocks up to 99.9% of automated attacks and makes users 99% less likely to be hacked, making it a critical security control for AgencyOS's multi-tenant environment.

MFA Implementation Strategy:

Authenti cation Fa ctor	Technology	Use Case	Security Level
Knowled ge Facto r	Password + PIN	Primary au thenticatio n	Standard
Possessi on Facto r	Microsoft Authenticator app pr oviding the best user experien ce with multiple modes includi ng passwordless, MFA push not ifications, and OATH codes	Secondary au thentica tion	High
Inherenc e Factor	Biometric authentication	Device-ba sed authe ntication	Very Hig h
Phishing -Resista nt	FIDO authenticators paired wit h W3C's Web Authentication A PI as the most common form o f phishing resistant authenticat ors, available as separate hard ware keys or embedded directl y into platforms	Administra tive acces s	Maximu m

Risk-Based MFA Implementation:

Risk-based authentication reduces the frequency of MFA prompts by only requiring MFA when the user is performing high-risk actions, such as logging in from a new device or location, or from locations considered high risk.



MFA Enforcement Policies:

User Role	MFA Requirement	Acceptable Methods	Bypass Conditions
Super Admin	Always required	Phishing-resistant authentication for users with elevated privileges	Emergency break-glass only
Admin/PM	Required for sensitive operations	Authenticator app, hardware token	Trusted device for 24 hours
Team Members	Risk-based enforcement	Authenticator app, SMS backup	Corporate network exemption
Client Users	Required for approvals	Authenticator app, email OTP	None

6.4.1.3 Session Management

AgencyOS implements secure session management with JWT-based authentication and comprehensive session lifecycle controls.

JWT Token Architecture:

Token Type	Expiration	Refresh Strategy	Storage Location
Access Token	15 minutes	Automatic refresh	Memory only
Refresh Token	7 days	Rotation on use	HTTP-only cookie
ID Token	15 minutes	Not refreshed	Memory only

Session Security Controls:

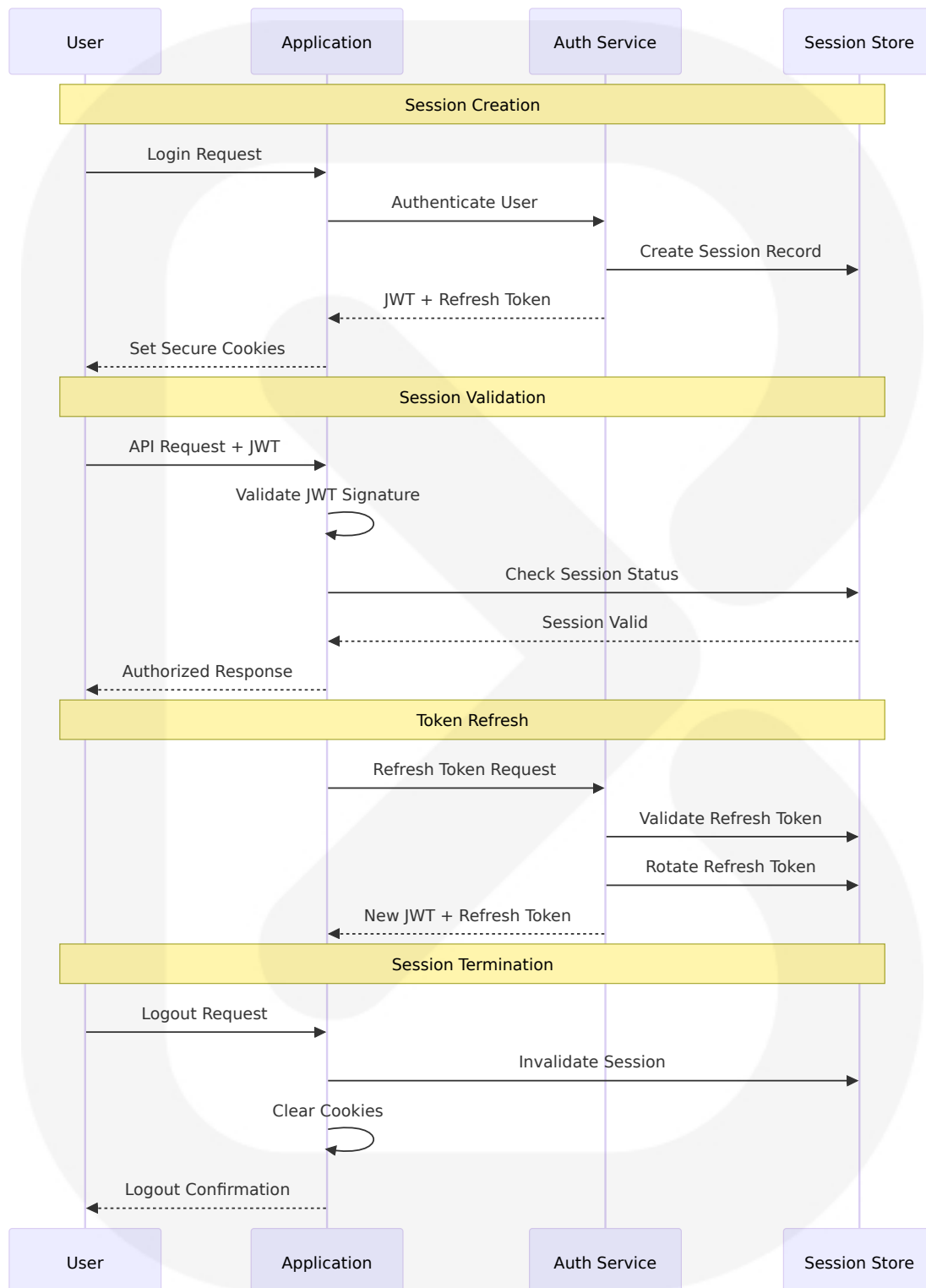
```
interface SessionSecurityConfig {
  // Token configuration
  accessTokenExpiry: '15m'
  refreshTokenExpiry: '7d'
  refreshTokenRotation: true

  // Session controls
  maxConcurrentSessions: 5
  sessionTimeout: '24h'
  slidingExpiration: true

  // Security headers
  secureFlag: true
  httpOnlyFlag: true
  sameSitePolicy: 'strict'

  // Multi-tenant context
  tenantIsolation: true
  crossTenantPrevention: true
}
```


Session Lifecycle Management:



6.4.1.4 Token Handling

JWT Security Implementation:

Security Aspect	Implementation	Validation
Signature Algorithm	RS256 (RSA with SHA-256)	Public key verification
Token Claims	Standard + custom claims	Audience, issuer, expiration validation
Key Rotation	Monthly automatic rotation	Graceful key transition support

Token Validation Process:

```
interface JWTValidationService {  
  // Core validation  
  validateSignature(token: string): Promise<boolean>  
  validateClaims(token: JWT): Promise<ClaimValidation>  
  checkExpiration(token: JWT): boolean  
  
  // Security checks  
  validateAudience(token: JWT, expectedAudience: string): boolean  
  validateIssuer(token: JWT, trustedIssuers: string[]): boolean  
  checkTokenBlacklist(jti: string): Promise<boolean>  
  
  // Multi-tenant validation  
  validateTenantContext(token: JWT, requestTenant: string): boolean  
  enforceProjectAccess(token: JWT, projectId: string):  
  Promise<boolean>  
}
```

6.4.1.5 Password Policies

Password Requirements:

Policy Element	Requirement	Enforcement
Minimum Length	12 characters	Client-side + server validation

Policy Element	Requirement	Enforcement
Complexity	Mixed case, numbers, symbols	Entropy-based scoring
History	Last 12 passwords remembered	Hashed comparison
Expiration	90 days for privileged accounts	Automated notifications

Password Security Implementation:

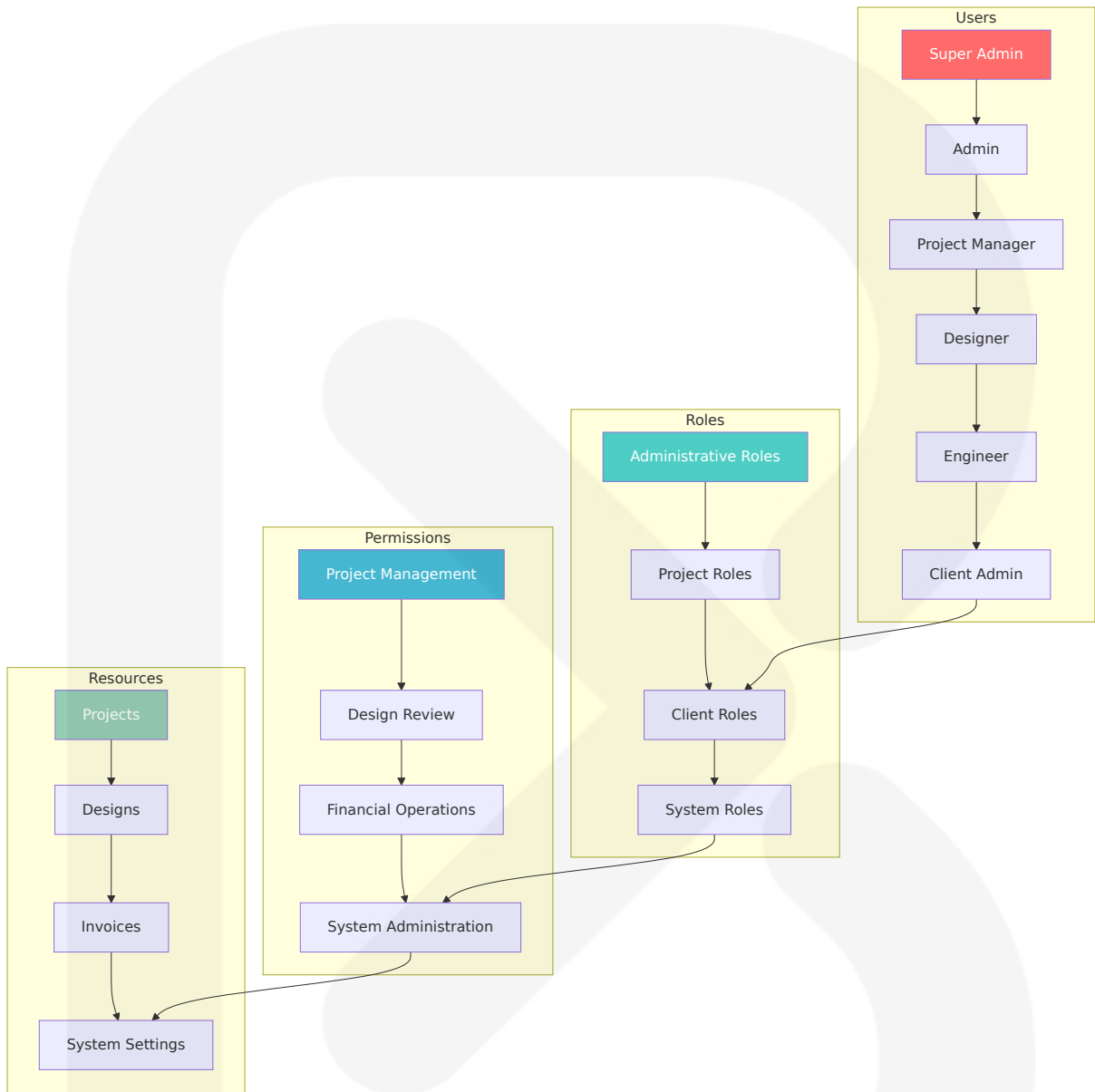
```
interface PasswordSecurityService {  
  // Hashing  
  hashPassword(password: string): Promise<string> // bcrypt with salt rounds 12  
  verifyPassword(password: string, hash: string): Promise<boolean>  
  
  // Policy enforcement  
  validatePasswordStrength(password: string): PasswordStrengthResult  
  checkPasswordHistory(userId: string, newPassword: string):  
  Promise<boolean>  
  enforcePasswordExpiry(userId: string): Promise<ExpiryStatus>  
  
  // Breach detection  
  checkCompromisedPassword(password: string): Promise<boolean>  
  generateSecurePassword(length: number): string  
}
```

6.4.2 Authorization System

6.4.2.1 Role-Based Access Control

Role-Based Access Control (RBAC) is a policy-neutral access control mechanism defined around roles and privileges. Rather than assigning permissions to individual users, RBAC assigns permissions to roles, and then users are assigned to those roles, simplifying the management of user permissions and improving security.

RBAC Architecture:



Role Hierarchy and Permissions:

Role Category	Roles	Key Permissions	Inheritance
Agency Internal	Super Admin, Admin, PM, Designer, Engineer, QA, Finance, Contractor	Full workspace access, project management, resource allocation	Hierarchical inheritance

Role Category	Roles	Key Permissions	Inheritance
Client Roles	Client Admin, Stakeholder/Reviewer, Billing Only, Viewer	Project visibility, approval workflows, invoice access	Role-specific permissions
System Roles	System Admin, Audit Viewer	Cross-tenant administration, compliance reporting	Elevated system access

Permission Matrix:

Resource	Super Admin	Admin	PM	Designer	Engineer	CA
Create Project	✓	✓	✓	X	X	X
Upload Design	✓	✓	✓	✓	X	X
Review Design	✓	✓	✓	✓	✓	✓
Approve Design	✓	✓	✓	X	X	✓

6.4.2.2 Permission Management

Dynamic Permission Evaluation:

```
interface PermissionEvaluationService {
  // Core permission checks
  hasPermission(
    userId: string,
    resource: string,
    action: string,
    context?: PermissionContext
  ): Promise<boolean>

  // Role-based evaluation
```

```

getUserRoles(userId: string, tenantId: string): Promise<Role[]>
getRolePermissions(roleId: string): Promise<Permission[]>

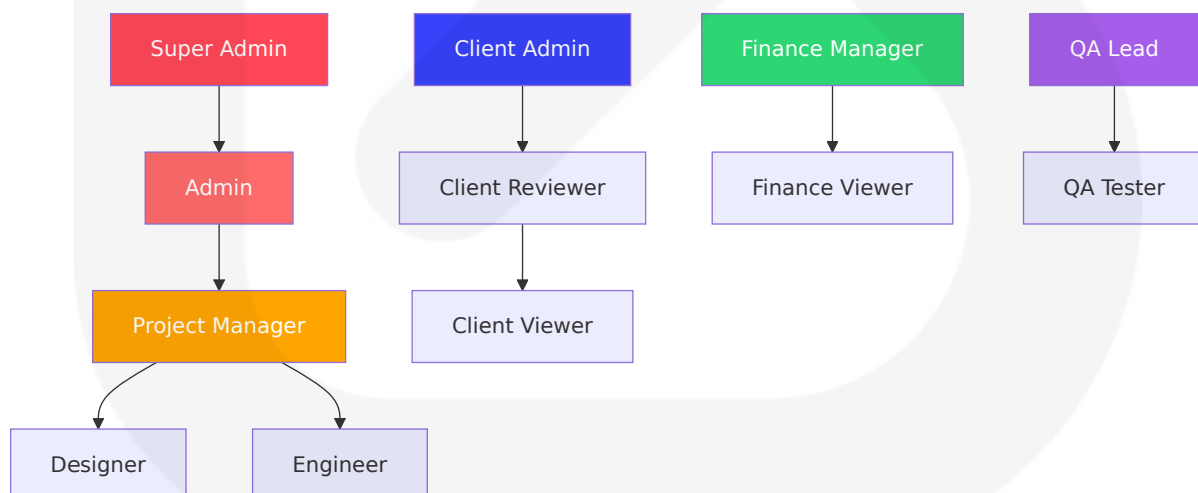
// Project-level overrides
getProjectPermissions(userId: string, projectId: string):
Promise<Permission[]>
hasProjectAccess(userId: string, projectId: string):
Promise<boolean>

// Attribute-based evaluation
evaluateAttributePolicy(
  user: User,
  resource: Resource,
  action: string,
  environment: Environment
): Promise<PolicyDecision>
}

```

Permission Inheritance Model:

Authorization systems define permissions as first-class concepts, allowing roles to include other roles. For example, the "editor" role can include the "viewer" role, with the "editor" relation type including all permissions from the "viewer" relation type and adding additional permissions.

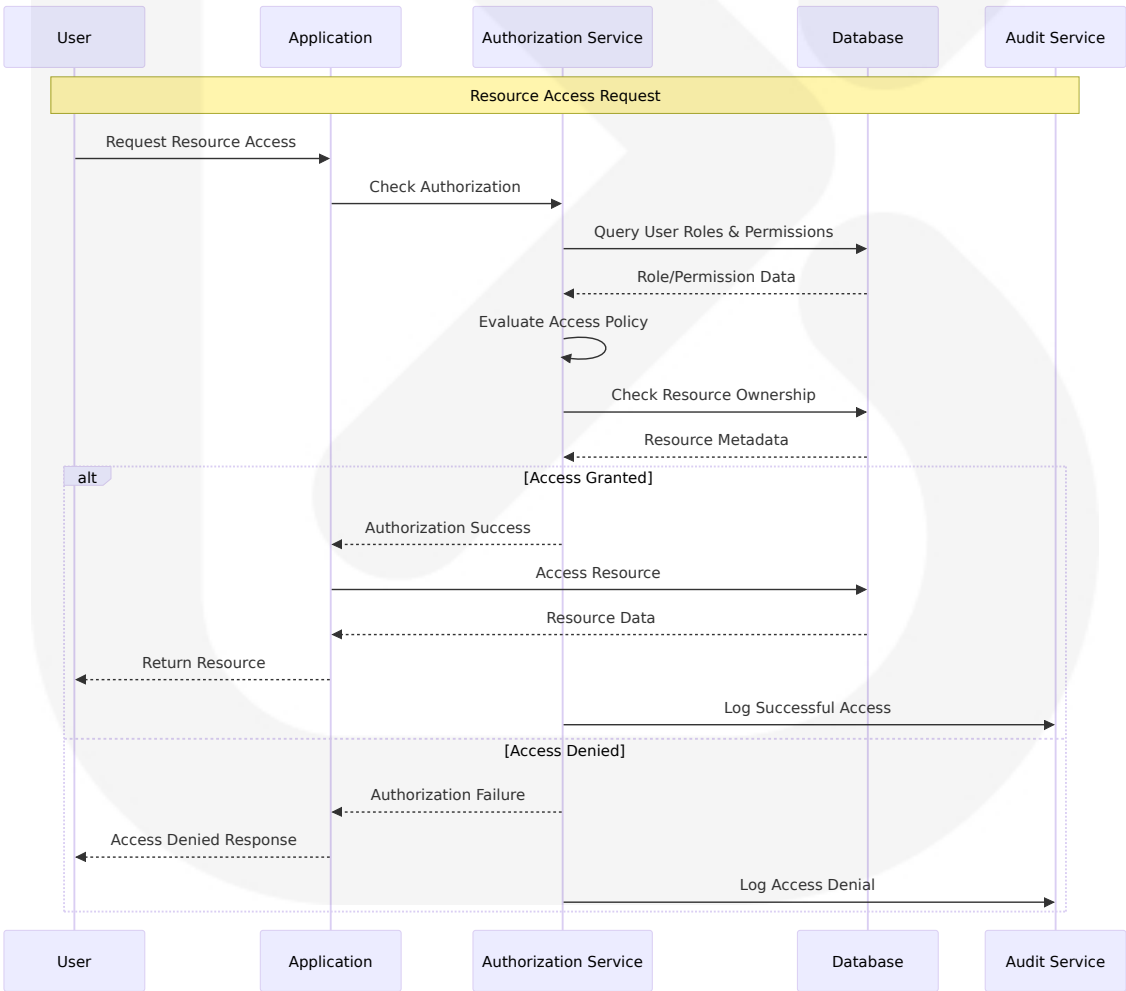


6.4.2.3 Resource Authorization

Multi-Tenant Resource Access Control:

Resource Type	Access Pattern	Tenant Isolation	Project Scoping
Projects	Tenant + project-level	Row-level security	User assignment required
Design Files	Project-based	S3 bucket prefixes	Version-controlled access
Financial Data	Role + tenant restricted	Encrypted at rest	Audit trail required
System Settings	Admin-only	Global or tenant-scoped	Configuration-dependent

Resource Authorization Flow:



6.4.2.4 Policy Enforcement Points

Authorization Enforcement Architecture:

Enforcement Point	Technology	Scope	Performance
API Gateway	NestJS Guards	All API endpoints	<50ms overhead
Database Layer	PostgreSQL RLS	Data access	Transparent
Frontend Components	React Context	UI element visibility	Client-side only
File Storage	S3 Bucket Policies	Asset access	Pre-signed URLs

Policy Decision Point Implementation:

```
interface PolicyDecisionPoint {  
  // Policy evaluation  
  evaluatePolicy(  
    subject: Subject,  
    resource: Resource,  
    action: Action,  
    environment: Environment  
  ): Promise<PolicyDecision>  
  
  // Policy management  
  createPolicy(policy: Policy): Promise<PolicyResult>  
  updatePolicy(policyId: string, updates: PolicyUpdate):  
    Promise<PolicyResult>  
  deletePolicy(policyId: string): Promise<void>  
  
  // Policy testing  
  testPolicy(policy: Policy, testCases: TestCase[]):  
    Promise<TestResult[]>  
  validatePolicy(policy: Policy): Promise<ValidationResult>  
}
```


6.4.2.5 Audit Logging

Comprehensive Authorization Audit Trail:

Event Type	Logged Information	Retention Period	Access Controls
Access Granted	User, resource, action, timestamp, IP	2 years	Admin + Audit roles
Access Denied	User, resource, action, reason, timestamp	2 years	Admin + Audit roles
Role Changes	Admin, target user, old/new roles, timestamp	7 years	Super Admin only
Permission Updates	Admin, permission changes, affected users	7 years	Super Admin only

Audit Log Schema:

```
interface AuthorizationAuditLog {  
  id: string  
  timestamp: Date  
  tenantId: string  
  userId: string  
  action: string  
  resource: {  
    type: string  
    id: string  
    attributes?: Record<string, any>  
  }  
  decision: 'PERMIT' | 'DENY'  
  reason?: string  
  context: {  
    ipAddress: string  
    userAgent: string  
    sessionId: string  
    requestId: string  
  }  
  metadata?: Record<string, any>  
}
```

6.4.3 Data Protection

6.4.3.1 Encryption Standards

AgencyOS implements comprehensive encryption standards following current best practices and regulatory requirements for data protection.

Encryption Algorithm Selection:

AES-256 encryption is extremely secure and is the most secure encryption algorithm available today, used extensively in government and military applications. The encryption has a key size of 256 bits, which is considered virtually uncrackable—even with the most advanced computing power and algorithms.

Data State	Algorithm	Key Size	Implementation
Data at Rest	AES-256 for all stored data, with some legacy systems using AES-128	256-bit	Database encryption, file storage
Data in Transit	TLS 1.2+ with AES-256-GCM	256-bit	HTTPS, API communications
Application Layer	AES-256-GCM	256-bit	Sensitive field encryption
Backup Data	AES-256 with independent DEK derived from Keystore and randomly generated per-file seed	256-bit	Backup encryption

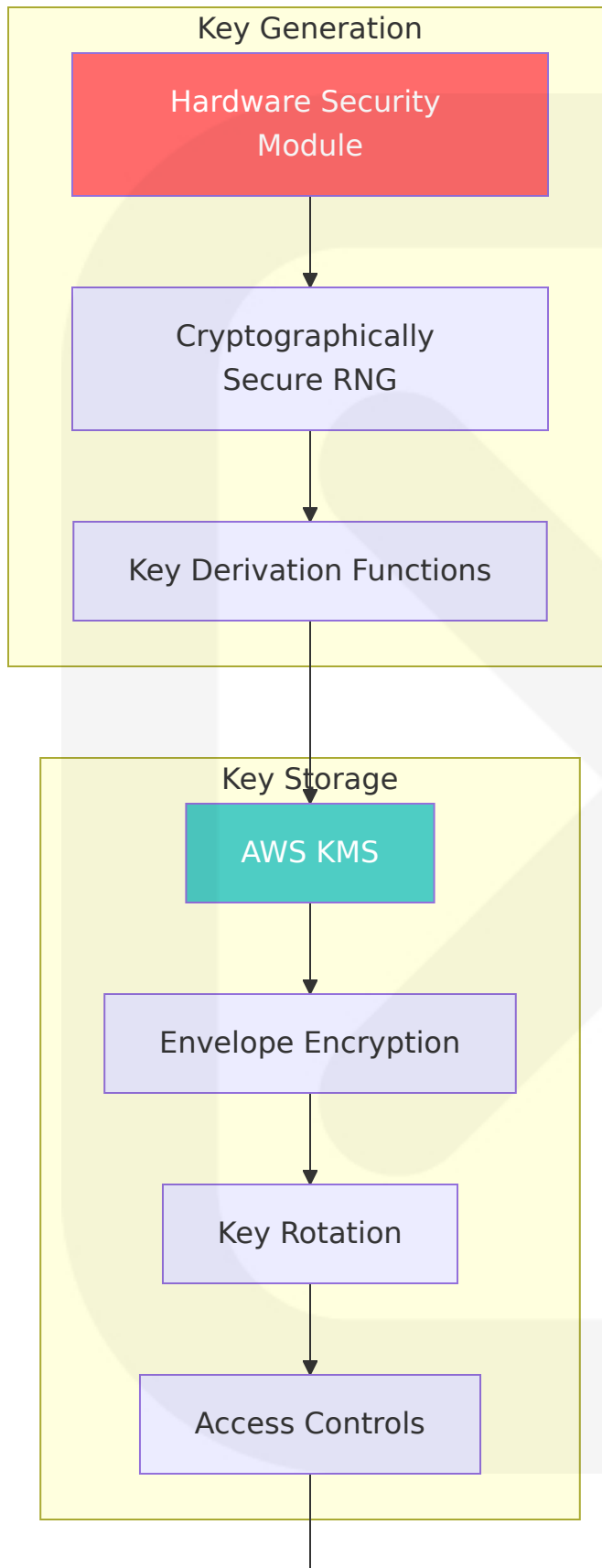
Quantum-Resistant Preparation:

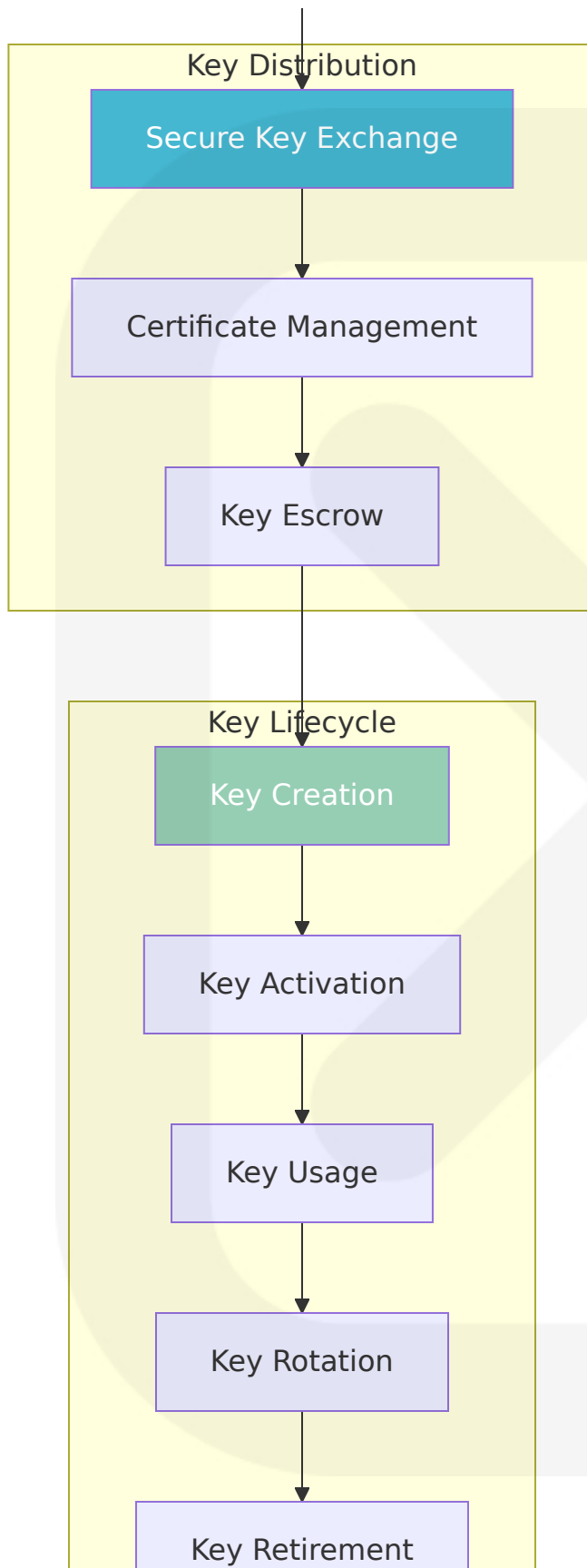
The impact of quantum attacks on symmetric cryptographic algorithms such as AES is unlikely to be felt for some time. However, for interoperability reasons, new cryptographic equipment and applications intended for use beyond 2030 should support AES-256.

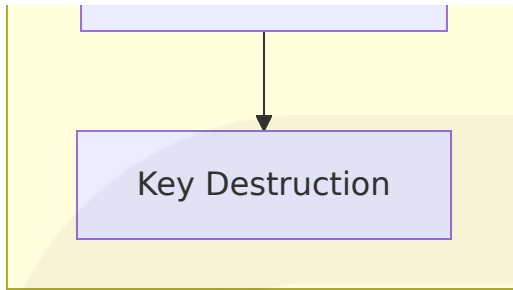
6.4.3.2 Key Management

Key Management Architecture:









Key Management Policies:

Key Type	Rotation Frequency	Storage Location	Access Controls
Master Keys	Annually	AWS KMS	Super Admin only
Data Encryption Keys	Monthly	Encrypted in database	System service accounts
JWT Signing Keys	Monthly	Secure key store	Authentication service
API Keys	On compromise	Encrypted configuration	Service-specific access

Envelope Encryption Implementation:

Data Encryption Keys (DEKs) are stored near the data they encrypt and are encrypted with (wrapped by) a Key Encryption Key (KEK), using envelope encryption technique.

```
interface KeyManagementService {  
  // Key generation  
  generateDataEncryptionKey(): Promise<EncryptionKey>  
  generateKeyEncryptionKey(): Promise<EncryptionKey>  
  deriveKey(masterKey: string, context: string): Promise<DerivedKey>  
  
  // Key operations  
  encryptData(data: Buffer, key: EncryptionKey):  
    Promise<EncryptedData>  
  decryptData(encryptedData: EncryptedData, key: EncryptionKey):  
    Promise<Buffer>  
  wrapKey(dek: EncryptionKey, kek: EncryptionKey): Promise<WrappedKey>
```

```
unwrapKey(wrappedKey: WrappedKey, kek: EncryptionKey):  
Promise<EncryptionKey>
```

```
// Key lifecycle  
rotateKey(keyId: string): Promise<KeyRotationResult>  
revokeKey(keyId: string): Promise<void>  
destroyKey(keyId: string): Promise<void>  
}
```

6.4.3.3 Data Masking Rules

Data Classification and Masking:

Data Classification	Masking Strategy	Implementation	Use Cases
PII (Personal)	Format-preserving encryption	AES-256-FF1	Names, addresses, phone numbers
Financial	Tokenization	Vault-based tokens	Credit cards, bank accounts
Sensitive Business	Dynamic masking	Role-based revelation	Project budgets, contracts
Public	No masking required	Plain text storage	Marketing materials, public docs

Data Masking Implementation:

```
interface DataMaskingService {  
  // Masking operations  
  maskPII(data: string, maskingType: MaskingType): string  
  maskFinancialData(data: string): string  
  dynamicMask(data: any, userRole: string): any  
  
  // Format-preserving encryption  
  encryptPreservingFormat(data: string, format: DataFormat): string  
  decryptPreservingFormat(encryptedData: string, format: DataFormat):  
  string  
  
  // Tokenization
```

```
tokenize(sensitiveData: string): Promise<Token>
detokenize(token: Token): Promise<string>

// Policy-based masking
applyMaskingPolicy(data: any, policy: MaskingPolicy): any
evaluateMaskingRules(data: any, userContext: UserContext):
MaskingRules
}
```

6.4.3.4 Secure Communication

Transport Layer Security:

Communication Type	Protocol	Cipher Suite	Certificate Management
Client-Server	TLS 1.2+	AES-256-GCM-SHA384	Let's Encrypt + Custom CA
Service-to-Service	mTLS	AES-256-GCM-SHA384	Internal PKI
Database Connections	TLS 1.2+	AES-256-GCM-SHA384	Certificate-based auth
File Transfers	SFTP/HTTPS	AES-256-GCM-SHA384	Pre-signed URLs

TLS Configuration:

```
interface TLSConfiguration {
  // Protocol settings
  minVersion: 'TLSv1.2'
  maxVersion: 'TLSv1.3'
  cipherSuites: [
    'TLS_AES_256_GCM_SHA384',
    'TLS_CHACHA20_POLY1305_SHA256',
    'TLS_AES_128_GCM_SHA256'
  ]

  // Certificate management
  certificateValidation: 'strict'
}
```



```
certificateRevocationCheck: true
ocspStapling: true

// Security headers
hsts: {
  maxAge: 31536000,
  includeSubDomains: true,
  preload: true
}

// Perfect forward secrecy
dhParams: 'rfc7919-ffdhe2048'
ecdhCurves: ['X25519', 'prime256v1']
}
```

6.4.3.5 Compliance Controls

Regulatory Compliance Framework:

Regulation	Scope	Controls	Audit Requirements
GDPR	EU personal data	Data minimization, consent, right to erasure	Annual compliance assessment
SOC 2 Type II	Security controls	Access controls, encryption, monitoring	Third-party audit
CCPA	California residents	Data transparency, opt-out rights	Self-assessment
HIPAA	Healthcare data	Administrative, physical, technical safeguards	Risk assessment

Data Retention and Deletion:

```
interface ComplianceService {
  // Data retention
  applyRetentionPolicy(dataType: string, tenantId: string):
  Promise<RetentionResult>
  scheduleDataDeletion(dataId: string, deletionDate: Date):
}
```

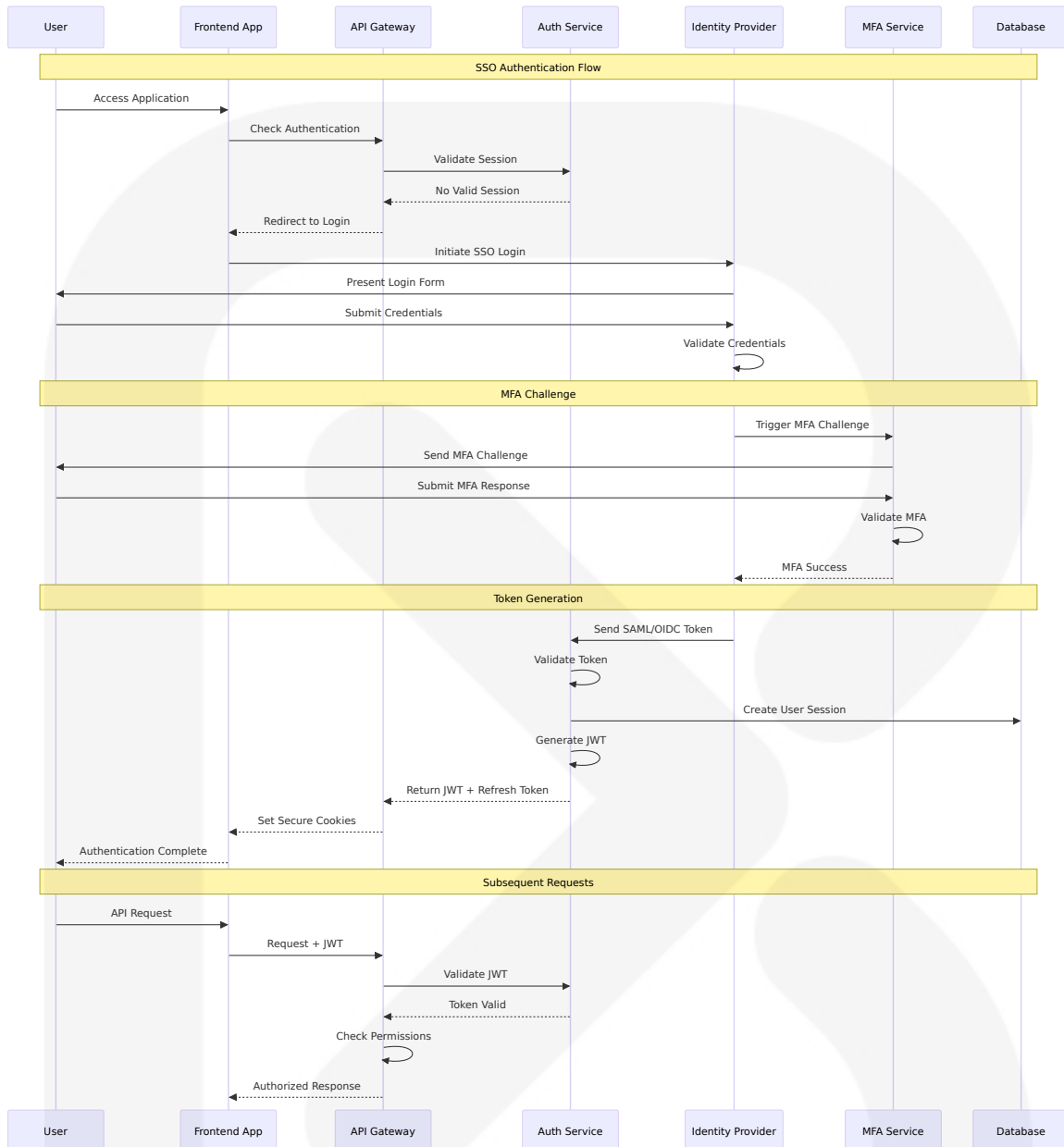
```
Promise<void>

  // GDPR compliance
  processDataSubjectRequest(request: DataSubjectRequest):
Promise<ComplianceResult>
    generateDataPortabilityExport(userId: string): Promise<DataExport>
    executeRightToErasure(userId: string): Promise<ErasureResult>

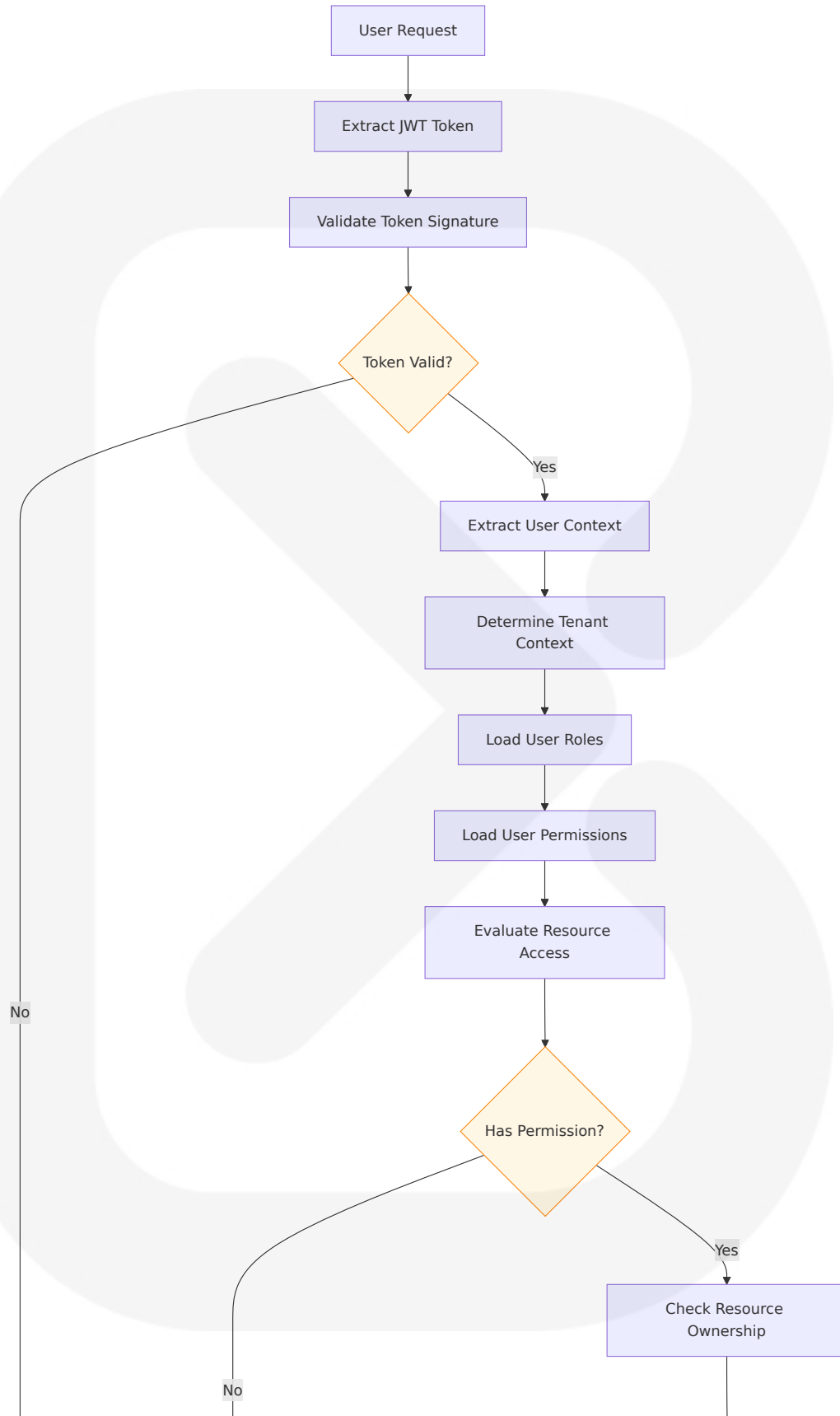
  // Audit and reporting
  generateComplianceReport(regulation: string, period: DateRange):
Promise<ComplianceReport>
    trackConsentStatus(userId: string): Promise<ConsentStatus>
    logDataProcessingActivity(activity: ProcessingActivity):
Promise<void>
}
```

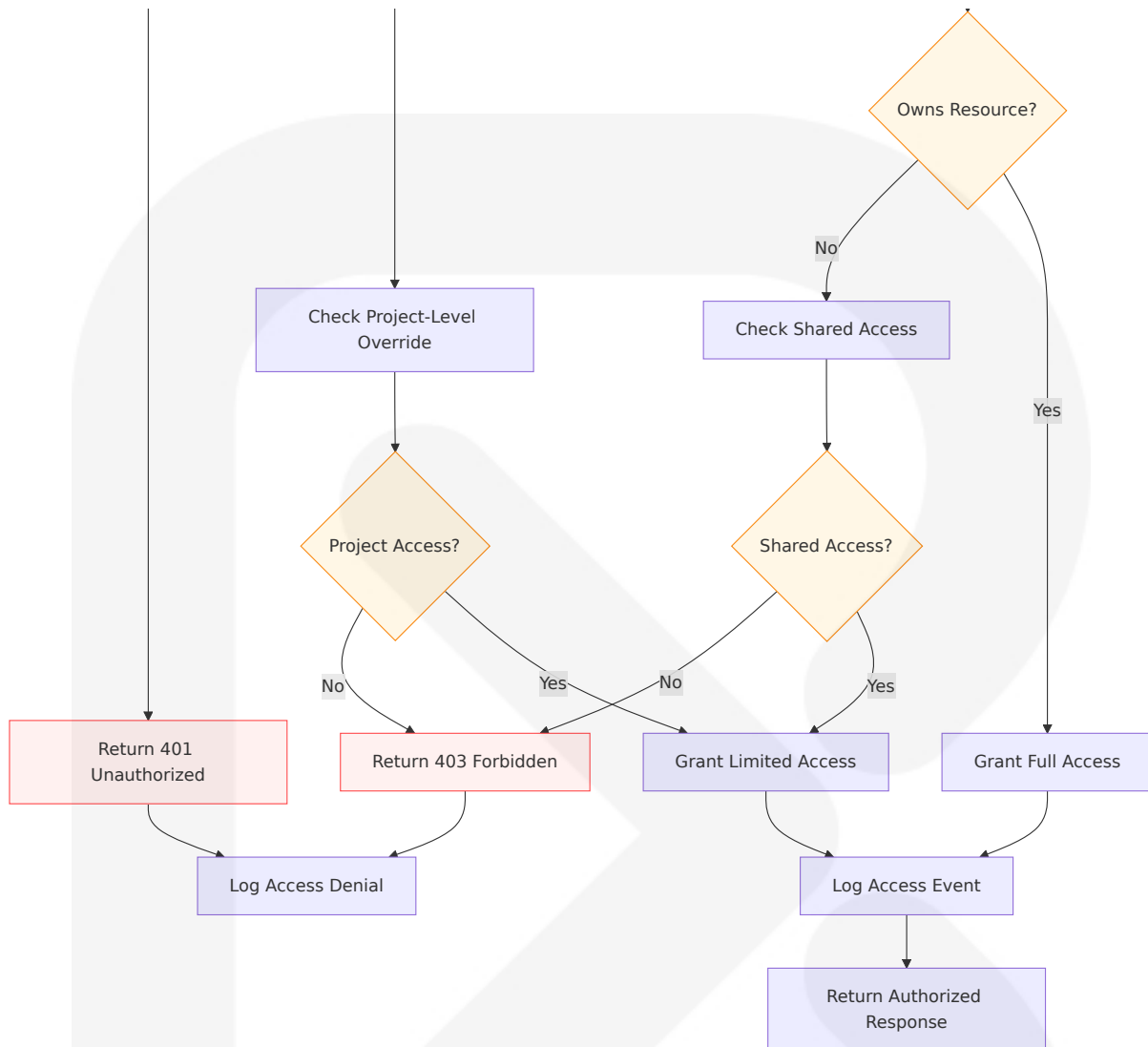
6.4.4 Security Architecture Diagrams

6.4.4.1 Authentication Flow Diagram

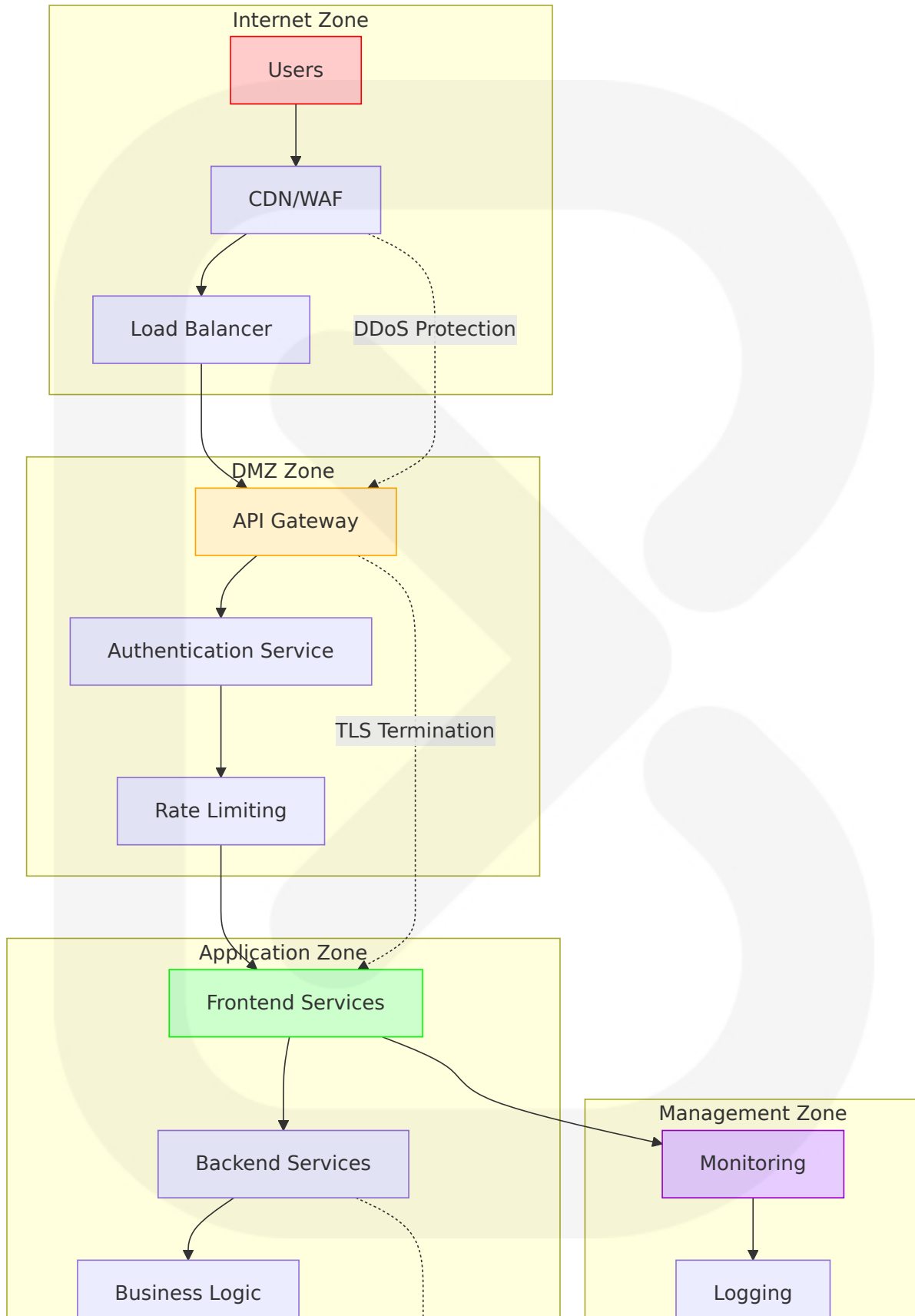


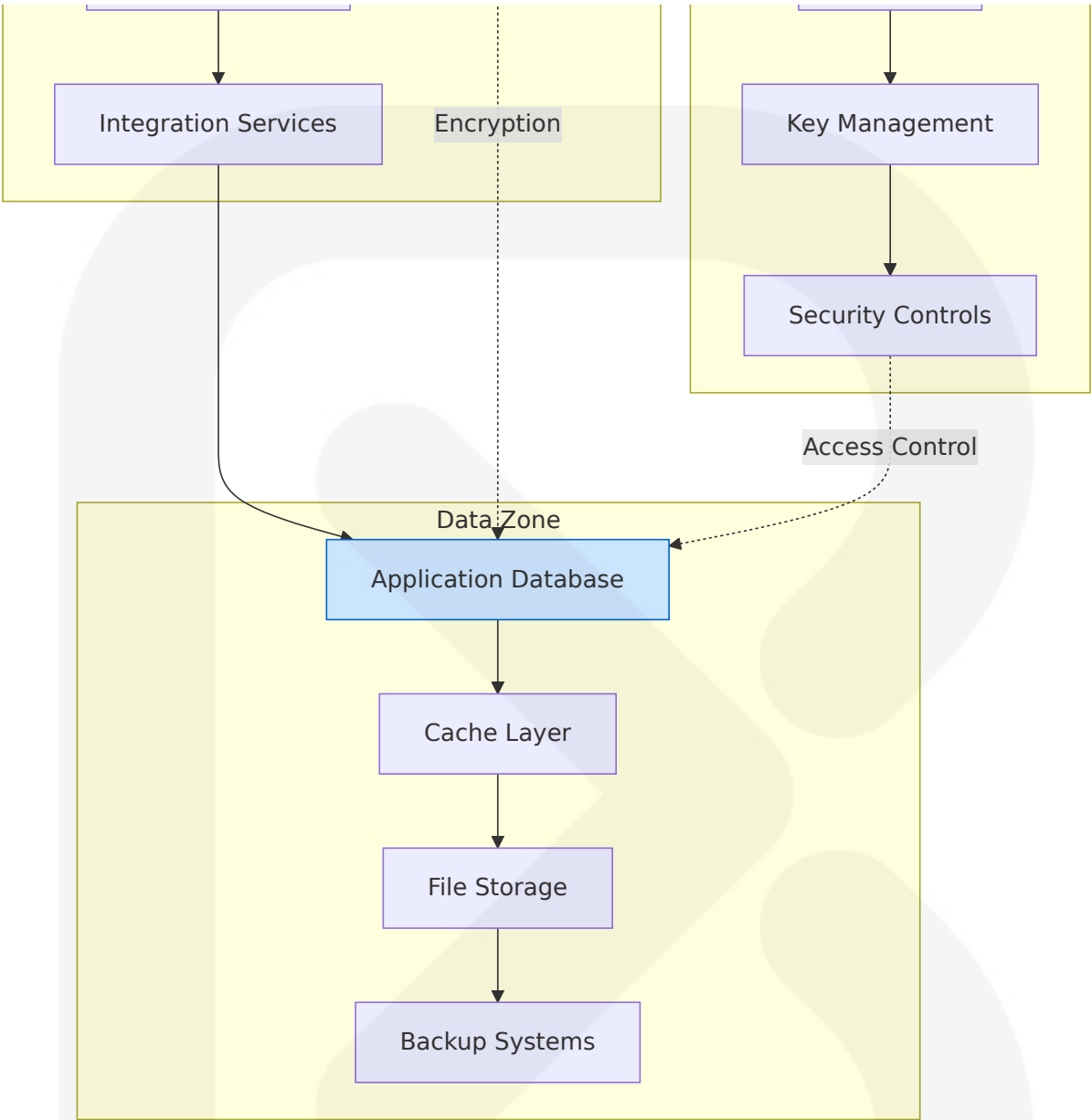
6.4.4.2 Authorization Flow Diagram





6.4.4.3 Security Zone Diagram





Security Zone Controls:

Zone	Security Controls	Access Requirements	Monitoring Level
Internet Zone	WAF, DDoS protection, rate limiting	Public access with filtering	High
DMZ Zone	TLS termination, authentication, authorization	Authenticated users only	High

Zone	Security Controls	Access Requirements	Monitoring Level
Application Zone	Service mesh, encryption in transit	Authorized services only	Medium
Data Zone	Encryption at rest, access logging	Database credentials required	High
Management Zone	Privileged access, audit logging	Admin credentials + MFA	Maximum

This comprehensive Security Architecture ensures AgencyOS maintains the highest levels of security while supporting the complex multi-tenant, multi-role requirements of modern digital agencies. The implementation follows industry best practices and regulatory compliance requirements, providing robust protection for both agency and client data.

6.5 Monitoring and Observability

6.5.1 Monitoring Infrastructure

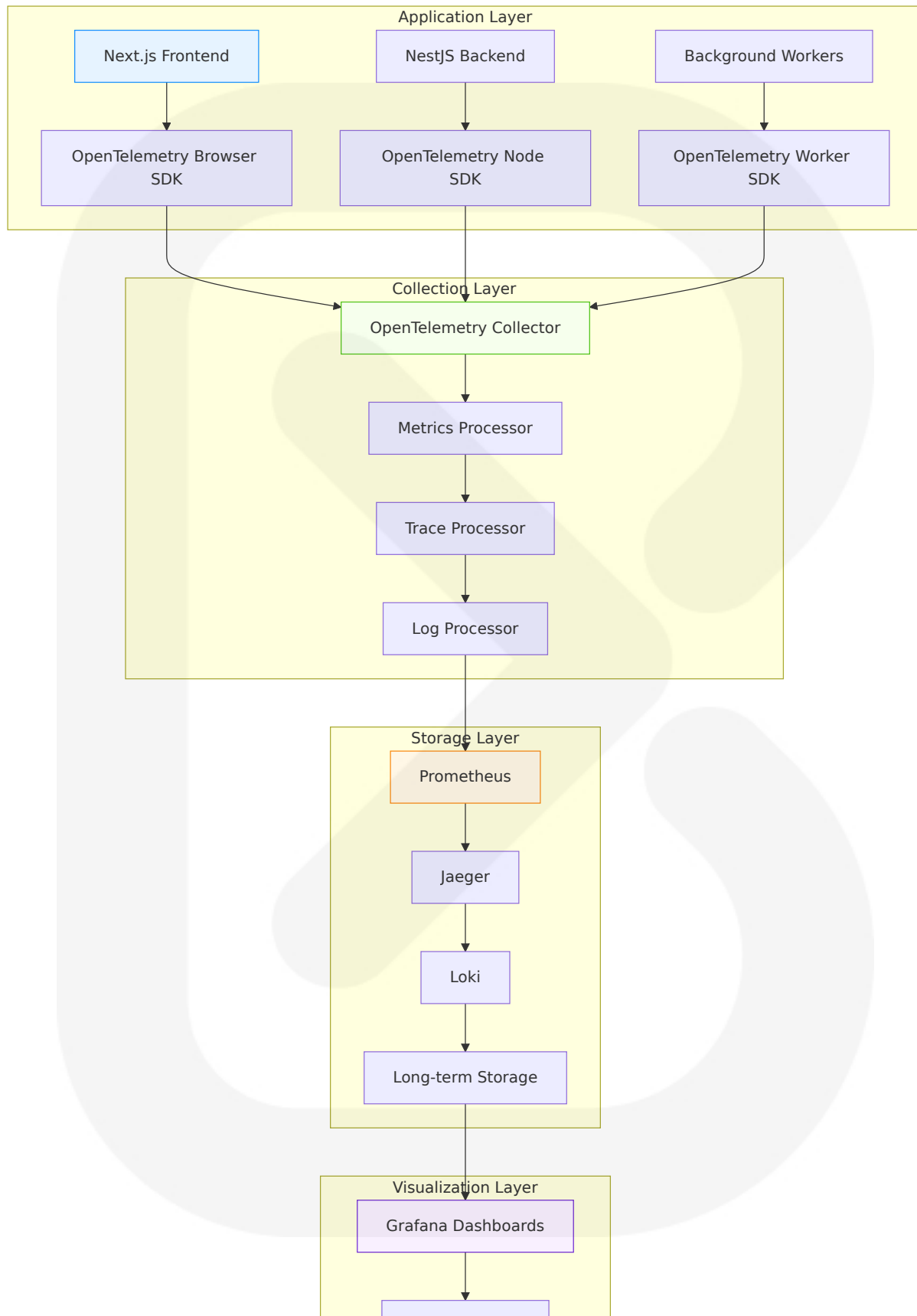
6.5.1.1 Metrics Collection Architecture

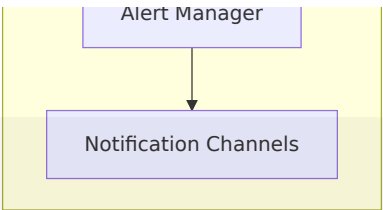
AgencyOS implements a comprehensive monitoring infrastructure built on OpenTelemetry as the vendor-neutral, open-source standard for collecting distributed traces, metrics, and logs, providing high-quality, ubiquitous, and portable telemetry to enable effective observability. The system leverages OpenTelemetry's general availability across all three pillars of observability (logs, metrics, and traces) to ensure comprehensive system visibility.

Metrics Collection Stack:

Component	Technology	Purpose	Collection Method
Application Metrics	OpenTelemetry SDK	Business and technical KPIs	Auto-instrumentation + custom metrics
Infrastructure Metrics	Prometheus + Node Exporter	System resource monitoring	Pull-based scraping
Database Metrics	PostgreSQL Exporter	Database performance tracking	Connection-based collection
Integration Metrics	Custom exporters	Third-party service health	API-based collection

OpenTelemetry Instrumentation Architecture:





Key Metrics Categories:

Metric Category	Examples	Collection Frequency	Retention Period
Business Metrics	Project completion rate, client satisfaction, revenue per project	Real-time	2 years
Application Metrics	API response times, error rates, throughput	15 seconds	90 days
Infrastructure Metrics	CPU, memory, disk usage, network I/O	30 seconds	30 days
Integration Metrics	Third-party API latency, success rates, quota usage	1 minute	60 days

6.5.1.2 Log Aggregation System

Prometheus is an open source monitoring system for which Grafana provides out-of-the-box support, forming the foundation of AgencyOS's log aggregation strategy combined with structured logging practices.

Centralized Logging Architecture:



Structured Logging Standards:

Log Level	Use Case	Retention	Format
ERROR	System errors, integration failures, security events	2 years	JSON with stack traces
WARN	Performance degradation, deprecated API usage	1 year	JSON with context

Log Level	Use Case	Retention	Format
INFO	Business events, user actions, system state changes	6 months	JSON with correlation IDs
DEBUG	Detailed execution flow, variable states	30 days	JSON with request tracing

6.5.1.3 Distributed Tracing Implementation

Distributed tracing lets you observe requests as they propagate through complex, distributed systems, improving the visibility of your application or system's health and letting you debug behavior that is difficult to reproduce locally.

Tracing Architecture:

Component	Technology	Sampling Rate	Trace Retention
Frontend Tracing	OpenTelemetry Browser	10% production, 100% staging	7 days
Backend Tracing	OpenTelemetry Node.js	5% production, 100% staging	14 days
Database Tracing	Auto-instrumentation	100% slow queries (>1s)	30 days
Integration Tracing	Custom spans	100% for errors, 20% success	14 days

Trace Correlation Strategy:

```
interface TraceContext {  
  traceId: string  
  spanId: string  
  tenantId: string  
  userId?: string  
  projectId?: string  
  requestId: string
```

```
    sessionId?: string
  }

  // Example trace implementation
  const tracer = trace.getTracer('agencyos-api', '1.0.0')

  async function processDesignReview(reviewId: string, context:
  TraceContext) {
    const span = tracer.startSpan('design-review.process', {
      attributes: {
        'agencyos.tenant.id': context.tenantId,
        'agencyos.user.id': context.userId,
        'agencyos.review.id': reviewId,
        'agencyos.operation': 'design-review-processing'
      }
    })

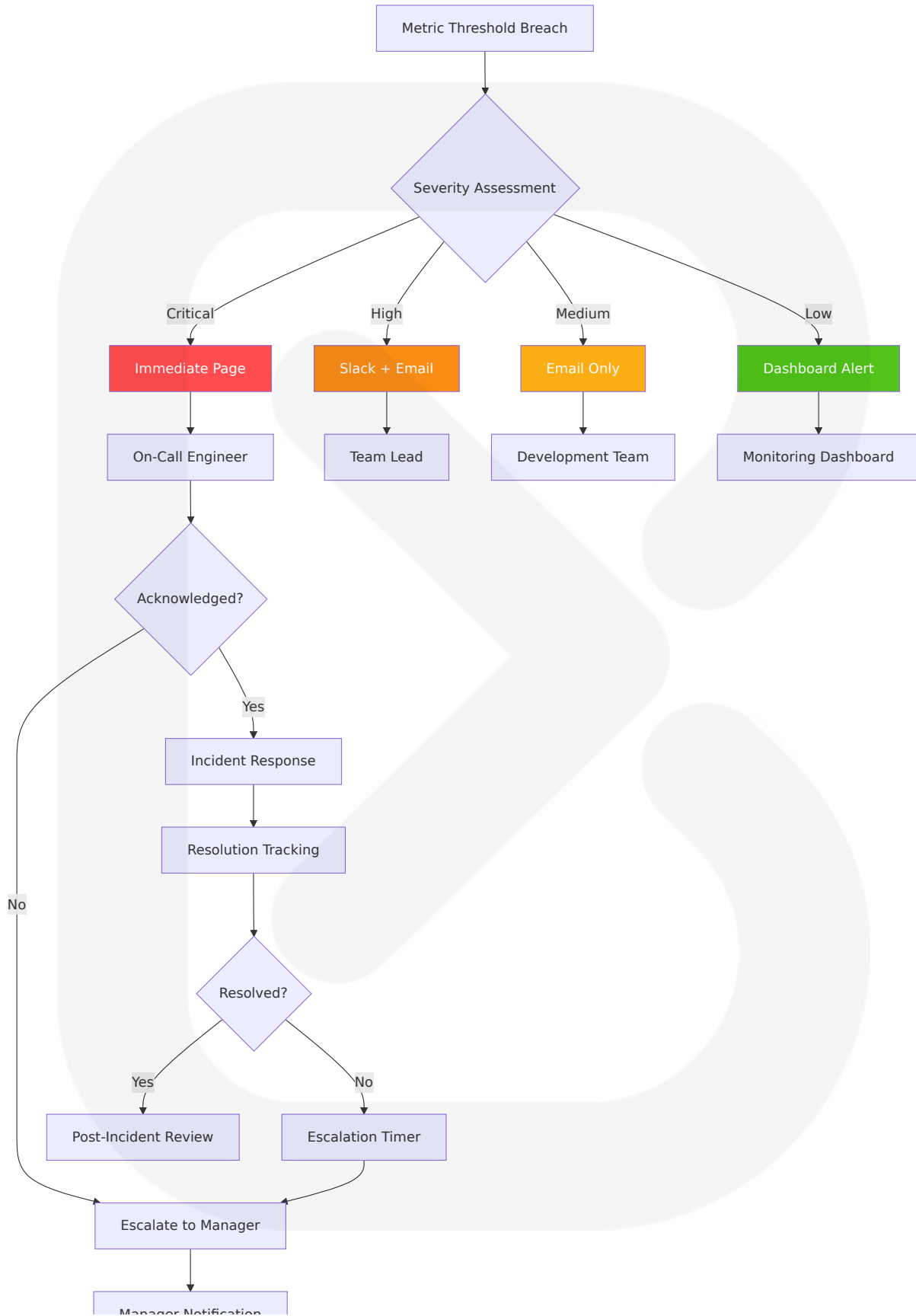
    try {
      // Business logic with nested spans
      await span.addEvent('review.validation.started')
      const validation = await validateReview(reviewId)
      await span.addEvent('review.validation.completed')

      return validation
    } catch (error) {
      span.recordException(error)
      span.setStatus({ code: SpanStatusCode.ERROR })
      throw error
    } finally {
      span.end()
    }
  }
}
```

6.5.1.4 Alert Management System

Implement proactive alerting mechanisms that can detect potential SLA breaches before they occur. Use tools that can analyze performance data in real-time and generate alerts based on predefined thresholds.

Alert Hierarchy and Routing:



Manager Notification

Alert Threshold Matrix:

Service Component	Critical	High	Medium	Low
API Response Time	p95 > 1000ms	p95 > 500ms	p95 > 300ms	p95 > 200ms
Error Rate	> 5%	> 2%	> 1%	> 0.5%
Database Connections	> 90% pool	> 80% pool	> 70% pool	> 60% pool
Memory Usage	> 90%	> 80%	> 70%	> 60%

6.5.1.5 Dashboard Design Strategy

Create dashboards to render system metrics monitored by Prometheus. When you install Prometheus and Node exporter, you will find recommended dashboards for use.

Dashboard Hierarchy:

Dashboard Level	Audience	Update Frequency	Key Metrics
Executive	Leadership, stakeholders	Daily	Business KPIs, SLA compliance, revenue metrics
Operational	DevOps, SRE teams	Real-time	System health, performance, alerts
Application	Development teams	Real-time	Application metrics, errors, traces
Business	Project managers, clients	Hourly	Project progress, utilization, client satisfaction

Dashboard Layout Standards:



6.5.2 Observability Patterns

6.5.2.1 Health Check Implementation

AgencyOS implements comprehensive health checks across all system components to ensure early detection of issues and maintain high availability.

Health Check Architecture:

Component	Check Type	Frequency	Timeout	Dependencies
API Gateway	HTTP endpoint	30 seconds	5 seconds	Database, Redis
Database	Connection pool	60 seconds	10 seconds	PostgreSQL primary
Cache Layer	Redis ping	30 seconds	3 seconds	Redis cluster
External APIs	Service availability	5 minutes	15 seconds	Third-party services

Health Check Response Format:

```
interface HealthCheckResponse {
  status: 'healthy' | 'degraded' | 'unhealthy'
  timestamp: string
  version: string
  uptime: number
  checks: {
    database: ComponentHealth
    cache: ComponentHealth
    integrations: ComponentHealth[]
    storage: ComponentHealth
  }
  metadata: {
    tenantCount: number
  }
}
```



```
    activeUsers: number
    requestsPerMinute: number
  }
}

interface ComponentHealth {
  status: 'healthy' | 'degraded' | 'unhealthy'
  responseTime: number
  lastChecked: string
  error?: string
  details?: Record<string, any>
}
```

6.5.2.2 Performance Metrics Framework

Golden Signals Monitoring:

Signal	Metric	Target	Measurement
Latency	API response time	p95 < 300ms	Request duration histograms
Traffic	Requests per second	Baseline + 20% capacity	Request rate counters
Errors	Error rate percentage	< 0.1%	Error ratio calculations
Saturation	Resource utilization	< 80% CPU/Memory	Resource usage gauges

Performance Monitoring Implementation:

```
// Custom metrics collection
const performanceMetrics = {
  // Request duration histogram
  httpRequestDuration: new Histogram({
    name: 'http_request_duration_seconds',
    help: 'Duration of HTTP requests in seconds',
    labelNames: ['method', 'route', 'status_code', 'tenant_id'],
    buckets: [0.1, 0.3, 0.5, 0.7, 1, 3, 5, 7, 10]
  }),
}
```

```
// Active user gauge
activeUsers: new Gauge({
  name: 'active_users_total',
  help: 'Number of active users by tenant',
  labelNames: ['tenant_id', 'user_type']
}),

// Business metrics
projectCompletionRate: new Gauge({
  name: 'project_completion_rate',
  help: 'Percentage of projects completed on time',
  labelNames: ['tenant_id', 'project_type']
})
}
```

6.5.2.3 Business Metrics Tracking

Key Business Indicators:

Metric Category	Specific Metrics	Collection Method	Business Impact
Project Delivery	On-time completion rate, scope creep percentage	Event-driven tracking	Client satisfaction, profitability
Client Engagement	Approval cycle time, feedback response rate	Workflow monitoring	Project velocity, client retention
Resource Utilization	Team utilization rate, project margin	Time tracking integration	Resource optimization, pricing
Financial Performance	DSO, invoice processing time, payment success rate	Financial system sync	Cash flow, revenue recognition

Business Metrics Dashboard:



6.5.2.4 SLA Monitoring Framework

SLA uptime results with this value (or higher) are good; they satisfy the SLA target. Values between this threshold and the error uptime percentage will cause a yellow warning in the SLA reports.

SLA Compliance Tracking:

SLA Component	Target	Warning Threshold	Critical Threshold	Measurement Window
System Availability	99.9% uptime	99.5%	99.0%	Monthly
API Performance	p95 < 300 ms	p95 > 400 ms	p95 > 600 ms	5-minute windows
Support Response	< 4 hours	> 6 hours	> 12 hours	Business hours
Data Recovery	RTO 60 min, RPO 15 min	RTO 90 min	RTO 120 min	Per incident

SLA Monitoring Implementation:

```
interface SLAMonitor {  
    // Availability monitoring  
    trackUptime(service: string, status: 'up' | 'down', timestamp:  
Date): void  
    calculateAvailability(service: string, period: TimePeriod): number  
  
    // Performance monitoring  
    trackResponseTime(endpoint: string, duration: number, timestamp:  
Date): void  
    calculatePerformanceSLA(endpoint: string, period: TimePeriod):  
PerformanceMetrics  
  
    // Support SLA tracking  
    trackSupportTicket(ticketId: string, createdAt: Date, respondedAt?:  
Date): void  
    calculateSupportSLA(period: TimePeriod): SupportMetrics  
}
```

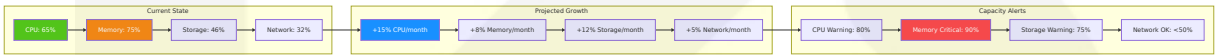
```
// Alert generation
checkSLACompliance(): SLAComplianceReport
generateSLAAlerts(violations: SLAViolation[]): void
}
```

6.5.2.5 Capacity Tracking System

Resource Capacity Monitoring:

Resource Type	Current Usage	Capacity Limit	Growth Rate	Alert Threshold
Database Connections	45/100	100 connections	+2/month	80 connections
API Rate Limits	750/1000 req/min	1000 req/min	+50/month	900 req/min
Storage Usage	2.3TB/5TB	5TB	+200GB/month	4TB
Memory Usage	12GB/16GB	16GB per instance	+1GB/month	14GB

Capacity Planning Dashboard:



6.5.3 Incident Response

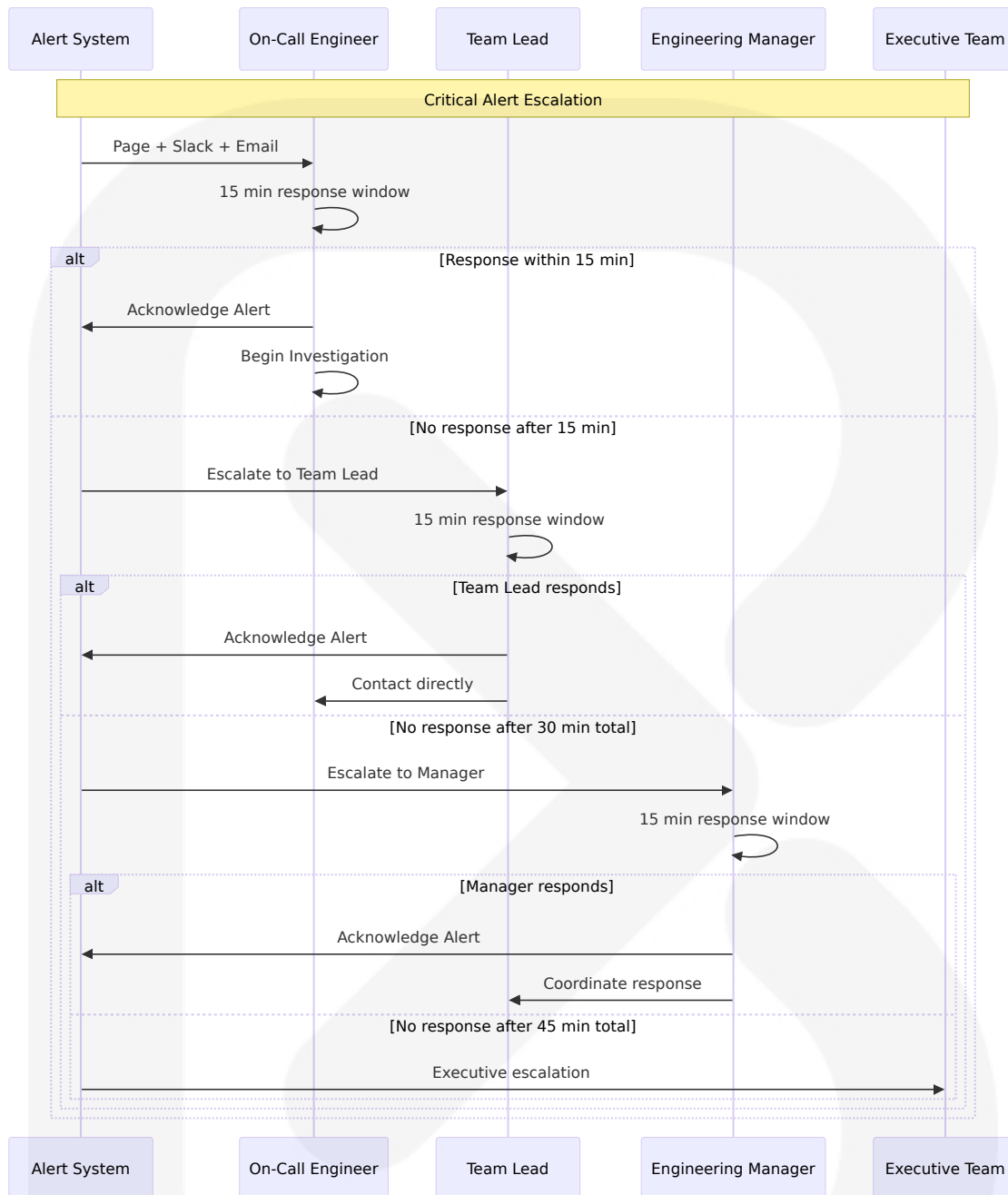
6.5.3.1 Alert Routing Strategy

Kick off different escalation paths as well as send stakeholder notifications and updates as SLAs are approaching or being breached.

Alert Routing Matrix:

Alert Severity	Primary Contact	Secondary Contact	Escalation Time	Notification Channels
Critical	On-call engineer	Team lead	15 minutes	Page, Slack, Email, SMS
High	Team lead	Engineering manager	30 minutes	Slack, Email
Medium	Development team	Project manager	2 hours	Slack, Email
Low	Development team	None	24 hours	Slack

Escalation Flow:



6.5.3.2 Escalation Procedures

Escalation Triggers:

Trigger Condition	Escalation Level	Response Time	Actions Required
No acknowledgment in 15 min	Level 1 → Level 2	Immediate	Contact backup on-call
No resolution in 1 hour	Level 2 → Level 3	15 minutes	Engage engineering manager
Customer-facing outage	Direct to Level 3	5 minutes	Immediate management notification
Security incident	Direct to Level 4	Immediate	Security team + executive notification

Escalation Contact Matrix:

Role	Primary	Backup	Contact Method	Availability
On-Call Engineer	Rotating schedule	Secondary on-call	PagerDuty, Phone	24/7
Team Lead	Engineering lead	Senior engineer	Slack, Phone	Business hours + critical
Engineering Manager	Development manager	CTO	Phone, Email	On-demand
Executive	CTO	CEO	Phone	Critical incidents only

6.5.3.3 Runbook Documentation

Incident Response Runbooks:

Incident Type	Runbook	MTTR Target	Automation Level
API Performance Degradation	Performance troubleshooting guide	30 minutes	Semi-automated diagnostics
Database Connection Issues	Database recovery procedures	15 minutes	Automated failover

Incident Type	Runbook	MTTR Target	Automation Level
Integration Failures	Third-party service recovery	45 minutes	Manual intervention
Security Incidents	Security response protocol	60 minutes	Automated containment

Runbook Template Structure:

Incident Runbook: [Incident Type]

Overview

- **Incident Type:** [Type]
- **Severity:** [Critical/High/Medium/Low]
- **MTTR Target:** [Time]
- **Owner:** [Team/Role]

Detection

- **Symptoms:** [Observable symptoms]
- **Monitoring:** [Alert conditions]
- **Verification:** [Confirmation steps]

Investigation

1. Initial Assessment

- Check system health dashboard
- Review recent deployments
- Examine error logs

2. Root Cause Analysis

- [Specific investigation steps]
- [Common causes checklist]

- [Diagnostic commands]

Resolution

1. Immediate Actions

- [Emergency mitigation steps]
- [Service restoration procedures]

2. Permanent Fix

- [Long-term resolution steps]
- [Prevention measures]

Communication

- **Internal:** [Team notification procedures]
- **External:** [Customer communication plan]
- **Escalation:** [When and how to escalate]

Post-Incident

- **Monitoring:** [Recovery verification]
- **Documentation:** [Incident report requirements]
- **Follow-up:** [Post-mortem scheduling]

6.5.3.4 Post-Mortem Process

****Post-Mortem Framework:****

Incident Severity	Post-Mortem Required	Timeline	Participants
--- --- --- ---			
Critical	Always	Within 48 hours	Full engineering team + stakeholders
High	Always	Within 1 week	Engineering team + affected stakeholders
Medium	If recurring	Within 2 weeks	Engineering team
Low	Optional	As needed	Individual contributor

****Post-Mortem Template:****

Post-Mortem: [Incident Title]

Incident Summary

- ****Date****: [Incident date]
- ****Duration****: [Total duration]
- ****Impact****: [Customer/business impact]
- ****Root Cause****: [Primary cause]

Timeline

Time	Event	Actions Taken
-----	-----	-----
[Time]	[Event description]	[Response actions]

Root Cause Analysis

What Happened

- [Detailed explanation of the incident]

Why It Happened

- [Root cause analysis]
- [Contributing factors]

How We Responded

- [Response effectiveness]
- [What worked well]
- [What could be improved]

Action Items

Action	Owner	Due Date	Priority
-----	-----	-----	-----
[Action description]	[Assignee]	[Date]	[High/Medium/Low]

Lessons Learned

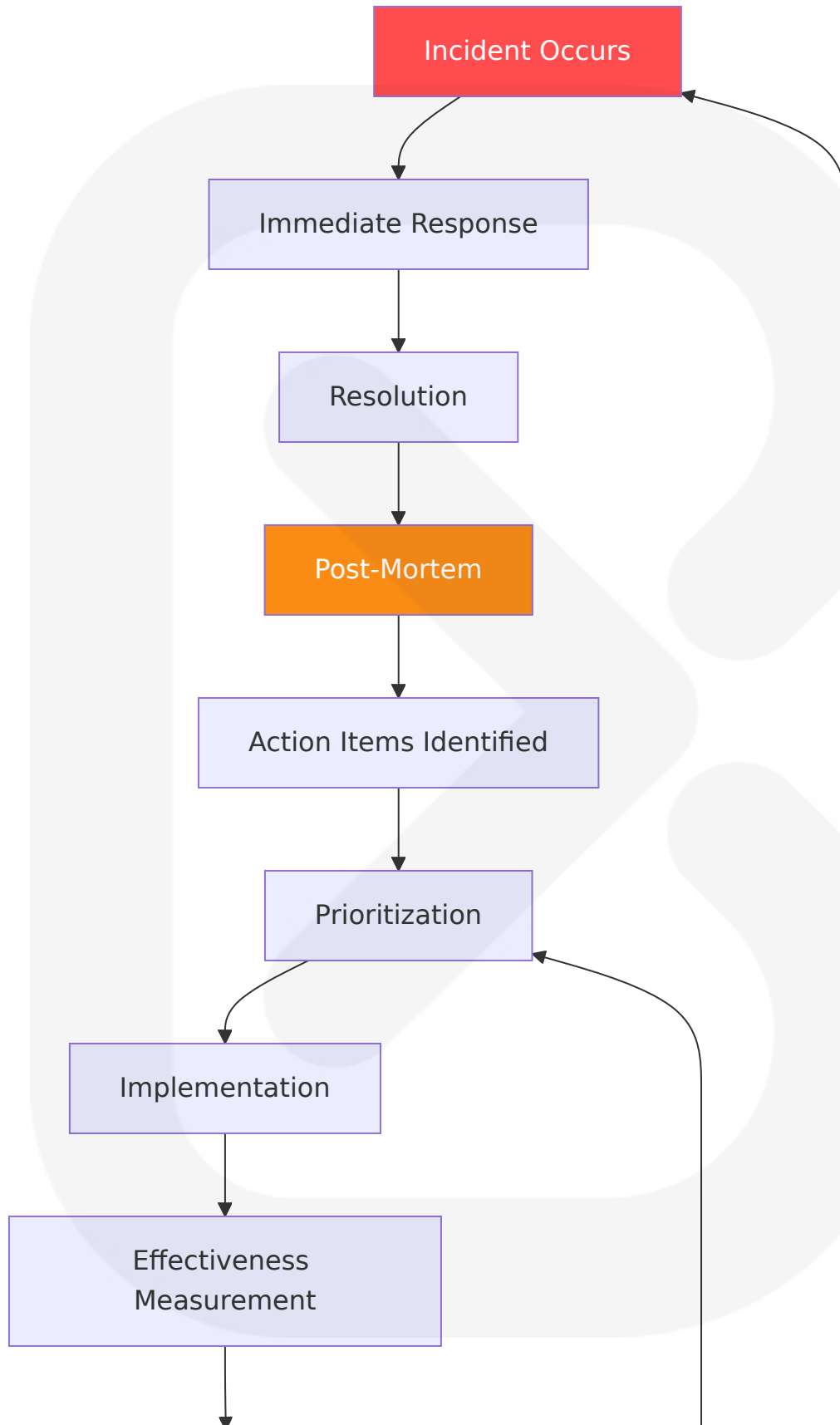
- [Key insights]
- [Process improvements]
- [Technical improvements]

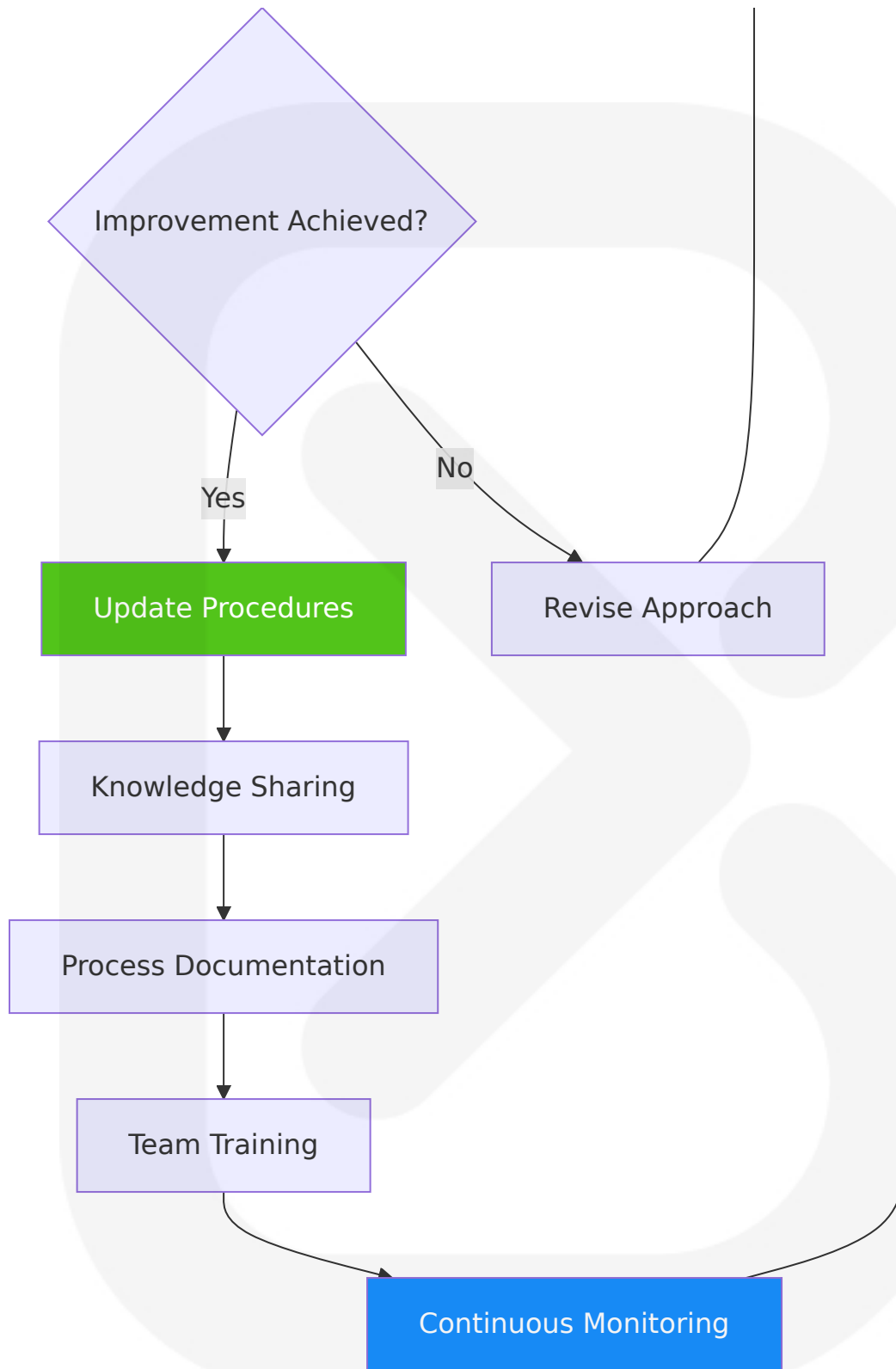
6.5.3.5 Improvement Tracking

Continuous Improvement Metrics:

Improvement Area	Metric	Current	Target	Tracking Method
MTTR Reduction	Mean time to resolution	45 minutes	30 minutes	Incident tracking system
Alert Accuracy	False positive rate	15%	<5%	Alert analysis
Runbook Effectiveness	Runbook usage rate	70%	>90%	Post-incident surveys
Prevention Success	Repeat incident rate	8%	<3%	Incident categorization

Improvement Process Flow:

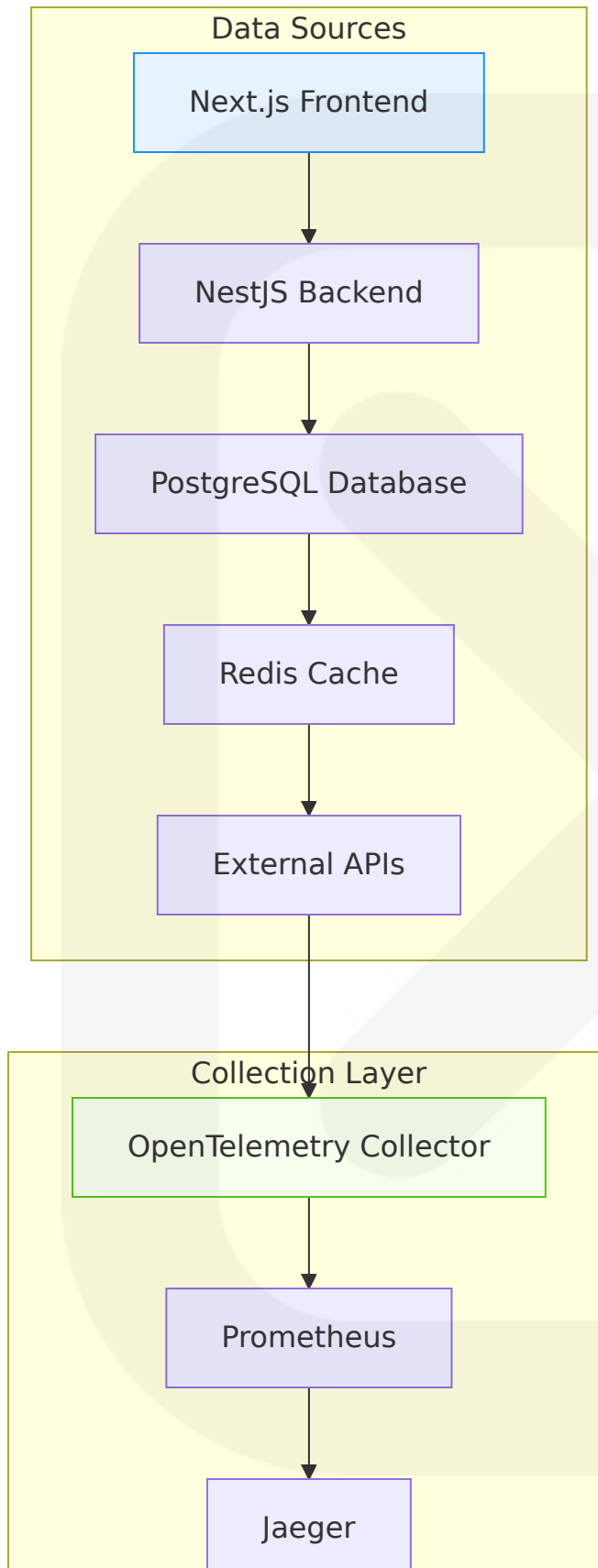


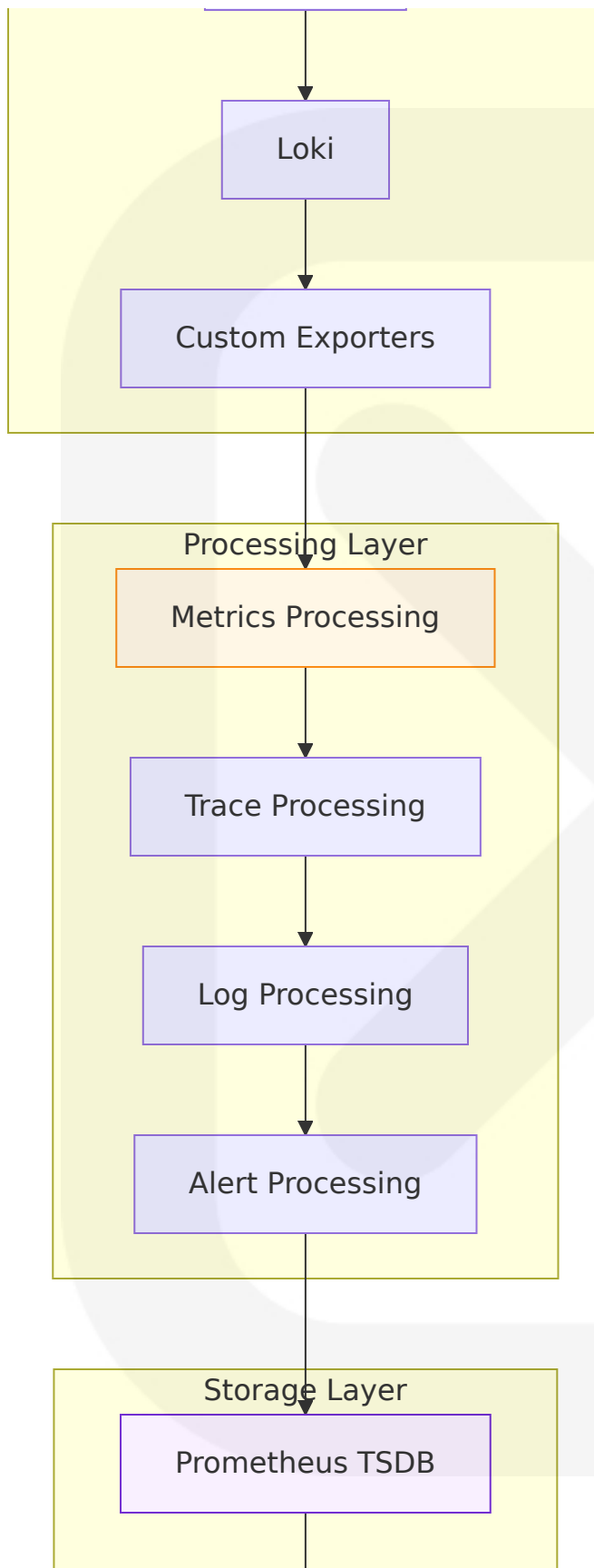


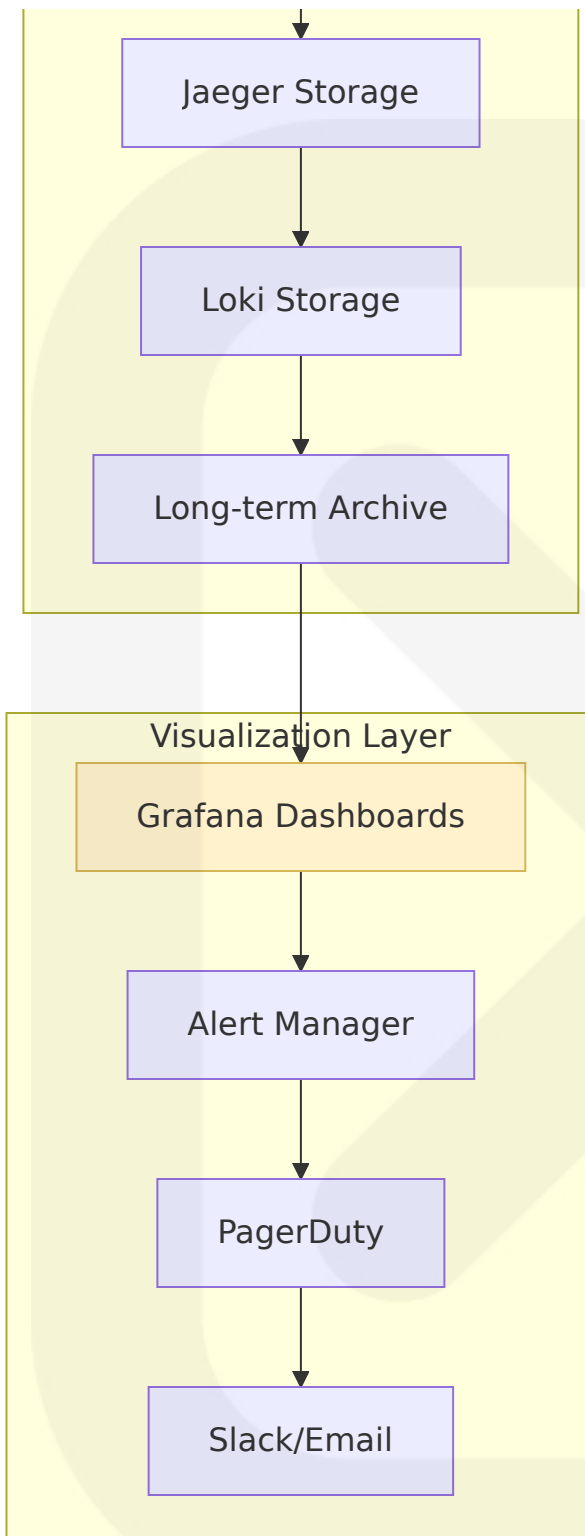
6.5.4 Monitoring Architecture Diagrams

6.5.4.1 Complete Monitoring Stack

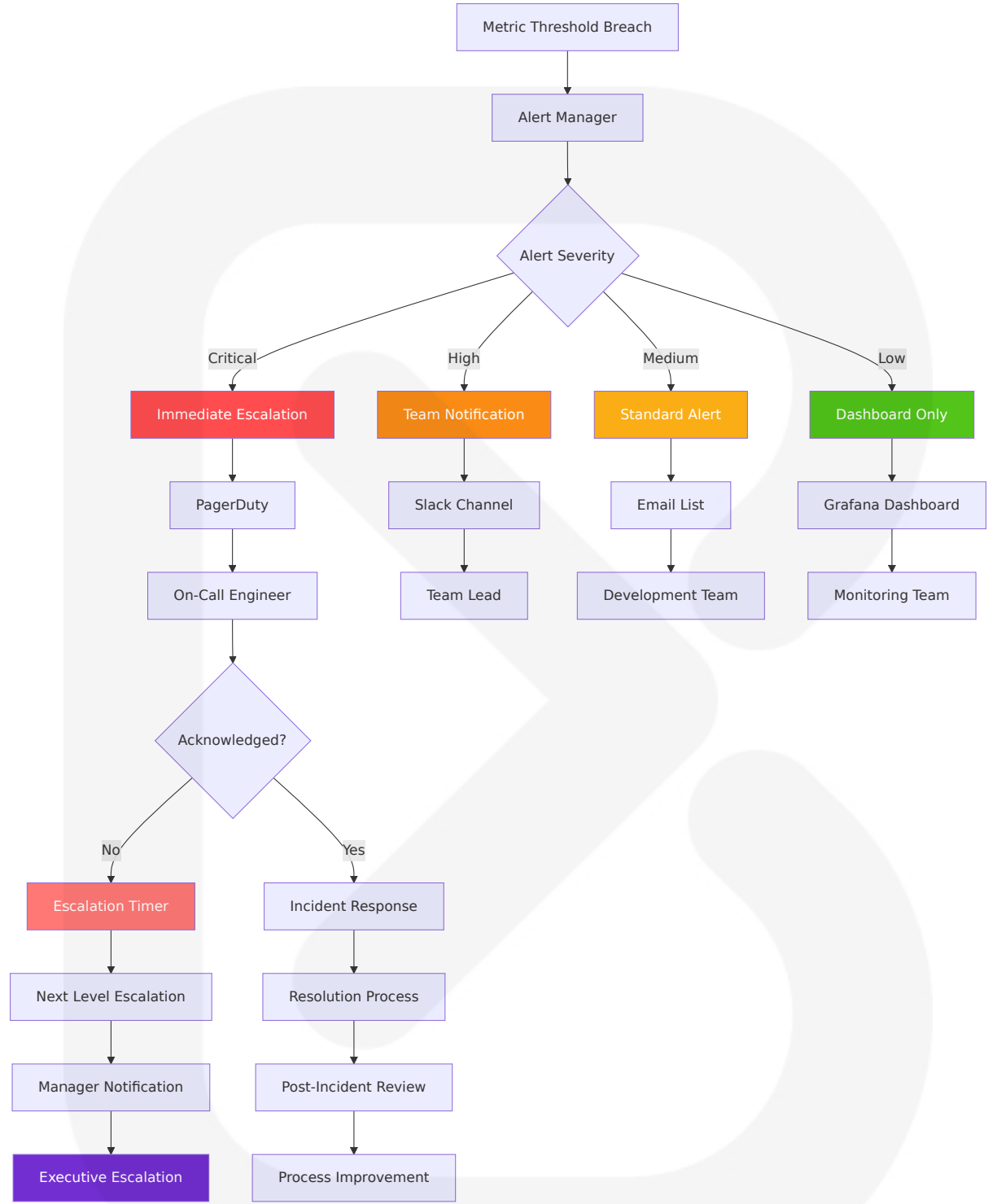








6.5.4.2 Alert Flow Architecture



6.5.4.3 SLA Monitoring Dashboard Layout



This comprehensive Monitoring and Observability architecture ensures AgencyOS maintains the highest levels of system reliability, performance, and business insight. AI implementations have been shown to reduce the mean time to resolution (MTTR) by up to 40%, and OpenTelemetry adoption has led to a 50% decrease in operational costs for many organizations, providing the foundation for proactive system management and continuous improvement.

6.6 Testing Strategy

6.6.1 TESTING APPROACH

6.6.1.1 Unit Testing

AgencyOS implements comprehensive unit testing across both frontend and backend components using modern testing frameworks optimized for the Next.js 15 and NestJS 11 technology stack.

Testing Frameworks and Tools:

Component	Framework	Version	Purpose	Justification
Frontend Testing	Jest with React Testing Library	Jest 29.x, RTL 16.x	Component and utility testing	Next.js 15 supports Jest and React Testing Library for unit testing
Backend Testing	NestJS Testing with Jest	Jest 29.x	Service and controller testing	NestJS provides integration with Jest out of the box

Component	Framework	Version	Purpose	Justification
Mocking Framework	Jest Mock Functions	Built-in	Dependency isolation	Jest provides assert functions and test-double utilities that help with mocking
Test Utilities	@testing-library/jest-dom	6.x	Enhanced assertions	DOM testing utilities for React components

Test Organization Structure:

```
src/
├── components/
│   ├── ProjectDashboard/
│   │   ├── ProjectDashboard.tsx
│   │   ├── ProjectDashboard.test.tsx
│   │   └── __mocks__/
│   └── DesignReview/
│       ├── DesignReview.tsx
│       ├── DesignReview.test.tsx
│       └── __mocks__/
├── services/
│   ├── project/
│   │   ├── project.service.ts
│   │   ├── project.service.spec.ts
│   │   └── __mocks__/
│   └── design-review/
│       ├── design-review.service.ts
│       ├── design-review.service.spec.ts
│       └── __mocks__/
└── utils/
    ├── validation/
    │   ├── validation.utils.ts
    │   └── validation.utils.test.ts
    └── formatting/
        ├── formatting.utils.ts
        └── formatting.utils.test.ts
```

Mocking Strategy:

Mock Type	Implementation	Use Case	Example
External APIs	MSW (Mock Service Worker)	Third-party service calls	Figma API, Stripe payments
Database Operations	Repository mock factory pattern	Data layer isolation	PostgreSQL queries, Redis operations
Authentication	JWT mock tokens	User context simulation	Role-based access testing
File Operations	In-memory file system	Asset management testing	Design file uploads, processing

Code Coverage Requirements:

Component Type	Coverage Target	Measurement	Enforcement
Business Logic	90% line coverage	Jest coverage reports	CI/CD quality gates
API Controllers	85% line coverage	Supertest integration	Automated testing pipeline
Utility Functions	95% line coverage	Unit test isolation	Pre-commit hooks
React Components	80% line coverage	React Testing Library	Component interaction testing

Test Naming Conventions:

```
// Service testing pattern
describe('ProjectService', () => {
  describe('createProject', () => {
    it('should create project with valid data', async () => {
      // Test implementation
    })

    it('should throw error when tenant context missing', async () => {
```

```
    // Error case testing
  })

  it('should enforce budget validation rules', async () => {
    // Business rule testing
  })
})

// Component testing pattern
describe('ProjectDashboard Component', () => {
  describe('when user has PM role', () => {
    it('should display project creation button', () => {
      // Role-based UI testing
    })

    it('should allow budget modifications', () => {
      // Permission-based testing
    })
  })

  describe('when user has client role', () => {
    it('should hide sensitive financial data', () => {
      // Data visibility testing
    })
  })
})
})
```

Test Data Management:

Data Type	Strategy	Implementation	Maintenance
User Fixtures	Factory pattern	User role combinations	Centralized test data
Project Data	Builder pattern	Complex project scenarios	Reusable test builders
Mock Responses	JSON fixtures	API response templates	Version-controlled fixtures
Database Seeds	Migration-based	Consistent test state	Automated cleanup

6.6.1.2 Integration Testing

Integration testing ensures seamless communication between AgencyOS components, external services, and database operations using containerized test environments.

Service Integration Test Approach:

Integration Layer	Testing Strategy	Tools	Scope
API Integration	Supertest for framework-agnostic API testing	Supertest 6.x	REST endpoint validation
Database Integration	Testcontainers with PostgreSQL	Testcontainers 1.19+	Data persistence testing
External Service Integration	Mock server integration	MSW, Nock	Third-party API simulation
Authentication Integration	JWT token validation	Custom auth middleware	Security boundary testing

API Testing Strategy:

```
// NestJS integration test example
describe('ProjectController (e2e)', () => {
  let app: INestApplication
  let projectService: ProjectService

  beforeEach(async () => {
    const moduleFixture: TestingModule = await
    Test.createTestingModule({
      imports: [AppModule],
    }).compile()

    app = moduleFixture.createNestApplication()
    projectService = moduleFixture.get<ProjectService>(ProjectService)
    await app.init()
  })
})
```

```
describe('POST /projects', () => {  
  it('should create project with valid payload', () => {  
    return request(app.getHttpServer())  
      .post('/projects')  
      .send(validProjectData)  
      .expect(201)  
      .expect((res) => {  
        expect(res.body.id).toBeDefined()  
        expect(res.body.tenantId).toBe(testTenantId)  
      })  
  })  
})  
})
```

Database Integration Testing:

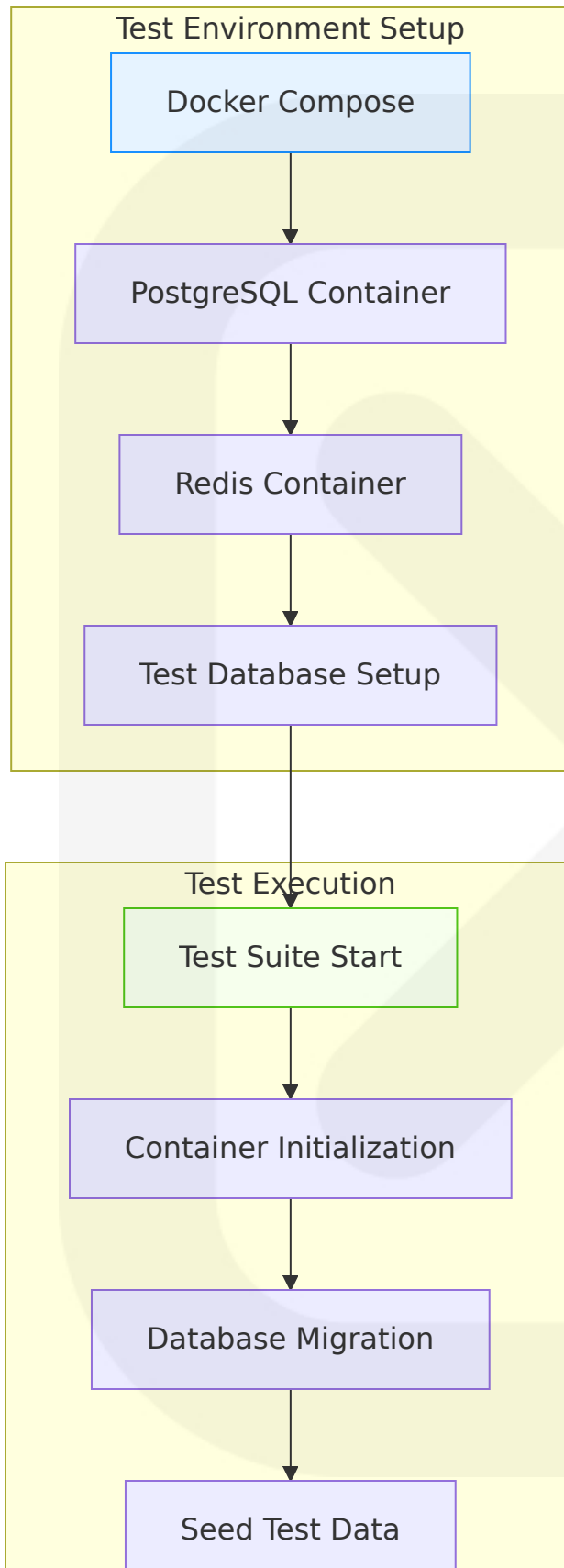
The basic idea is to create a template database before running the tests and then create a new database from the template database for each test, ensuring complete test isolation.

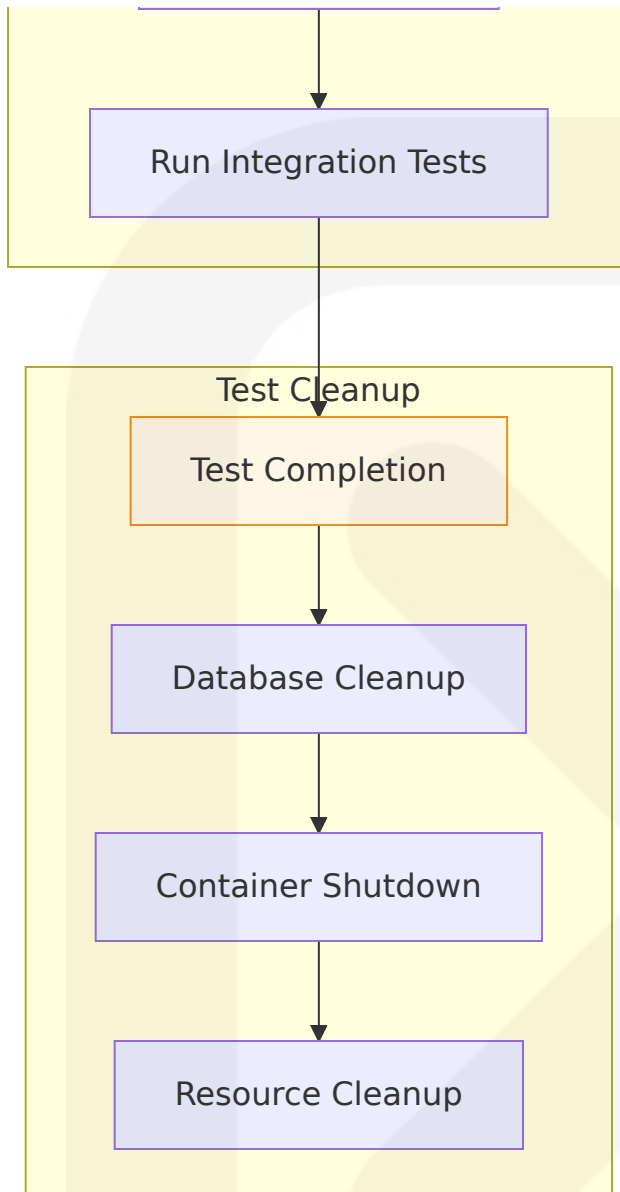
Database Operation	Test Approach	Implementation	Performance
Multi-tenant Isolation	Docker container with memory disk for PostgreSQL data storage	Testcontainers PostgreSQL	<500ms setup time
Row-Level Security	Policy validation testing	Custom RLS test utilities	Tenant boundary verification
Migration Testing	Schema evolution validation	Automated migration testing	Forward/backward compatibility
Connection Pooling	Concurrent access testing	Load simulation	Connection limit validation

External Service Mocking:

Service	Mock Strategy	Test Scenarios	Validation
Figma API	Response simulation	File access, webhook processing	API contract compliance
Stripe Payments	Webhook simulation	Payment success/failure flows	Financial transaction integrity
GitHub Integration	Repository mocking	PR creation, status updates	Development workflow validation
Slack Notifications	Message interception	Notification delivery testing	Communication flow verification

Test Environment Management:





6.6.1.3 End-to-End Testing

Playwright is a testing framework that lets you automate Chromium, Firefox, and WebKit with a single API for End-to-End (E2E) testing, providing comprehensive user journey validation for AgencyOS.

E2E Test Scenarios:

User Journey	Test Scope	Browser Coverage	Validation Points
Client Onboarding	Lead capture → SOW signature → deposit → project start	Chromium, Firefox, WebKit	Multi-step workflow completion
Design Review Process	Upload → review → feedback → approval	Cross-browser compatibility	Real-time collaboration features
Project Delivery	Development → UAT → client sign-off → invoice	Mobile responsive testing	End-to-end business process
Payment Processing	Invoice generation → payment → confirmation	Security validation	Financial transaction flow

UI Automation Approach:

Playwright will simulate a user navigating your application using three browsers: Chromium, Firefox and Webkit, requiring your Next.js server to be running.

```
// Playwright E2E test example
import { test, expect } from '@playwright/test'

test.describe('Design Review Workflow', () => {
  test('should complete design review and approval process', async ({
    page }) => {
    // Navigate to project dashboard
    await page.goto('/projects/test-project-id')

    // Upload design file
    await page.click('[data-testid="upload-design"]')
    await page.setInputFiles('input[type="file"]', 'test-design.pdf')

    // Create review session
    await page.fill('[data-testid="review-title"]', 'Homepage Design Review')
    await page.click('[data-testid="create-review"]')
```

```
// Verify review creation
await expect(page.locator('[data-testid="review-status"]')).toContainText('Pending Review')

// Simulate client approval
await page.click('[data-testid="approve-design"]')
await expect(page.locator('[data-testid="review-status"]')).toContainText('Approved')
})
})
```

Test Data Setup/Teardown:

Data Category	Setup Strategy	Teardown Strategy	Isolation Method
User Accounts	Factory-generated test users	Automated cleanup after test suite	Unique email domains per test run
Project Data	Template-based project creation	Database transaction rollback	Tenant-specific test data
File Assets	Temporary file generation	File system cleanup	Isolated test directories
External Service State	Mock service reset	State restoration	Service virtualization

Performance Testing Requirements:

k6 is an open-source tool and cloud service that makes load testing easy for developers and QA engineers, built for developer happiness.

Performance Test Type	Tool	Target Metrics	Acceptance Criteria
API Load Testing	k6 open source framework for performance testing	Response time, throughput	p95 < 300ms, 1000 req/min
Database Performance	k6 + PostgreSQL monitoring	Query execution time	p95 < 50ms for queries

Performance Test Type	Tool	Target Metrics	Acceptance Criteria
File Upload Performance	Playwright + k6	Upload completion time	10MB files < 30 seconds
Real-time Features	WebSocket load testing	Connection stability	1000 concurrent connections

Cross-Browser Testing Strategy:

Playwright configuration supports three browsers: Chromium, Firefox, and Webkit for comprehensive cross-browser testing.

Browser	Version Support	Test Coverage	Specific Validations
Chromium	Latest stable	Full test suite	WebRTC features, modern APIs
Firefox	Latest stable	Core functionality	Standards compliance
WebKit	Latest stable	Critical paths	Safari-specific behaviors
Mobile Safari	iOS simulation	Responsive design	Touch interactions

6.6.2 TEST AUTOMATION

6.6.2.1 CI/CD Integration

AgencyOS implements comprehensive test automation integrated into the CI/CD pipeline using GitHub Actions, ensuring consistent quality gates and automated feedback loops.

Automated Test Triggers:

Trigger Event	Test Suite	Execution Time	Quality Gate
Pull Request	Unit + Integration tests	< 10 minutes	90% pass rate required
Main Branch Push	Full test suite + E2E	< 30 minutes	100% critical tests pass
Nightly Build	Performance + Security tests	< 60 minutes	SLA compliance validation
Release Candidate	Complete test matrix	< 90 minutes	Zero critical defects

GitHub Actions Workflow:

```
name: AgencyOS Test Pipeline
on:
  pull_request:
    branches: [main, develop]
  push:
    branches: [main]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [20.x]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Run unit tests
        run: pnpm test:unit --coverage

      - name: Upload coverage reports
```

```
    uses: codecov/codecov-action@v3
    with:
      file: ./coverage/lcov.info

integration-tests:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:15-alpine
      env:
        POSTGRES_PASSWORD: test
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5
  steps:
    - uses: actions/checkout@v4
    - name: Run integration tests
      run: pnpm test:integration

e2e-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Install Playwright
      run: npx playwright install --with-deps
    - name: Run E2E tests
      run: pnpm test:e2e
    - name: Upload test results
      uses: actions/upload-artifact@v3
      if: failure()
      with:
        name: playwright-report
        path: playwright-report/
```

Parallel Test Execution:

Test Category	Parallelization Strategy	Resource Allocation	Execution Time
Unit Tests	Jest worker processes	4 CPU cores	3-5 minutes
Integration Tests	Database per worker	2 CPU cores per worker	8-12 minutes
E2E Tests	Browser instance per test	1 CPU core per browser	15-25 minutes
Performance Tests	Isolated k6 instances	Dedicated test environment	10-20 minutes

Test Reporting Requirements:



Failed Test Handling:

Failure Type	Automatic Actions	Manual Intervention	Recovery Process
Unit Test Failures	Block PR merge, notify author	Code review required	Fix and re-run tests
Integration Failures	Rollback deployment, alert team	Infrastructure investigation	Environment restoration
E2E Test Failures	Screenshot capture, video recording	Manual test execution	Test script updates
Performance Degradation	Alert on-call engineer	Performance analysis	Optimization implementation

Flaky Test Management:

Detection Method	Mitigation Strategy	Tracking	Resolution
Statistical Analysis	Test retry with exponential backoff	Flaky test dashboard	Root cause analysis

Detection Method	Mitigation Strategy	Tracking	Resolution
Historical Data	Quarantine unreliable tests	Success rate monitoring	Test stabilization
Pattern Recognition	Environment-specific test suites	Failure categorization	Infrastructure improvements
Manual Reporting	Developer feedback integration	Issue tracking system	Collaborative debugging

6.6.3 QUALITY METRICS

6.6.3.1 Code Coverage Targets

AgencyOS maintains comprehensive code coverage standards across all system components to ensure thorough testing and minimize production defects.

Coverage Requirements by Component:

Component Type	Line Coverage	Branch Coverage	Function Coverage	Statement Coverage
Business Logic Services	90%	85%	95%	90%
API Controllers	85%	80%	90%	85%
Database Repositories	80%	75%	85%	80%
Utility Functions	95%	90%	100%	95%
React Components	80%	70%	85%	80%
Integration Adapters	75%	70%	80%	75%

Test Success Rate Requirements:

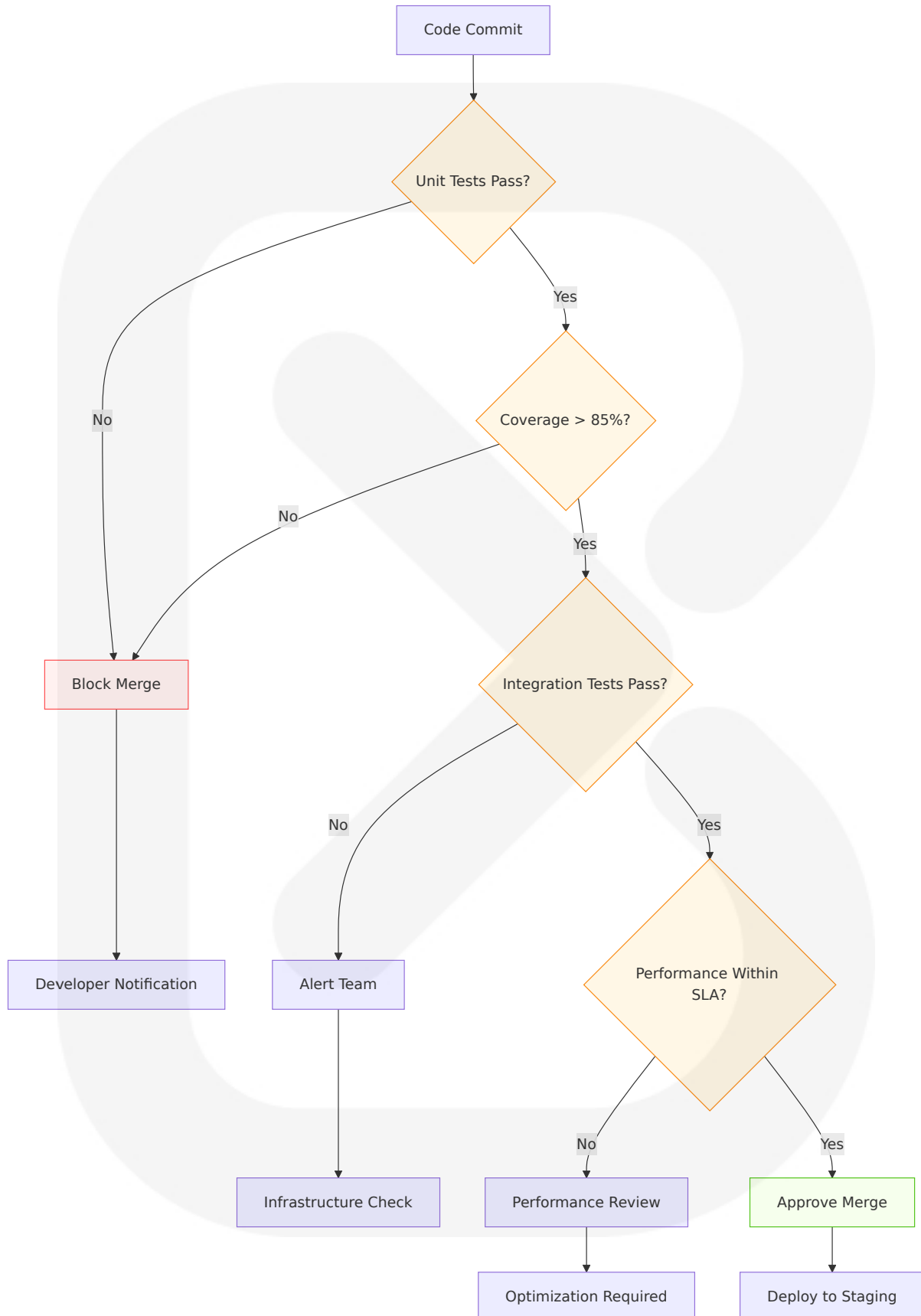
Test Category	Success Rate Target	Measurement Window	Alert Threshold
Unit Tests	100%	Per commit	Any failure blocks merge
Integration Tests	98%	Daily average	<95% triggers investigation
E2E Tests	95%	Weekly average	<90% requires immediate action
Performance Tests	90%	Monthly average	<85% indicates system degradation

Performance Test Thresholds:

Example SLOs: 99% of APIs returning product information respond in less than 600ms, 99.99% of failed log-in requests respond in less than 1000ms.

Performance Metric	Target Threshold	Measurement Method	Failure Action
API Response Time	p95 < 300 ms	k6 load testing simulating multiple users accessing the API	Performance optimization required
Database Query Time	p95 < 50ms	Query execution monitoring	Index optimization needed
Page Load Time	p95 TTI < 2.5s	Lighthouse CI integration	Frontend optimization required
File Upload Speed	10MB < 30 seconds	End-to-end upload testing	Infrastructure scaling needed

Quality Gates:

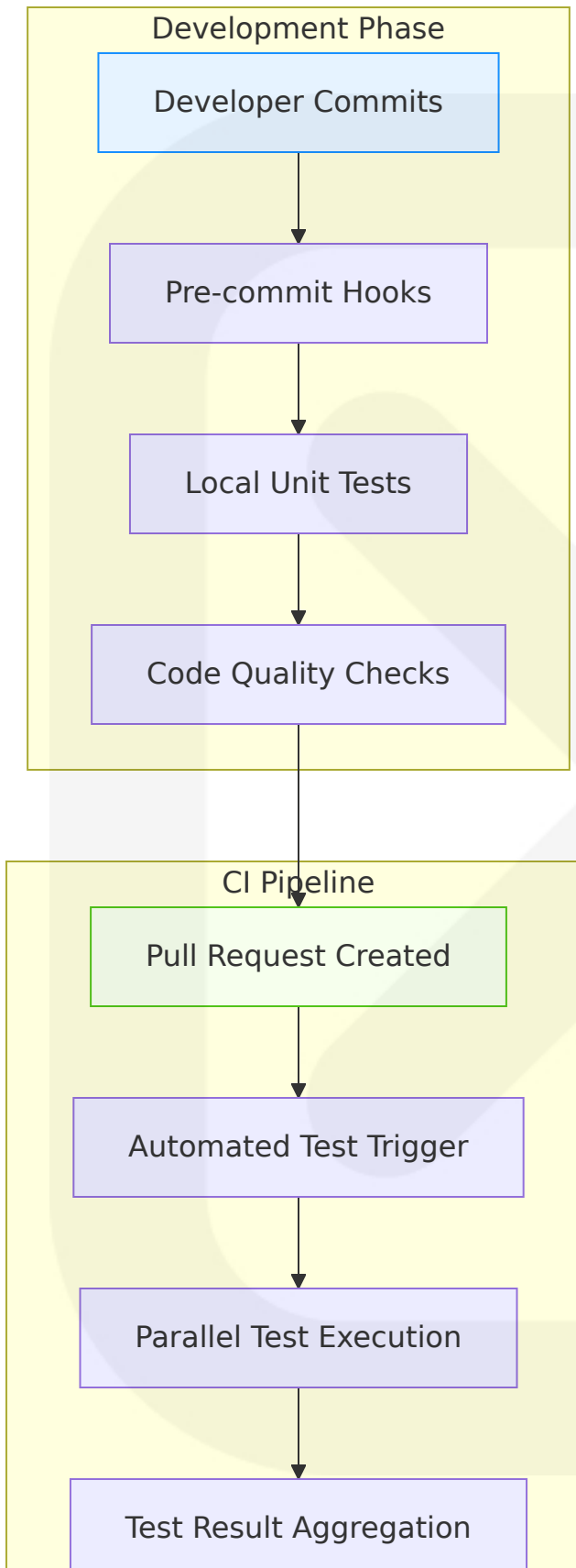


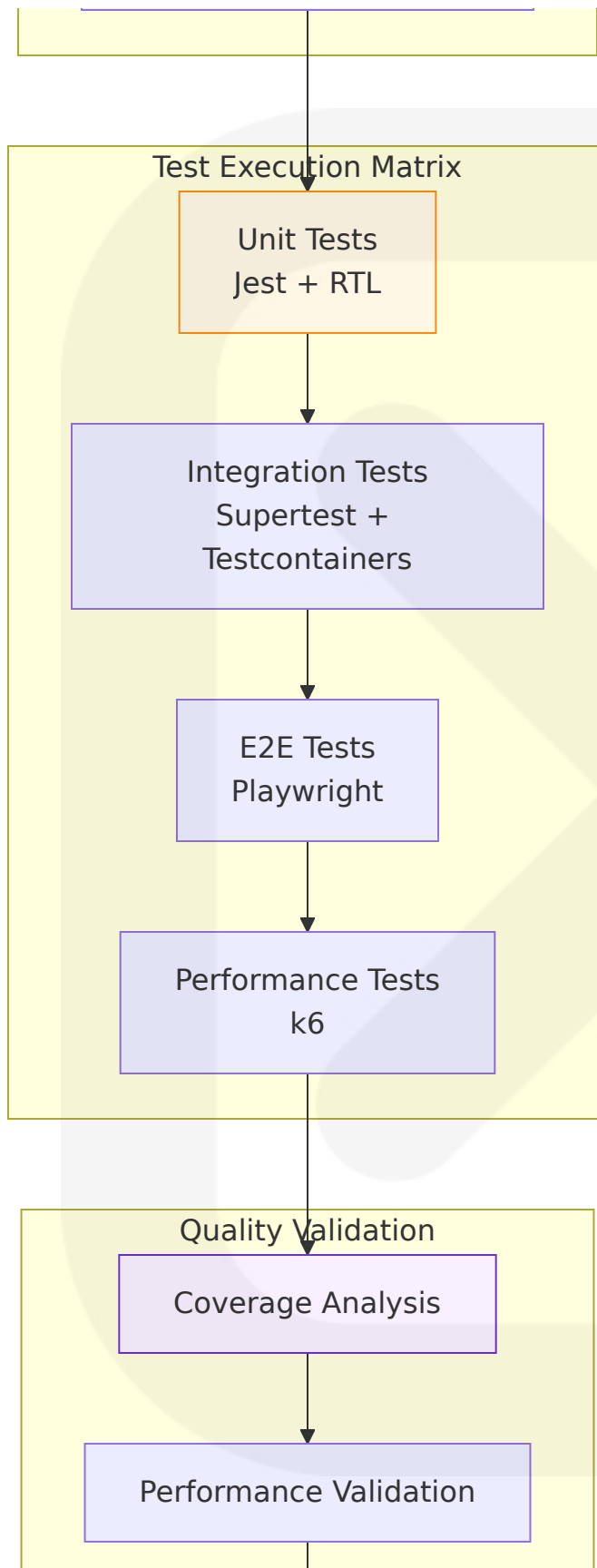
Documentation Requirements:

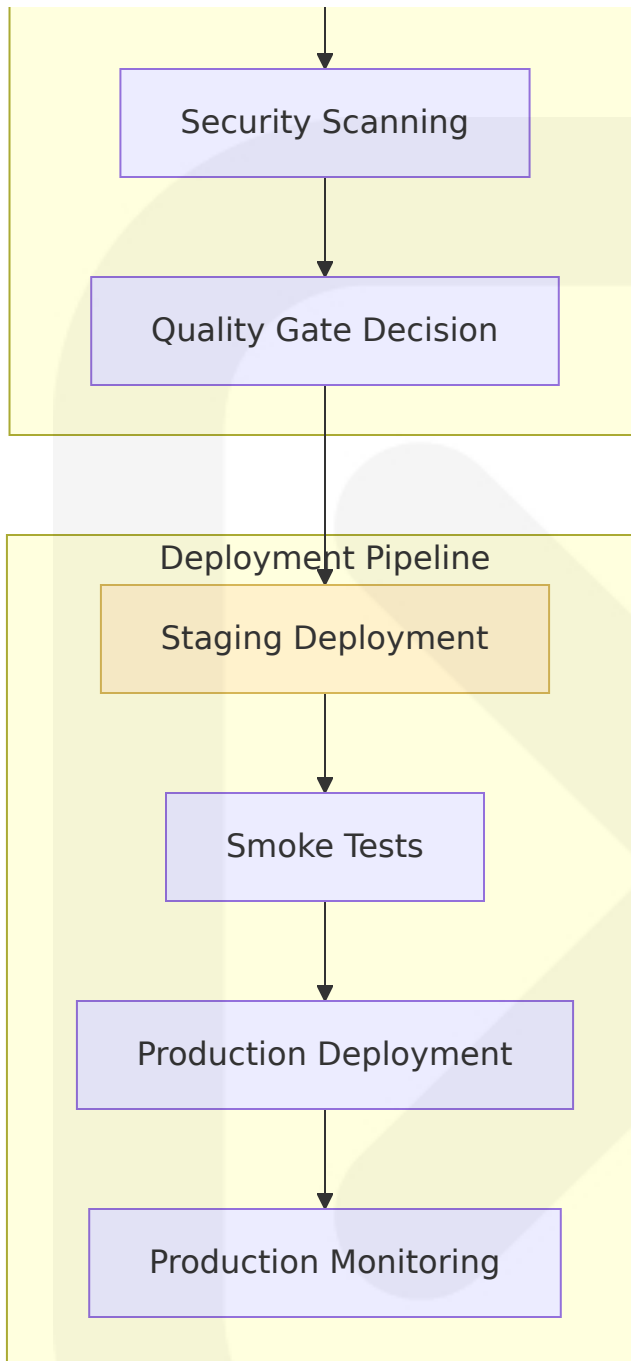
Documentation Type	Coverage Requirement	Update Frequency	Quality Standard
API Documentation	100% endpoint coverage	Every API change	OpenAPI 3.0 compliance
Test Case Documentation	Critical path coverage	Sprint completion	Behavior-driven descriptions
Performance Baselines	All SLA metrics documented	Monthly updates	Historical trend analysis
Security Test Results	Complete vulnerability scan	Weekly security tests	OWASP compliance validation

6.6.4 TEST EXECUTION FLOW

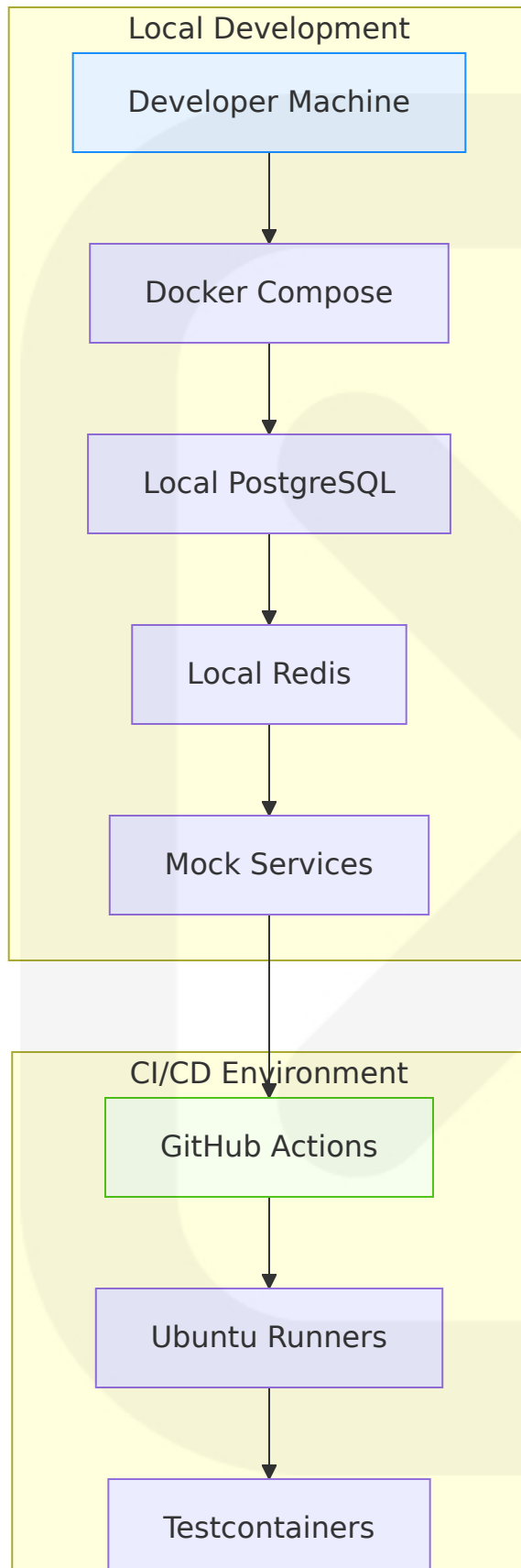
6.6.4.1 Test Execution Architecture

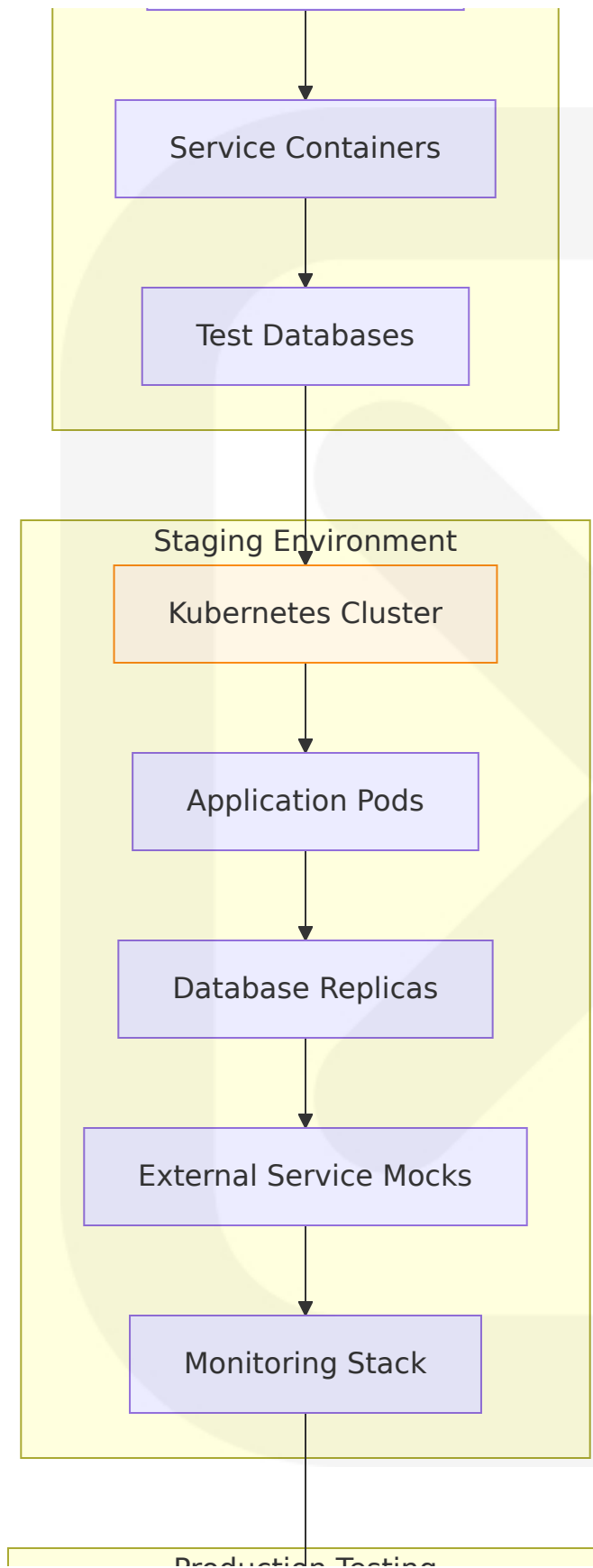


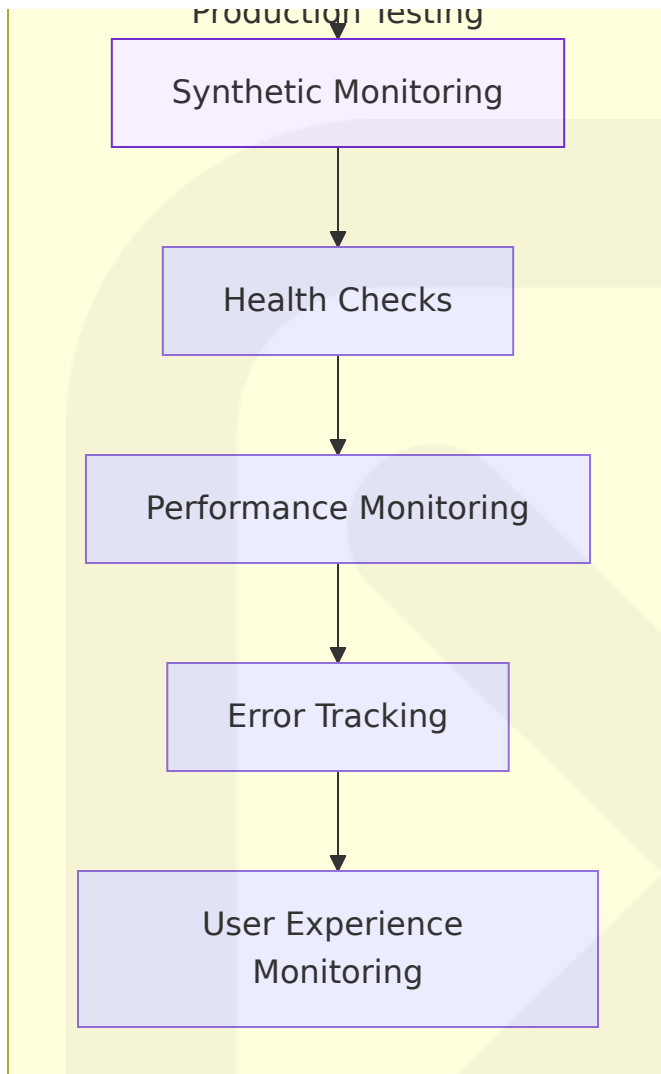




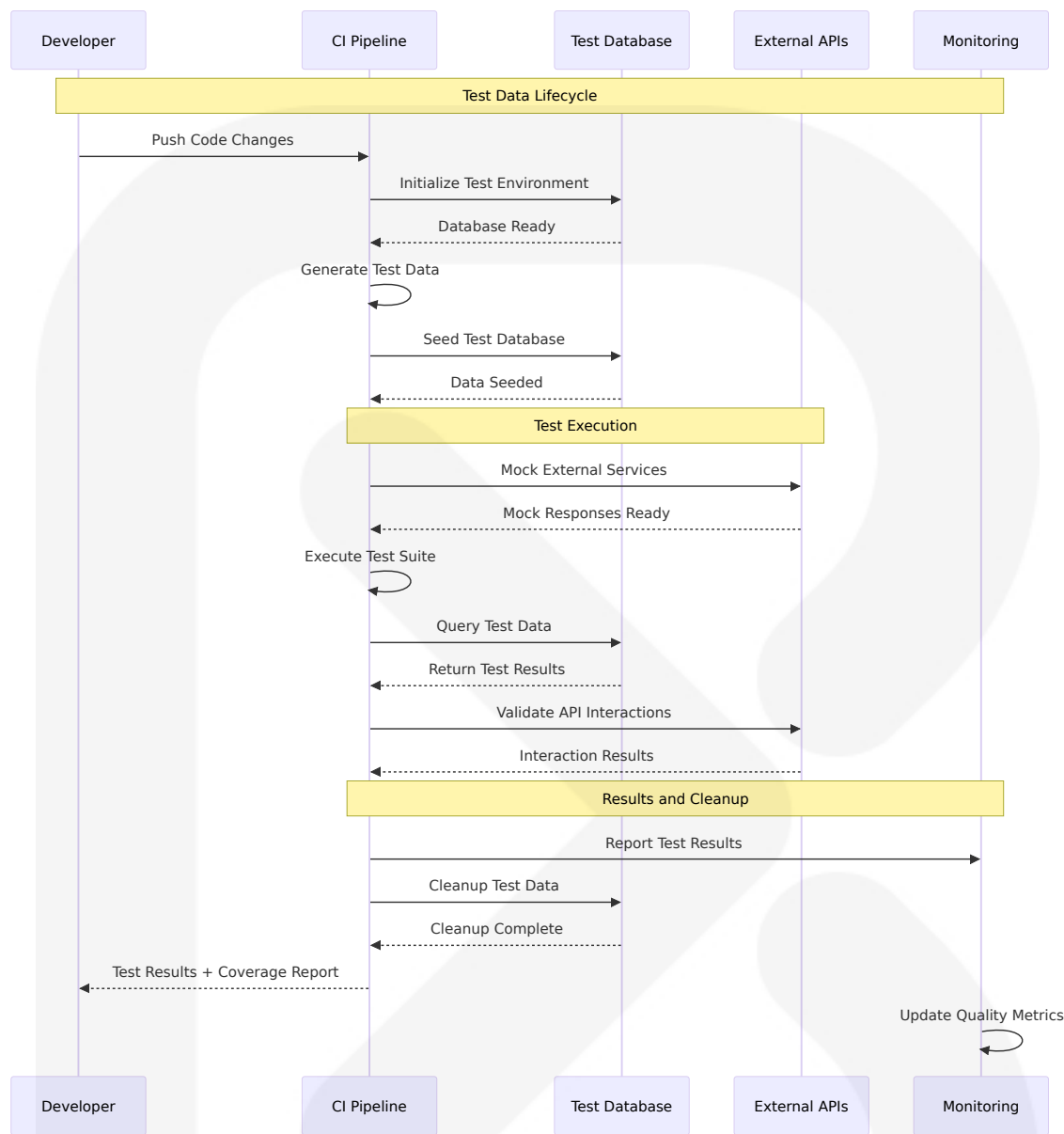
6.6.4.2 Test Environment Architecture







6.6.4.3 Test Data Flow Diagram



This comprehensive Testing Strategy ensures AgencyOS maintains the highest quality standards through automated testing, continuous integration, and comprehensive quality metrics. The strategy leverages modern testing tools and practices to provide confidence in system reliability, performance, and security while supporting the complex multi-tenant, multi-role requirements of digital agency operations.

7. User Interface Design

7.1 CORE UI TECHNOLOGIES

7.1.1 Frontend Technology Stack

AgencyOS implements a modern, component-driven user interface built on Next.js 15 with App Router architecture, React Server Components, and seamless full-stack capabilities. In version 15, the App Router uses React 19 RC, and we've also introduced backwards compatibility for React 18 with the Pages Router based on community feedback.

Primary UI Framework:

Technology	Version	Purpose	Implementation Benefits
React	18.2+	UI Library	Next.js 15 maintains backward compatibility for the Pages Router with React 18, providing stable foundation for complex UI components
Next.js	15.x	Full-Stack Framework	App Router architecture, React Server Components, and seamless full-stack capabilities
TypeScript	5.3+	Type Safety	Enhanced developer experience and compile-time error detection
TanStack Query	5.x	Data Fetching	React Query is compatible with React v18+ and works with ReactDOM and React Native

Component Architecture:

Component Library	Version	Purpose	Accessibility
Radix UI	1.x	Headless Components	Components adhere to the WAI-ARIA design patterns where possible

Component Library	Version	Purpose	Accessibility
		nts	le. We handle many of the difficult implementation details related to accessibility, including aria and role attributes, focus management, and keyboard navigation.
Tailwind CSS	3.4+	Styling Framework	A utility-first CSS framework packed with classes like flex, pt-4, text-center and rotate-90 that can be composed to build any design, directly in your markup.
React Hook Form	7.x	Form Management	Performant form handling with minimal re-renders
Zod	3.x	Schema Validation	Runtime type validation for form data and API responses

7.1.2 Design System Foundation

Headless UI Architecture:

Headless UI is a term for libraries and utilities that provide the logic, state, processing and API for UI elements and interactions, but do not provide markup, styles, or pre-built implementations. The hardest parts of building complex UIs usually revolve around state, events, side-effects, data computation/management. By removing these concerns from the markup, styles and implementation details, our logic and components can be more modular and reusable.

Component Design Philosophy:

| Design Principle | Implementation | Benefits |

|---|---|---|---|

| **Accessibility First** | Our goal is to create a well-funded, open-source component library that the community can use to build accessible design

systems. Components adhere to the WAI-ARIA design patterns where possible. | WCAG 2.2 AA compliance |

| **Utility-First Styling** | A utility-first CSS framework packed with classes like flex, pt-4, text-center and rotate-90 that can be composed to build any design, directly in your markup. | Rapid prototyping and consistent design |

| **Component Composition** | Radix UI primitives with custom styling | Flexible, reusable components |

| **Type Safety** | TypeScript interfaces for all components | Compile-time validation and IntelliSense |

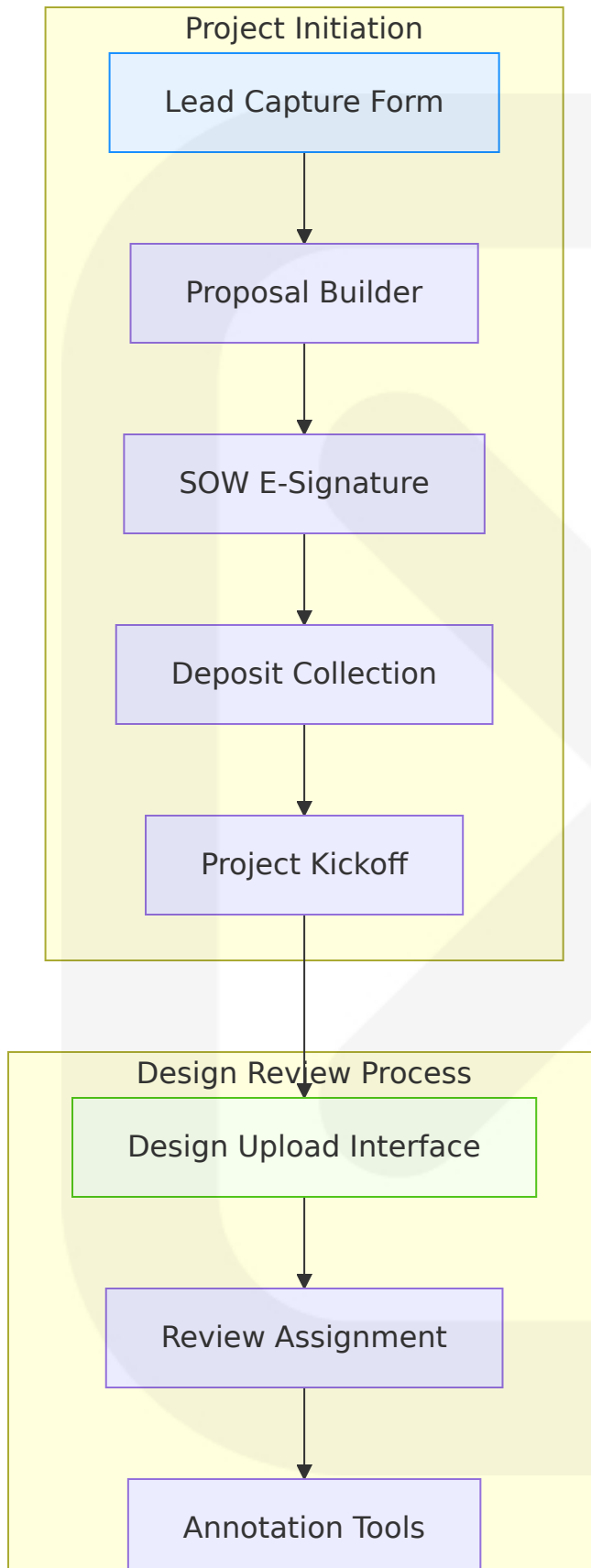
7.2 UI USE CASES

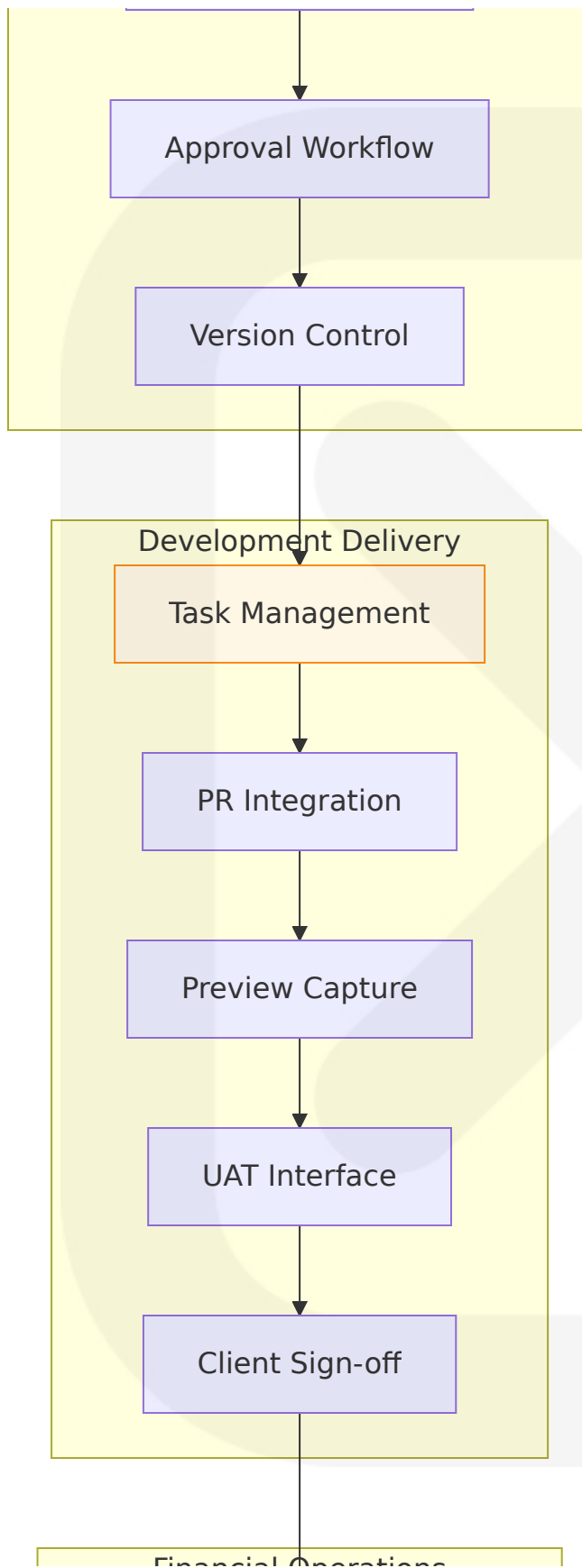
7.2.1 Primary User Workflows

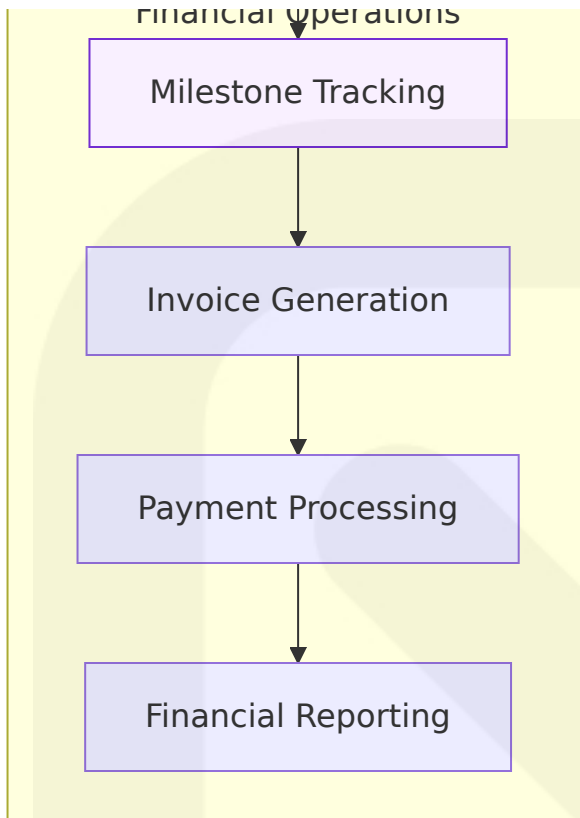
Agency Internal Users:

User Role	Primary UI Workflows	Key Interfaces
Project Manager	Project creation, sprint planning, budget tracking, client communication	Project Dashboard, Resource Planning, Budget Overview
Designer	Design upload, version management, client feedback review	Design Review Interface, Asset Library, Approval Workflows
Engineer	Task management, code integration, deployment tracking	Development Dashboard, PR Integration, UAT Management
Client Admin	Project oversight, design approval, invoice management	Client Portal, Approval Interface, Financial Dashboard

Core User Journey Flows:







7.2.2 Multi-Tenant Interface Requirements

Workspace Customization:

Customization Level	Features	Implementation
Branding	Custom logos, color schemes, typography	CSS custom properties with Tailwind theme extension
Domain Configuration	Custom subdomains, white-label client portals	Next.js middleware for domain routing
Feature Flags	Tenant-specific feature availability	React Context with server-side feature detection
Role-Based UI	Dynamic interface based on user permissions	Conditional rendering with RBAC integration

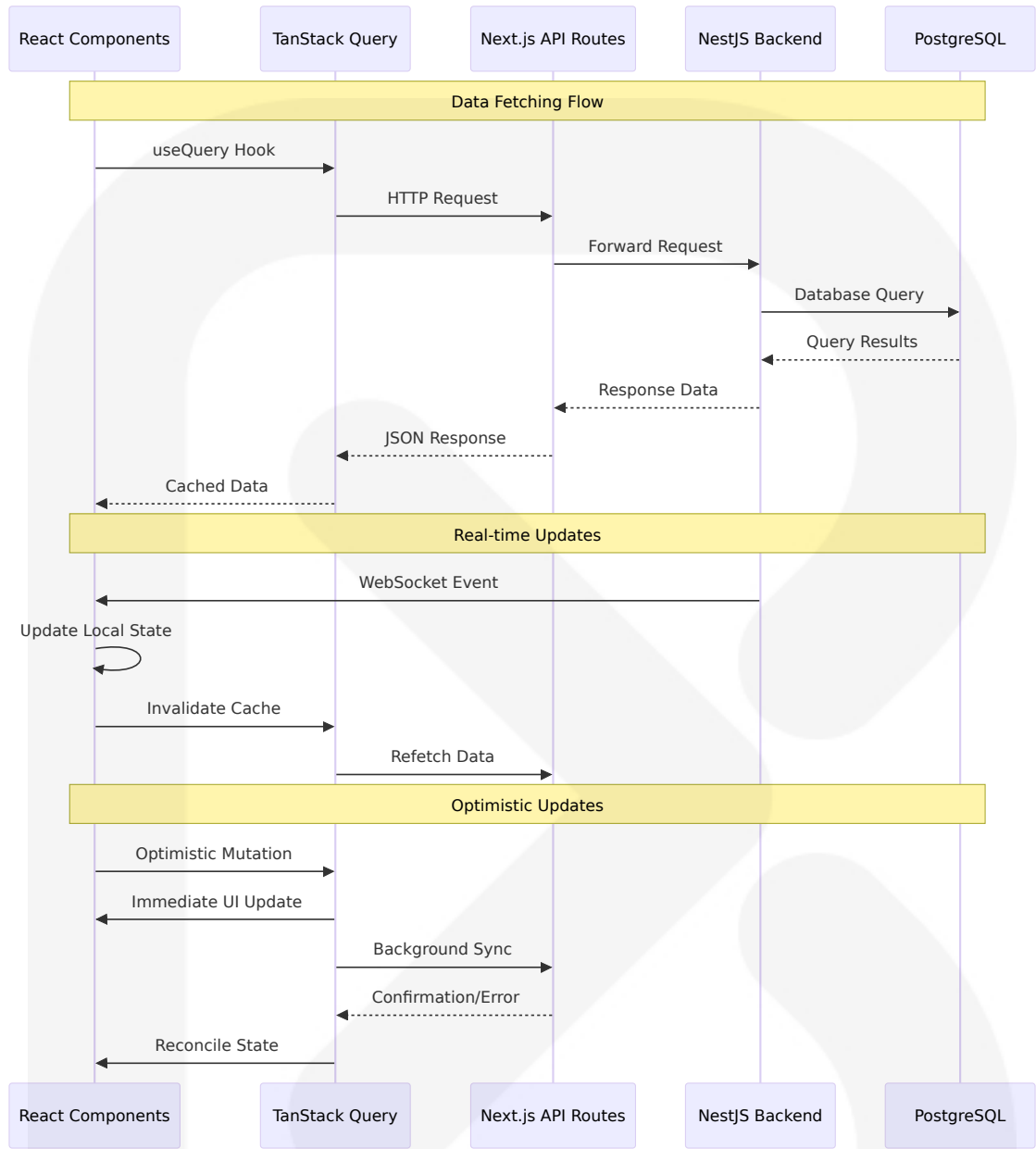
Responsive Design Requirements:

Breakpoint	Target Devices	UI Adaptations
Mobile	320px - 768px	Collapsible navigation, touch-optimized controls, simplified layouts
Tablet	768px - 1024px	Sidebar navigation, grid-based layouts, touch and mouse support
Desktop	1024px+	Full feature set, multi-column layouts, keyboard shortcuts

7.3 UI/BACKEND INTERACTION BOUNDARIES

7.3.1 Data Flow Architecture

Client-Server Communication Patterns:



State Management Boundaries:

State Type	Location	Technology	Synchronization
UI State	Client-side	React useState/useReducer	Local component state
Server State	Client cache	TanStack Query	Background synchronization

State Type	Location	Technology	Synchronization
Form State	Client-side	React Hook Form	Validation on submit
Global State	Client context	React Context	Event-driven updates

7.3.2 API Integration Patterns

RESTful API Design:

Endpoint Pattern	HTTP Method	UI Integration	Error Handling
/api/projects	GET, POST	Project listing, creation forms	Toast notifications, form validation
/api/projects/{id}	GET, PUT, DELETE	Project details, editing	Optimistic updates with rollback
/api/design-reviews	GET, POST	Design review interface	Real-time status updates
/api/integrations/{service}	POST, PUT	Integration configuration	Connection status indicators

Real-time Communication:

```
// WebSocket integration for real-time updates
interface RealtimeConnection {
  // Design review collaboration
  subscribeToReview(reviewId: string): void
  publishComment(reviewId: string, comment: Comment): void

  // Project status updates
  subscribeToProject(projectId: string): void
  publishStatusUpdate(projectId: string, status: ProjectStatus): void

  // Notification system
  subscribeToNotifications(userId: string): void
```

```
markNotificationRead(notificationId: string): void  
}
```

7.3.3 Authentication Integration

JWT Token Management:

Token Type	Storage	Refresh Strategy	UI Integration
Access Token	Memory only	Automatic refresh	Axios interceptors
Refresh Token	HTTP-only cookie	Rotation on use	Silent refresh
Session State	React Context	Server validation	Route protection

Role-Based UI Rendering:

```
// Permission-based component rendering  
interface RoleBasedComponent {  
  requiredPermissions: Permission[]  
  fallbackComponent?: React.ComponentType  
  children: React.ReactNode  
}  
  
// Usage example  
<RoleBasedComponent requiredPermissions={['PROJECT_EDIT']}>  
  <ProjectEditForm />  
</RoleBasedComponent>
```

7.4 UI SCHEMAS

7.4.1 Component Data Models

Project Dashboard Schema:

```
interface ProjectDashboardData {
  project: {
    id: string
    name: string
    status: 'proposal' | 'active' | 'completed' | 'on_hold'
    progress: number
    budget: {
      allocated: number
      spent: number
      remaining: number
      currency: string
    }
    timeline: {
      startDate: Date
      endDate: Date
      milestones: Milestone[]
    }
    team: TeamMember[]
    client: ClientInfo
  }
  metrics: {
    tasksCompleted: number
    totalTasks: number
    hoursLogged: number
    estimatedHours: number
    approvalCycleTime: number
  }
  recentActivity: Activity[]
}
```

Design Review Schema:

```
interface DesignReviewData {
  review: {
    id: string
    title: string
    status: 'pending' | 'in_review' | 'approved' | 'changes_requested'
    createdAt: Date
    deadline?: Date
    figmaFileKey?: string
    figmaNodeId?: string
  }
}
```

```

    }
    versions: DesignVersion[]
    comments: ReviewComment[]
    approvers: Approver[]
    annotations: Annotation[]
  }

  interface ReviewComment {
    id: string
    authorId: string
    content: string
    coordinates?: { x: number; y: number }
    timestamp: Date
    resolved: boolean
    replies: ReviewComment[]
  }

```

7.4.2 Form Validation Schemas

Project Creation Form:

```

import { z } from 'zod'

const ProjectCreationSchema = z.object({
  name: z.string().min(1, 'Project name is required').max(100),
  clientId: z.string().uuid('Invalid client ID'),
  description: z.string().optional(),
  budget: z.object({
    amount: z.number().positive('Budget must be positive'),
    currency: z.enum(['USD', 'EUR', 'GBP']).default('USD')
  }),
  timeline: z.object({
    startDate: z.date(),
    endDate: z.date(),
    milestones: z.array(z.object({
      name: z.string().min(1),
      dueDate: z.date(),
      deliverables: z.array(z.string())
    }))
  })
}).refine(data => data.endDate > data.startDate, {
  message: 'End date must be after start date'
})

```



```

    })),
    team: z.array(z.object({
      userId: z.string().uuid(),
      role: z.enum(['pm', 'designer', 'engineer', 'qa']),
      allocation: z.number().min(0).max(100)
    }))
  })
})

```

Design Review Form:

```

const DesignReviewSchema = z.object({
  title: z.string().min(1, 'Review title is required'),
  projectId: z.string().uuid(),
  taskId: z.string().uuid().optional(),
  figmaFileKey: z.string().optional(),
  reviewType: z.enum(['design', 'prototype', 'final']),
  approvers: z.array(z.string().uuid()).min(1, 'At least one approver required'),
  deadline: z.date().optional(),
  instructions: z.string().optional(),
  files: z.array(z.object({
    name: z.string(),
    url: z.string().url(),
    type: z.enum(['pdf', 'image', 'video', 'figma'])
  })))
})

```

7.4.3 API Response Schemas

Standardized API Response Format:

```

interface APIResponse<T> {
  success: boolean
  data?: T
  error?: {
    code: string
    message: string
    details?: Record<string, any>
  }
}

```

```
meta?: {
  pagination?: {
    page: number
    limit: number
    total: number
    totalPages: number
  }
  timestamp: string
  requestId: string
}
}

// Usage examples
type ProjectListResponse = APIResponse<{
  projects: Project[]
}>

type ProjectDetailResponse = APIResponse<{
  project: Project
  permissions: Permission[]
}>
```

7.5 SCREENS REQUIRED

7.5.1 Agency Internal Screens

Dashboard and Overview Screens:

Screen Name	Purpose	Key Components	Responsive Behavior
Main Dashboard	System overview, quick actions	Project cards, metrics widgets, activity feed	Stacked layout on mobile, grid on desktop
Project Dashboard	Individual project management	Timeline, budget tracker, team overview, task board	Tabbed interface on mobile
Resource Planning	Team utilization and capacity	Calendar view, utilization charts, capacity	Simplified view on mobile

Screen Name	Purpose	Key Components	Responsive Behavior
	ty	city planning	
Financial Overview	Revenue, margins, cash flow	Charts, invoice status, payment tracking	Scrollable cards on mobile

Project Management Screens:

Screen Name	Purpose	Key Components	User Roles
Project Creation	New project setup	Multi-step form, client selection, team assignment	Admin, PM
Sprint Planning	Task organization and planning	Drag-and-drop task board, sprint timeline	PM, Team Leads
Task Management	Individual task tracking	Task details, time tracking, status updates	All team members
Budget Tracking	Financial project oversight	Budget vs. actual, expense tracking, alerts	PM, Finance

Design and Review Screens:

Screen Name	Purpose	Key Components	Integration Points
Design Upload	Asset management and versioning	File upload, metadata entry, version control	Figma API, S3 storage
Design Review Interface	Collaborative design feedback	Annotation tools, comment threads, approval buttons	Figma integration, WebSocket
Asset Library	Centralized asset management	Search, filtering, usage tracking, rights management	File storage, metadata DB

Screen Name	Purpose	Key Components	Integration Points
Approval Workflows	Structured approval processes	Approval chains, status tracking, notifications	Email, Slack integration

7.5.2 Client Portal Screens

Client-Facing Interface:

Screen Name	Purpose	Key Components	Access Control
Client Dashboard	Project overview for clients	Project status, timeline, recent updates	Client Admin, Reviewer
Design Review Portal	Client design feedback interface	Design viewer, annotation tools, approval buttons	Client Admin, Reviewer
File Access	Document and asset downloads	File browser, download tracking, access logs	Role-based file access
Invoice Portal	Billing and payment management	Invoice viewer, payment processing, history	Client Admin, Billing

Communication Screens:

Screen Name	Purpose	Key Components	Notification Integration
Message Center	Project communication hub	Message threads, file attachments, read receipts	Email, Slack notifications
Meeting Scheduler	Appointment coordination	Calendar integration, availability, meeting links	Zoom, Calendly integration

Screen Name	Purpose	Key Components	Notification Integration
Support Tickets	Issue tracking and resolution	Ticket creation, status tracking, SLA timers	Automated escalation

7.5.3 Administrative Screens

System Administration:

Screen Name	Purpose	Key Components	Security Level
Tenant Management	Workspace configuration	Branding settings, feature flags, user limits	Super Admin only
User Management	Account and role administration	User creation, role assignment, permissions	Admin, Super Admin
Integration Settings	Third-party service configuration	API keys, webhook setup, connection testing	Admin level
Audit Logs	System activity monitoring	Log viewer, filtering, export capabilities	Admin, Compliance

Reporting and Analytics:

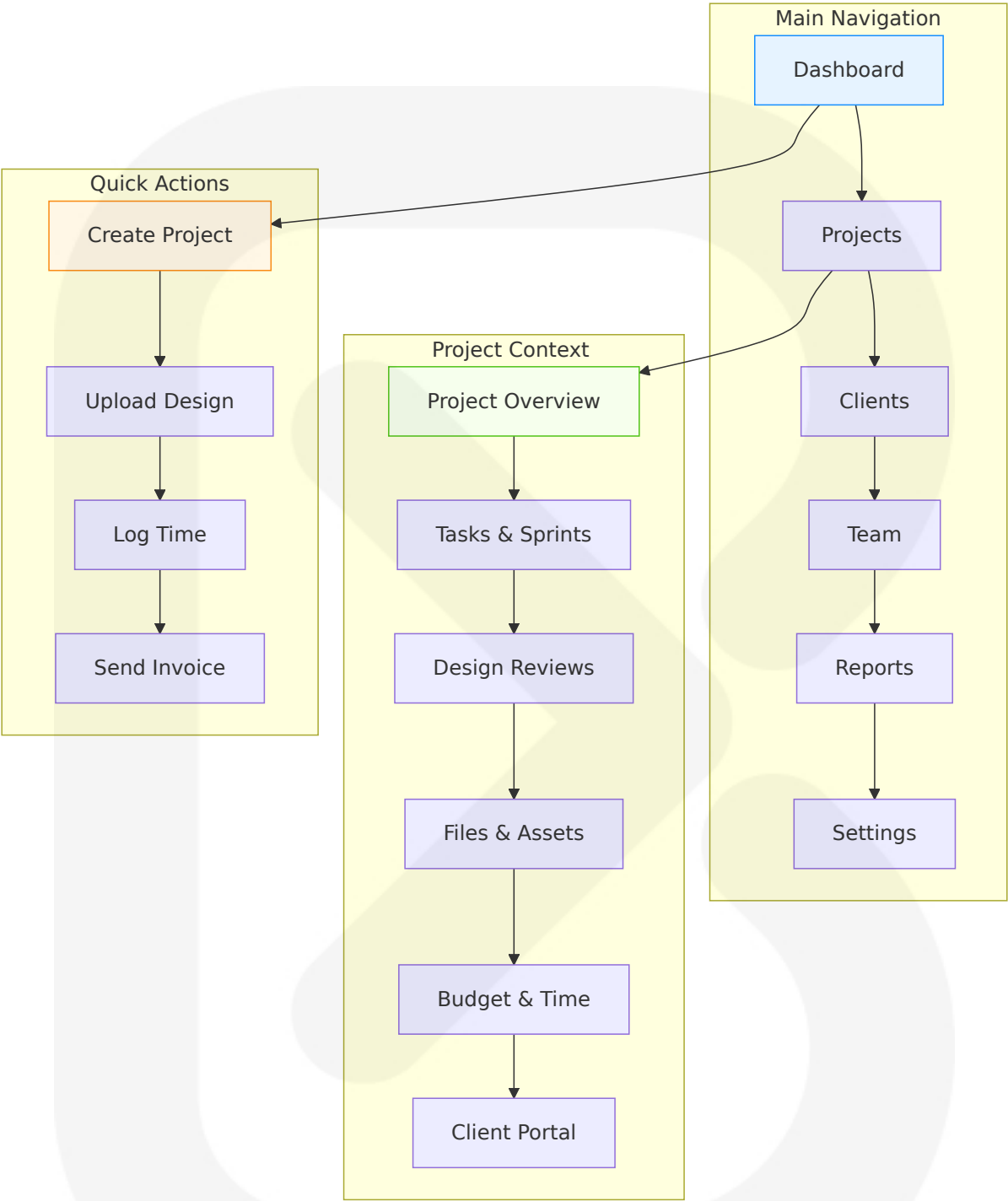
Screen Name	Purpose	Key Components	Data Sources
Performance Dashboard	System metrics and KPIs	Charts, alerts, trend analysis	Application metrics, business data
Financial Reports	Revenue and profitability analysis	P&L reports, margin analysis, forecasting	Financial data, time tracking

Screen Name	Purpose	Key Components	Data Sources
Client Health	Relationship and satisfaction metrics	Satisfaction scores, engagement metrics, risk indicators	Project data, feedback systems

7.6 USER INTERACTIONS

7.6.1 Navigation Patterns

Primary Navigation Structure:



Responsive Navigation Behavior:

Screen Size	Navigation Pattern	Interaction Method
Mobile (< 768px)	Bottom tab bar + hamburger menu	Touch gestures, swipe navigation

Screen Size	Navigation Pattern	Interaction Method
Tablet (768px - 1024px)	Collapsible sidebar	Touch and mouse support
Desktop (> 1024px)	Persistent sidebar + top navigation	Keyboard shortcuts, mouse interaction

7.6.2 Form Interactions

Multi-Step Form Patterns:

Form Type	Steps	Validation Strategy	Save Behavior
Project Creation	1. Basic Info, 2. Team & Budget, 3. Timeline, 4. Review	Step-by-step validation	Auto-save drafts
Design Review Setup	1. Files & Details, 2. Reviewers, 3. Instructions	Real-time validation	Immediate save
Client Onboarding	1. Company Info, 2. Contacts, 3. Preferences	Progressive validation	Checkpoint saves

Form Interaction Patterns:

```
// Form state management with React Hook Form
interface FormInteractionPattern {
  // Real-time validation
  validateOnChange: boolean
  validateOnBlur: boolean

  // Auto-save functionality
  autoSaveInterval: number
  autoSaveOnChange: boolean

  // Progress indication
  showProgress: boolean
  allowStepSkipping: boolean

  // Error handling
  showInlineErrors: boolean
```



```
scrollToFirstError: boolean  
}
```

7.6.3 Real-Time Collaboration Features

Design Review Collaboration:

Interaction Type	Implementation	User Feedback	Conflict Resolution
Live Cursors	WebSocket position updates	Colored cursor indicators	Position smoothing
Simultaneous Annotations	Real-time comment sync	Immediate visual feedback	Timestamp-based ordering
Approval Status	Live status updates	Status badges, notifications	Last action wins
Version Control	Automatic version creation	Version history timeline	Merge conflict detection

Collaborative Editing Patterns:

```
interface CollaborationFeatures {  
  // Real-time presence  
  showActiveCursors: boolean  
  displayUserAvatars: boolean  
  
  // Conflict resolution  
  optimisticUpdates: boolean  
  conflictResolutionStrategy: 'last-write-wins' | 'manual-merge'  
  
  // Notification system  
  realTimeNotifications: boolean  
  emailDigestFrequency: 'immediate' | 'hourly' | 'daily'  
}
```

7.6.4 Keyboard and Accessibility Interactions

Keyboard Navigation Support:

Interaction	Keyboard Shortcut	Screen Reader Support	Focus Management
Quick Search	Cmd/Ctrl + K	Search landmark, results announced	Focus trap in search modal
Create Project	Cmd/Ctrl + N	Button role, action announced	Focus on first form field
Save Changes	Cmd/Ctrl + S	Save status announced	Focus remains on current field
Navigate Tabs	Arrow keys	Tab role, selected state	Roving tabindex

Accessibility Implementation:

```
interface AccessibilityFeatures {  
  // WCAG 2.2 AA Compliance  
  colorContrastRatio: number // >= 4.5:1  
  focusIndicators: boolean  
  keyboardNavigation: boolean  
  screenReaderSupport: boolean  
  
  // Internationalization  
  rightToLeftSupport: boolean  
  languageSelection: string[]  
  dateTimeLocalization: boolean  
  
  // Motor accessibility  
  clickTargetSize: number // >= 44px  
  dragAndDropAlternatives: boolean  
  timeoutExtensions: boolean  
}
```

7.7 VISUAL DESIGN CONSIDERATIONS

7.7.1 Design System Architecture

Color System:

Color Category	Usage	Tailwind Classes	Accessibility
Primary	Brand colors, CTAs, active states	bg-blue-600 , text-blue-600	4.5:1 contrast ratio minimum
Secondary	Supporting actions, highlights	bg-gray-600 , text-gray-600	WCAG AA compliant
Semantic	Success, warning, error states	bg-green-600 , bg-red-600	Color-blind friendly palette
Neutral	Text, borders, backgrounds	bg-gray-50 to bg-gray-900	Sufficient contrast at all levels

Typography Scale:

Text Style	Tailwind Classes	Usage	Responsive Behavior
Display	text-4xl lg:text-6xl	Page headers, hero text	Scales down on mobile
Heading	text-xl lg:text-2xl	Section headers	Maintains hierarchy
Body	text-base lg:text-lg	Main content	Readable at all sizes
Caption	text-sm	Metadata, labels	Consistent across breakpoints

7.7.2 Component Visual Patterns

Card-Based Layout System:

```
interface CardDesignPattern {
  // Visual hierarchy
  elevation: 'flat' | 'raised' | 'floating'
  borderRadius: 'none' | 'sm' | 'md' | 'lg'

  // Content organization
  header: boolean
  footer: boolean
  padding: 'compact' | 'comfortable' | 'spacious'

  // Interactive states
  hover: boolean
  focus: boolean
  selected: boolean
}
```

Data Visualization Patterns:

Chart Type	Use Case	Library	Responsive Design
Progress Bars	Project completion, budget usage	Custom CSS + Tailwind	Stacked on mobile
Time Series	Performance metrics, usage trends	Chart.js/D3	Simplified mobile view
Status Indicators	Health checks, approval states	Custom components	Icon-based on mobile
Comparison Charts	Budget vs. actual, team utilization	Chart.js	Horizontal layout mobile

7.7.3 Responsive Design Strategy

Breakpoint System:

Breakpoint	Screen Size	Layout Strategy	Component Behavior
Mobile	320px - 767px	Single column, stacked content	Simplified interfaces, touch-optimized

Breakpoint	Screen Size	Layout Strategy	Component Behavior
Tablet	768px - 1023px	Two-column, sidebar navigation	Hybrid touch/mouse interactions
Desktop	1024px - 1439px	Multi-column, persistent navigation	Full feature set, keyboard shortcuts
Large Desktop	1440px+	Wide layouts, additional sidebars	Enhanced data density

Mobile-First Design Principles:

```
/* Mobile-first responsive design with Tailwind */
.project-card {
  @apply w-full p-4 mb-4;
  @apply md:w-1/2 md:p-6;
  @apply lg:w-1/3 lg:p-8;
  @apply xl:w-1/4;
}

.navigation {
  @apply fixed bottom-0 w-full bg-white border-t;
  @apply md:static md:w-64 md:h-screen md:border-r md:border-t-0;
}
```

7.7.4 Dark Mode and Theming

Theme System Implementation:

Theme Variant	Color Palette	Usage Context	Implementation
Light Mode	High contrast, bright backgrounds	Default, daytime usage	CSS custom properties
Dark Mode	Reduced eye strain, dark backgrounds	Low-light environments	dark: Tailwind variants
High Contrast	Maximum accessibility	Accessibility requirements	WCAG AAA compliance

Theme Variant	Color Palette	Usage Context	Implementation
Custom Branding	Client-specific colors	White-label deployments	Dynamic CSS variables

Theme Switching Logic:

```
interface ThemeSystem {
  // Theme detection
  systemPreference: 'light' | 'dark'
  userPreference: 'light' | 'dark' | 'system'

  // Custom theming
  brandColors: {
    primary: string
    secondary: string
    accent: string
  }

  // Accessibility
  highContrast: boolean
  reducedMotion: boolean
  fontSize: 'small' | 'medium' | 'large'
}
```

7.7.5 Animation and Micro-Interactions

Motion Design Principles:

Animation Type	Duration	Easing	Purpose
Page Transitions	300ms	ease-in-out	Smooth navigation
Modal Animations	200ms	ease-out	Attention focusing
Loading States	Continuous	linear	Progress indication

Animation Type	Duration	Easing	Purpose
Hover Effects	150ms	ease-in-out	Interactive feedback

Micro-Interaction Patterns:

```
interface MicroInteractions {  
  // Button interactions  
  hoverScale: boolean  
  clickFeedback: boolean  
  loadingStates: boolean  
  
  // Form interactions  
  focusAnimations: boolean  
  validationFeedback: boolean  
  successConfirmations: boolean  
  
  // Data updates  
  optimisticUpdates: boolean  
  errorRecovery: boolean  
  refreshAnimations: boolean  
}
```

This comprehensive User Interface Design specification ensures AgencyOS delivers a modern, accessible, and highly functional interface that supports the complex workflows of digital agencies while maintaining excellent user experience across all device types and user roles. The design system leverages Tailwind Plus collection of beautiful, fully responsive UI components, designed and developed by us, the creators of Tailwind CSS. It's got hundreds of ready-to-use examples to choose from, and is guaranteed to help you find the perfect starting point for what you want to build.

8. Infrastructure

8.1 DEPLOYMENT ENVIRONMENT

8.1.1 Target Environment Assessment

Environment Type and Architecture

AgencyOS is designed as a **cloud-native, multi-tenant SaaS platform** deployed on Amazon Web Services (AWS) with a hybrid cloud approach that supports both managed services and containerized workloads. The architecture prioritizes scalability, security, and operational efficiency while maintaining cost optimization for the target market of digital agencies.

Primary Environment Characteristics:

Environment Aspect	Specification	Justification
Deployment Model	Cloud-native on AWS	Scalability, managed services, global reach
Architecture Pattern	Microservices-ready monolith	Development velocity with scaling flexibility
Containerization	Docker with Kubernetes orchestration	Portability, scaling, resource efficiency
Multi-tenancy	Shared infrastructure, isolated data	Cost efficiency with security compliance

Geographic Distribution Requirements

AgencyOS implements a **multi-region deployment strategy** to serve global digital agencies with optimal performance and compliance requirements.

Regional Distribution Strategy:

Region	Primary Purpose	Target Markets	Compliance Requirements
US East (N. Virginia)	Primary production region	North American agencies	SOC 2 Type II, C CPA
EU West (Ireland)	European operations	European agencies	GDPR, data residency
Asia Pacific (Sydney)	APAC operations	Australian/NZ agencies	Local data sovereignty
US West (Oregon)	Disaster recovery	Backup and failover	Business continuity

Resource Requirements Analysis

Based on the expected user base of 5-500 users per organization and the complex multi-tenant architecture, AgencyOS requires substantial compute, memory, and storage resources.

Compute Requirements:

Component	Instance Type	vCPU	Memory	Storage	Scaling Strategy
Application Servers	t3.large → c5.2xlarge	2-8 vCPU	8-16 GB	100 GB SSD	Horizontal auto-scaling
Database Primary	r5.xlarge → r5.4xlarge	4-16 vCPU	32-128 GB	1 TB SSD	Vertical scaling + read replicas
Cache Layer	r5.large → r5.2xlarge	2-8 vCPU	16-64 GB	Memory-optimized	Redis cluster scaling
Background Workers	c5.large → c5.xlarge	2-4 vCPU	4-8 GB	50 GB SSD	Queue-based auto-scaling

Storage Requirements:

Storage Type	Initial Capacity	Growth Rate	Retention Policy	Implementation
Database Storage	500 GB	50 GB/month	7 years transactional data	EBS gp3 with auto-scaling
File Assets	1 TB	100 GB/month	Indefinite with lifecycle	S3 with Intelligent Tiering
Backup Storage	2 TB	150 GB/month	90 days active, 7 years archive	S3 Glacier for long-term
Log Storage	100 GB	20 GB/month	2 years audit logs	CloudWatch Logs + S3

Compliance and Regulatory Requirements

AgencyOS must meet stringent compliance requirements to serve enterprise clients and handle sensitive business data.

Compliance Framework:

Regulation	Scope	Implementation Requirements	Audit Frequency
SOC 2 Type II	Security, availability, confidentiality	Access controls, encryption, monitoring	Annual third-party audit
GDPR	EU personal data processing	Data minimization, consent, right to erasure	Continuous compliance monitoring
CCPA	California resident data	Transparency, opt-out rights, data deletion	Self-assessment with legal review
WCAG 2.2 AA	Accessibility compliance	UI/UX accessibility standards	Quarterly accessibility testing

8.1.2 Environment Management

Infrastructure as Code (IaC) Approach

AgencyOS implements a comprehensive Infrastructure as Code strategy using **Terraform** for infrastructure provisioning and **AWS Systems Manager** for configuration management.

IaC Technology Stack:

Component	Technology	Version	Purpose
Infrastructure Provisioning	Terraform	1.6+	AWS resource creation and management
Configuration Management	AWS Systems Manager	Latest	Application configuration and secrets
State Management	Terraform Cloud	Enterprise	Remote state with collaboration
Policy as Code	Open Policy Agent (OPA)	0.58+	Infrastructure compliance validation

Terraform Module Structure:

```
# Root module structure
modules/
├─ vpc/                # Network infrastructure
├─ eks/                # Kubernetes cluster
├─ rds/                # Database infrastructure
├─ elasticache/        # Redis cache
├─ s3/                 # Object storage
├─ cloudfront/          # CDN distribution
├─ route53/             # DNS management
├─ iam/                 # Identity and access management
├─ monitoring/          # CloudWatch and alerting
└─ security/            # Security groups and NACLs

#### Environment-specific configurations
environments/
├─ dev/
└─ staging/
```

```
| production/
| dr/
# Disaster recovery
```

Configuration Management Strategy

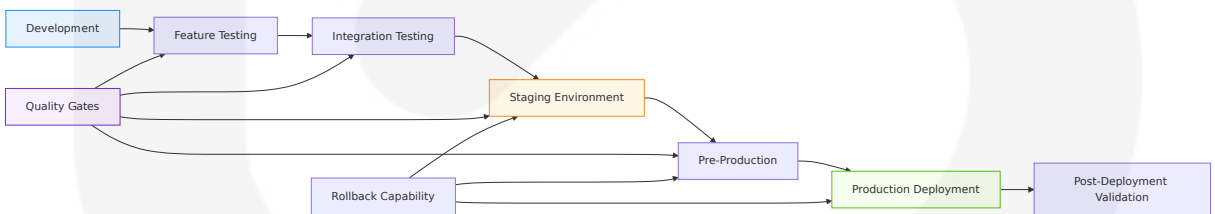
Configuration Hierarchy:

Configuration Level	Technology	Scope	Update Frequency
Infrastructure Config	Terraform variables	AWS resources, networking	Monthly releases
Application Config	AWS Parameter Store	Feature flags, API keys	Weekly updates
Runtime Config	Kubernetes ConfigMaps	Application settings	Daily deployments
Secrets Management	AWS Secrets Manager	Database credentials, API tokens	On rotations schedule

Environment Promotion Strategy

AgencyOS implements a **progressive deployment strategy** with automated promotion gates and comprehensive testing at each stage.

Environment Promotion Flow:



Promotion Criteria:

Environment	Promotion Requirements	Validation Tests	Approval Process
Dev → Staging	All unit tests pass, code review approved	Automated test suite, security scan	Automated promotion

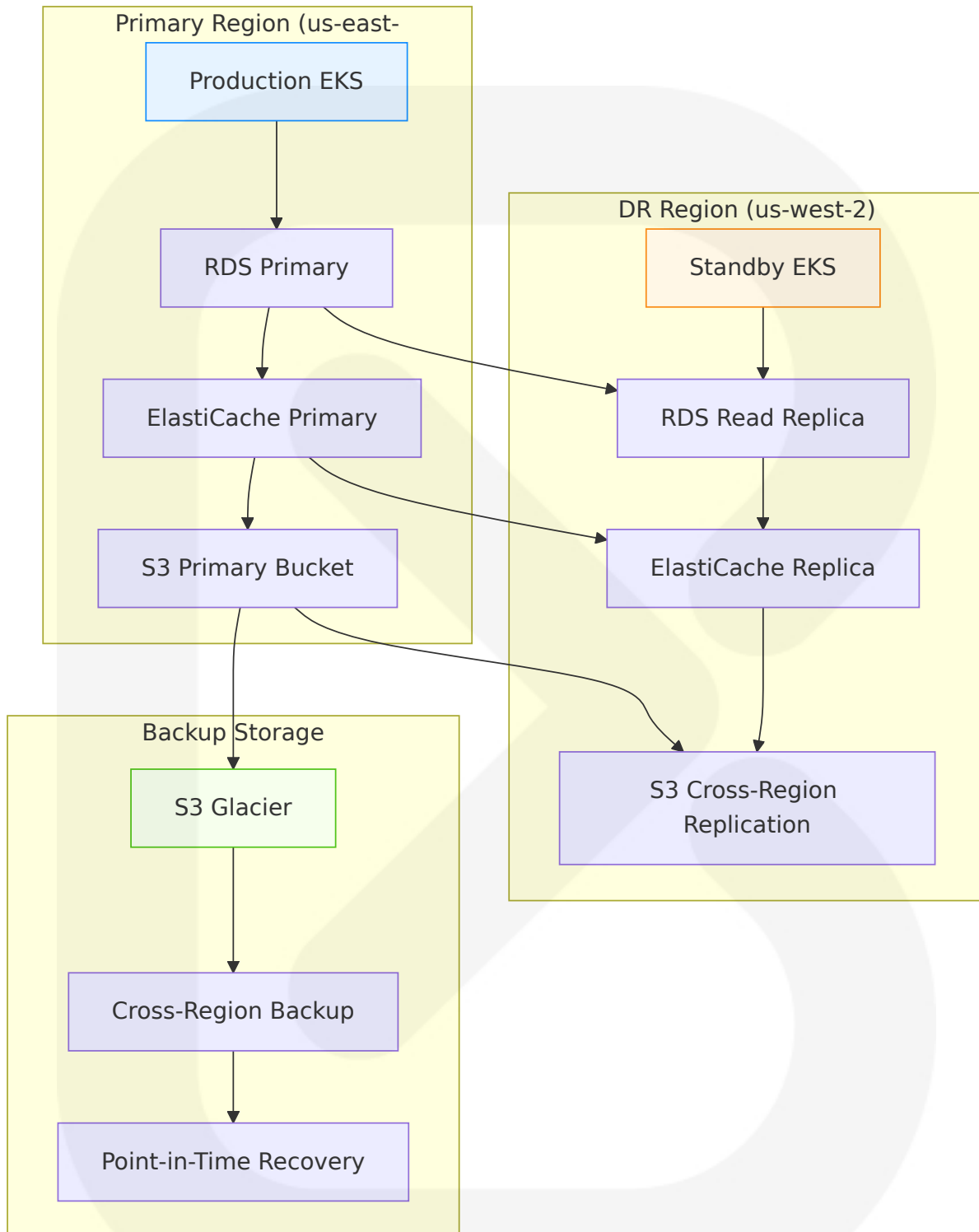
Environment	Promotion Requirements	Validation Tests	Approval Process
Staging → Pre-Prod	Integration tests pass, performance benchmarks met	Load testing, security validation	Engineering manager approval
Pre-Prod → Production	UAT complete, business stakeholder sign-off	Full regression testing, disaster recovery test	CTO approval required

Backup and Disaster Recovery Plans

Comprehensive Backup Strategy:

Data Category	Backup Frequency	Retention Period	Recovery Time Objective (RTO)	Recovery Point Objective (RPO)
Databases	Continuous WAL + daily snapshots	30 days active, 7 years archive	60 minutes	15 minutes
Application State	Hourly Redis snapshots	7 days	30 minutes	1 hour
File Assets	Real-time S3 replication	Indefinite with lifecycle	15 minutes	5 minutes
Configuration	Git-based versioning	Indefinite	10 minutes	0 (version controlled)

Disaster Recovery Architecture:



8.2 CLOUD SERVICES

8.2.1 Cloud Provider Selection and Justification

Amazon Web Services (AWS) Selection Rationale:

AgencyOS leverages AWS as the primary cloud provider based on comprehensive evaluation of technical capabilities, compliance certifications, and cost optimization opportunities.

Evaluation Criteria	AWS Advantages	Business Impact
Service Breadth	200+ services covering all requirements	Reduced vendor lock-in, integrated solutions
Global Infrastructure	31 regions, 99 availability zones	Low latency, data residency compliance
Compliance Certifications	SOC 2, GDPR, HIPAA, FedRAMP	Simplified compliance for enterprise clients
Kubernetes Maturity	Kubernetes 1.33 is now available in Amazon EKS. Kubernetes 1.34 is now available in Amazon EKS.	Latest Kubernetes features with managed operations

8.2.2 Core Services Required

Compute Services

Service	Version/Type	Purpose	Configuration
Amazon EKS	Kubernetes 1.33 + (latest available: 1.34)	Container orchestration	Multi-AZ cluster with managed node groups

Service	Version/Type	Purpose	Configuration
EC2 Instances	t3.large to c5.4xlarge	Application workloads	Auto Scaling Groups with mixed instance types
AWS Fargate	Latest	Serverless containers	Background jobs and batch processing
Lambda Functions	Node.js 20 runtime	Event-driven processing	Webhook handlers, scheduled tasks

Database Services

Service	Configuration	Purpose	High Availability
Amazon RDS PostgreSQL	15.x with Multi-AZ	Primary application database	Automatic failover, read replicas
Amazon ElastiCache Redis	7.2+ cluster mode	Session storage, caching	Multi-AZ with automatic failover
Amazon OpenSearch	2.11+	Full-text search, analytics	Multi-AZ deployment with dedicated masters
Amazon S3	Standard + Intelligent Tiering	Object storage, backups	Cross-region replication

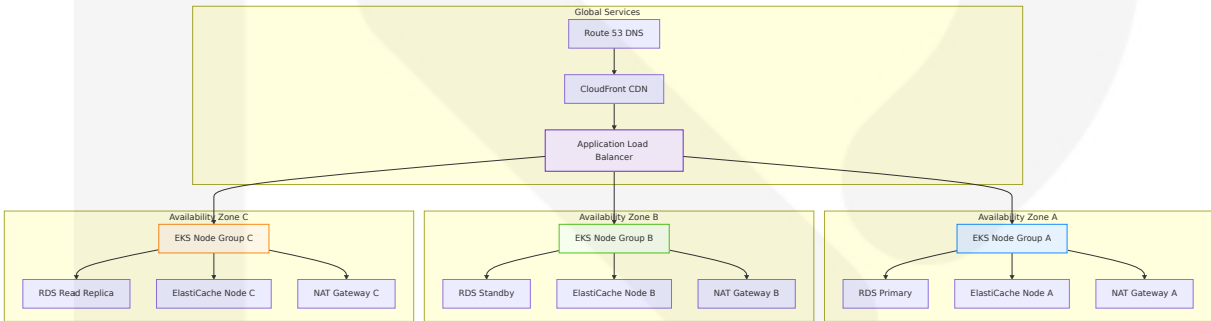
Networking and Security Services

Service	Implementation	Security Features	Compliance
Amazon VPC	Multi-AZ with private/public subnets	Network ACLs, Security Groups	Network isolation
AWS WAF	CloudFront integration	DDoS protection, rate limiting	OWASP Top 10 protection

Service	Implementation	Security Features	Compliance
AWS Certificate Manager	Wildcard SSL certificates	Automatic renewal	TLS 1.2+ enforcement
AWS Secrets Manager	Automatic rotation	Encrypted credential storage	Audit logging

8.2.3 High Availability Design

Multi-AZ Architecture:



Service Level Objectives (SLOs):

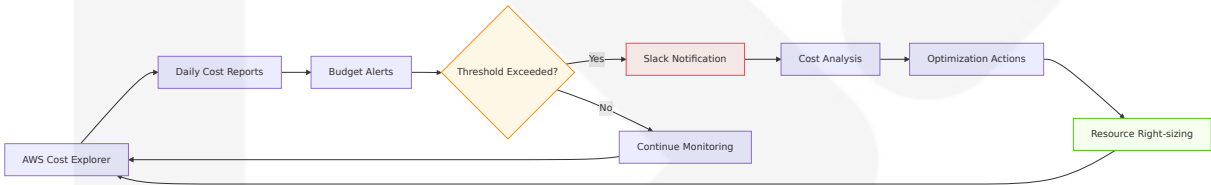
Service Component	Availability Target	MTTR	MTBF	Monitoring
Application Tier	99.9%	15 minutes	720 hours	EKS health checks
Database Tier	99.95%	5 minutes	1440 hours	RDS automated failover
Cache Tier	99.9%	10 minutes	720 hours	ElastiCache cluster monitoring
CDN/Edge	99.99%	2 minutes	8760 hours	CloudFront global distribution

8.2.4 Cost Optimization Strategy

Multi-Dimensional Cost Optimization:

Optimization Strategy	Implementation	Expected Savings	Monitoring
Reserved Instances	1-year term for predictable workloads	30-40% compute savings	AWS Cost Explorer
Spot Instances	Non-critical batch processing	50-70% savings	Spot Fleet management
Auto Scaling	CPU/memory-based scaling policies	20-30% resource optimization	CloudWatch metrics
S3 Intelligent Tiering	Automatic storage class transitions	15-25% storage savings	S3 Storage Class Analysis

Cost Monitoring and Alerting:



Monthly Cost Projections:

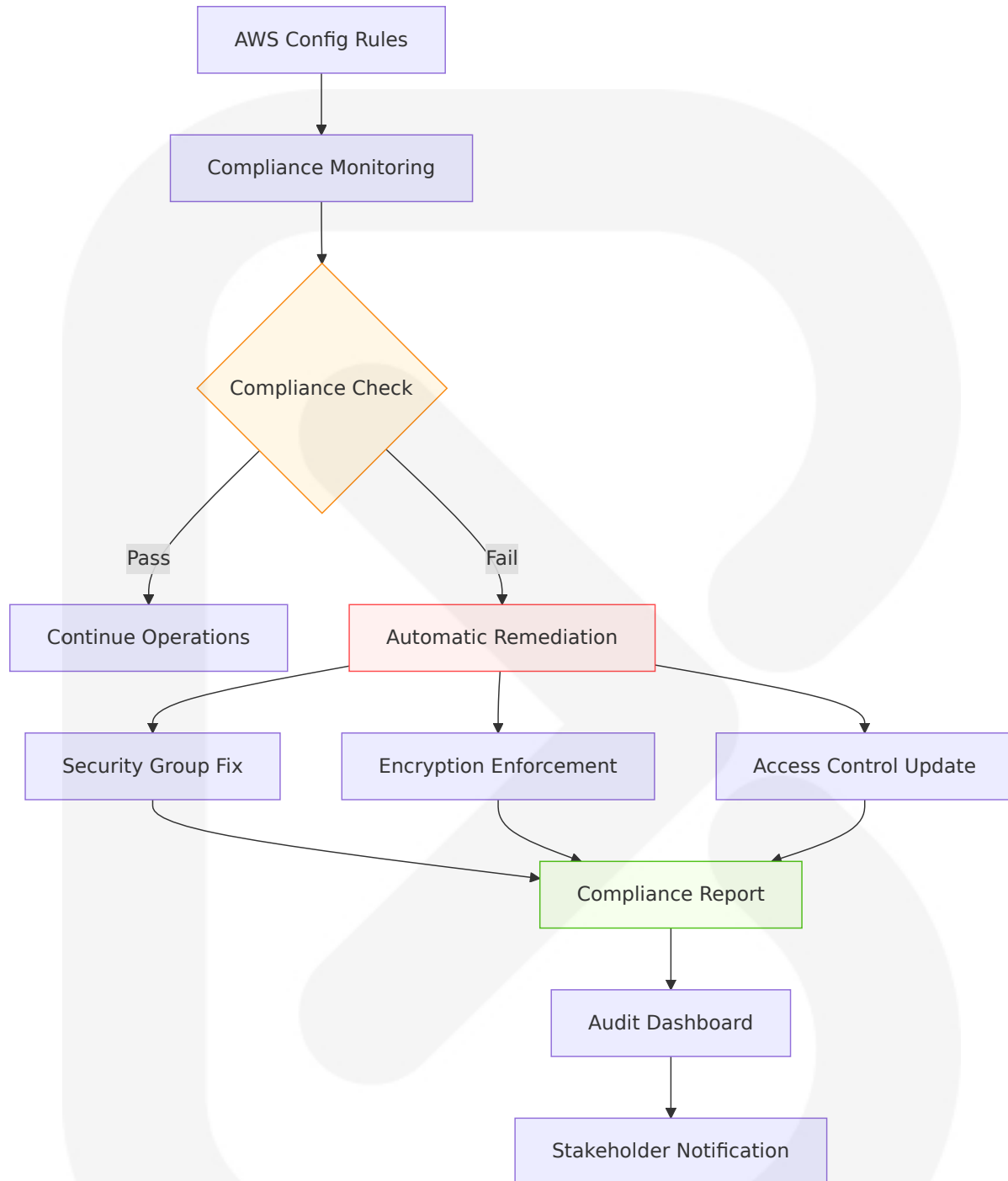
Service Category	Startup (100 users)	Growth (1000 users)	Scale (5000 users)
Compute (EKS + EC2)	\$800/month	\$3,200/month	\$12,000/month
Database (RDS + ElastiCache)	\$400/month	\$1,200/month	\$4,000/month
Storage (S3 + EBS)	\$200/month	\$800/month	\$2,500/month
Networking (CloudFront + ALB)	\$150/month	\$500/month	\$1,500/month
Total Estimated	\$1,550/month	\$5,700/month	\$20,000/month

8.2.5 Security and Compliance Considerations

AWS Security Services Integration:

Security Do main	AWS Servi ce	Implementation	Compliance Be nefit
Identity Man agement	AWS IAM + SSO	Role-based acces s with MFA	SOC 2 access co ntrols
Data Encrypt ion	AWS KMS	Envelope encrypt ion for all data	GDPR data prote ction
Network Sec urity	AWS WAF + Shield	DDoS protection, traffic filtering	Security baselin e compliance
Audit Loggin g	AWS CloudTr ail	All API calls logge d	Compliance audi t trail

Compliance Automation:



8.3 CONTAINERIZATION

8.3.1 Container Platform Selection

Docker Containerization Strategy:

AgencyOS utilizes Docker Engine version 27.0 as the primary containerization platform, providing consistent deployment environments across development, staging, and production.

Container Platform Justification:

Selection Criteria	Docker Advantages	AgencyOS Benefits
Industry Standard	Most widely adopted containerization platform	Extensive community support, tooling ecosystem
Development Experience	Consistent local development environment	Reduced "works on my machine" issues
Kubernetes Integration	Native Kubernetes container runtime	Seamless orchestration with Amazon EKS
Security Features	Rootless containers, image scanning	Enhanced security posture for multi-tenant architecture

8.3.2 Base Image Strategy

Multi-Stage Build Architecture:

AgencyOS implements a comprehensive base image strategy optimized for security, performance, and maintainability.

Base Image Selection:

Component	Base Image	Version	Justification
Node.js Applications	node:20-alpine	20.x LTS	Minimal attack surface, long-term support
PostgreSQL	postgres:15-alpine	15.x	Lightweight, security-focused

Component	Base Image	Version	Justification
Redis	redis:7-alpine	7.2+	Memory-optimized, minimal footprint
Nginx	nginx:alpine	Latest stable	High-performance reverse proxy

Multi-Stage Dockerfile Example:

```
# Build stage
FROM node:20-alpine AS builder
WORKDIR /app

#### Install dependencies
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force

#### Copy source code
COPY . .

#### Build application
RUN npm run build

#### Production stage
FROM node:20-alpine AS production
WORKDIR /app

#### Create non-root user
RUN addgroup -g 1001 -S nodejs && \
  adduser -S nextjs -u 1001

#### Copy built application
COPY --from=builder --chown=nextjs:nodejs /app/dist ./dist
COPY --from=builder --chown=nextjs:nodejs /app/node_modules
./node_modules

#### Security hardening
RUN apk --no-cache add dumb-init && \
  rm -rf /var/cache/apk/*

USER nextjs
EXPOSE 3000
```

```
ENTRYPOINT ["dumb-init", "--"]
CMD ["node", "dist/main.js"]
```

8.3.3 Image Versioning Approach

Semantic Versioning for Container Images:

Versioning Pattern	Usage	Example	Deployment Strategy
Semantic Version	Production releases	agencyos-api:1.2.3	Blue-green deployments
Git SHA	Development builds	agencyos-api:abc123f	Feature branch testing
Environment Tags	Environment-specific	agencyos-api:staging-latest	Continuous deployment
Date-based	Backup/rollback	agencyos-api:2024-01-15	Disaster recovery

Image Tagging Strategy:

```
# Production release tagging
docker tag agencyos-api:latest agencyos-api:1.2.3
docker tag agencyos-api:latest agencyos-api:1.2
docker tag agencyos-api:latest agencyos-api:1
docker tag agencyos-api:latest agencyos-api:production-latest

#### Development build tagging
docker tag agencyos-api:latest agencyos-api:${GIT_SHA}
docker tag agencyos-api:latest agencyos-api:dev-${BUILD_NUMBER}
```

8.3.4 Build Optimization Techniques

Container Build Performance Optimization:

Optimization Technique	Implementation	Performance Gain	Maintenance Impact
Layer Caching	Strategic COPY ordering	50-80% build time reduction	Requires careful Dockerfile structure
Multi-stage Builds	Separate build and runtime stages	60-70% image size reduction	Increased Dockerfile complexity
Dependency Caching	npm ci with package-lock.json	40-60% dependency install time	Requires lock file maintenance
Base Image Optimization	Alpine Linux variants	80-90% image size reduction	Limited package availability

Build Cache Optimization:

```
# Optimize layer caching by copying package files first
COPY package*.json ./
RUN npm ci --only=production

#### Copy source code after dependencies (changes less frequently)
COPY src/ ./src/
COPY public/ ./public/

#### Build application (most likely to change)
RUN npm run build
```

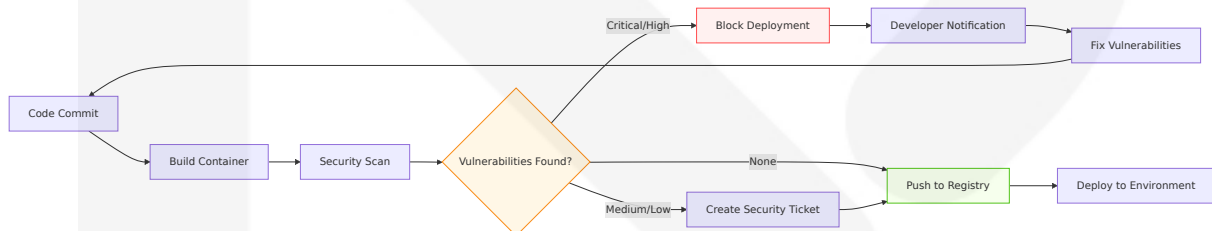
8.3.5 Security Scanning Requirements

Comprehensive Container Security Strategy:

Security Layer	Tool/Service	Scan Frequency	Action Threshold
Base Image Vulnerabilities	AWS ECR Image Scanning	Every push	High/Critical CVEs block deployment

Security Layer	Tool/Service	Scan Frequency	Action Threshold
Dependency Scanning	npm audit + Sn yk	Daily scheduled	Medium+ vulnerabilities require review
Runtime Security	Falco + AWS GuardDuty	Continuous monitoring	Anomalous behavior triggers alerts
Compliance Scanning	Docker Bench + CIS Benchmarks	Weekly	Non-compliance blocks production

Security Scanning Pipeline:



Container Security Hardening:

```
# Security hardening practices
FROM node:20-alpine

#### Update packages and remove package manager
RUN apk update && apk upgrade && \
    apk add --no-cache dumb-init && \
    rm -rf /var/cache/apk/*

#### Create non-root user
RUN addgroup -g 1001 -S appgroup && \
    adduser -S appuser -u 1001 -G appgroup

#### Set secure file permissions
COPY --chown=appuser:appgroup . /app
WORKDIR /app

#### Drop privileges
USER appuser
```

```
#### Use init system for proper signal handling
ENTRYPOINT ["dumb-init", "--"]
CMD ["node", "server.js"]
```

8.4 ORCHESTRATION

8.4.1 Orchestration Platform Selection

Amazon EKS (Elastic Kubernetes Service) Selection:

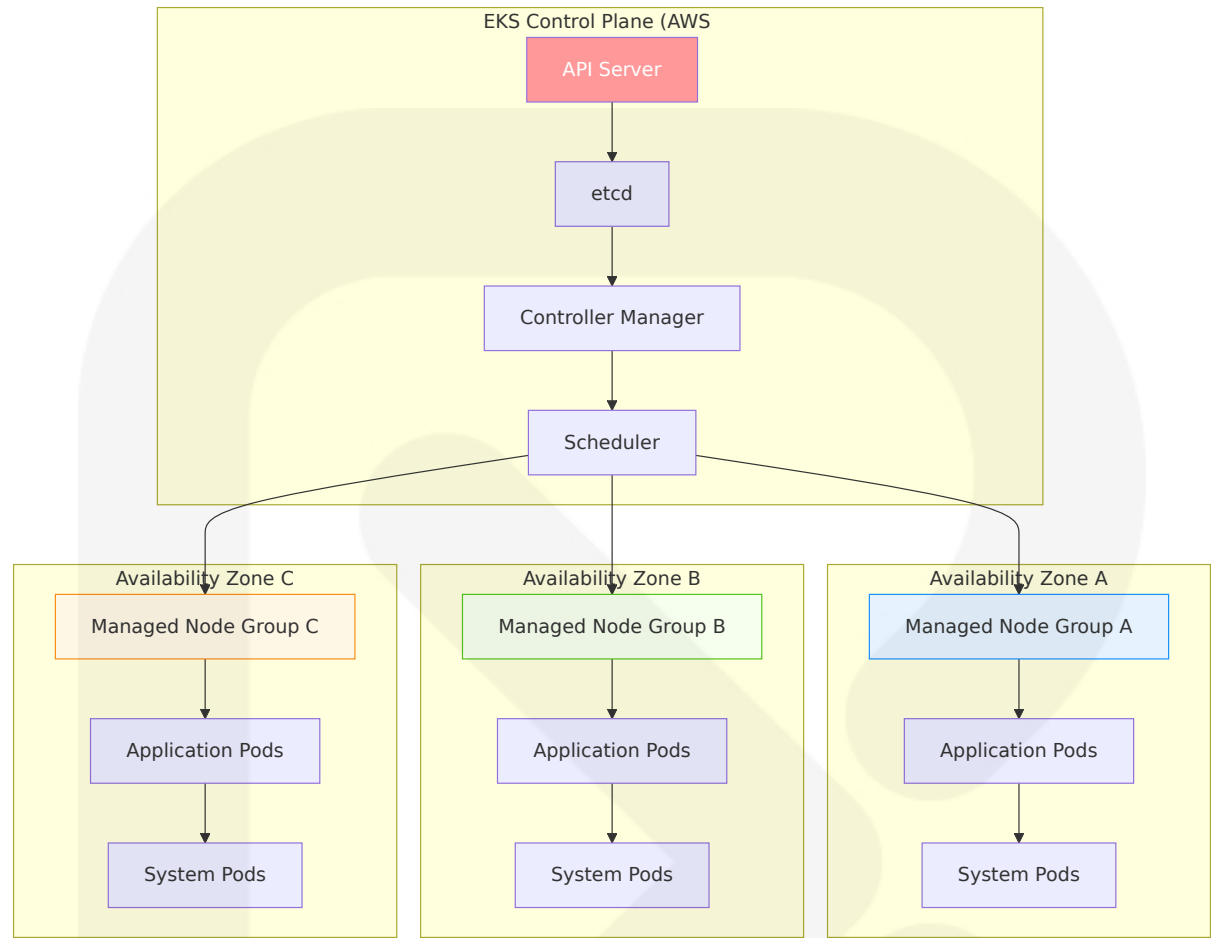
AgencyOS leverages Amazon EKS with Kubernetes 1.33+ (latest available: 1.34) for container orchestration, providing enterprise-grade Kubernetes management with AWS integration.

EKS Selection Rationale:

Evaluation Criteria	EKS Advantages	AgencyOS Benefits
Managed Control Plane	AWS manages Kubernetes masters	Reduced operational overhead, automatic updates
AWS Service Integration	Native integration with AWS services	Simplified networking, storage, and security
Enterprise Features	RBAC, network policies, pod security	Multi-tenant security requirements
Compliance Certifications	SOC 2, HIPAA, PCI DSS compliance	Simplified compliance for enterprise clients

8.4.2 Cluster Architecture

Multi-AZ EKS Cluster Design:



Node Group Configuration:

Node Group	Instance Types	Scaling Policy	Purpose
System Nodes	t3.medium (2 v CPU, 4GB RAM)	Fixed: 3 nodes	System pods, monitoring, logging
Application Nodes	c5.large to c5.2xlarge	Auto-scaling: 3-20 nodes	AgencyOS application workloads
Batch Processing	c5.xlarge with Spot instances	Auto-scaling: 0-10 nodes	Background jobs, file processing
Database Workloads	r5.large (memory-optimized)	Auto-scaling: 2-6 nodes	Redis, search, analytics

8.4.3 Service Deployment Strategy

Deployment Patterns and Strategies:

Deployment Type	Strategy	Use Case	Rollback Time
Blue-Green	Complete environment switch	Major releases, database migrations	< 5 minutes
Canary	Gradual traffic shifting	Feature releases, A/B testing	< 10 minutes
Rolling Update	Pod-by-pod replacement	Minor updates, configuration changes	< 15 minutes
Recreate	Stop all, then start new	Stateful services, maintenance windows	< 30 minutes

Kubernetes Deployment Configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: agencyos-api
  namespace: production
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 1
  selector:
    matchLabels:
      app: agencyos-api
  template:
    metadata:
      labels:
        app: agencyos-api
        version: v1.2.3
    spec:
      containers:
        - name: api
          image: agencyos/api:1.2.3
          ports:
```

```
- containerPort: 3000
resources:
  requests:
    memory: "512Mi"
    cpu: "250m"
  limits:
    memory: "1Gi"
    cpu: "500m"
livenessProbe:
  httpGet:
    path: /health
    port: 3000
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 3000
  initialDelaySeconds: 5
  periodSeconds: 5
```

8.4.4 Auto-Scaling Configuration

Horizontal Pod Autoscaler (HPA) Configuration:

Service Component	Scaling Metric	Min Replicas	Max Replicas	Target Utilization
API Server	CPU + Memory	3	20	70% CPU, 80% Memory
Background Workers	Queue Length	1	15	10 jobs per pod
WebSocket Server	Connection Count	2	10	1000 connections per pod
File Processor	CPU Utilization	0	8	80% CPU

Vertical Pod Autoscaler (VPA) Configuration:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: agencyos-api-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: agencyos-api
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
      - containerName: api
        minAllowed:
          cpu: 100m
          memory: 256Mi
        maxAllowed:
          cpu: 2
          memory: 4Gi
```

Cluster Autoscaler Configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cluster-autoscaler
  namespace: kube-system
spec:
  template:
    spec:
      containers:
        - image: k8s.gcr.io/autoscaling/cluster-autoscaler:v1.21.0
          name: cluster-autoscaler
          command:
            - ./cluster-autoscaler
            - --v=4
            - --stderrthreshold=info
            - --cloud-provider=aws
            - --skip-nodes-with-local-storage=false
            - --expander=least-waste
```

```
- --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,k8s.io/cluster-autoscaler/agencyos-cluster
- --balance-similar-node-groups
- --scale-down-delay-after-add=10m
- --scale-down-unneeded-time=10m
```

8.4.5 Resource Allocation Policies

Resource Quotas and Limits:

Namespace	CPU Limit	Memory Limit	Storage Limit	Pod Limit
production	50 cores	200 GB	1 TB	200 pods
staging	20 cores	80 GB	500 GB	100 pods
development	10 cores	40 GB	200 GB	50 pods
monitoring	8 cores	32 GB	100 GB	30 pods

Quality of Service (QoS) Classes:

```
# Guaranteed QoS - Critical services
resources:
  requests:
    memory: "1Gi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "500m"

#### Burstable QoS - Standard services
resources:
  requests:
    memory: "512Mi"
    cpu: "250m"
  limits:
    memory: "2Gi"
    cpu: "1000m"
```

```
#### BestEffort QoS - Background jobs
#### No resource requests or limits specified
```

Pod Disruption Budgets:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: agencyos-api-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: agencyos-api
```

8.5 CI/CD PIPELINE

8.5.1 Build Pipeline

Source Control Integration and Triggers:

AgencyOS implements a comprehensive CI/CD pipeline using GitHub Actions with automated triggers and quality gates to ensure reliable software delivery.

Build Trigger Configuration:

Trigger Event	Pipeline Action	Quality Gates	Deployment Target
Pull Request	Build + Test + Security Scan	Unit tests, code coverage, security scan	None (validation only)
Main Branch Push	Full CI/CD Pipeline	All tests, performance benchmarks	Staging environment

Trigger Event	Pipeline Action	Quality Gates	Deployment Target
Release Tag	Production Pipeline	Complete test suite, manual approval	Production environment
Scheduled	Nightly builds, dependency updates	Security scans, dependency audit	Development environment

GitHub Actions Workflow Architecture:

```
name: AgencyOS CI/CD Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  release:
    types: [published]

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [20.x]

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Run linting
```

```
run: pnpm lint

- name: Run unit tests
  run: pnpm test:unit --coverage

- name: Run integration tests
  run: pnpm test:integration

- name: Build application
  run: pnpm build

- name: Security scan
  uses: securecodewarrior/github-action-add-sarif@v1
  with:
    sarif-file: security-scan-results.sarif
```

Build Environment Requirements

Containerized Build Environment:

Build Component	Technology	Version	Purpose
Base Runner	Ubuntu Latest	22.04 LTS	GitHub Actions runner environment
Node.js Runtime	Node.js	20.x LTS	Application build and test execution
Package Manager	pnpm	8.x	Fast, disk space efficient package management
Container Runtime	Docker	27.0	Container image building and testing

Dependency Management

Automated Dependency Management:

```
# Dependabot configuration
version: 2
updates:
```

```
- package-ecosystem: "npm"
  directory: "/"
  schedule:
    interval: "weekly"
  open-pull-requests-limit: 10
  reviewers:
    - "engineering-team"
  assignees:
    - "tech-lead"
  commit-message:
    prefix: "deps"
    include: "scope"
```

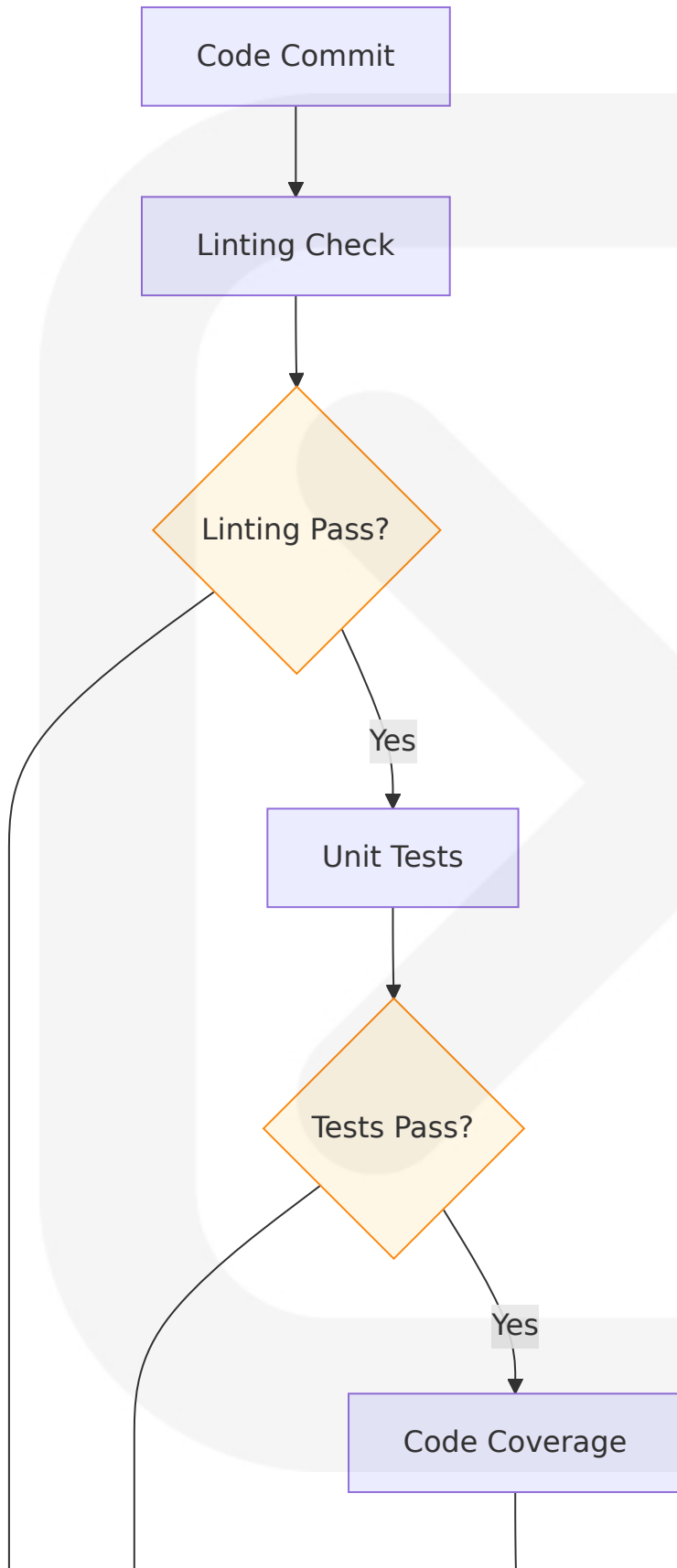
Artifact Generation and Storage

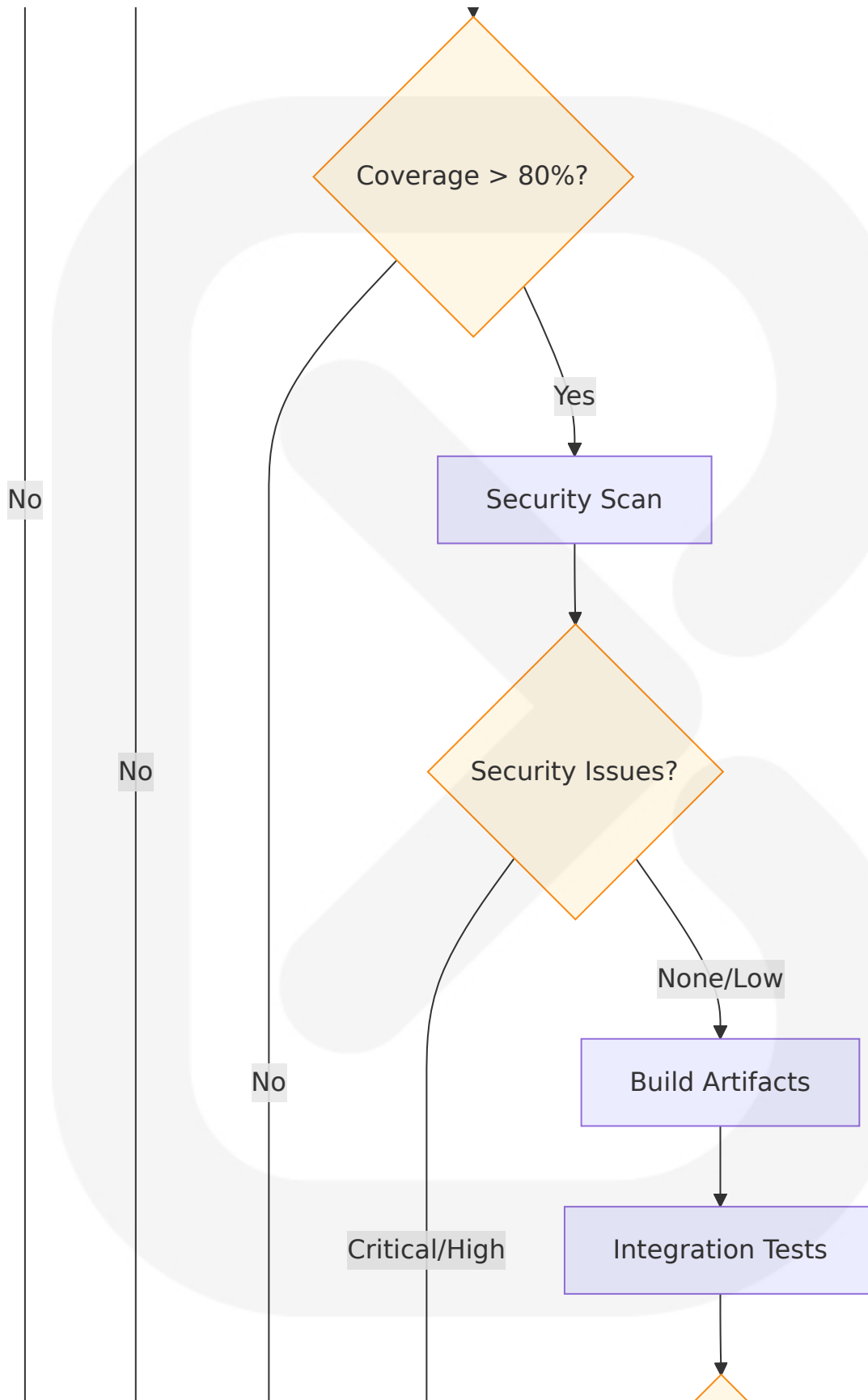
Build Artifact Strategy:

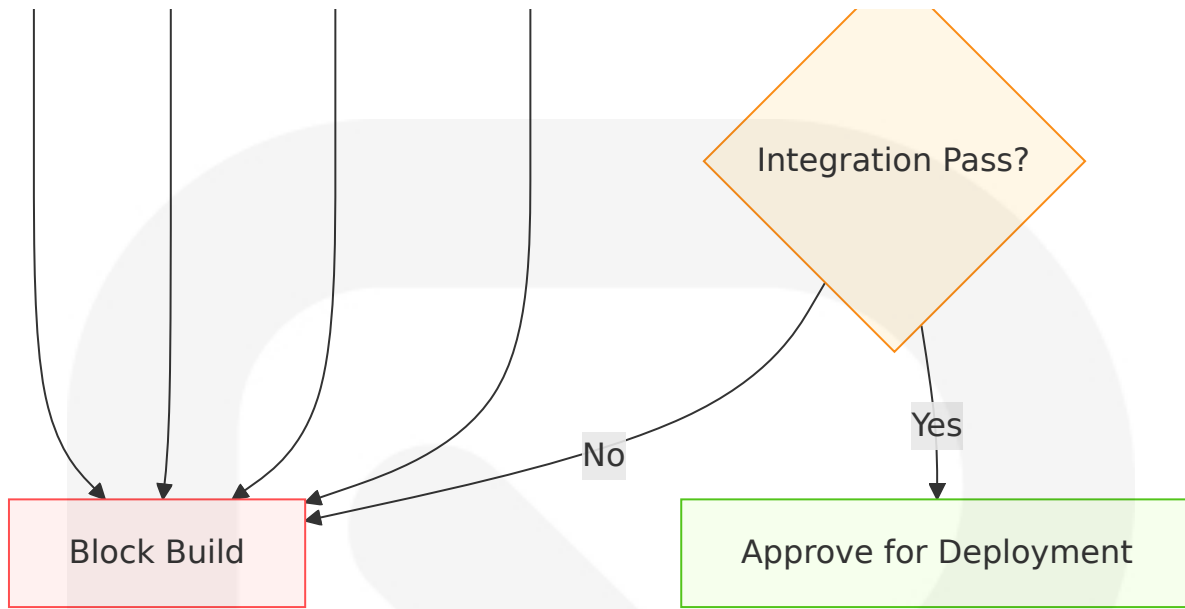
Artifact Type	Storage Location	Retention Policy	Access Control
Container Images	Amazon ECR	30 days for dev, 1 year for prod	IAM-based access
Test Reports	GitHub Actions Artifacts	90 days	Team members only
Security Scan Results	GitHub Security Tab	2 years	Security team access
Build Logs	CloudWatch Logs	30 days	Engineering team access

Quality Gates

Comprehensive Quality Gate Framework:







8.5.2 Deployment Pipeline

Deployment Strategy Implementation

Blue-Green Deployment for Production:

AgencyOS implements blue-green deployment strategy for zero-downtime production releases with immediate rollback capability.

```
# Blue-Green Deployment Configuration
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: agencyos-api
spec:
  replicas: 6
  strategy:
    blueGreen:
      activeService: agencyos-api-active
      previewService: agencyos-api-preview
      autoPromotionEnabled: false
      scaleDownDelaySeconds: 30
      prePromotionAnalysis:
        templates:
          - templateName: success-rate
```

```
args:
  - name: service-name
    value: agencyos-api-preview
postPromotionAnalysis:
  templates:
    - templateName: success-rate
  args:
    - name: service-name
      value: agencyos-api-active
```

Environment Promotion Workflow

Progressive Environment Promotion:

Environment	Promotion Trigger	Validation Requirements	Approval Process
Development	Feature branch merge	Unit tests pass	Automated
Staging	Main branch push	Integration tests + security scan	Automated
Pre-Production	Manual trigger	Load testing + UAT	Engineering Manager
Production	Release tag	Full regression + business approval	CTO + Product Owner

Rollback Procedures

Automated Rollback Mechanisms:

```
# Rollback configuration with health checks
spec:
  strategy:
    blueGreen:
      postPromotionAnalysis:
        templates:
          - templateName: error-rate
        args:
          - name: error-threshold
```

```
      value: "5"
    - name: duration
      value: "5m"
  failurePolicy:
    action: rollback
    conditions:
    - type: AnalysisFailed
      status: "True"
```

Rollback Decision Matrix:

Failure Type	Detection Time	Rollback Method	Recovery Time
Health Check Failure	< 30 seconds	Automatic traffic switch	< 2 minutes
Error Rate Spike	< 2 minutes	Automatic rollback	< 5 minutes
Performance Degradation	< 5 minutes	Manual rollback decision	< 10 minutes
Business Logic Error	Variable	Manual rollback	< 15 minutes

Post-Deployment Validation

Comprehensive Validation Framework:

```
# Post-deployment validation tests
apiVersion: batch/v1
kind: Job
metadata:
  name: post-deployment-validation
spec:
  template:
    spec:
      containers:
      - name: validation
        image: agencyos/validation-suite:latest
        command:
```



```
- /bin/sh
- -c
- |
# Health check validation
curl -f http://agencyos-api:3000/health || exit 1

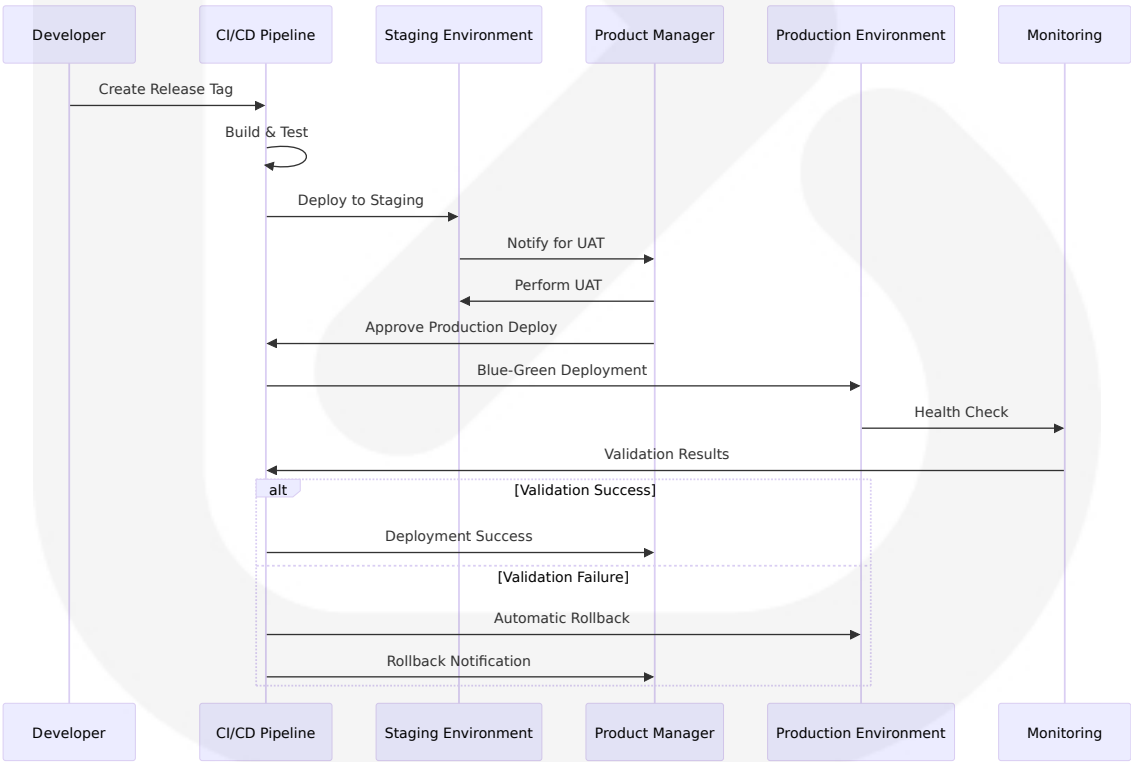
# API endpoint validation
curl -f http://agencyos-api:3000/api/v1/projects || exit 1

# Database connectivity
npm run test:db-connection || exit 1

# Integration validation
npm run test:integration:smoke || exit 1
restartPolicy: Never
```

Release Management Process

Release Coordination Workflow:



8.6 INFRASTRUCTURE MONITORING

8.6.1 Resource Monitoring Approach

Comprehensive Infrastructure Monitoring Stack:

AgencyOS implements a multi-layered monitoring approach using AWS native services combined with open-source tools for comprehensive visibility into infrastructure performance and health.

Monitoring Architecture:

Monitoring Layer	Technology	Metrics Collected	Alerting Threshold
Infrastructure	CloudWatch + DataDog	CPU, Memory, Disk, Network	>80% utilization
Application	Prometheus + Grafana	Request rates, response times, errors	p95 >300ms
Kubernetes	Kubernetes Metrics Server	Pod/Node health, resource usage	Pod restart > 3 times
Database	RDS Performance Insights	Query performance, connections	>80% connection pool

CloudWatch Dashboard Configuration:

```
{
  "widgets": [
    {
      "type": "metric",
      "properties": {
        "metrics": [
          ["AWS/EKS", "cluster_failed_request_count", "ClusterName",
            "agencyos-cluster"],
          ["AWS/RDS", "CPUUtilization", "DBInstanceIdentifier",
            "agencyos-db"],
          ["AWS/ElastiCache", "CPUUtilization", "CacheClusterId",
            "agencyos-redis"]
        ]
      }
    }
  ]
}
```

```
    ],
    "period": 300,
    "stat": "Average",
    "region": "us-east-1",
    "title": "Infrastructure Health Overview"
  }
}
]
```

8.6.2 Performance Metrics Collection

Key Performance Indicators (KPIs) Monitoring:

Metric Category	Specific Metrics	Collection Method	Business Impact
Application Performance	API response time, throughput, error rate	Prometheus + custom metrics	User experience, SLA compliance
Infrastructure Health	CPU, memory, disk I/O, network	CloudWatch agents	System stability, capacity planning
Database Performance	Query execution time, connection pool usage	RDS Performance Insights	Data access performance
User Experience	Page load time, interaction latency	Real User Monitoring (RUM)	Client satisfaction metrics

Custom Metrics Implementation:

```
// Application metrics collection
import { register, Counter, Histogram, Gauge } from 'prom-client'

// API request metrics
const httpRequestDuration = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code', 'tenant_id'],
```

```
    buckets: [0.1, 0.3, 0.5, 0.7, 1, 3, 5, 7, 10]
  })

  // Business metrics
  const activeProjects = new Gauge({
    name: 'agencyos_active_projects_total',
    help: 'Total number of active projects',
    labelNames: ['tenant_id']
  })

  const designReviewCycleTime = new Histogram({
    name: 'agencyos_design_review_cycle_seconds',
    help: 'Time taken for design review cycle completion',
    labelNames: ['tenant_id', 'project_type'],
    buckets: [3600, 7200, 14400, 28800, 86400, 172800, 259200] // 1h to
    3d
  })
```

8.6.3 Cost Monitoring and Optimization

AWS Cost Management Strategy:

Cost Category	Monitoring Tool	Optimization Strategy	Target Savings
Compute Costs	AWS Cost Explorer	Reserved Instances, Spot Instances	30-40%
Storage Costs	S3 Storage Class Analysis	Intelligent Tiering, Lifecycle Policies	20-30%
Data Transfer	CloudWatch + Cost Explorer	CDN optimization, regional placement	15-25%
Database Costs	RDS Performance Insights	Right-sizing, read replicas	20-30%

Cost Alerting Configuration:

```
# AWS Budget Alert Configuration
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: cost-monitoring-config
data:
  budget-alerts.json: |
    {
      "budgets": [
        {
          "budgetName": "AgencyOS-Monthly-Budget",
          "budgetLimit": {
            "amount": "10000",
            "unit": "USD"
          },
          "timeUnit": "MONTHLY",
          "budgetType": "COST",
          "costFilters": {
            "Service": ["Amazon Elastic Kubernetes Service", "Amazon
RDS", "Amazon S3"]
          },
          "notifications": [
            {
              "notificationType": "ACTUAL",
              "comparisonOperator": "GREATER_THAN",
              "threshold": 80,
              "thresholdType": "PERCENTAGE"
            }
          ]
        }
      ]
    }
```

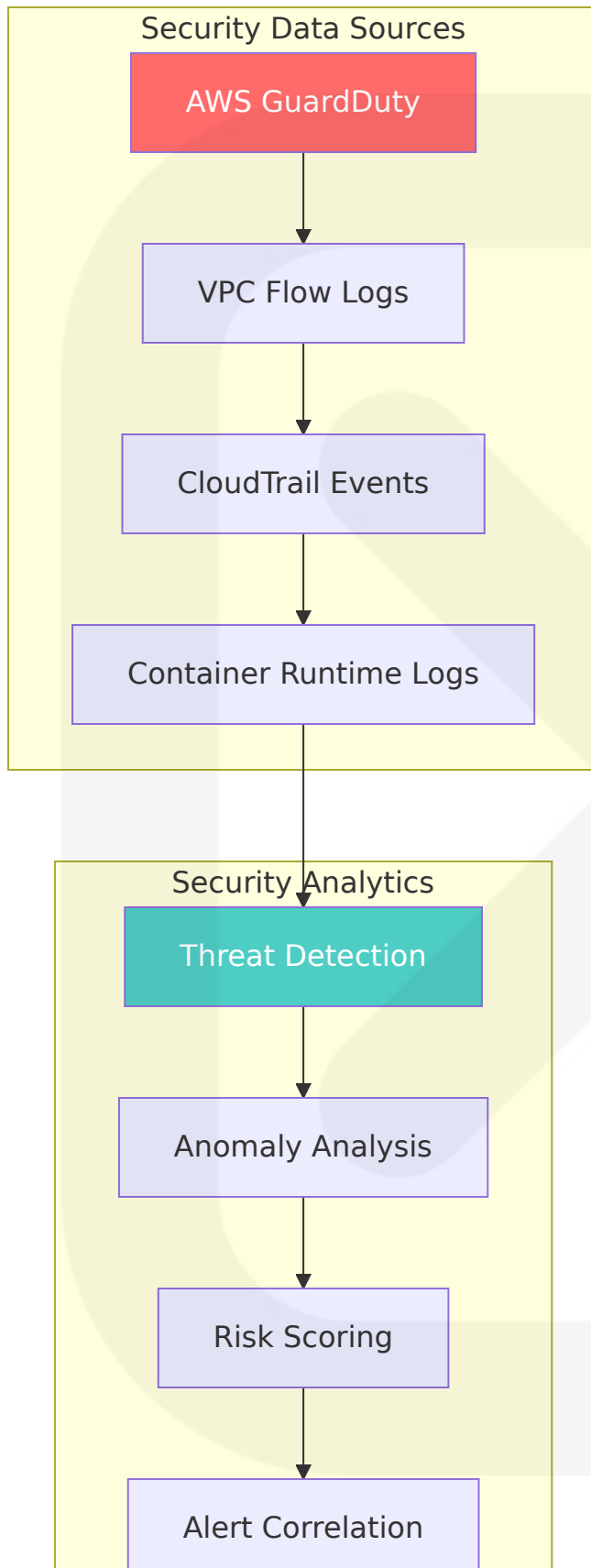
8.6.4 Security Monitoring

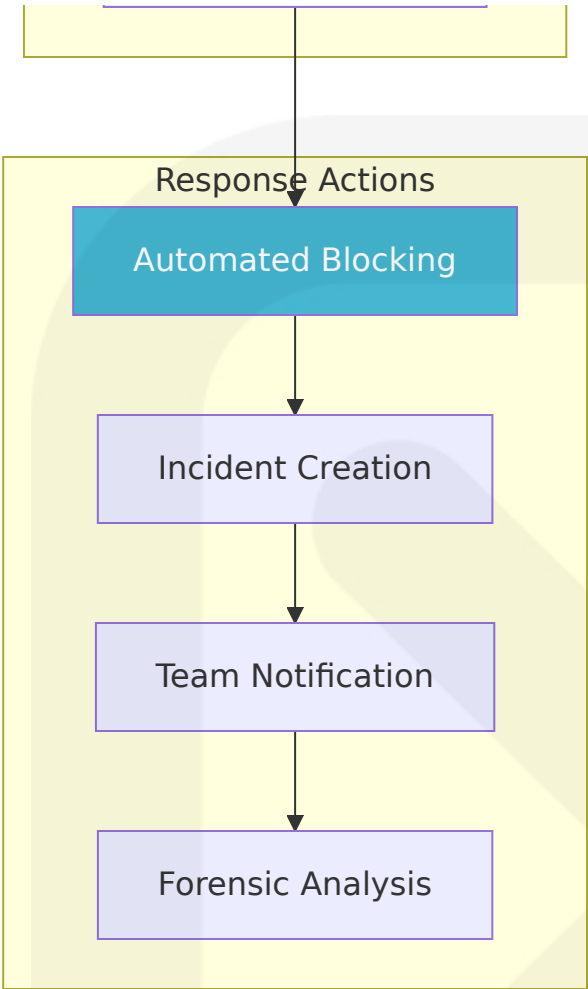
Security Event Detection and Response:

Security D omain	Monitoring Tool	Detection Criteri a	Response Actio n
Access An omalies	AWS GuardD uty	Unusual login patt erns, privilege esc alation	Automatic accou nt lockout

Security Domain	Monitoring Tool	Detection Criteria	Response Action
Network Security	AWS WAF + VPC Flow Logs	DDoS attacks, suspicious traffic	Traffic blocking, rate limiting
Container Security	Falco + AWS Security Hub	Runtime anomalies, privilege escalation	Pod isolation, alert escalation
Data Access	CloudTrail + Config	Unauthorized data access, configuration changes	Immediate notification, access revocation

Security Monitoring Dashboard:





8.6.5 Compliance Auditing

Automated Compliance Monitoring:

Compliance Framework	Monitoring Scope	Validation Frequency	Remediation SLA
SOC 2 Type II	Access controls, encryption, monitoring	Continuous	24 hours
GDPR	Data processing, retention, access rights	Daily	72 hours
CCPA	Data transparency, opt-out mechanisms	Weekly	7 days

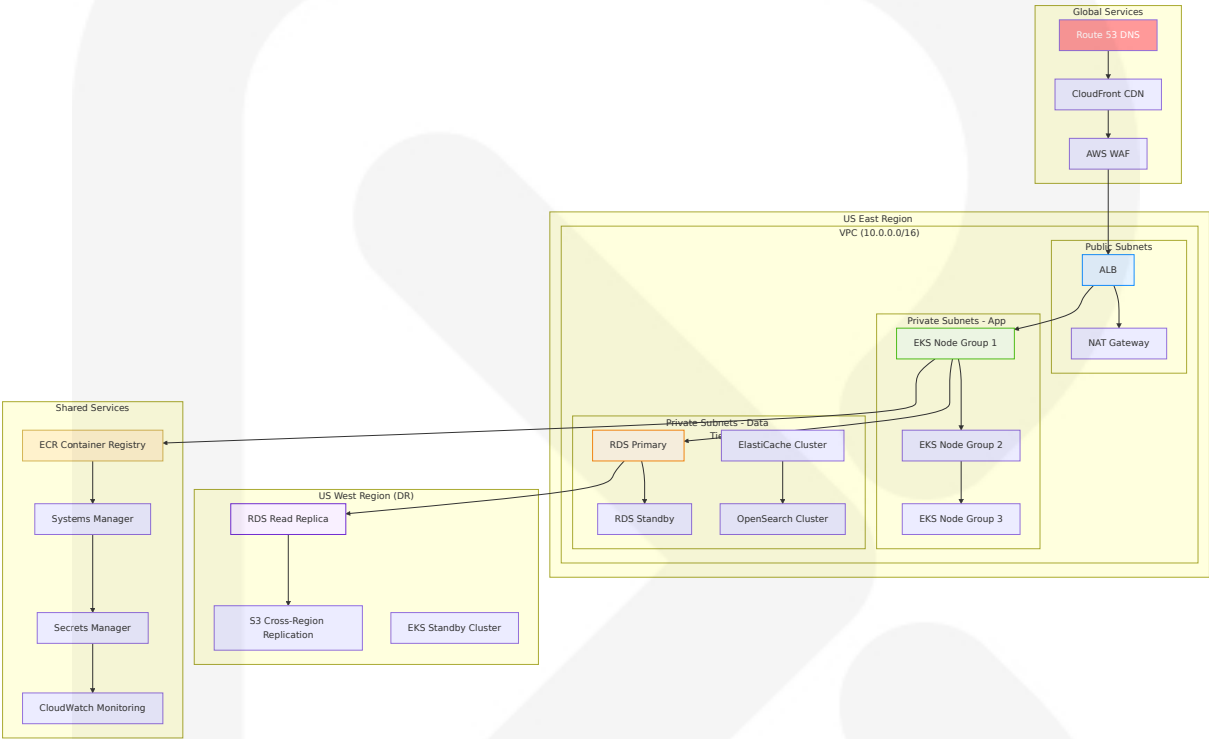
Compliance Framework	Monitoring Scope	Validation Frequency	Remediation SLA
CIS Benchmarks	Infrastructure hardening	Daily	48 hours

Compliance Automation Framework:

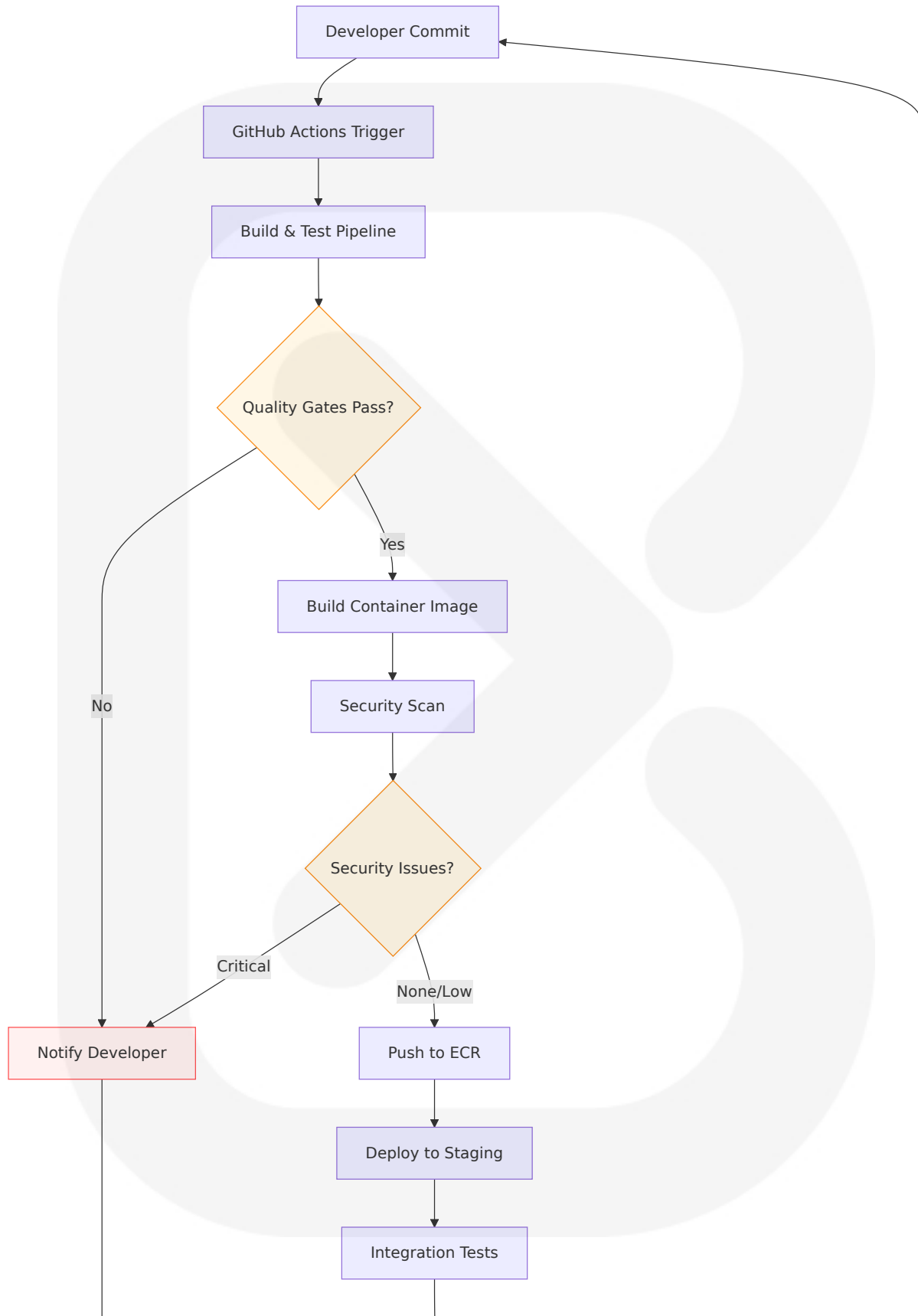
```
# AWS Config Rules for Compliance
apiVersion: v1
kind: ConfigMap
metadata:
  name: compliance-rules
data:
  config-rules.json: |
    {
      "configRules": [
        {
          "configRuleName": "encrypted-volumes",
          "source": {
            "owner": "AWS",
            "sourceIdentifier": "ENCRYPTED_VOLUMES"
          },
          "scope": {
            "complianceResourceTypes": ["AWS::EC2::Volume"]
          }
        },
        {
          "configRuleName": "rds-encryption-enabled",
          "source": {
            "owner": "AWS",
            "sourceIdentifier": "RDS_STORAGE_ENCRYPTED"
          },
          "scope": {
            "complianceResourceTypes": ["AWS::RDS::DBInstance"]
          }
        }
      ]
    }
```

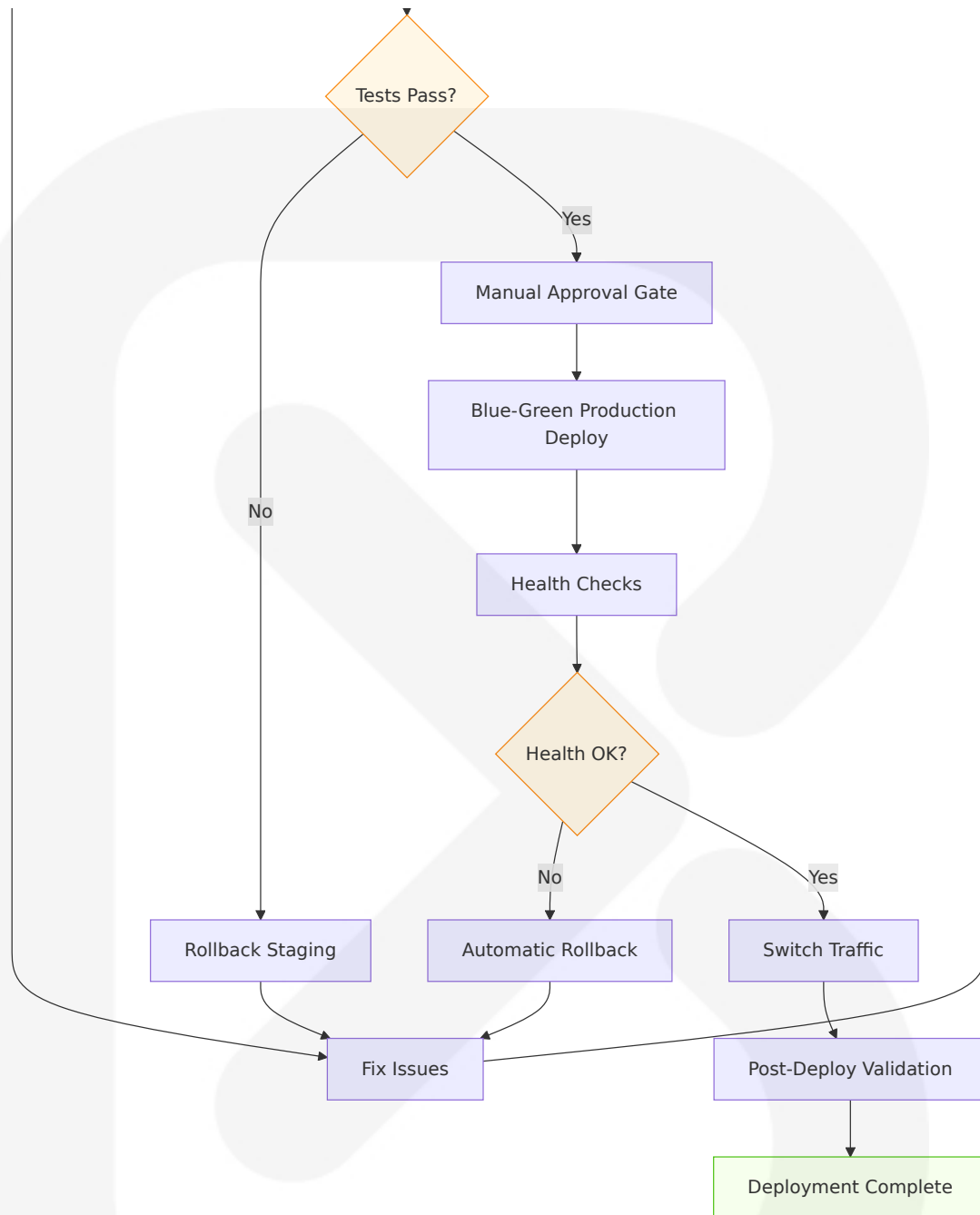
8.7 INFRASTRUCTURE ARCHITECTURE DIAGRAMS

8.7.1 Complete Infrastructure Architecture

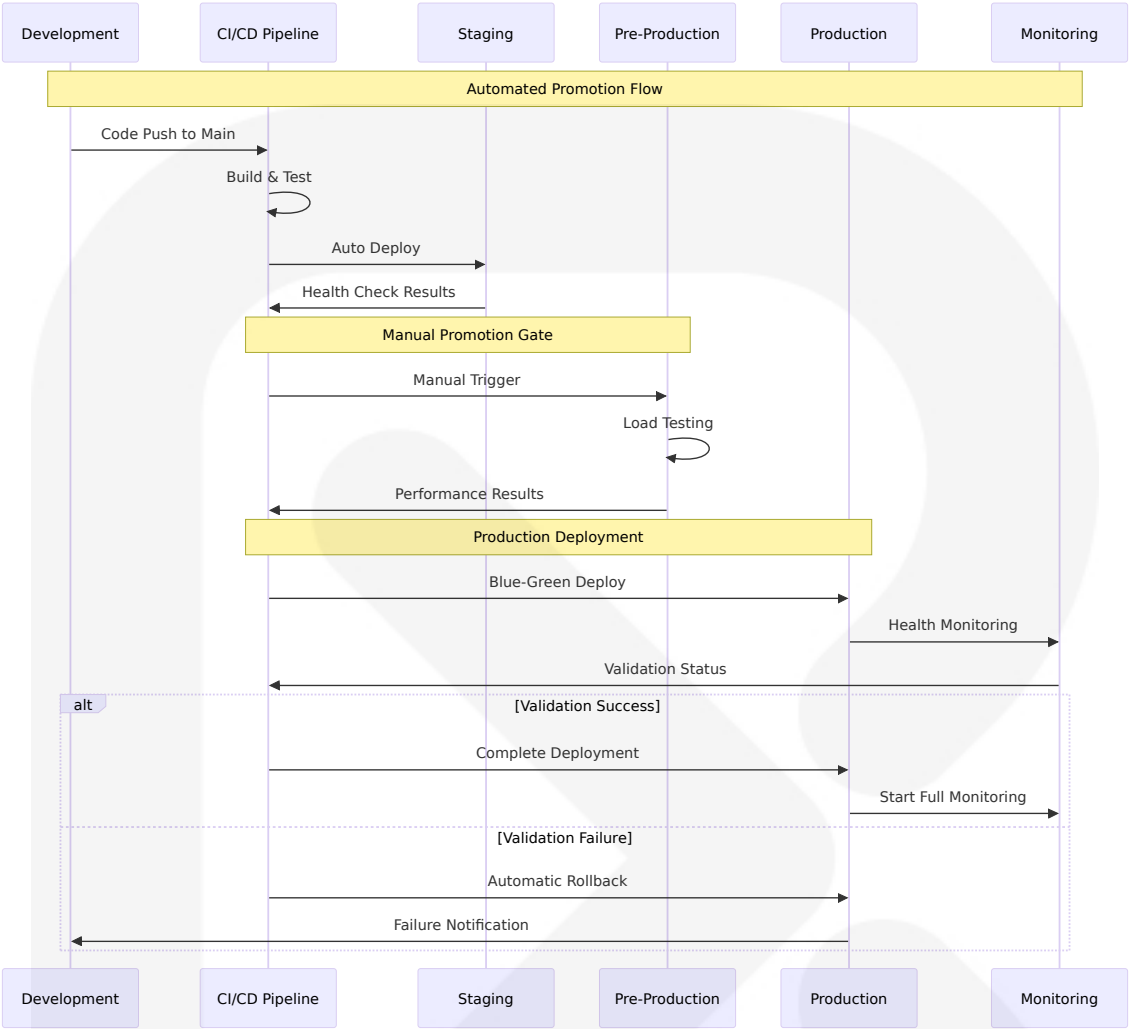


8.7.2 Deployment Workflow Diagram

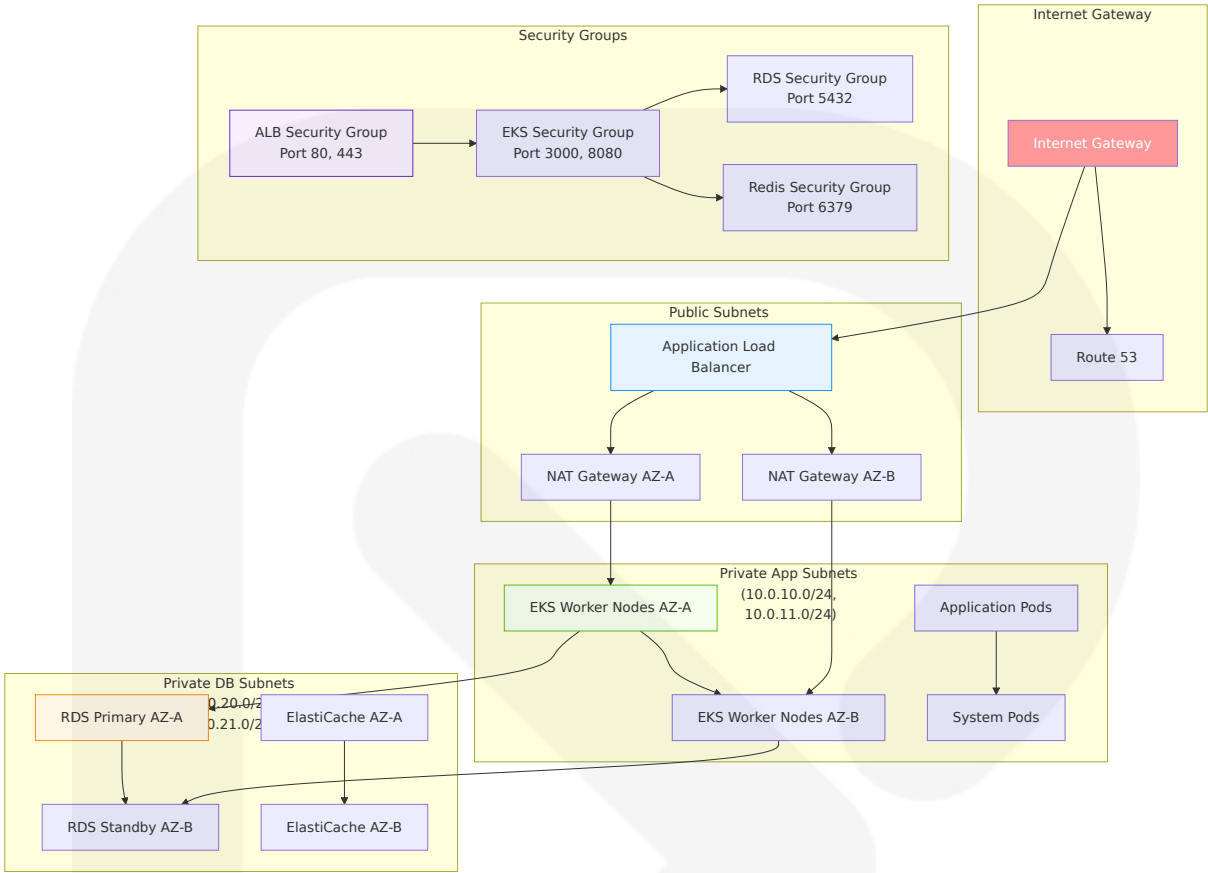




8.7.3 Environment Promotion Flow



8.7.4 Network Architecture Diagram



8.8 INFRASTRUCTURE COST ESTIMATES

8.8.1 Monthly Cost Breakdown by Service

Detailed Cost Analysis by Growth Stage:

Service Category	Startup (100 users)	Growth (1,000 users)	Scale (5,000 users)	Enterprise (10,000+ users)
Compute (EKS + EC2)	\$800/month	\$3,200/month	\$12,000/month	\$25,000/month
Database (RDS + ElastiCache)	\$400/month	\$1,200/month	\$4,000/month	\$8,500/month

Service Category	Startup (100 users)	Growth (1,000 users)	Scale (5,000 users)	Enterprise (10,000+ users)
Storage (S3 + EBS)	\$200/month	\$800/month	\$2,500/month	\$5,000/month
Networking (CloudFront + ALB)	\$150/month	\$500/month	\$1,500/month	\$3,000/month
Monitoring & Security	\$100/month	\$300/month	\$800/month	\$1,500/month
Data Transfer	\$50/month	\$200/month	\$700/month	\$1,500/month
Total Monthly Cost	\$1,700/month	\$6,200/month	\$21,500/month	\$44,500/month
Annual Cost	\$20,400/year	\$74,400/year	\$258,000/year	\$534,000/year

8.8.2 Cost Optimization Strategies

Reserved Instance Savings Analysis:

Instance Type	On-Demand Cost	1-Year Reserved	3-Year Reserved	Savings Potential
c5.2xlarge	\$280/month	\$182/month	\$140/month	35-50% savings
r5.xlarge	\$240/month	\$156/month	\$120/month	35-50% savings
t3.large	\$67/month	\$43/month	\$33/month	36-51% savings

Spot Instance Integration:

Workload Type	Spot Savings	Risk Mitigation	Implementation
Background Jobs	60-70% cost reduction	Queue-based retry logic	Spot Fleet with mixed instances
Development/Testing	50-60% cost reduction	Acceptable interruption	Spot instances for non-prod
Batch Processing	70-80% cost reduction	Checkpointing and resume	Kubernetes Spot node groups

8.8.3 Resource Sizing Guidelines

Compute Resource Recommendations:

Component	Minimum Spec	Recommended Spec	Maximum Spec	Scaling Trigger
API Servers	t3.medium (2 vCPU, 4GB)	c5.large (2 vCPU, 4GB)	c5.2xlarge (8 vCPU, 16GB)	CPU >70%
Databases	r5.large (2 vCPU, 16GB)	r5.xlarge (4 vCPU, 32GB)	r5.4xlarge (16 vCPU, 128GB)	Connection >80%
Cache	t3.micro (2 vCPU, 1GB)	r5.large (2 vCPU, 16GB)	r5.2xlarge (8 vCPU, 64GB)	Memory >80%
Workers	t3.small (2 vCPU, 2GB)	c5.large (2 vCPU, 4GB)	c5.xlarge (4 vCPU, 8GB)	Queue depth >50

This comprehensive Infrastructure specification ensures AgencyOS can scale efficiently from startup to enterprise levels while maintaining security, compliance, and cost optimization. The cloud-native architecture leverages AWS managed services to reduce operational overhead while providing the flexibility and scalability required for a multi-tenant SaaS platform serving digital agencies globally.

9. Appendices

9.1 ADDITIONAL TECHNICAL INFORMATION

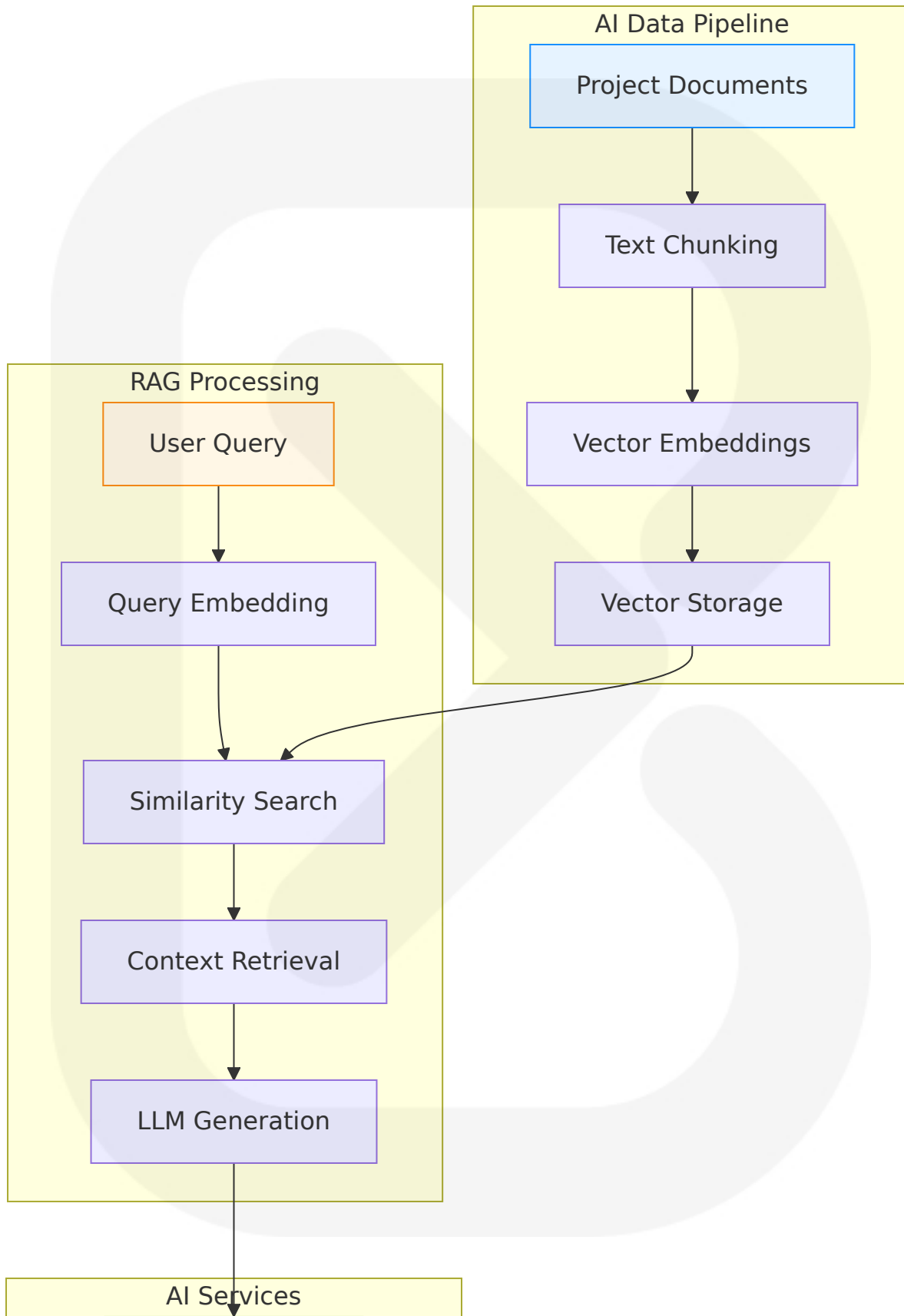
9.1.1 AI Integration ArchitectureAgencyOS incorporates AI capabilities through a comprehensive Retrieval-Augmented Generation (RAG) architecture that enhances project management workflows with intelligent assistance for brief drafting, meeting summaries, and risk detection.

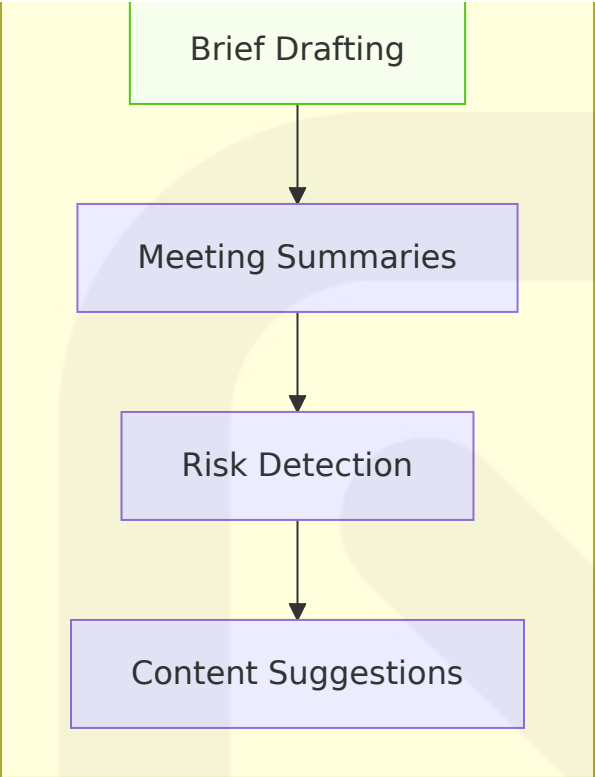
AI Integration Components:

Compon ent	Technology	Purpose	Impleme ntation
Vector E mbeddi ngs	Embeddings are a way to represent words, sentences, or even entire documents as dense vectors in a high-dimensional space. The purpose of embeddings is to capture the semantic meaning of text, such that words or phrases with similar meanings are located closer to each other in this vector space.	Semantic search across project documents, SOWs, and knowledge base	OpenAI text-embedding-3-large

Component	Technology	Purpose	Implementation
Vector Database	Meilisearch with vector support	In RAG, embeddings are used to retrieve relevant documents or pieces of information from a large corpus based on a user's query. The retrieved information is then used to generate a coherent and contextually accurate response.	Project knowledge retrieval
Document Processing	The RAG architecture consists of three key processes: Ingestion: Building a knowledge base by indexing data from multiple sources. Retrieval: Retrieving relevant information using semantic search methods. Generation: Combining the retrieved information with the model's output to produce a coherent and fact-based response.	Automated content analysis	Text chunking and embedding pipeline
LLM Integration	OpenAI GPT-4 API	Brief generation, meeting summaries, risk analysis	API-based integration with context injection

AI-Enhanced Features:





9.1.2 Feature Flag ManagementAgencyOS implements a comprehensive feature flag management system to enable controlled feature rollouts, A/B testing, and rapid deployment cycles without code changes.

Feature Flag Implementation Strategy:

Platform	Use Case	Justification	Implementation
Launch Darkly	Production feature management	Feature flags are a software development concept that allow you to enable or disable a feature without modifying the source code or requiring a re-deploy.	SaaS-based with enterprise features
Unleash	Self-hosted alternative	Unleash is an open-source library for feature management.	Open-source with custom

Platform	Use Case	Justification	Implementation
	ve	It simplifies your development workflow, accelerates software delivery, and empowers teams to control how and when they roll out new features to end users.	omization options

Feature Flag Categories:

Flag Type	Purpose	Lifecycle	Examples
Release Toggles	This type of feature flag is suitable for Trunk based development. In trunk-based development all the developers commit to a shared branch called trunk. Release toggles allow adding incomplete features to the shared branch.	Temporary	New design review interface, enhanced reporting
Experiment Toggles	They are used to perform A/B Testing. The flag would evaluate either true or false based on the characteristics of the user.	Short-term	UI variations, pricing experiments
Operational Toggles	They are used for operational aspects of the system For instance, the restaurant system can disable cash on delivery payments in order to support contactless delivery due to the pandemic.	Long-term	Maintenance mode, integration toggles
Kill Switch	A permanent flag that enables or disables non-core functionality.	Permanent	Third-party service fallbacks, resource-intensive features

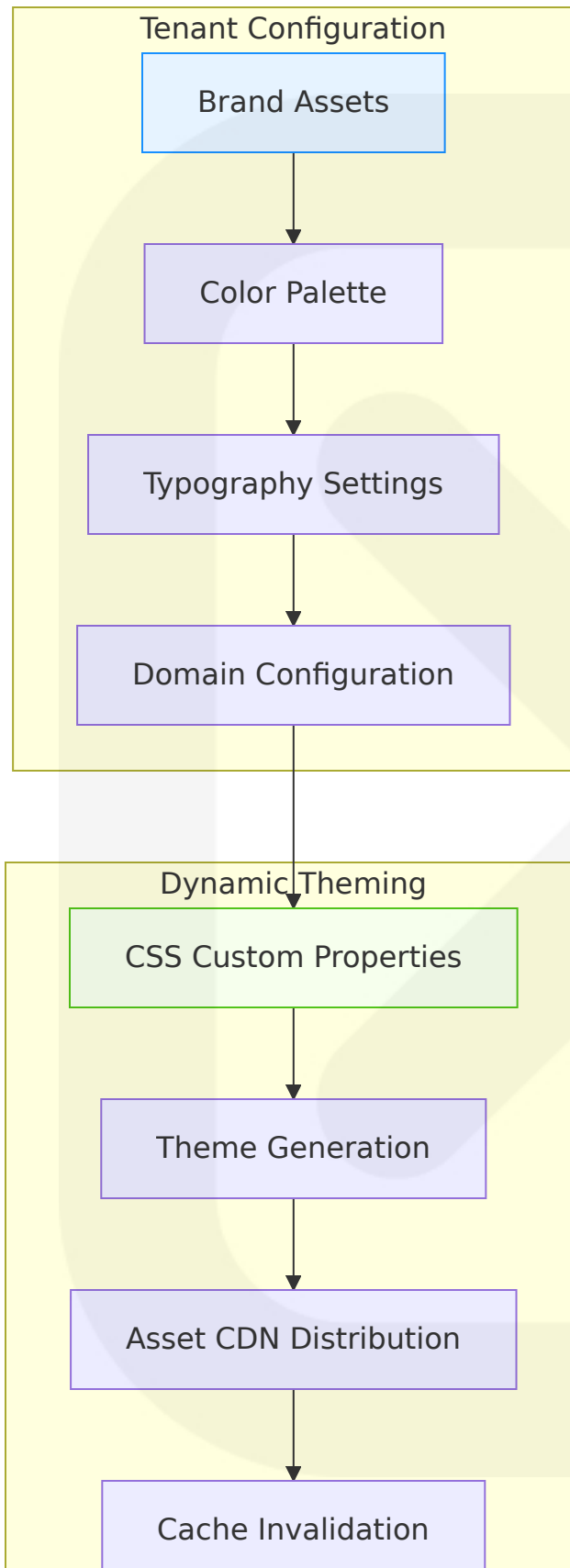
9.1.3 White-Label Client Portal Configuration

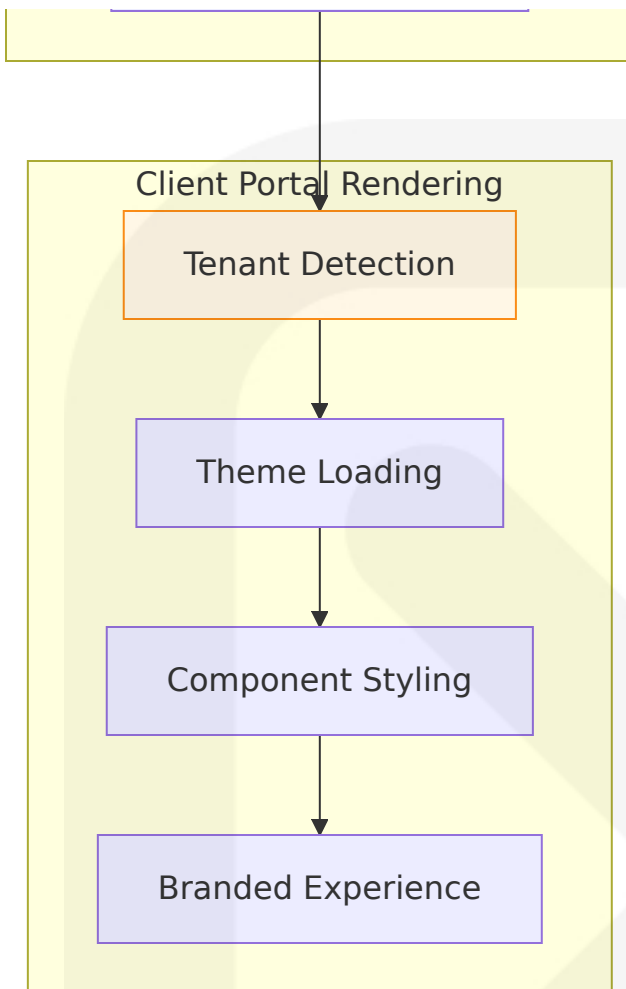
AgencyOS supports comprehensive white-labeling capabilities for client portals, enabling agencies to provide branded experiences that align with their corporate identity.

White-Label Configuration Options:

Customization Level	Features	Implementation	Client Impact
Visual Branding	Custom logos, color schemes, typography, favicon	CSS custom properties, dynamic theme loading	Seamless brand experience
Domain Configuration	Custom subdomains, CNAME records	Next.js middleware routing, SSL certificate management	Professional domain presence
Content Customization	Custom messaging, terms of service, privacy policy	CMS-like content management	Localized legal compliance
Feature Visibility	Selective feature exposure, custom navigation	Feature flag integration, role-based UI	Tailored functionality

Multi-Tenant Branding Architecture:





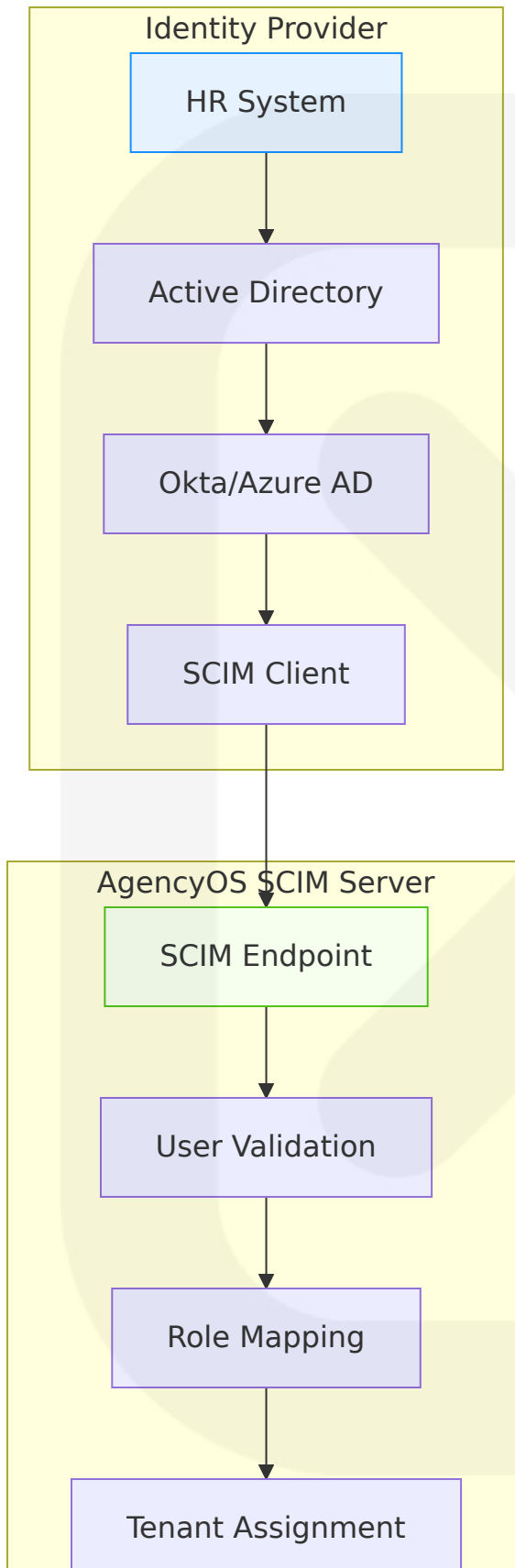
9.1.4 SCIM Provisioning

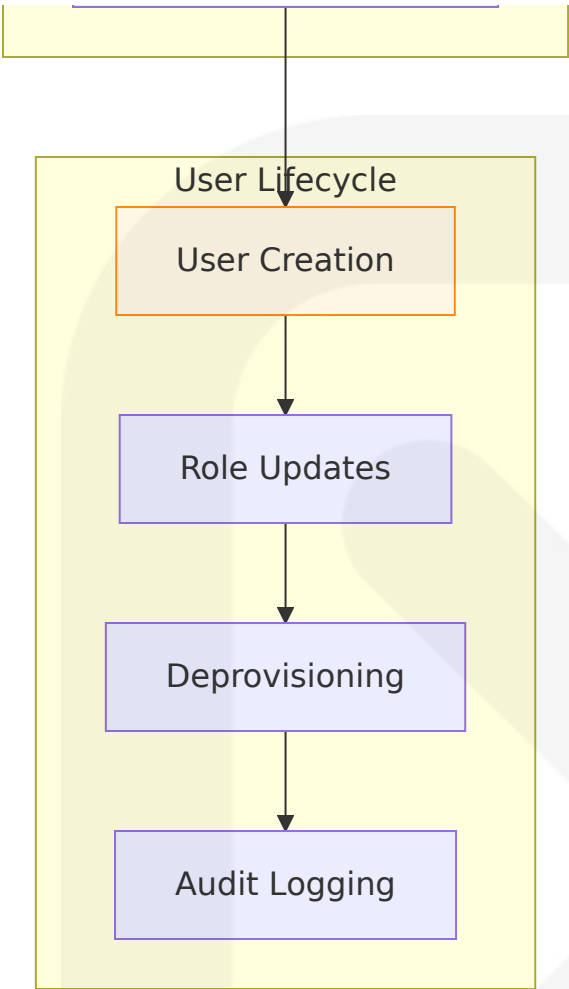
IntegrationAgencyOS implements SCIM (System for Cross-domain Identity Management) provisioning to enable automated user lifecycle management and seamless integration with enterprise identity providers.

SCIM Implementation Architecture:

Component	Technology	Purpose	Integration Points
SCIM Server	Users can enable SCIM provisioning for their existing SAML applications by configuring SCIM connections through the Stytlch dashboard or API.	User provisioning automation	Identity providers, HR systems
User Schema	SCIM provisioning uses a schema, which defines what data can be stored and managed for certain users and resources. For example, a specific username and email address must be stored with the affiliated user.	Standardized user attributes	Multi-tenant user management
Provisioning Operations	Create: This operation provisions new user and makes a record of their identity on the SCIM data store. Read: The read operation fetches user identities from the data store. Update: This makes changes to the user identities e.g user changing roles when an employee is promoted. Delete: The delete operation removes existing resource types from the SCIM data store e.g deleting employee records.	CRUD operations	Automated user life cycle

SCIM Integration Benefits:





9.1.5 Advanced Search Implementation

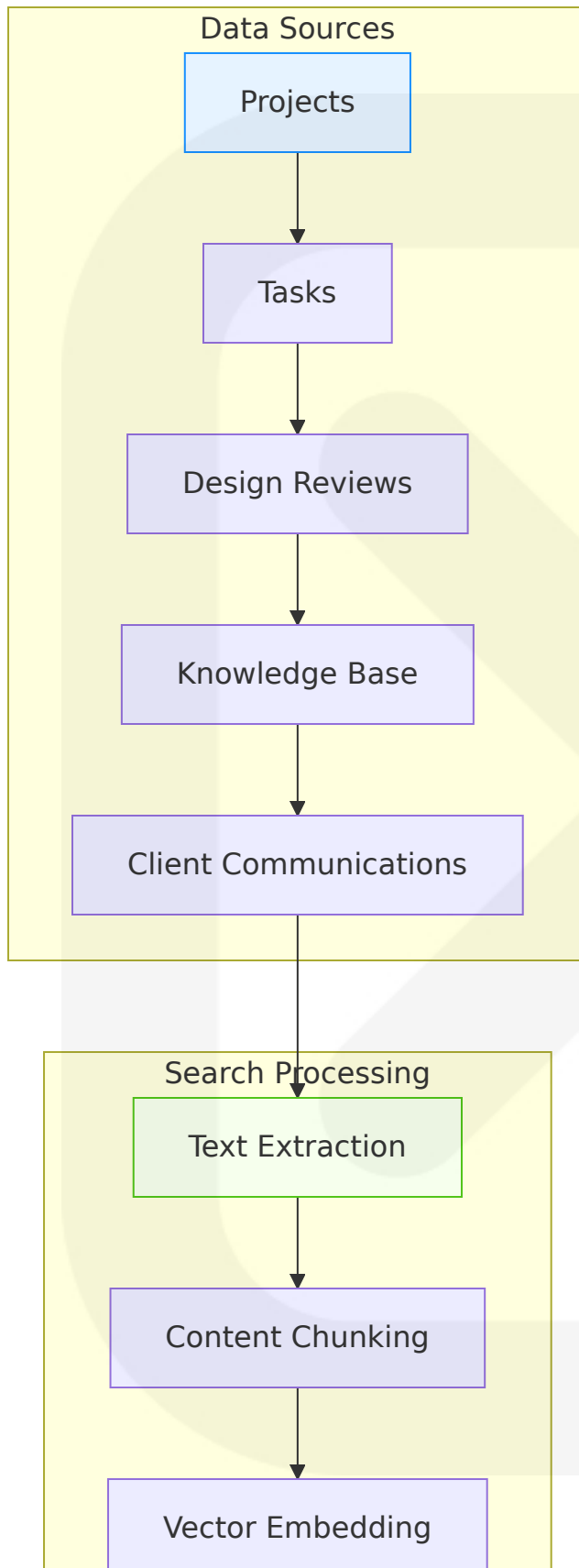
AgencyOS implements sophisticated search capabilities using Meilisearch with vector search support for semantic project discovery and knowledge retrieval.

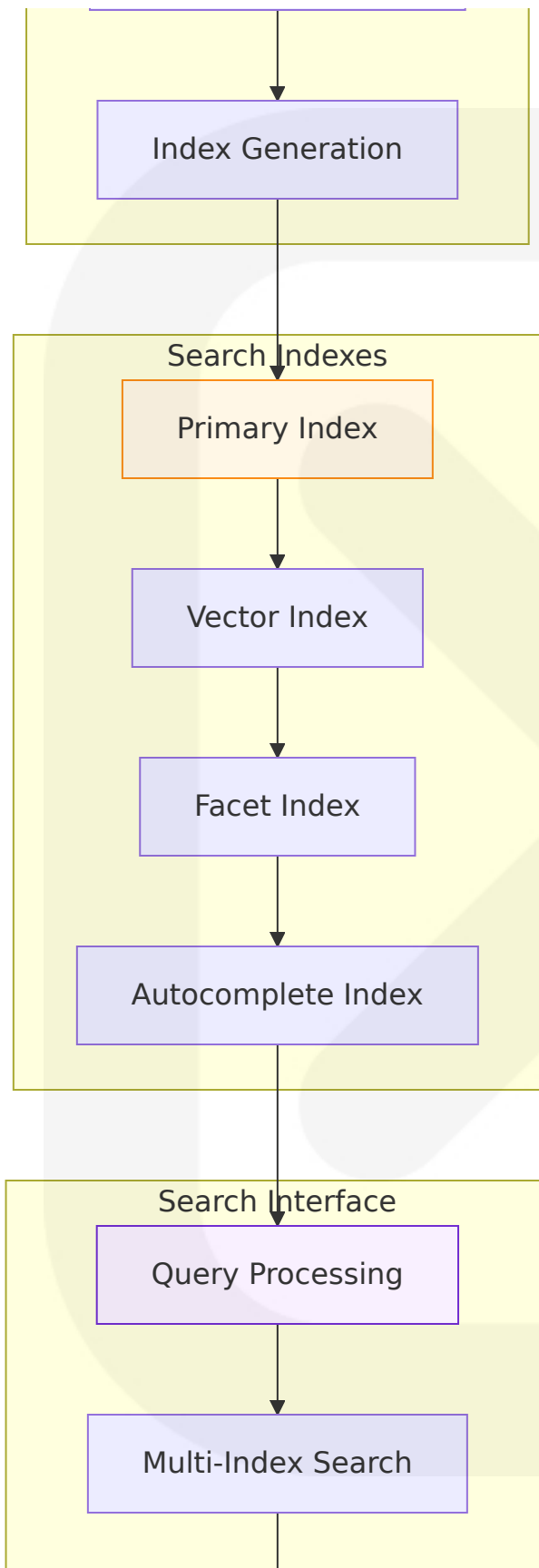
Search Architecture Components:

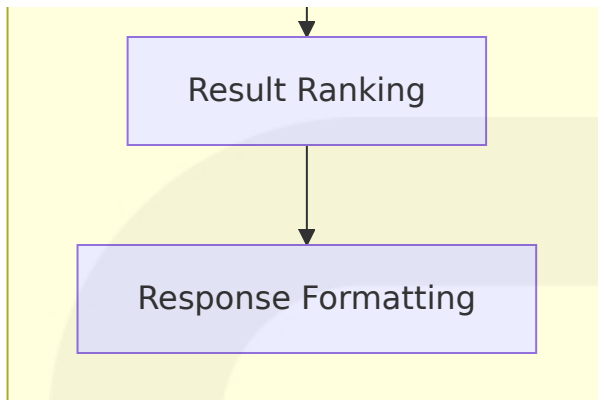
Search Type	Technology	Use Case	Performance Target
Full-Text Search	Meilisearch with typo tolerance	Project names, client information, task descriptions	<500ms p95
Semantic Search	Vector embeddings + similarity	Knowledge base, past SOW retrieval	<1s p95

Search Type	Technology	Use Case	Performance Target
	earch		
Faceted Search	Meilisearch filters	Project status, date ranges, team members	<200ms p95
Autocomplete	Meilisearch instant search	Real-time search suggestions	<100ms p95

Search Index Strategy:







9.2 GLOSSARY

Term	Definition
Agency	A digital design and development service provider that manages client projects and deliverables
Approval Cycle	The time duration from design submission to final client approval
Asset Library	Centralized repository for storing and managing design files, documents, and project deliverables
Budget Guardrails	Automated alerts and controls that prevent project budget overruns
Change Request (CR)	Formal process for modifying project scope, timeline, or budget
Client Portal	Dedicated interface for clients to view project status, approve deliverables, and manage invoices
Design Review	Collaborative process for reviewing and approving design deliverables with annotation capabilities
DSO (Days Sales Outstanding)	Average number of days it takes to collect payment after a sale
Multi-Tenant	Software architecture where a single instance serves multiple organizations with data isolation

Term	Definition
Project Workspace	Centralized environment for managing all aspects of a client project
Scope Creep	Uncontrolled expansion of project requirements beyond the original agreement
SOW (Statement of Work)	Legal document defining project scope, deliverables, timeline, and payment terms
UAT (User Acceptance Testing)	Final testing phase where clients validate deliverables before approval
Utilization Rate	Percentage of billable hours worked compared to total available hours
White-Label	Customizable branding that allows agencies to present the platform as their own

9.3 ACRONYMS

Acronym	Expanded Form
ABAC	Attribute-Based Access Control
API	Application Programming Interface
CDN	Content Delivery Network
CI/CD	Continuous Integration/Continuous Deployment
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
DSO	Days Sales Outstanding
EKS	Elastic Kubernetes Service
GDPR	General Data Protection Regulation
HPA	Horizontal Pod Autoscaler
IAM	Identity and Access Management
IaC	Infrastructure as Code

Acronym	Expanded Form
JWT	JSON Web Token
KPI	Key Performance Indicator
MFA	Multi-Factor Authentication
MTBF	Mean Time Between Failures
MTTR	Mean Time To Recovery
OIDC	OpenID Connect
ORM	Object-Relational Mapping
PWA	Progressive Web Application
QoS	Quality of Service
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RLS	Row-Level Security
RPO	Recovery Point Objective
RTO	Recovery Time Objective
SAML	Security Assertion Markup Language
SCIM	System for Cross-domain Identity Management
SDK	Software Development Kit
SLA	Service Level Agreement
SLO	Service Level Objective
SOC	Service Organization Control
SOW	Statement of Work
SSO	Single Sign-On
TTI	Time to Interactive
UAT	User Acceptance Testing
VPA	Vertical Pod Autoscaler

Acronym	Expanded Form
WAF	Web Application Firewall
WCAG	Web Content Accessibility Guidelines